Oracle SOA Suite 12*c*: Essential Concepts

Activity Guide

D84365GC10

Edition 1.0

August 2014

D87597

ORACLE

Author
Iris Li

Technical Contributors and Reviewers
David Mills, Jay Kasi, Drishya Tm, William Prewitt, Phoebe He, Mary Peek, Simone Geib

Editor
Malavika Jinka

Graphic Designer
Seema Bopaiah

Publishers
Jobi Varghese, Nita Brozowski

This book was published using: Oracle Tutor

Table of Contents

Practices for Lesson 1: Introduction

Chapter 1

Practices for Lesson 1

Practices Overview

There are no practices for this lesson titled “Oracle SOA Suite 12c: Essential Concepts Edition 1 – Course Introduction”.

General Notes

There are no notes.

Practices for Lesson 2: Introducing Service-Oriented Architecture Concepts

Chapter 2

Practice 2-1: Understanding the Problems

In this course, all practices are based on a case study about a fictitious company, AviTrec, which is experiencing business challenges while adopting SOA. This practice covers the business challenges and project requirements. Your task is to provide solutions by applying the knowledge that you gain in this practice. As we proceed through the lessons and practices, you will gain a better understanding about Oracle SOA Suite business solutions.

Company Background

AviTrec.com is an e-commerce website where people can buy and sell outdoor gear and sporting goods. The company has embarked upon a SOA program to align with its business goals of improving customer satisfaction and increasing market share. Like other companies adopting SOA, AviTrec faces a series of business challenges:

- It has interactions with external parties (including business partners).
- It needs to create more efficient business processes across departments that have automated and manual tasks.
- It needs to implement security.
- It faces changing requirements with ever shorter times-to-market.
- It hopes to gain more real-time insight into the current state of affairs.

Problem Statement

AviTrec's current order management system is based on a single-purpose, monolithic application. One key issue in this system is that credit card payments are often denied for various, sometimes minor reasons, such as an expired date.

Because the process to correct these issues varies across its order entry systems, AviTrec lacks a consistent process to follow up and resolve customer problems. Orders that are lost and delayed in the system cause customer dissatisfaction.

AviTrec plans to expand its business by taking on more online business partners, but its existing system cannot accommodate multichannel business partners.

Project Requirements and Solution Technologies

The following table summarizes the requirements for improving the order management system:

Requirement	Solution Technology
Provide a consistent interface to all order entry applications for credit validation.	
Do not disrupt order processing operations after credit checks are outsourced to a third-party credit provider. (Currently, credit is checked by internal departments.)	
During holiday seasons, allow manual overrides if a customer's total order amount is marginally over the daily limit.	
Ensure that the order processing system is accessible through multiple protocols, data formats, and client types: • Support access through RESTful APIs	

• Interface with trading partners and provide electronic data interface (EDI) support • Receive orders through a different channel as batch CSV files over FTP	
Collect analytic data to gain business insights into customer order requests.	

To the best of your knowledge, provide the solutions in the Solution Technology column. At this point, you do not need to know the solutions for all requirements. In the following lessons, you will learn more about SOA technologies and find the solutions for each requirement.

Solution 2-1: Understanding the Problems

Requirement	Solution Technology
Provide a consistent interface to all order entry applications for credit validation.	WSDL, web service
Do not disrupt order processing operations after credit checks are outsourced to a third-party credit provider. (Currently, credit is checked by internal departments.)	OSB
During holiday seasons, allow manual overrides if a customer's total order amount is marginally over the daily limit.	Business rules Human workflow
Ensure that the order processing system is accessible through multiple protocols, data formats, and client types: a. Support access through RESTful APIs b. Interface with trading partners and provide electronic data interface (EDI) support c. Receive orders through a different channel as batch CSV files	a. REST adapter b. B2B c. File adapter, Mediator
Collect analytic data to gain business insights into customer order requests.	BAM

Practices for Lesson 3: Getting Started with Oracle SOA Suite 12c

Chapter 3

Practices for Lesson 3

Practices Overview

Your lab machine is preinstalled with the Quick Start distribution. For this course, you will use JDeveloper's Integrated WebLogic Server's default domain for all the practices.

In this lesson's practices, you first configure the Integrated WebLogic Server default domain, then deploy a SOA composite application to the integrated server, and then use Oracle Enterprise Manager 12*c* Fusion Middleware Control console to initiate the deployed SOA composite and inspect the composite's instance.

Practice 3-1: Configuring the Integrated WebLogic Server's Default Domain in JDeveloper

Overview

By default, the Oracle SOA Suite Quick Start installation contains Oracle JDeveloper and an Integrated WebLogic Server.

In this practice, you will launch JDeveloper and configure the Integrated WebLogic Server's default domain in JDeveloper.

Tasks

1. Preparing your environment

   Set the `JDEV_USER_DIR` environment variable to define the location of the default domain. This will also determine where to save new applications created in Oracle JDeveloper.

   Open a terminal window, enter the following command:

   ```
   export JDEV_USER_DIR=$HOME/domains/DefaultDomain
   ```

   Note: If you do not set JDEV_USER_DIR, the domain is created at the default location:

   ```
   $HOME/.jdeveloper/system12.1.3.x.x.x.xxxx/DefaultDomain
   ```

2. Launching JDeveloper

   a. In the same terminal window, start JDeveloper by entering the command:

   ```
   $MW_HOME/jdeveloper/jdev/bin/jdev
   ```

   As Oracle JDeveloper launches, you will get three prompts.

   b. When prompted for role, select **Studio Developer** as your role, deselect the “Always prompt for role selection on startup” option, and click **OK**.

   c. When prompted, say **No** to import preferences from a previous JDeveloper installation.

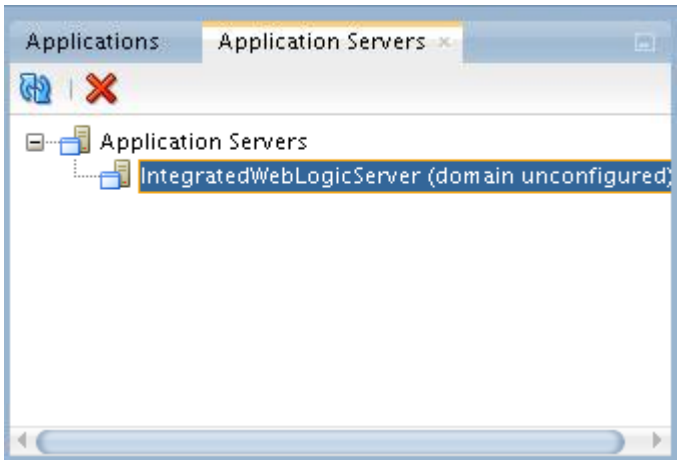
   d. When prompted about Oracle Usage Tracking, check whether or not you want Oracle JDeveloper to send automated usage reports to Oracle and press OK.

   JDeveloper has fully launched at this point.

3. Creating Default Domain and launching the Integrated WebLogic Server

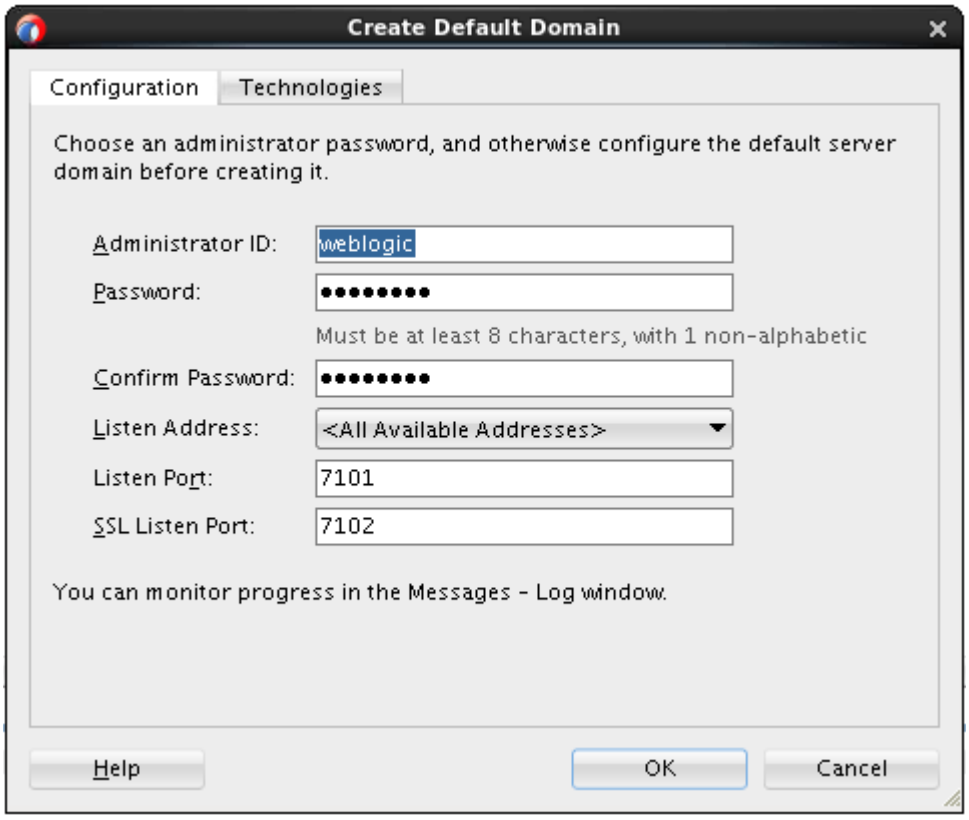
   a. From the JDeveloper main menu, select Window > Application Servers.

   b. In the Application Servers view, expand the Application Servers node, you should see a domain unconfigured integrated WebLogic server as shown in the following image:



   c. Right-click the IntegratedWeblogicServer node, and, from the context menu, select Create Default Domain.

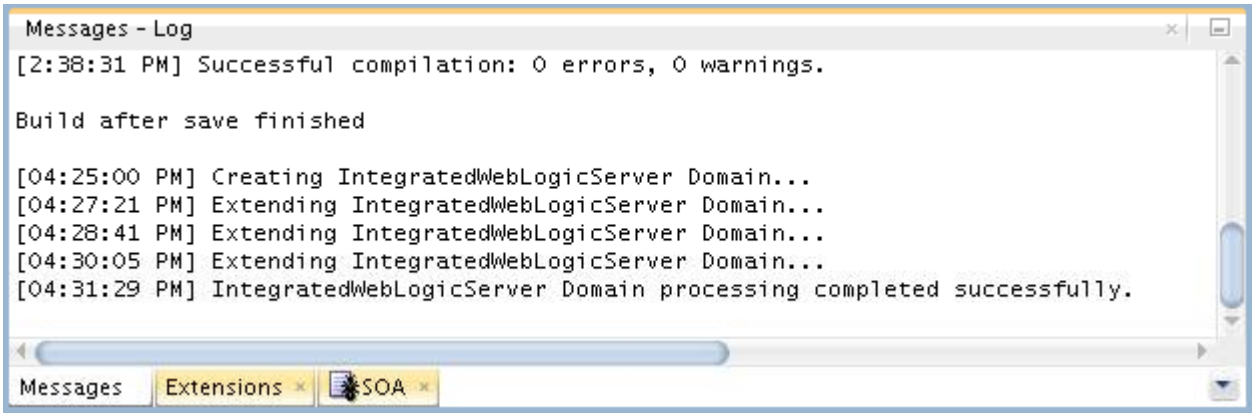
d. In the Create Default Domain wizard, on the Configuration tab page, enter **`welcome1`** as password, and accept the defaults for all other fields. Click **OK**.



e. The domain creation process starts and it takes several minutes.

You can track the progress in the Messages window pane. This window should automatically open at the bottom of the JDeveloper screen. If it is not there, you can open it by selecting Window from the top bar menu and Log from the drop-down menu.

f. When you see the following messages appear in the log, the default domain has been created successfully.



4. In the Application Servers view, right-click the IntegratedWeblogicServer node, and, from the context menu, select Start Server Instance. This will take a few minutes. You should see a “SOA Platform is running and accepting requests” message in the Running: IntegratedWebLogicServer – Log window.
5. To verify that the default domain is configured properly and the integrated server is started, perform the following steps:

a. Open a Firefox browser, access Enterprise Manager Fusion Middleware Control with the following URL:

   `http://localhost:7101/em`

b. Log in using the username and password that you specified when configuring the default domain:

   User Name: `weblogic`

   Password: `welcome1`

   When you first log in, Fusion Middleware Control is displayed with the target navigation pane on the left and the domain home page on the right.

c. You can expand SOA > service-bus > default and SOA > soa-infra > default. At this point both folders are empty.

Practice 3-2: Deploying and Testing a SOA Composite

Overview

Once AviTrec identified that the existing credit card payment process caused lots of problems in their system, they created a new payment composite to provide a consistent interface for all order entry applications for credit validation.

The composite will validate credit card payments and return the payment status. If the payment is denied, the order will not be processed.

In this practice, you deploy the ValidatePayment composite application to the integrated server, and then initiate an instance and examine the process flow in Fusion Middleware Control console.

Assumption

The Integrated WebLogic Server is up and running.

Tasks

High level steps:

- Deploying the ValidatePayment composite application using a command
- Testing and examining the deployed composite

Deploying the composite

1. In a terminal window, enter the following commands:

```
cd /labs/lessons/lesson03
ls -l
./deploy.sh sca_ValidatePayment_rev1.0.jar
```

2. The build and deployment information will be displayed in the console. You should see a message showing both build and deploying executed successfully.

Testing and examining the deployed composite

For ValidatePayment composite, all available credit cards are stored in the database, including payment type, card number, expiry date, card name and daily limit.

The validation process includes three steps:

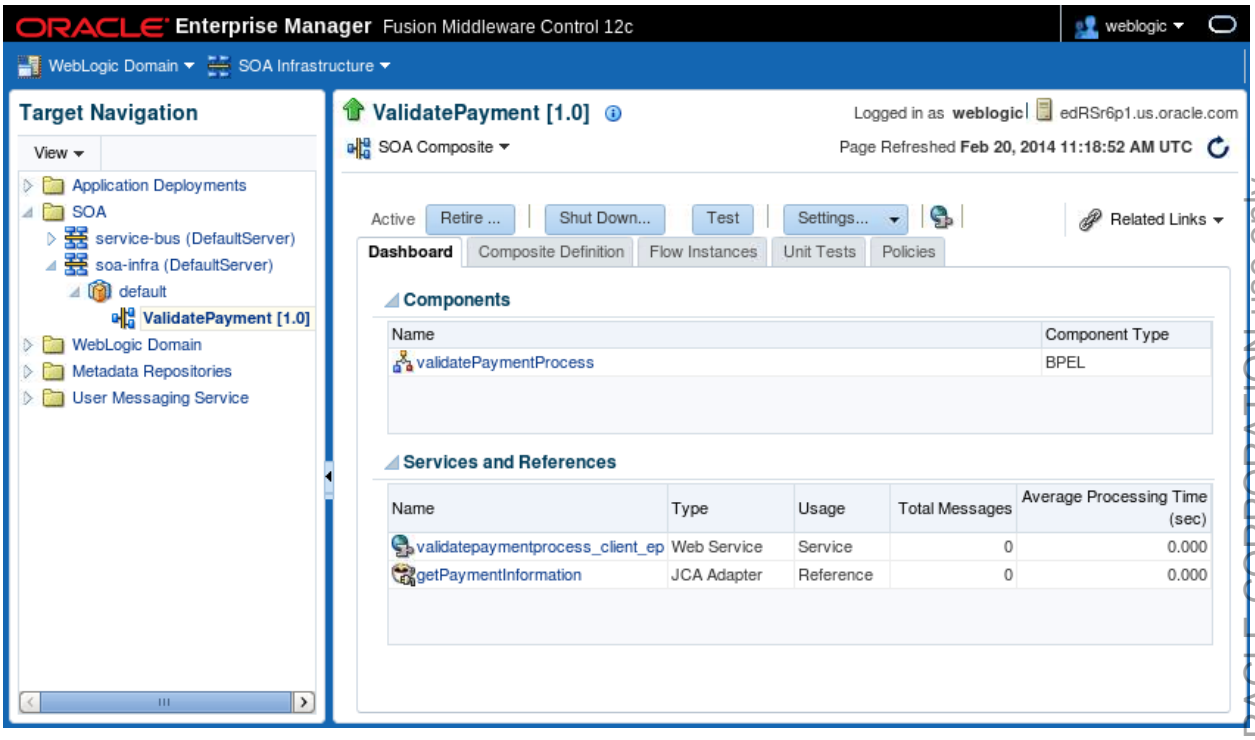
1) First, the payment information is retrieved from the database, using the credit card number quoted in the order message as the key. If there is no data available with this credit card number, payment is denied.
2) If data for the credit card number is available, the expiry date in the database record is compared to the expiry date listed in the order message. If they are not the same, the payment is also denied.
3) The last check compares whether the total order amount is less than the daily limit on the credit card in the database.

If all tests are successful, the payment is authorized. Otherwise it is denied.

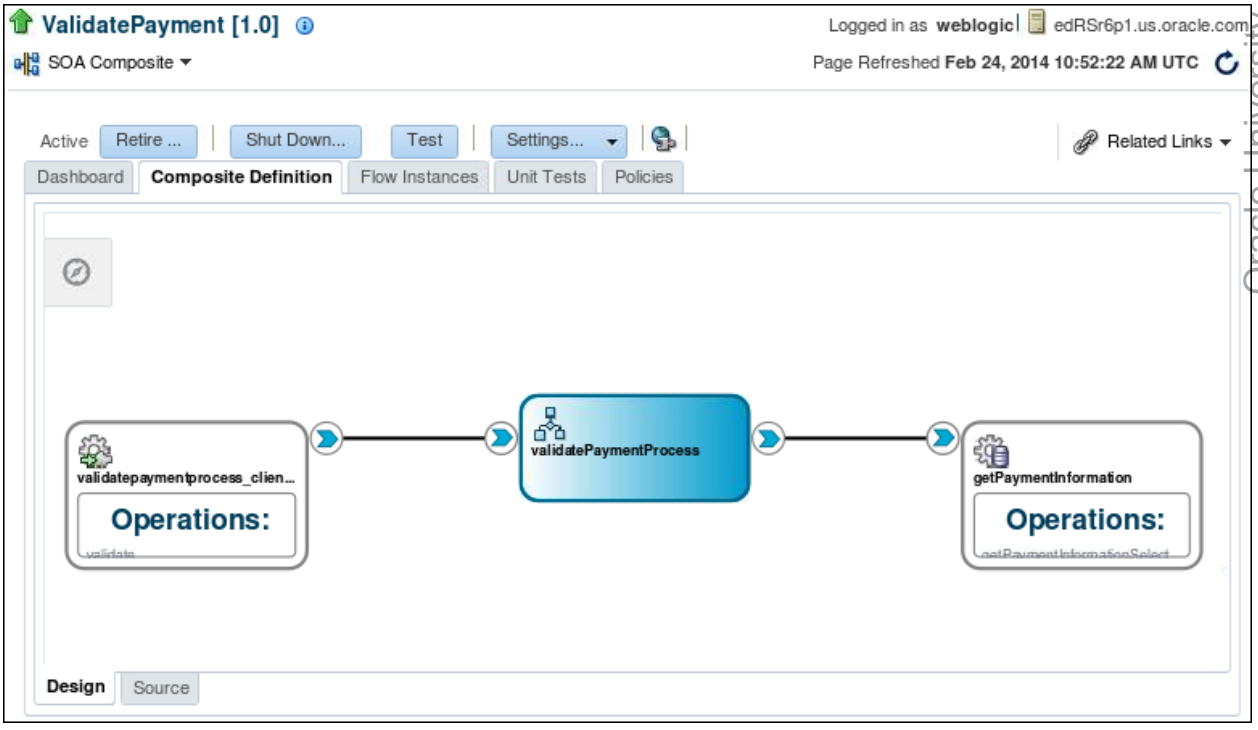
3. Examine the SOA composite application.
   a. Go to the Fusion Middleware Control console, and, in Target Navigation panel, expand SOA > soa-infra > default. Now you should see the deployed ValidatePayment [1.0] composite application.

b. Click the ValidatePayment [1.0] link. The home page (Dashboard tab page) for the selected SOA composite application is displayed. You can view the service components and service and reference binding components included in the composite.

Note: This page also enables you to perform composite management tasks such as retire, activate, shut down, start up, and test.

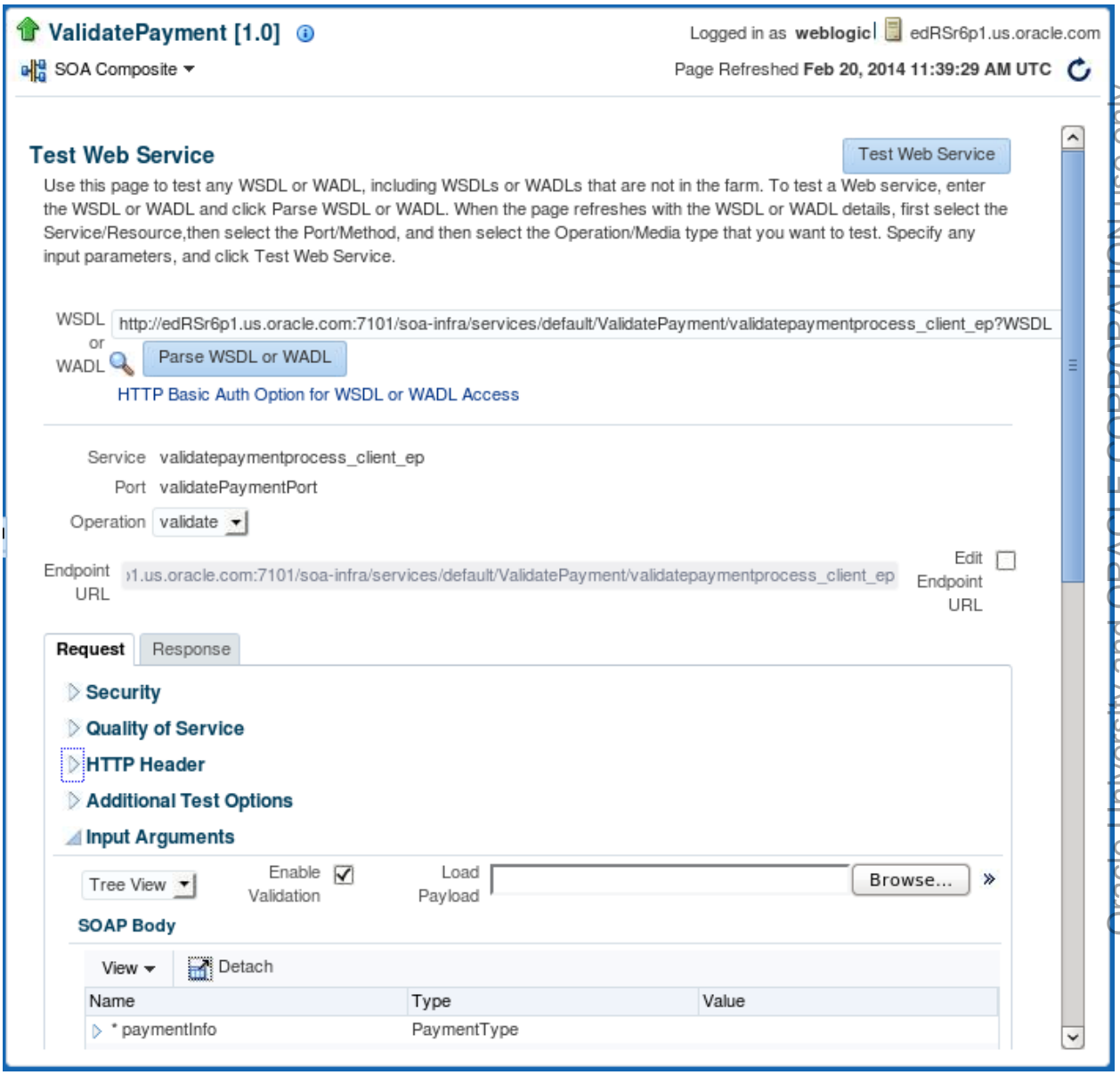


c. Click the Composite Definition tab. You should see a graphical display of the SOA composite applications designed in Oracle JDeveloper 12*c*. This view is similar to the display of the SOA composite application in Oracle JDeveloper.



Here you can see that the implementation of this service uses a BPEL process to receive the incoming payment information and then retrieve the credit card data from the database and perform the tests outlined in step 2.

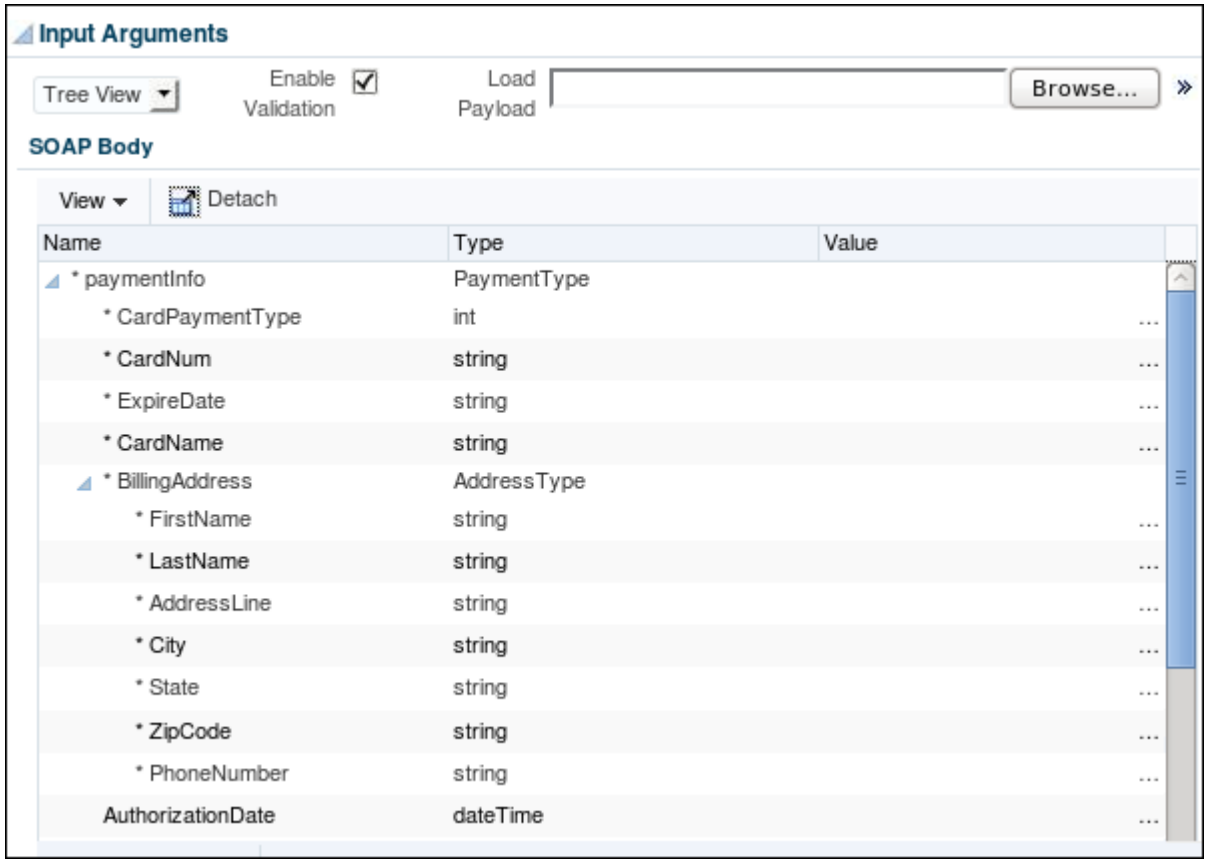
4. Initiate and test the SOA composite application.
   a. On the ValidatePayment [1.0] home page, click Test.
   b. On the Test Web Service page, accept all default settings. Make sure that the Request tab is selected.



   c. Scroll down to the **Input Arguments** section.

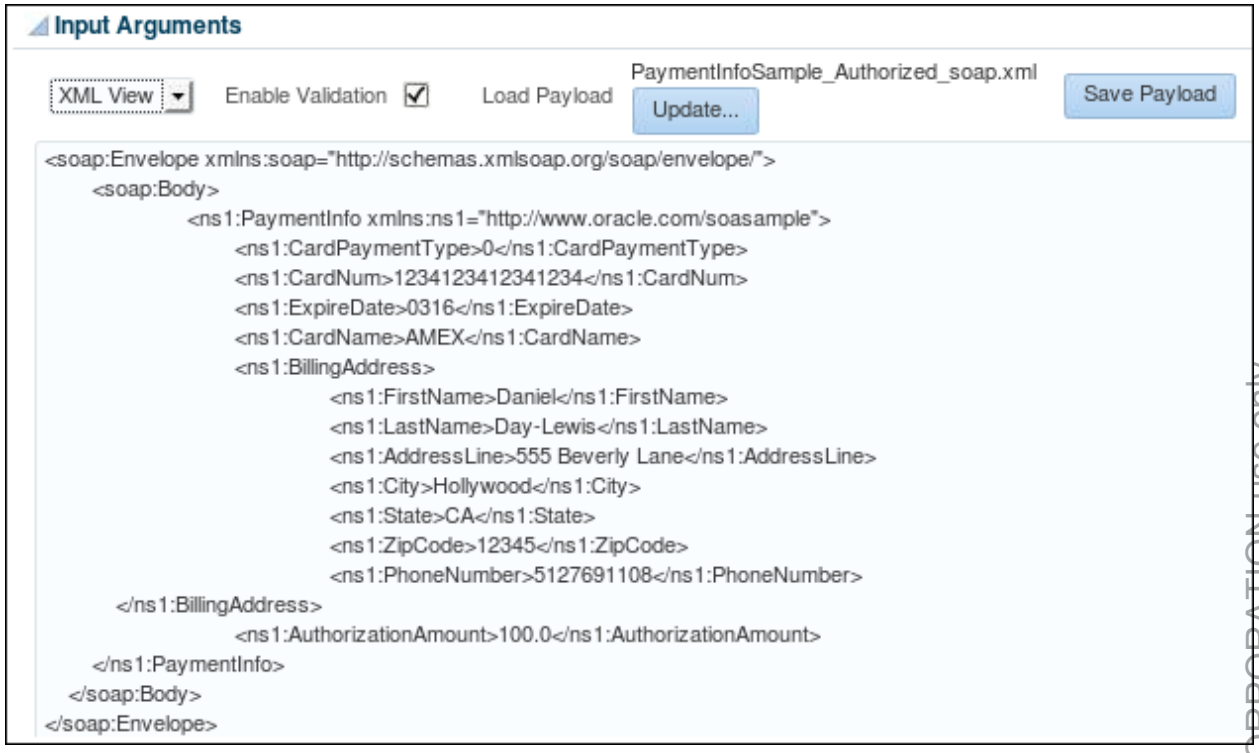
      To enter XML payload data, you have options of selecting Tree View (default) or XML View.

      - Tree View: Displays a graphical interface of text fields in which to enter information.
      - XML View: Displays the XML file format for inserting values. You can paste the raw XML payload of your message into this field.

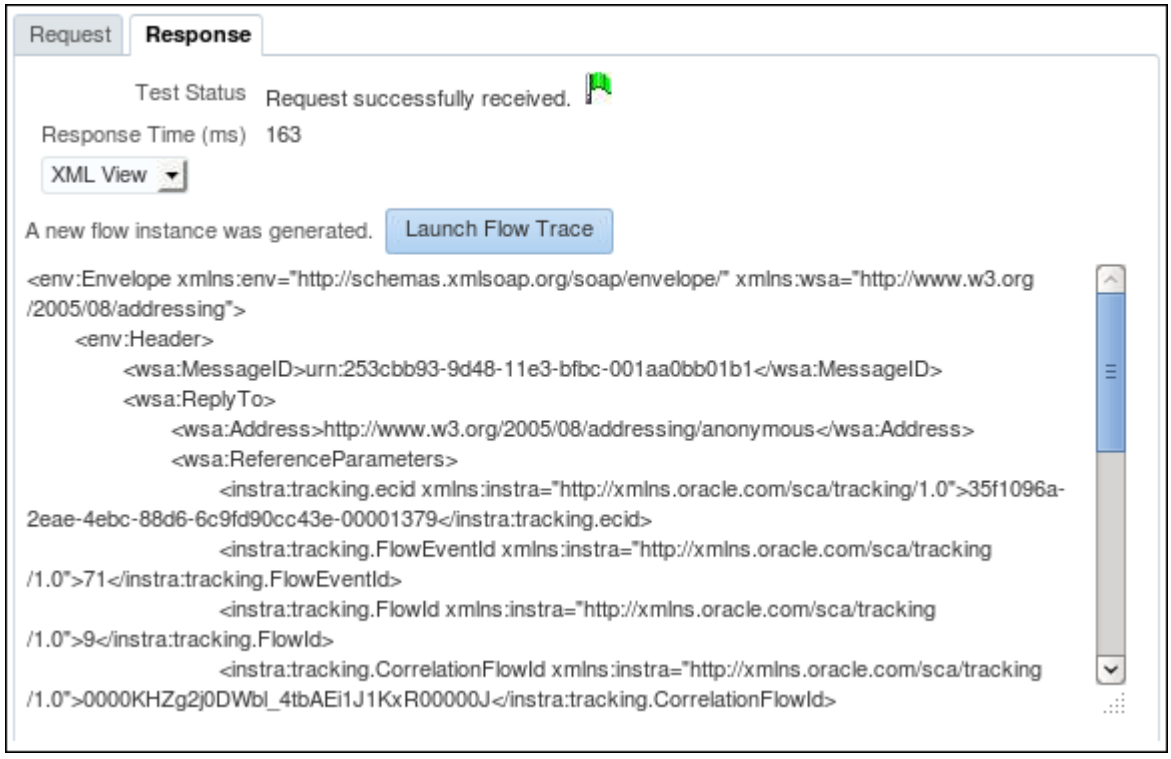


d. Select XML View. You should see the web service invocation payload in XML format.
Tip: You can easily copy-and-paste your data into the XML View.

e. To load the test message, click Browse next to the Load Payload field, and select the `/labs/resources/sample_input/PaymentInfoSample_Authorized_soap.xml` message.
This message already includes the SOAP envelope needed for the web service test.



f. Click Test Web Service. When the composite completes, the screen switches to the Response tab and the returned value is shown (similar to the screenshot below).



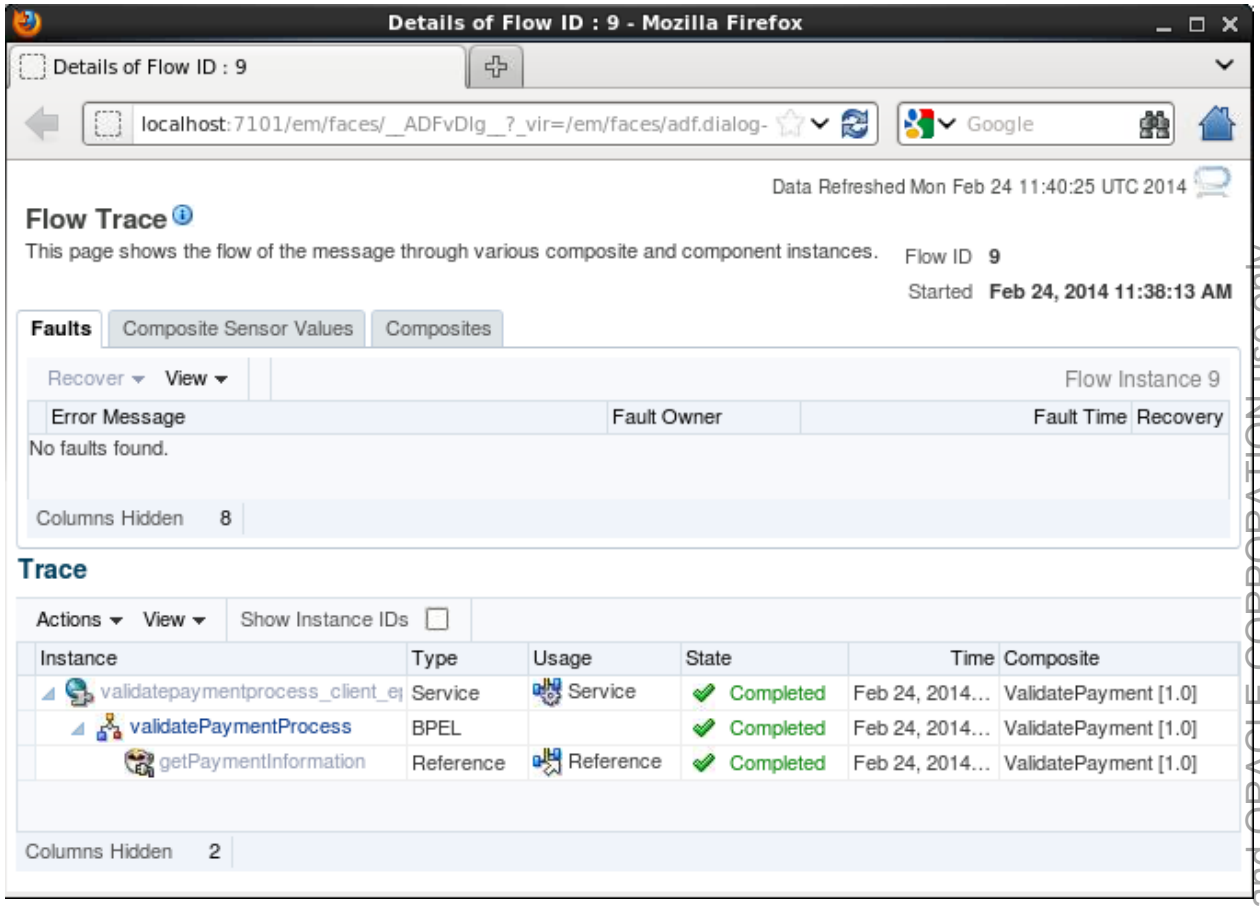
g. Review the response message, and you should see that the payment status is Authorized.

Note: You can also switch to Tree View to view the result.

5. From here, you launch the Flow Trace of this instance.

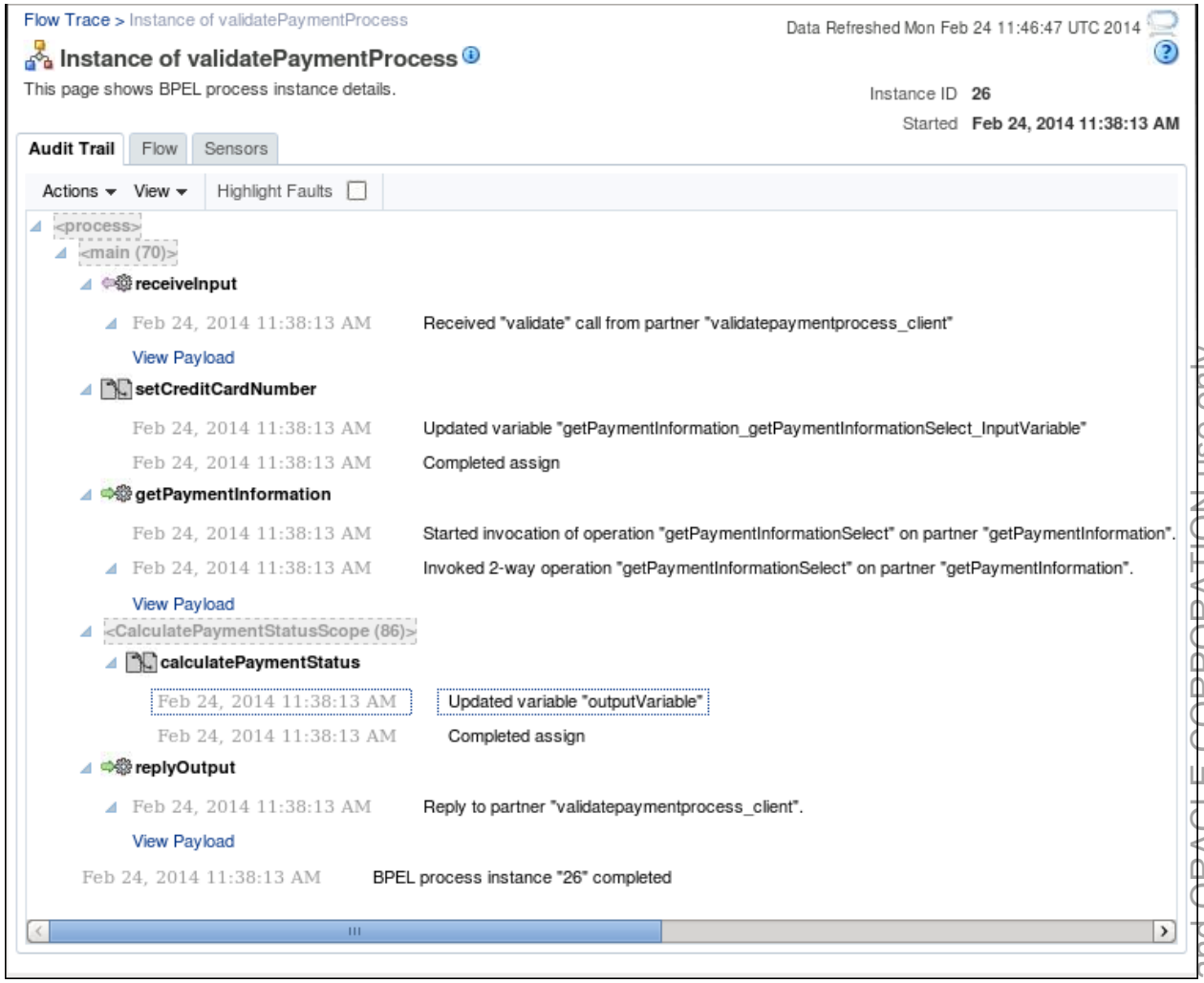
a. Click Launch Flow Trace.

The Flow Trace window opens. You can see the flow of the composite and the status of each service, component, and reference.



b. Click the BPEL process link to view the detailed execution trail.

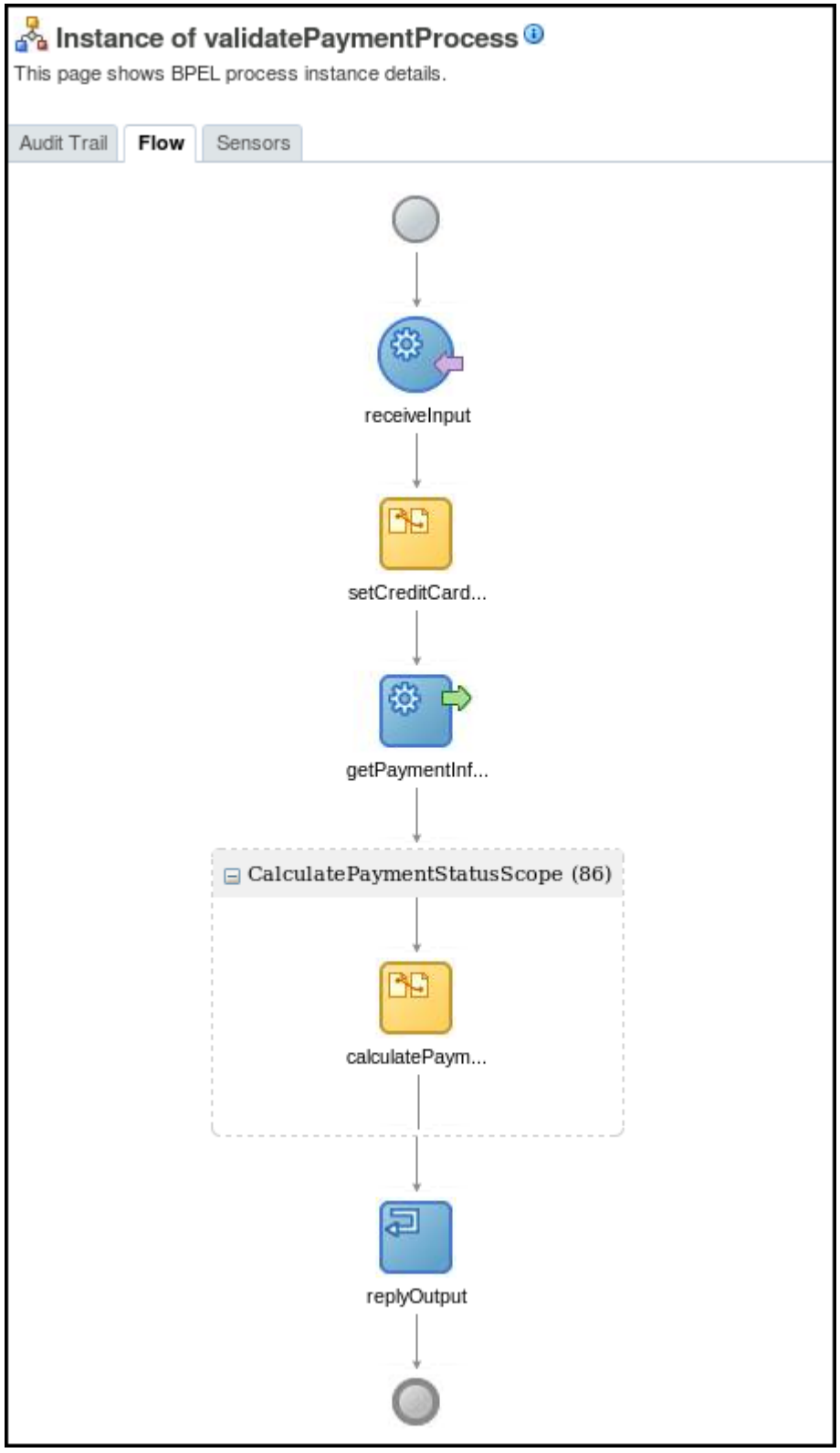
The audit trail of the BPEL process activities is displayed in a new window.



1) Click ViewPayload under receiveInput. What is the Authorization Amount?
2) Click ViewPayload under getPaymentInformation. Examine the message and answer the following questions:
   - What is the input variable?
   - What is the output variable?
   - What is the dailyLimit of the credit card?

   Tip: Make a note of the values of input and output variables. This information will be used when you examine the composite structure/assembly model (in order to understand how the composite work) in later practices.
3) Click ViewPayload under replyOutput to view the payment status.

c. You can also click the Flow tab to view the process flow.



Practices for Lesson 4: Building SOA Composite Applications

Chapter 4

Practices for Lesson 4

Practices Overview

In previous practices, you deployed and tested the ValidatePayment composite application on the integrated server, and got an idea of how the validation process works. In these practices, you will examine the structure/assembly model of this composite application in JDeveloper, and get familiar with its service interface described in the WSDL file.

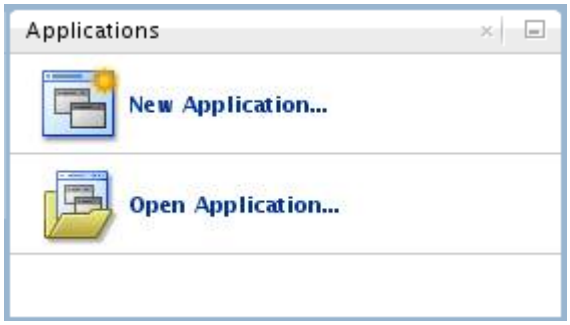
Practice 4-1: Examining the Composite Application in JDeveloper

Overview

In this practice, you examine the Oracle SOA Suite 12*c* composite that validates a credit card payment. This service will be leveraged in a later practice, when you build the ProcessOrder composite.

Tasks

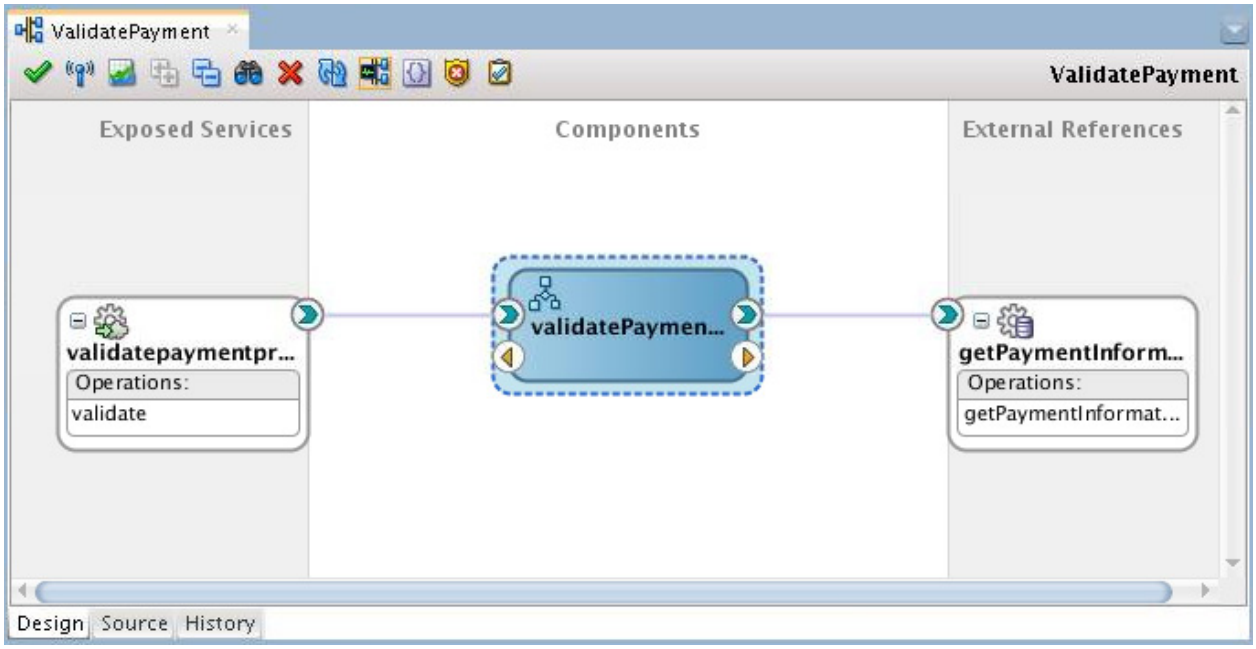
1. Open JDeveloper if it is not open. (You can use the JDeveloper shortcut on the desktop.)
2. In the Applications pane, click **Open Application** (or you can select **File** > **Open**).



3. In the Open Application(s) dialog box, navigate to the `/labs/lessons/Lesson04/po_composite/` directory, and open the `po_composite.jws` file.
4. If prompted for migrating files, select Yes. The ValidatePayment project folder is displayed on the left side in the Application pane.
5. Expand ValidatePayment > SOA folder, you should see a collection of application files and artifacts.

   The SOA folder has a number of subfolders with default names, which hold common SOA artifacts, like XML schemas, WSDL files, transformation-related files and events. You will see new subfolders created when creating new components. For example, when you create a new BPEL component, a `BPEL` folder will be created, which includes all `.bpel` files. The structure and names of the subfolders can be customized to your liking, as long as all folders are located under SOA.
6. Located directly under the SOA folder, you should see a file named `ValidatePayment`. (In previous releases, it was just shown as `composite.xml`.) This file contains the XML source defining the structure of the composite. Double-click the `ValidatePayment` file. The Composite Editor opens and presents the Design view of the composite.

   Note: On the bottom of the editor pane are three tabs: Design, Source, and History. The Design and Source tabs provide different views of the same source.



In the Design view, you can see the components from which the composite application is assembled and how they are related. Try to answer the following questions:

a. This composite is assembled from what service components?

   Answer: The composite only contains one service component, named `validatePaymentProcess`. It is implemented in BPEL 2.0. You can tell the component type by moving the mouse over the component.

   Note: Optionally, double-click the ValidatePaymentProcess BPEL process (This opens up the `validatePaymentProcess.bpel` file) to open the BPEL designer and view the BPEL activities.

b. Which service provided by the composite is accessible from outside the composite's domain?

   Answer: It is the service entry point, `validatePaymentProcess_client_ep`.

c. How does the BPEL service component access the credit card database?

   Answer: Through the `getPaymentInformation` service configured by a database adapter.

Examining the Exposed Service

7. Double-click the `validatepaymentprocess_client_ep` icon in the Exposed Services lane. In the Update Service dialog box, you can see that the service contract is defined in a WSDL document named `ValidatePaymentWrapper.wsdl`. A wrapper WSDL is a WSDL that imports other WSDLs, each having a single port type. In this case, it only contains one WSDL, `ValidatePayment.wsdl`. At the heart of the WSDL contract for a web service is the portType element, which contains operation definitions.

   Note: All WSDL files are located in project's SOA > WSDLs folder.

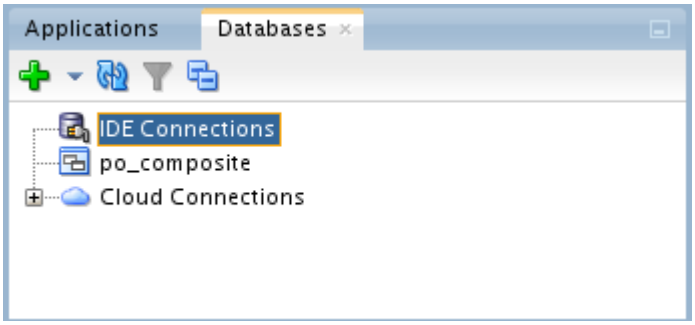
   Close the Update Service dialog box when you are done.

Examining the External Reference

In the External References lane is `getPaymentInformation`, a database adapter. This adapter enables a BPEL process to communicate with Oracle databases or third-party databases through JDBC.

The BPEL process calls the getPaymentInformation database adapter, and the DB adapter will query the table and fetch field values.

8. The adapter is already configured for the composite, but the database connection required by the adapter configuration is not available in your development environment. Therefore, you need to import the database connection. To do so, perform the following steps:
   a. From the JDeveloper main menu, select Window > Database >Databases. The Databases window is displayed in the upper-left corner.



   The po_composite connection appears in the view. Under po_composite, there is an empty folder PO.

   b. Right-click po_composite, and select Import Connections.

   The Import Connections wizard is displayed.

   c. In the Source File screen (Step 1):
      1) Click Browse.
      2) In the Choose Connections File dialog, navigate to `/labs/lessons/lesson04`, and select `PO_DBConnection.xml`.

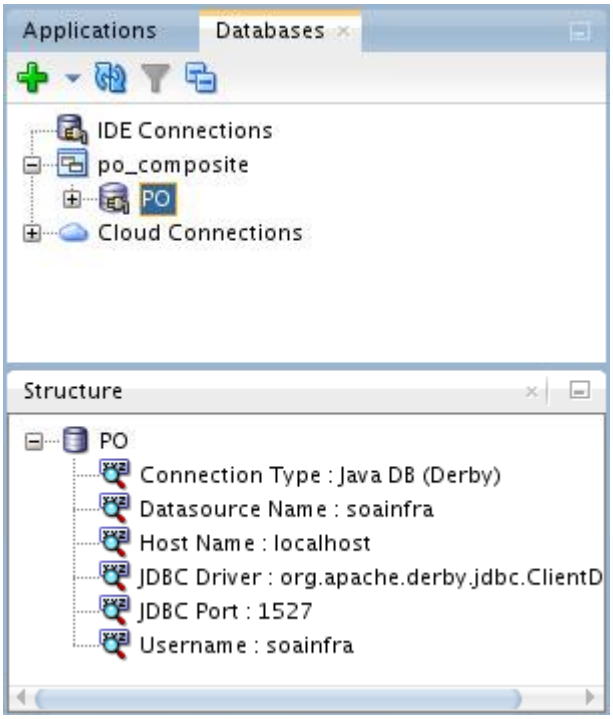
         Back in the Wizard's Source file screen, the Filename field is populated with the file path.
      3) Click Next.

   d. In the Password Handling screen (Step 2):
      1) Select the "Remove all passwords from the exported connections" option.
      2) Click Next.

   e. In the Select Connection screen (Step 3):
      1) Check PO_DBConnection.xml.
      2) Click Next.

   f. Review the summary, and click Finish.

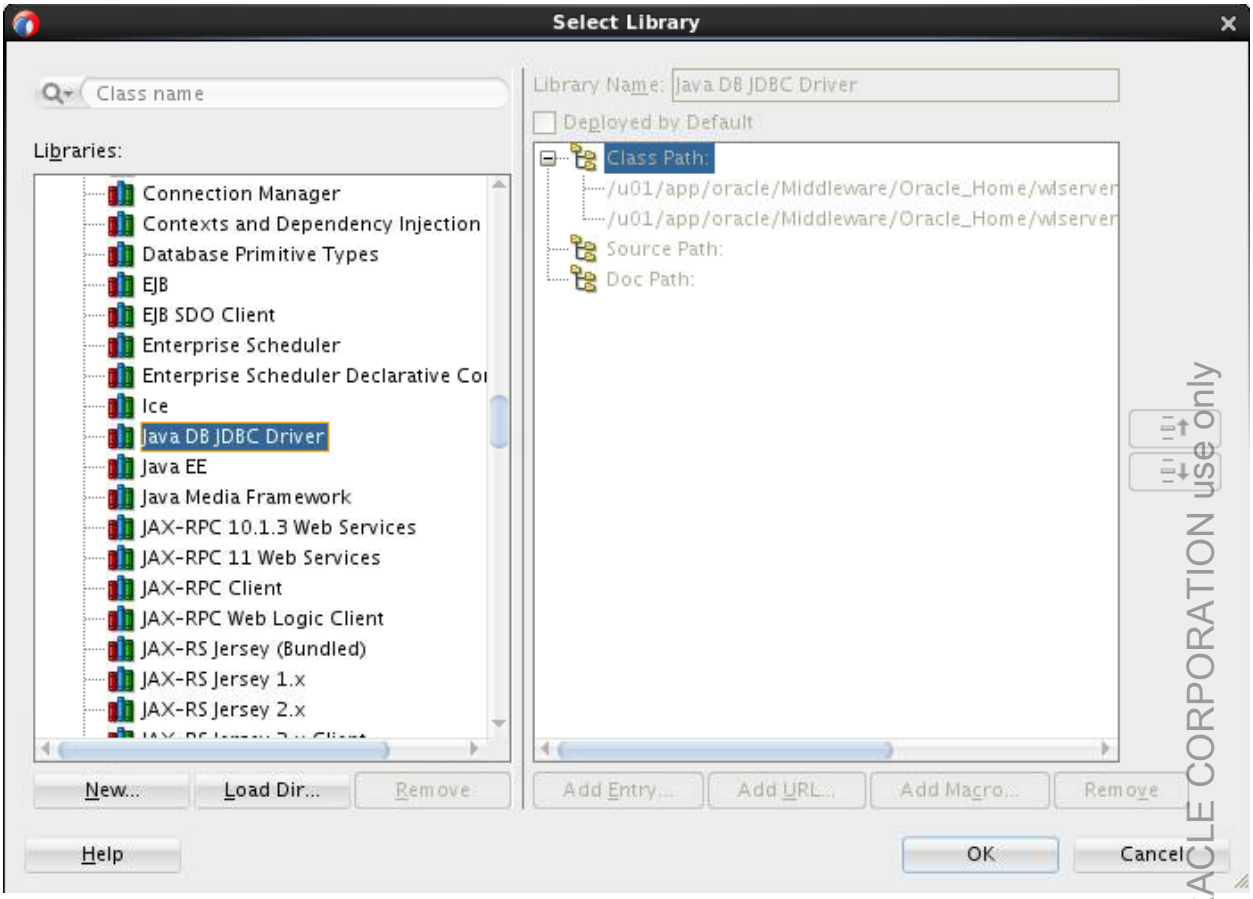
   g. Click PO. You should see the connection details in the Structure panel as shown below:



h. Expand the PO folder.

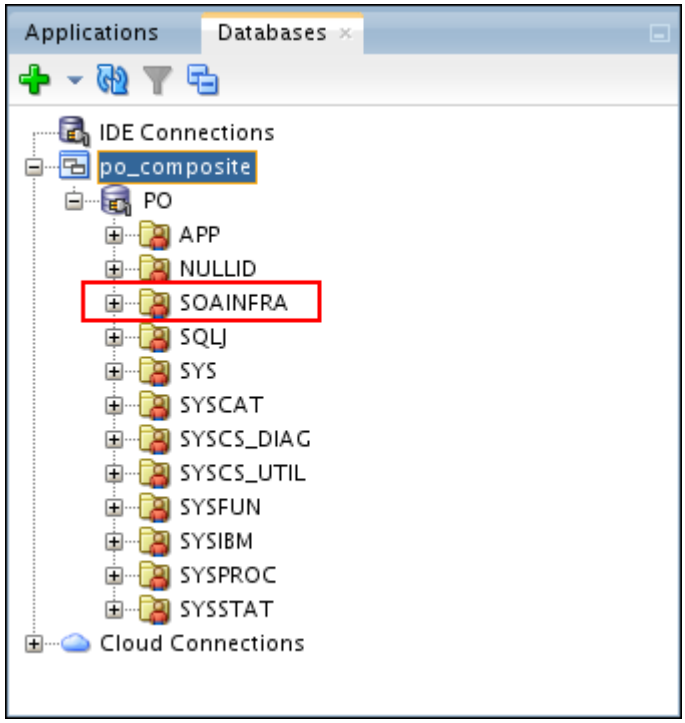
Note: If you encounter the error “Driver class not found,” do the following:

1) Right-click the PO folder, and select Properties from the context menu.
   The Edit Database Connection window is displayed.
2) Click the browser icon next to the empty Library field.
   The Select Library window is displayed.
3) In the Libraries field, scroll down, locate, and select **Java DB JDBC Driver**.

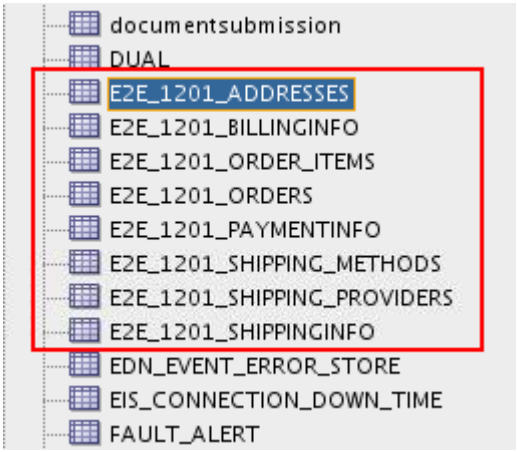


4) Click OK.
5) In the Edit Database Connection window, the Library filed is populated with your selection. Click OK.

Now you should able to see the database schemas via the PO connection.



All sample data for the labs is stored in tables with name pattern E2E_1201_xxxxx in the SOAINFRA schema.



i. Close the Database window.

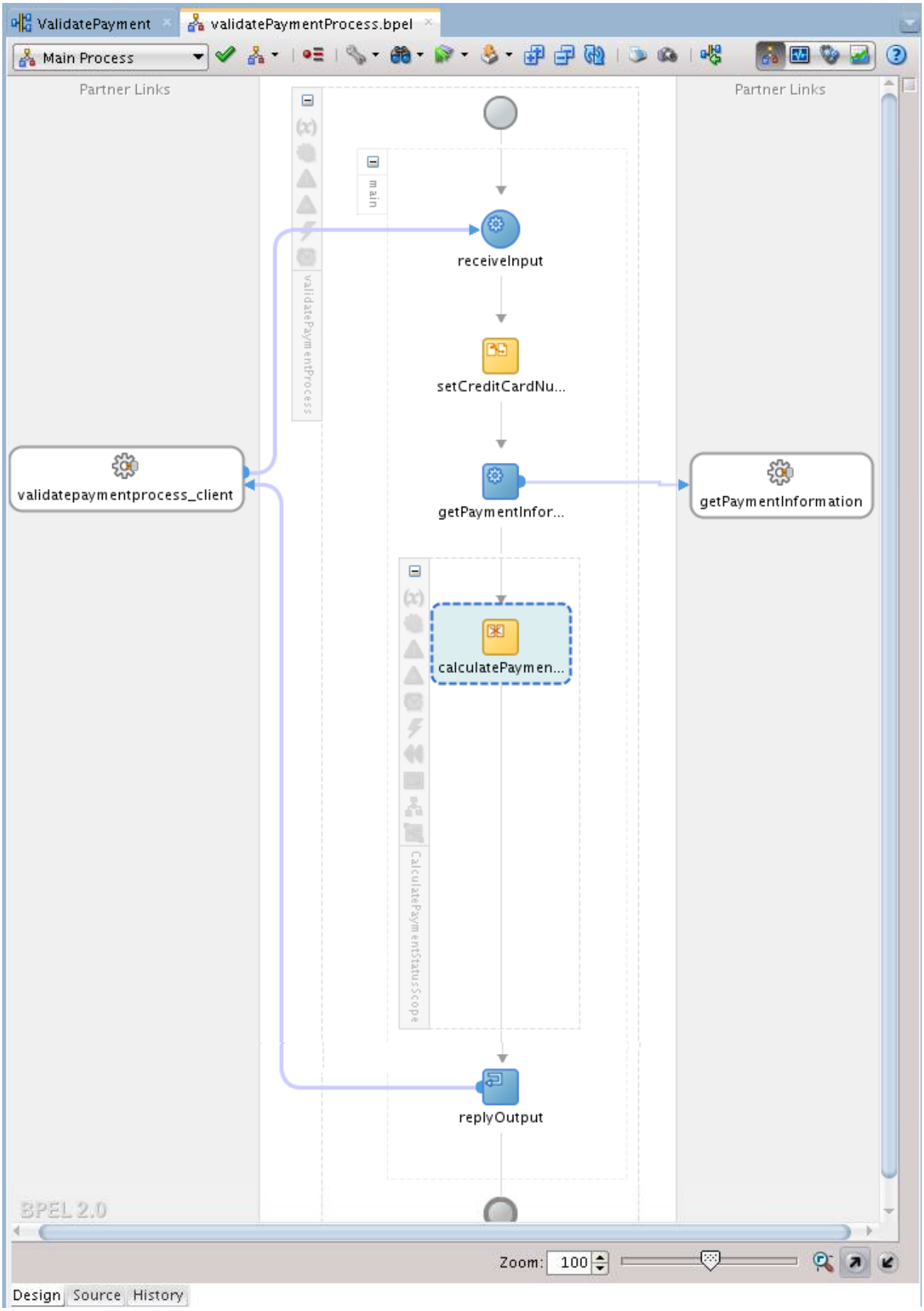
9. In the Composite Editor window, double-click the `getPaymentInformation` icon in the External Reference lane. The Oracle Database Adapter Configuration wizard is displayed. You can go through the steps to view the configurations, but take no further actions. Adapter will be covered later, in the lesson titled “Connecting with Binding Components.”

   The adapter processes your choices and generates a service that implements the operation you specified, which is a WSDL file, `getPaymentInformation.wsdl`, to represent that service.

   Close the wizard when you are done.

Examining the BPEL service component

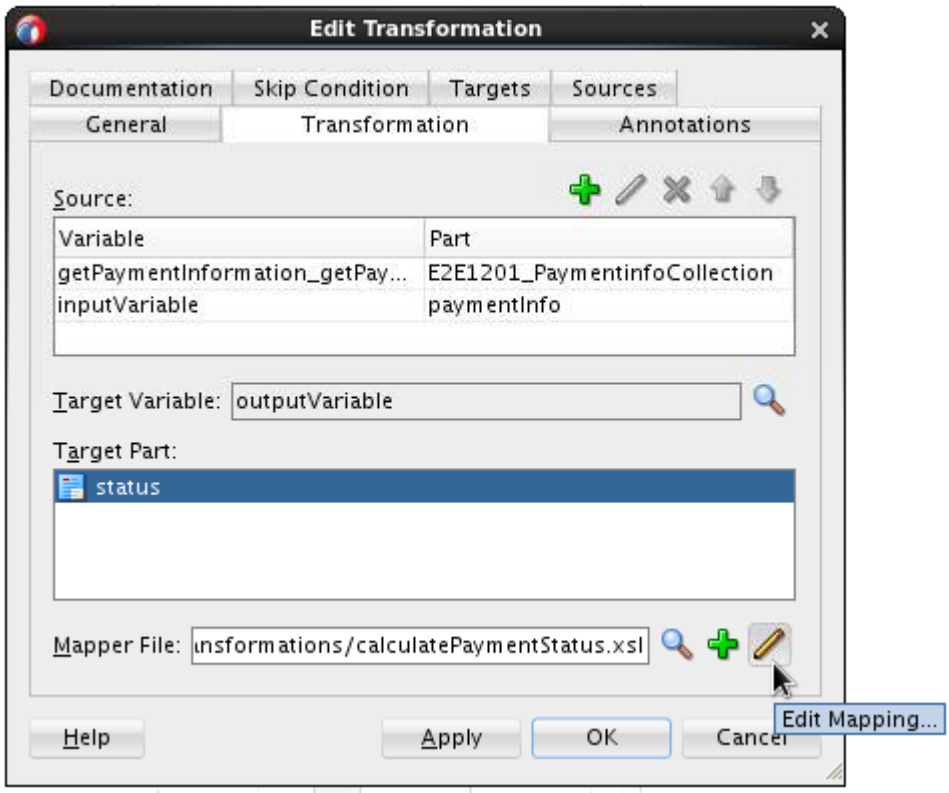
10. Double-click the `validatePaymentProcess` BPEL process component. The BPEL process editor opens and presents the design view of the BPEL process. Use the following image as a guide:



The Design view offers a graphical overview of the BPEL process activities, functions they provide, and the logic flow. It looks very similar to the flow you see in the Fusion Middleware Control console.

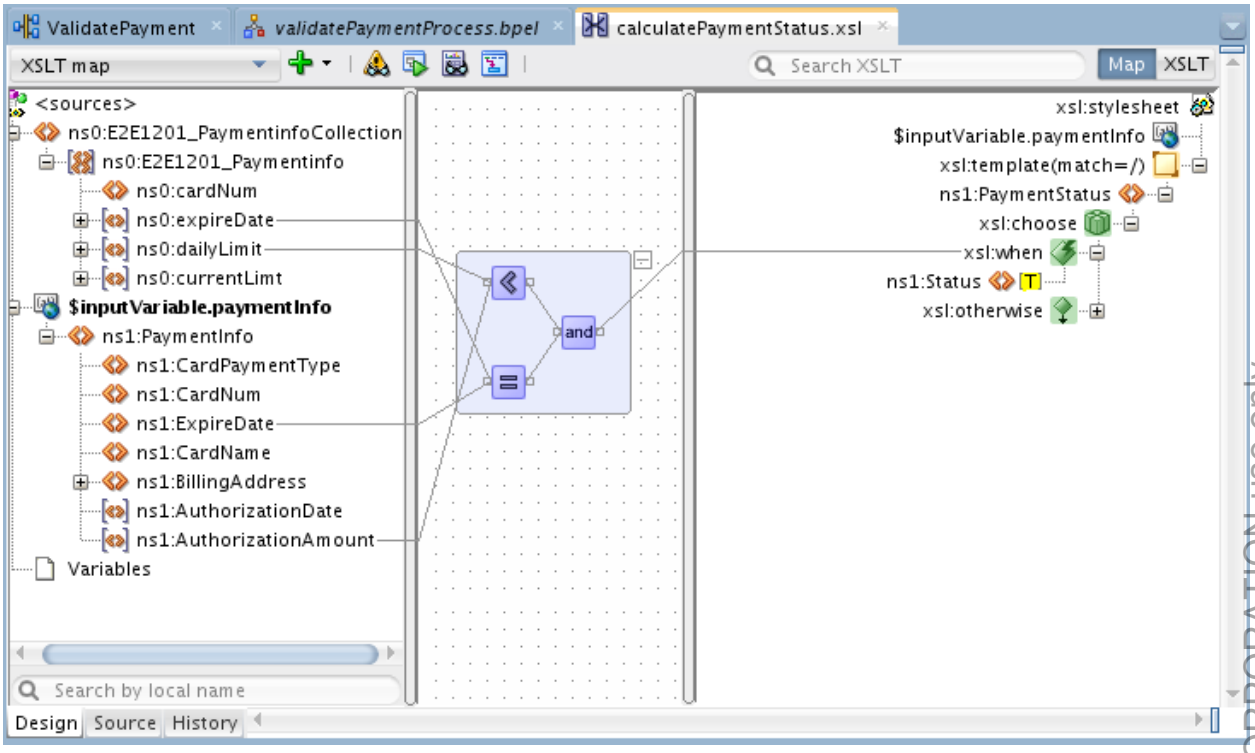
The validatePaymentProcess BPEL process basically receives the order request with credit card information, uses the credit card number as the key to retrieve payment information from the database, and then compares whether the total order amount is less than the daily limit on the credit card in the database and the card's expiration date listed in the order message is the same as the expiry date in the database record. The comparison of daily limit and expiration date is done by an XSLT transformation. The result will be replied to the client.

a. Double-click the `calculatePaymentStatus` transformation activity. The Edit Transformation dialog is displayed.



The transformation is defined in the `calculatePaymentStatus.xsl` stylesheet.

b. Click the Edit Mapping icon next to the Mapper File field. The XSLT Map Editor is displayed.



The transformation expects two inputs: the payment information retrieved from the database, and the total order amount. The output is the status field in the process output message, which will either be set to “Denied” or “Authorized.”
On the left (source) side, you see the structure of the two input data. On the right is the XSD structure of the output data. The mapping (and, therefore, the XSLT transformation) is created by connecting nodes from the source with the corresponding nodes in the target.

c. When you are done, close the XSLT Map Editor and the BPEL process editor.

Practice 4-2: Inspecting the Service Interface (Contract) WSDL File

Overview

In this practice you analyze the technical aspects of the service interface (contract) according to WSDL, and the XSD document that is used by the WSDL document to define the message structure.

There are three WSDL files in the WSDLs subfolder:

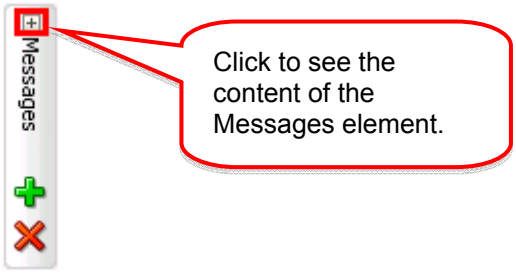
- `ValidatePayment.wsdl`: Representing the ValidatePayment service
- `ValidatePaymentWrapper.wsdl`: A wrapper file that includes ValidatePayment.wsdl
- `getPaymentInformation.wsdl`: Representing the getPaymentInformation service (adapter)

Tasks

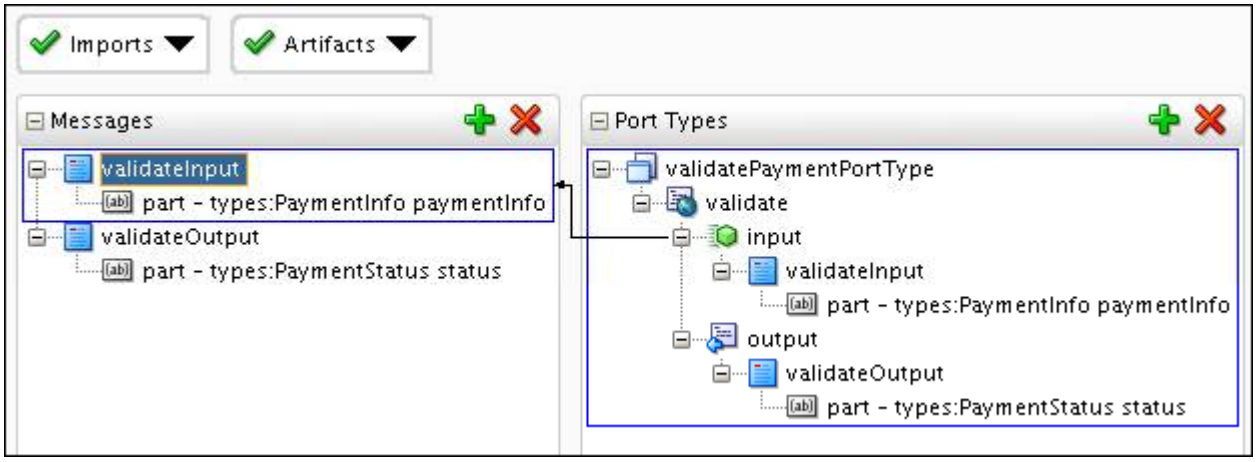
1. In the ValidatePayment > SOA > WSDLs folder, double-click the `ValidatePayment.wsdl` file. The WSDL editor opens and presents the design view of the WSDL document.

   In this graphical overview of the WSDL file, you can explore the structure and relationship of the elements.

2. You should see the contents of Port Types, Bindings / Partner Link Types and Services. The WSDL document contains the portType element, a set of operations (in this case, an operation named `validate`), and the abstract messages (input, output, and fault) involved with those operations. Click the Show Contents icon on the Message tag.



3. A message can consist of multiple parts. Each part can be seen to represent a parameter in the operation request or response. Click validatePaymentPortType > validate> input element in the Port Types pane, a link appears showing the relationship.



Message part can be seen to represent a parameter in the operation request or response. A message can consist of multiple parts.

Note: In general, it is recommended to stick with single-part messages because they are less complex and less likely to have you run into tool limitations.

4. Click the Source tab at the bottom of the WSDL editor. The source view of the WSDL document presents.
5. Locate the `<types>` section. The type definitions can contain XSD-style elements or import one or more external XSD documents. In this case, the WSDL document imports message definitions using an external XSD document, `CanonicalOrder.xsd`.

```
<wsdl:definitions targetNamespace="http://www.oracle.com/ValidatePayment"
                  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
                  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
                  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
                  xmlns:tns="http://www.oracle.com/ValidatePayment"
                  xmlns:types="http://www.oracle.com/soasample">
  <wsdl:types>
    <xsd:schema>
      <xsd:import schemaLocation="../Schemas/CanonicalOrder.xsd" namespace="http://www.
    </xsd:schema>
  </wsdl:types>
```

Examine the XML Schema

Now you will open the supplied XML schema file using the XML schema (XSD) editor. This enables you to view the structure of a message payload.

When composing applications using services, it is important that the services use a common vocabulary or language. All services base their interface on one data model. The benefits of this are:

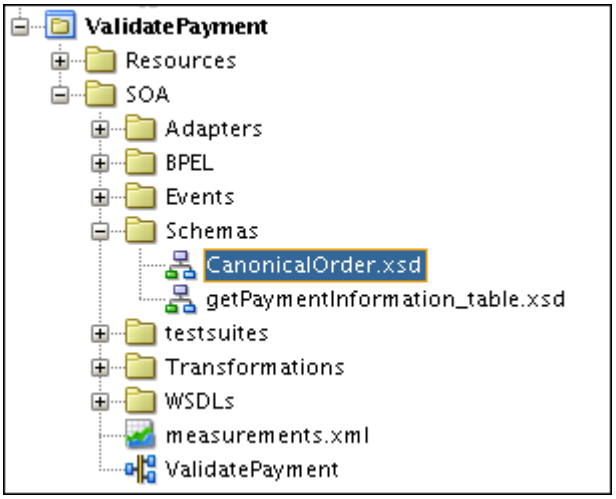
- Standardizing the service contracts so that consumers will encounter the same data structures in all services
- Lowering the number of message formats an application has to know about so that the number of error-prone transformations can be reduced.

Based on that, AviTrec uses the `CanonicalOrder.xsd` schema to define the message structure of various services in their order management system.

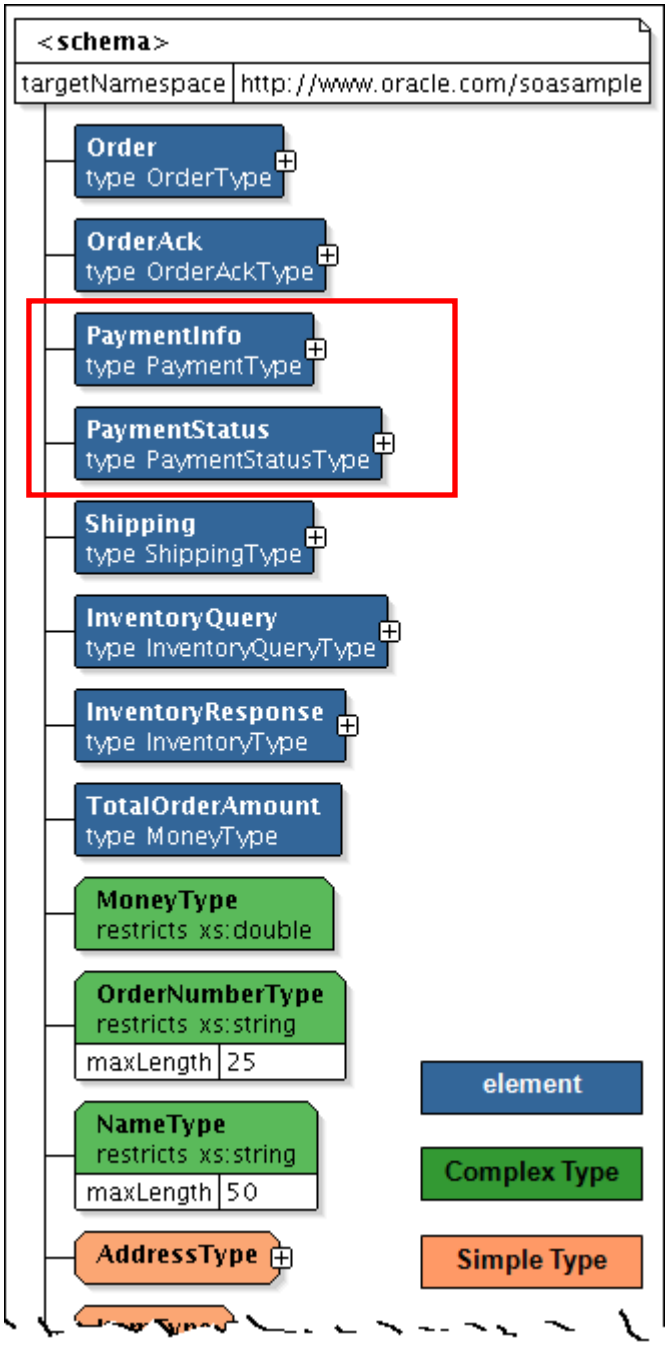
6. You can view the XSD file by using one of the following two ways:
   a. In the design view of the WSDL editor, click the Import drop-down menu, and select the `CanonicalOrder.xsd` file.

   Or

   b. Double-click the file in the project's Schemas folder:

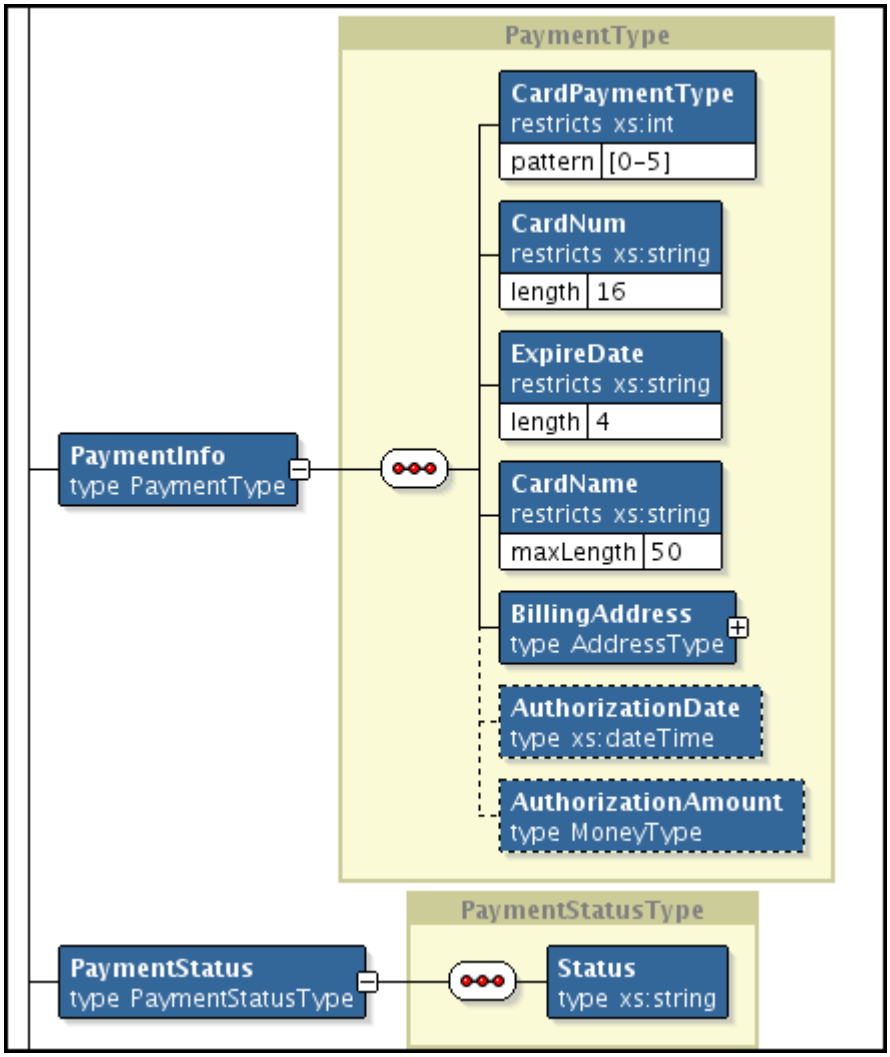


7. The XSD editor opens and presents the design view of the XSD document. You can view the UML class model and entity relationship.



Note that the WSDLs of ValidatePayment composite only contains `PaymentInfo` and `PaymentStatus` two elements to define data structure.

8. Expand `PaymentInfo` and `PaymentStatus` elements to view their structure.



Note: In the previous practice, the test file (`PaymentInfoSample_Authorized_soap.xml`) contained one `PaymentInfo` element.

9. Click the Source tab at the bottom of the XSD editor to view the source code of the XSD document.

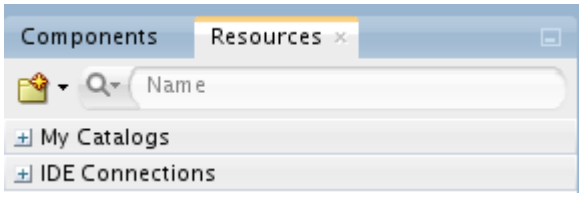
Practice 4-3: Populating the SOA-MDS Repository with Source Data

Overview

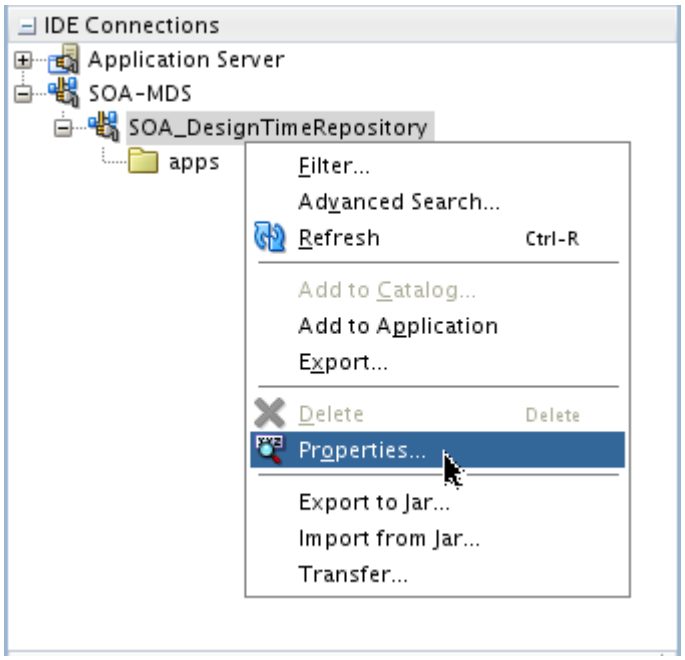
In this practice, you learn how to share design time artifacts such as WSDL, XSD and XQuery files of ValidatePayment with other composite applications by using the MDS repository.

Viewing the default MDS repository configuration

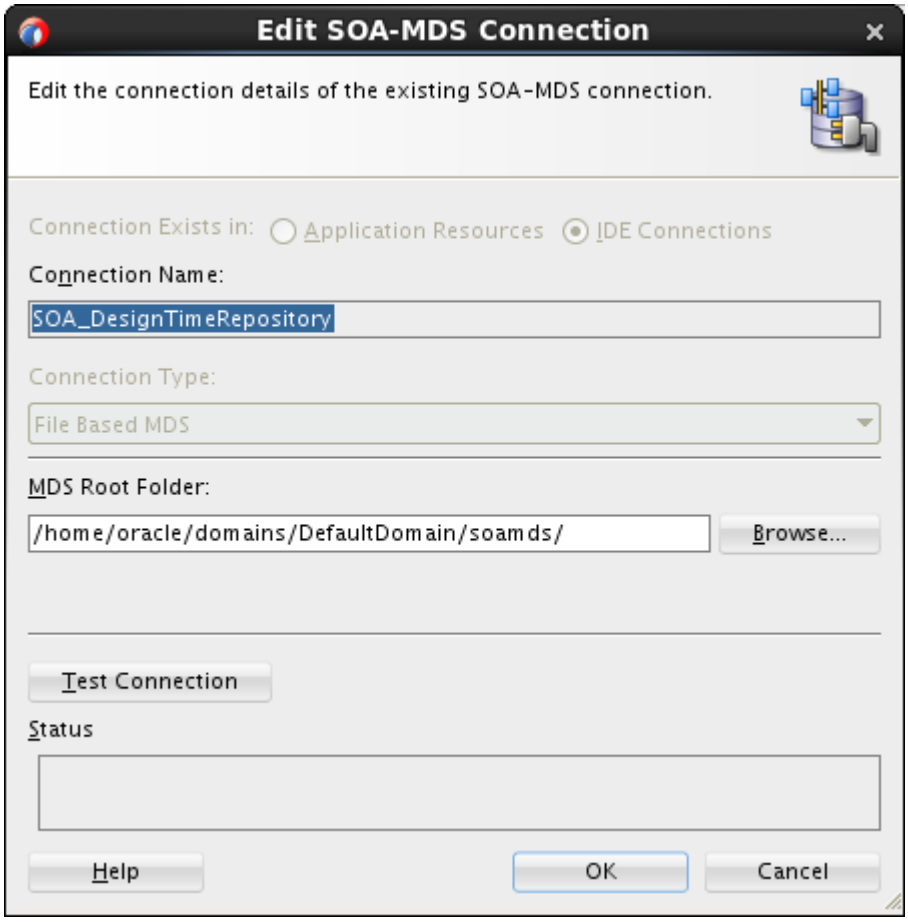
1. Go to JDeveloper and, from the Window main menu, select Resources.
2. In the Resources window, expand IDE Connections.



3. In the IDE Connections panel, expand SOA-MDS. You should see the automatically created (when creating a SOA composite) connection named `SOA_DesignTimeRepository` and the `/apps` folder in the repository. At this point, the folder is empty.
4. Right-click the SOA_DesignTimeRepository connection, and select Properties.



The Edit SOA-MDS Connection dialog box is displayed. It resembles the following image:



The default SOA-MDS connection name is `SOA_DesignTimeRepository`. The default repository root folder is `/home/oracle/domains/DefaultDomain/soamds/`.

5. Click **Browse**. The Choose Directory dialog box is displayed. You can see that there is an `apps` folder in the default repository root folder. This folder is automatically created.

   The connection name cannot be changed, but the default repository location can be modified to point to another folder or source control location.

   Leave this dialog box open; you will use it shortly.

Customizing the MDS repository folder

6. Use `/labs` as the repository root folder, and add the `/apps` folder and a folder for ValidatePayment. Perform the following steps:
   a. Create the `apps` folder.

      Open a terminal window, and enter the following commands:

      ```
      >cd /labs
      >mkdir apps
      ```

   b. Create a folder for ValidatePayment to share its artifacts.

      ```
      >cd apps
      >mkdir ValidatePayment
      ```

   c. Copy the following three folders from `/labs/lessons/lesson04/po_composite/ValidatePayment/SOA/` into the share data folder `/apps/ValidatePayment`:

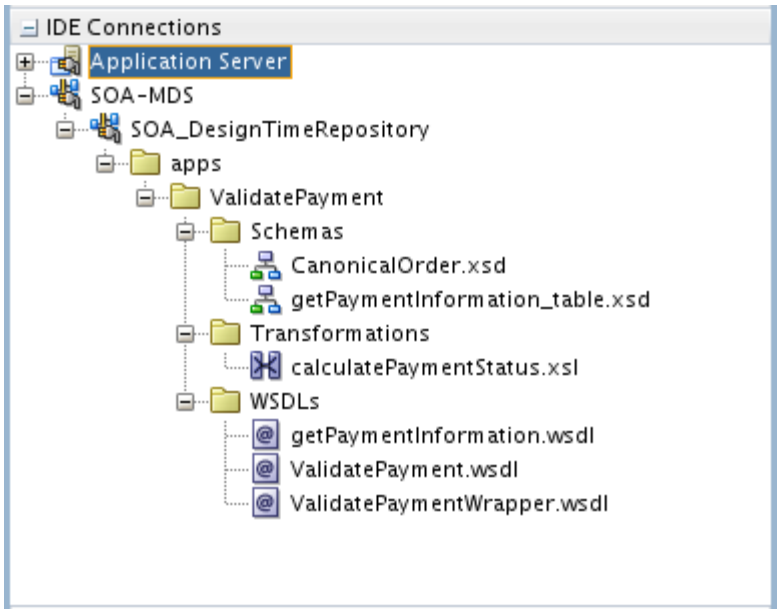
- `Schemas`
- `Transformations`
- `WSDLs`

The copy command is:

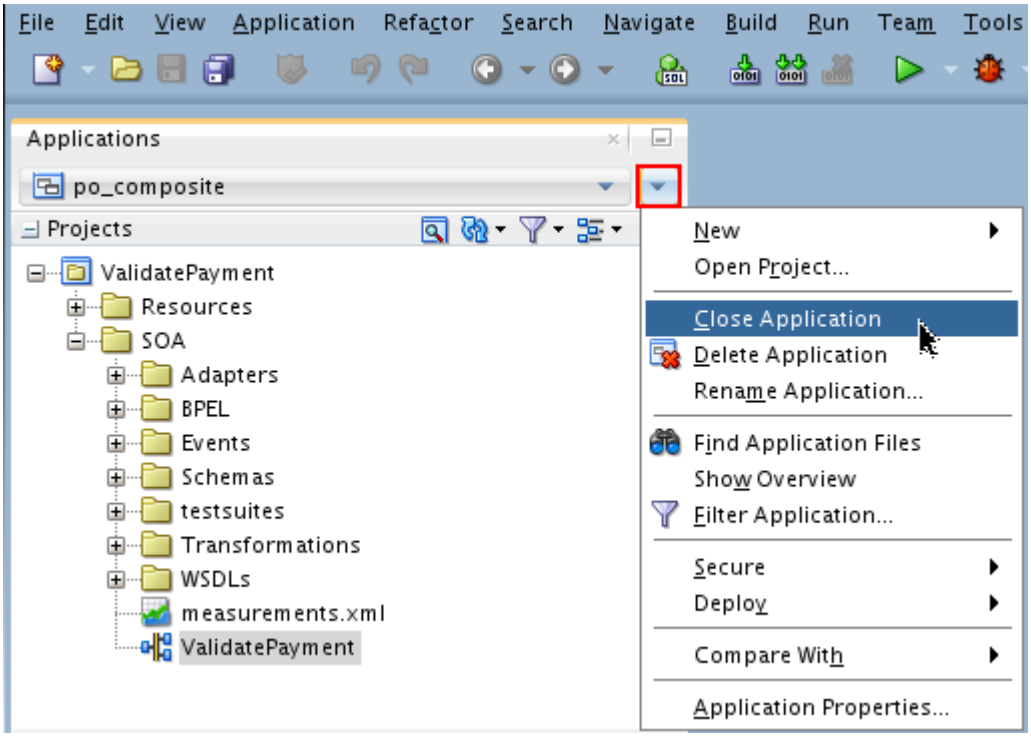
```
>cd /labs/lessons/lesson04/po_composite/ValidatePayment/SOA/
>cp -R Schemas /labs/apps/ValidatePayment/
```

Do the same for the other two folders.

7. Go back to JDeveloper and, in the Choose Directory dialog box that you opened before, select `/labs` folder, and click OK.
8. The Resources window is updated with the new MDS repository content, the artifacts from the ValidatePayment composite.



9. When you are done, close the application and all open windows of this project.



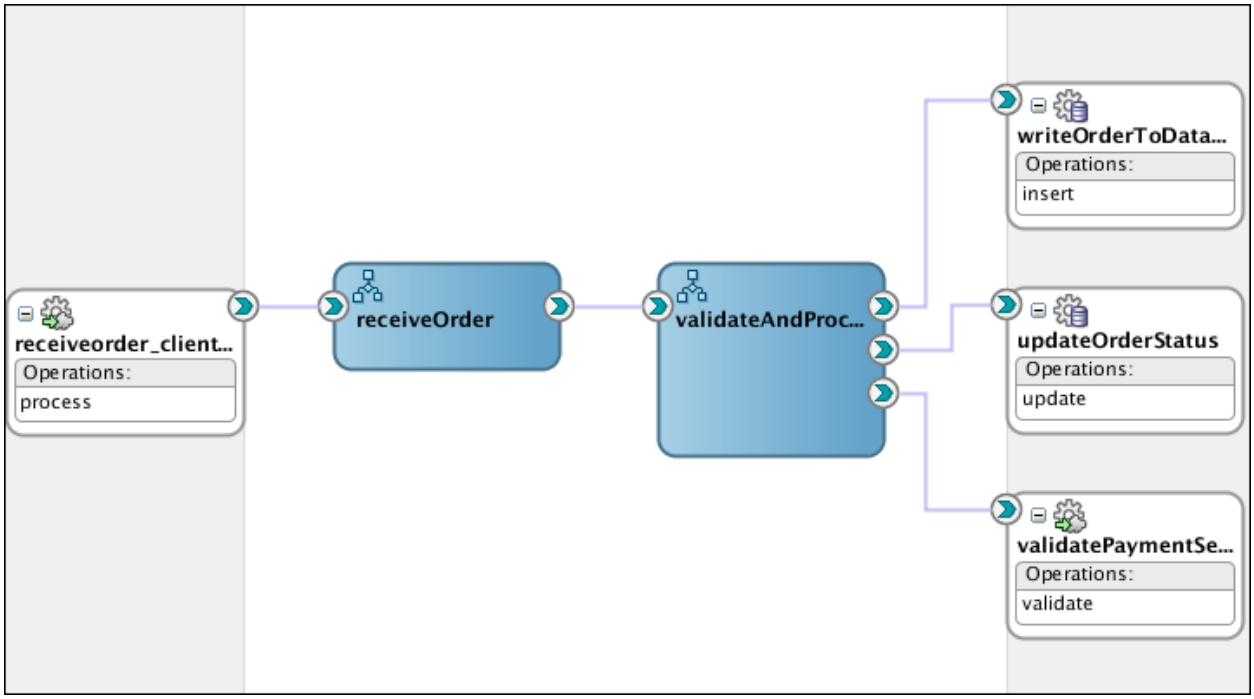
Practices for Lesson 5: Orchestrating Services with BPEL Process Components

Chapter 5

Practices for Lesson 5

Practices Overview

In these practices, you build the basis of the new order processing application for AviTrec, referred to as `ProcessOrder`. The composite accepts new purchase orders, approves them, and forwards them to the fulfillment system. You use a project template to implement the basic order processing scenario, add a call to the payment validation service, and update the order status in the database based on the outcome of the payment validation. The order status update will be converted to a BPEL subprocess to make it easily re-usable. Once completed, the composite will look like this image:



High-Level Steps

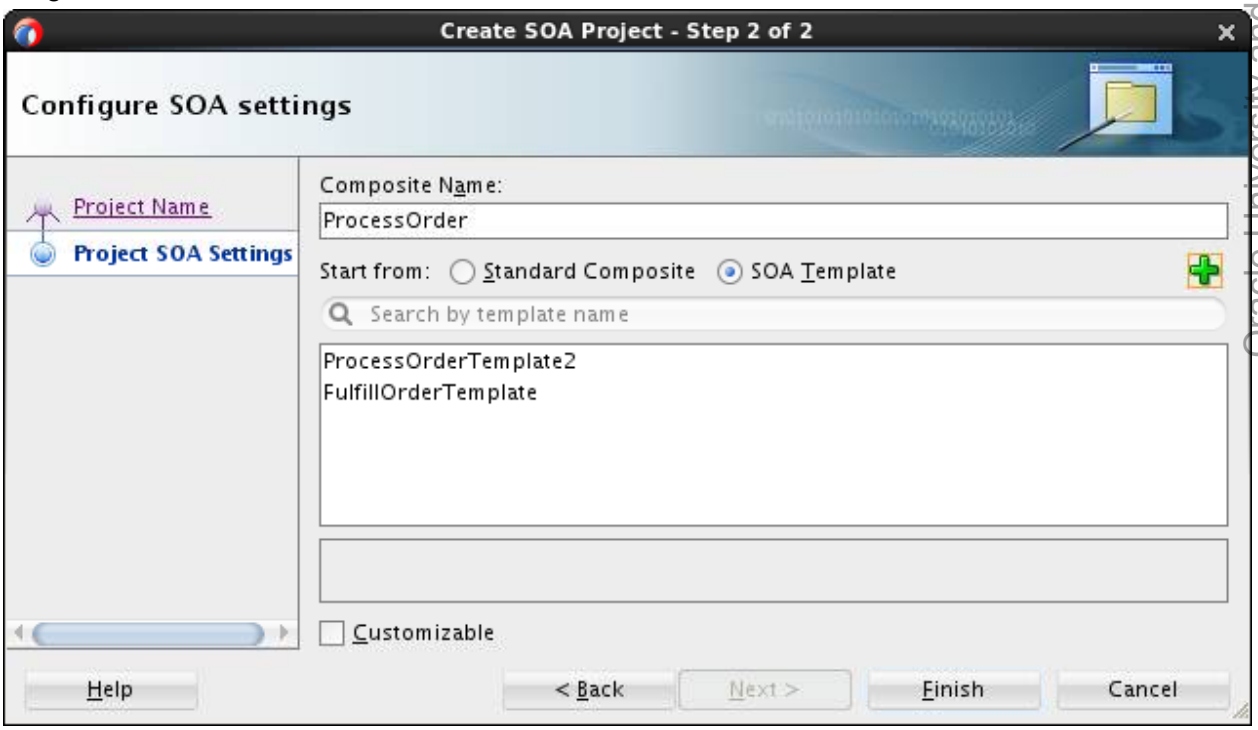
- Import SOA template.
- Invoke the `validatePayment` service to validate credit card payment before the order is processed.
- If the payment is denied, the order status is set to Denied and the processing is stopped.
- If the payment is authorized, the order status is set to Authorized and the order is processed.
- For authorized order, when the order processing is finished, the order status is set to ReadyForShip.

Practice 5-1: Creating a New Project from a Template

Overview

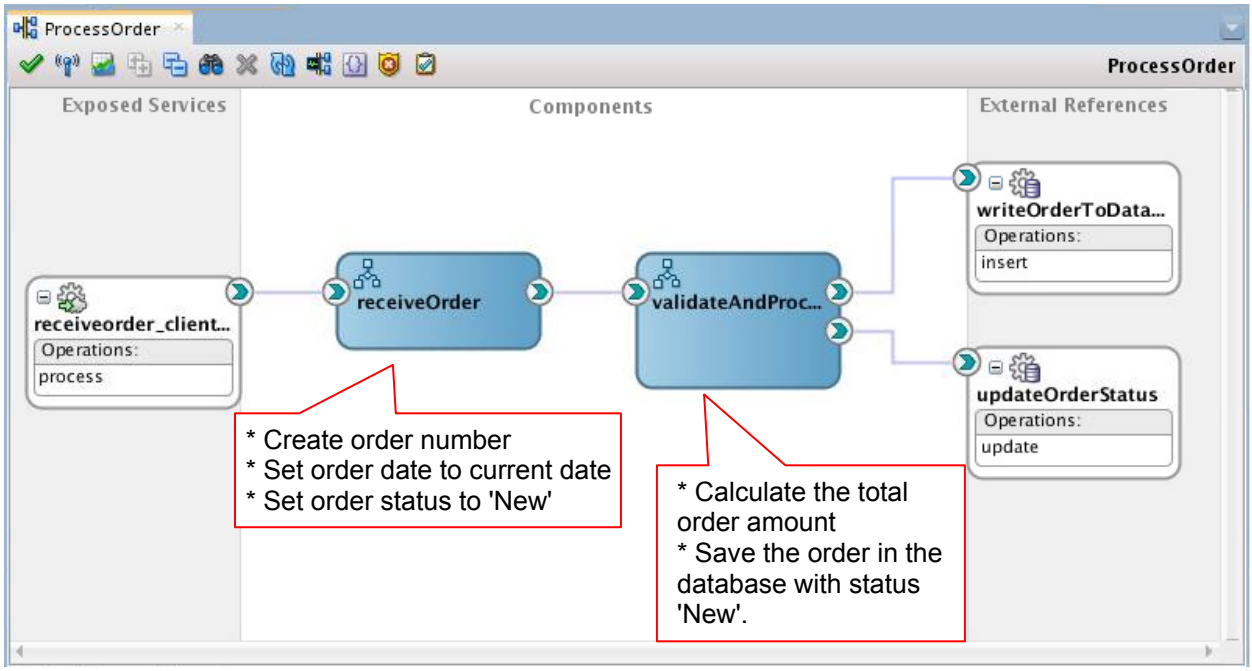
In this section, you create the `ProcessOrder` composite (order processing service) by importing a SOA project template with a number of components already predefined.

1. Create a new project from a project template.
   a. In JDeveloper, in the Applications pane, click Open Application (or you can select File > Open).
   b. In the Open Application(s) dialog box, navigate to the `/labs/lessons/Lesson05/po_composite/` directory, and open the `po_composite.jws` file.
   c. If prompted for migrating files, select Yes. The application opens with one project ValidatePayment folder.
   d. Select **File > New > Project** from the JDeveloper main menu.
      The New Gallery dialog box opens.
   e. Select **SOA Project**, and Click **OK**.
      The Create SOA Project wizard is displayed.
   f. Name the project `ProcessOrder`, and click **Next**.
      The wizard advances to step 2: the Configure SOA settings screen is displayed.
   g. Select **SOA template**, and click the "+" icon.
      The Choose Directory dialog box is displayed.
   h. Navigate to the `/labs/resources/templates` folder, and select it.
      You should see a list of templates populated in the field as shown in the following image:



   i. Select **ProcessOrdertemplate2**, and click **Finish**.

Wait for a moment, you should see a new project with predefined components created.



These components can:

- Receive an order through a web service call
- Create an order number, set the order date to the current date, and set the order status to New
- Calculate the total order amount
- Save the order in the database with the status New
- Return an acknowledgement to the client with the order number

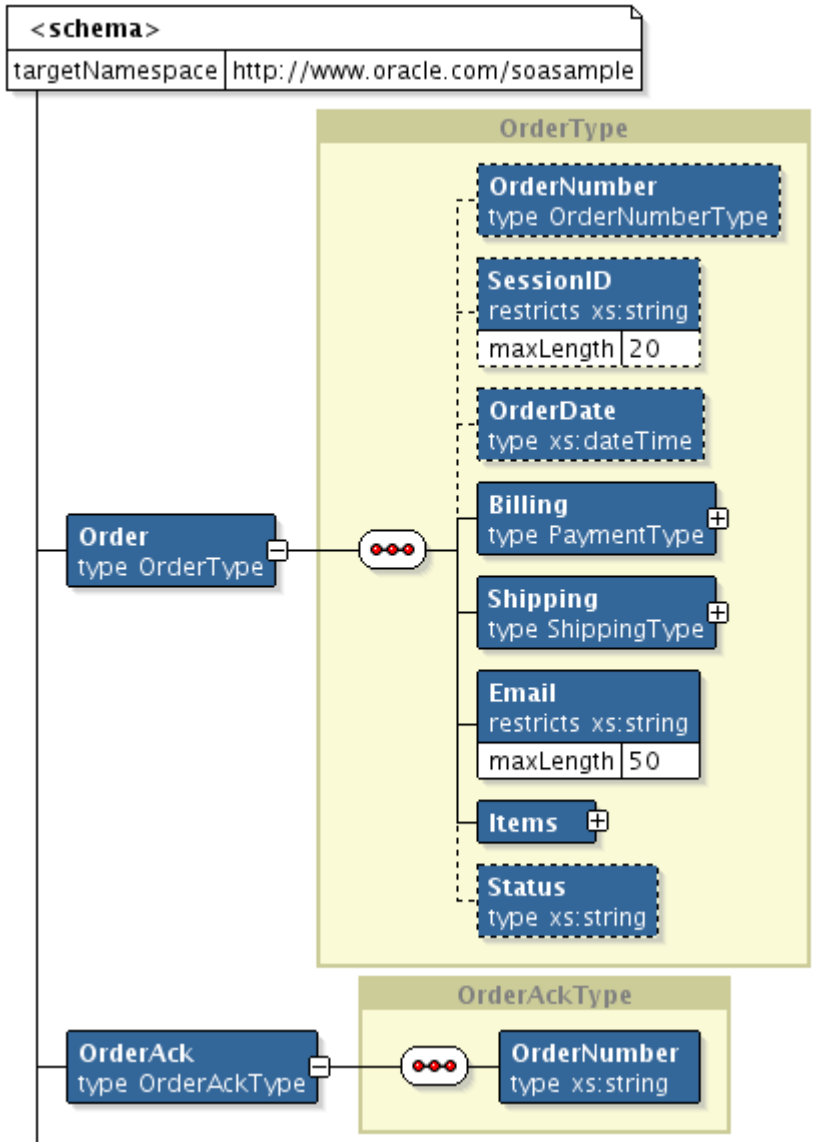
2. Examine the part of the XML schema that defines the data structure of an order
   a. Open the `CanonicalOrder.xsd` file in the project's `SOA/Schemas` folder.

      The XSD editor opens and presents the design view of the XSD document.

      This schema file is the same as you saw in previous lab. However, in this practice, you will examine the `Order` and `OrderAck` elements, which are contained in WSDLs of the ProcessOrder composite to define the data structure.

      Note: Knowing the data structure makes it easy to understand the variables and the Assign and transformation activities used in a BPEL process.

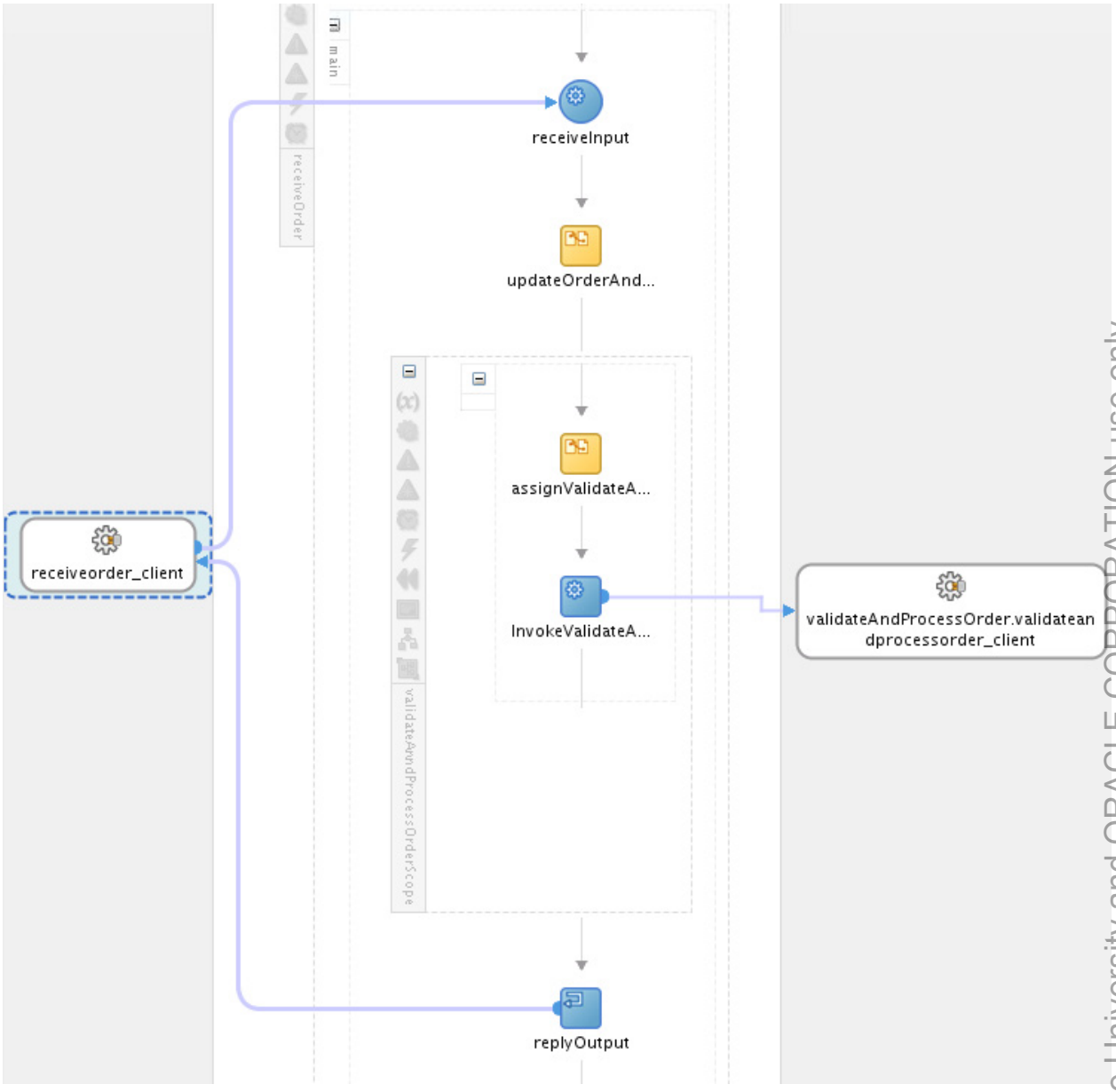
   b. Expand the `Order` and `OrderAck` elements.



Note that the `Billing` element also has a `PaymentType`.

c. The Billing, Shipping and Items elements all contain a child element. You can expand them and examine further.

3. Inspect the BPEL Process Service Components.
   a. Double-click the receiveOrder component to open it in the BPEL Editor.



You can see that this is a synchronous BPEL Process. It receives an order request from the client, creates an order number, sets the order date to the current date, sets the order status to New, copies the data to the payload of the validateAndProcessOrder BPEL process, and then invokes it. Finally, it returns an acknowledgement to the client with the order number.

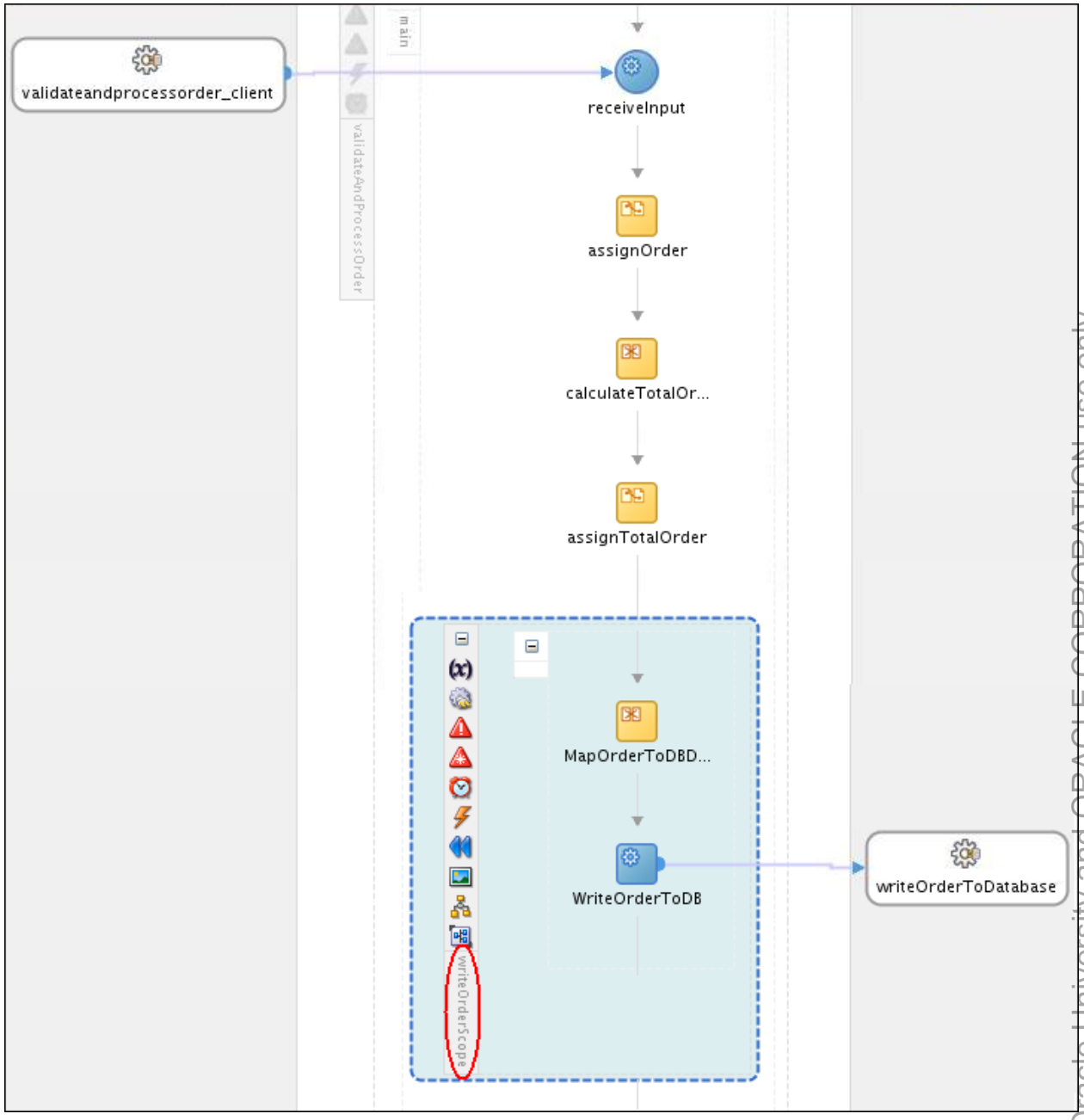
b. Double-click `updateOrderAndAck`, an Assign activity, to open it in the editor. You can examine how it implements the above functionality by using XML expressions.

   Can you tell how the order number gets generated?

c. Close the Edit Assign window.

d. Close the receiveOrder BPEL editor window.

e. Go back to the ProcessOrder window, and double-click the validateAndProcessOrder component to open it in the BPEL Editor.



You can see that this is a one-way BPEL Process. Let's take a look at what has been defined in the process.

- After assigning the order to a process variable, `validateAndProcessOrder` calculates the total order amount, which is used to validate the payment. Remember, this was an input to the payment validation service.
- The calculation is done through an XSLT map.
- An assign activity then adds the total order amount to the order message.
- The `writeOrderScope` includes all activities involved in writing the order to the database. If the payment is denied, we will cancel the order and update the order status in the database accordingly.

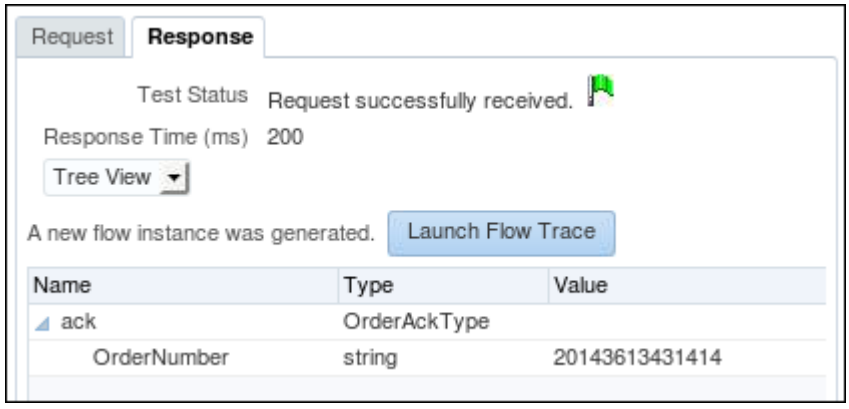
- The last step in the predefined process is to update the status in the database with the value of the status element in the order message. This will be re-used several times in this process:
  - To update the order status with the outcome of the payment validation (Denied or Authorized)
  - To update the status to ReadyForShip if the payment has been authorized and the order process has been completed.

f. You can examine the activities. When you are done, close the validateAndProcessOrder BPEL editor window.

Deploying and testing the project

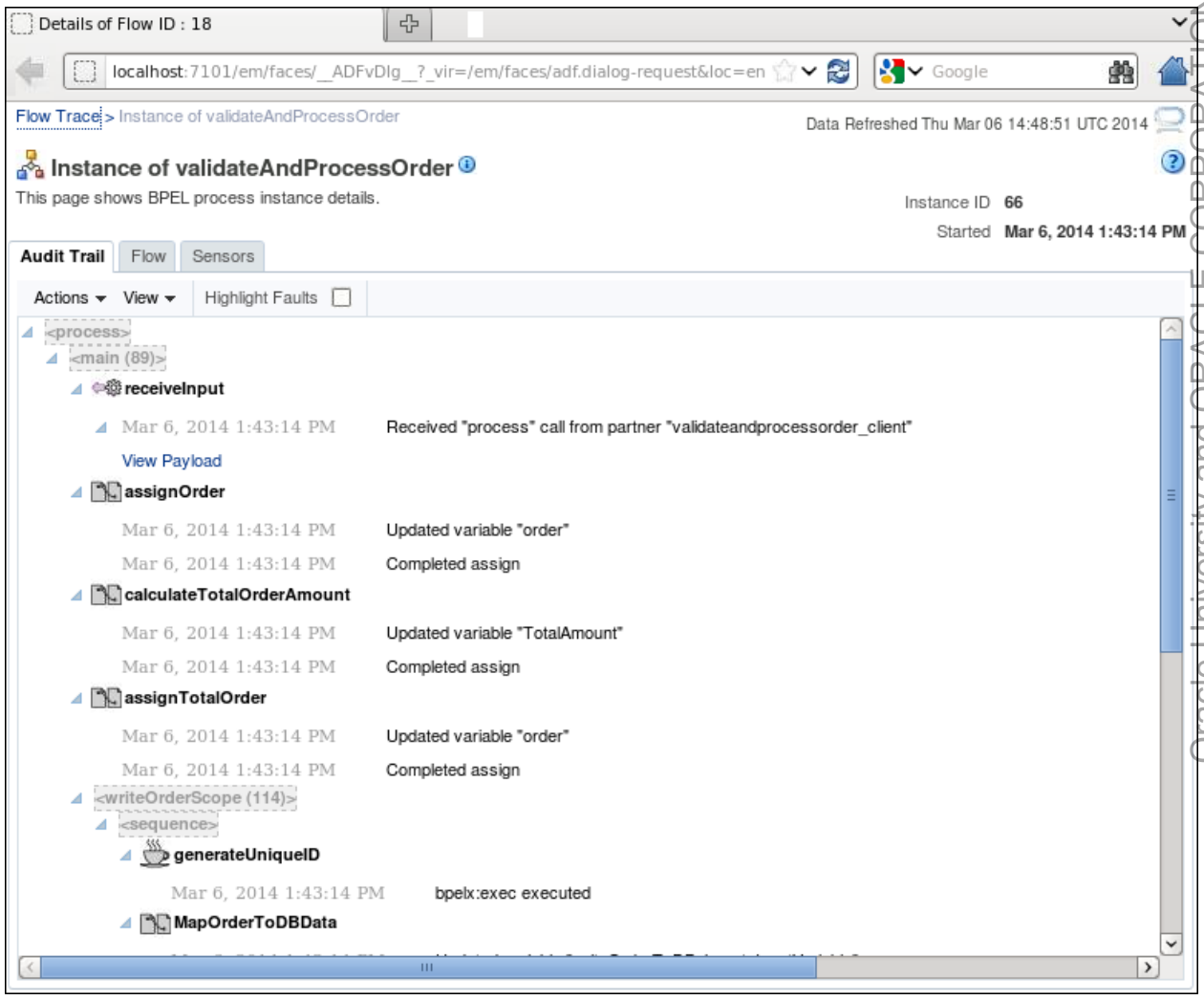
4. Start the Integrated WebLogic Server (if it is not already running).
5. Deploy the project.
   a. Right-click `ProcessOrder` in the project menu.
   b. Select **Deploy > ProcessOrder**.
      The Deploy ProcessOrder dialog box is displayed.
   c. Select the **Deploy to Application Server** option, and click **Next**.
      The SOA Deployment Configuration dialog box opens.
   d. Accept the default revision ID 1.0, select the **Overwrite any existing composites with the same revision ID** option, and click **Next**.
   e. Select **IntegratedWebLogicServer**, and click **Next**.
   f. Click **Next**.
      The deployment summary is displayed.
   g. Click **Finish**.

   The application is built and deployed. In the SOA log, at the bottom of JDeveloper, the message BUILD SUCCESSFUL is displayed, and the deployment begins. In the Deployment log, you can view the details of the deployment.
6. Test the project.
   a. Switch to the Fusion Middleware Control console and, in Target Navigation panel, expand SOA > soa-infra > default. Now you should see the newly deployed ProcessOrder composite application in the folder.
   b. Select the ProcessOrder composite.
   c. Click the **Test** button at the top of the screen.
      The Test page opens.
   d. Scroll down to the **Input Arguments** section.
   e. Click **Browse**. In the File Upload window, select the `/labs/resources/sample_input/OrderSample_soap.xml` file, and open it.
      The Order element in the SOAP Body area is populated with the order data.
   f. Click **Test Web Service**. When the composite completes, the screen switches to the Response tab showing an order number (similar to the following screenshot).



g. Click **Launch Flow Trace**.

You can view the detailed Audit Trail of both BPEL processes: receiveOrder instance and valideAndProcessOrder instance.

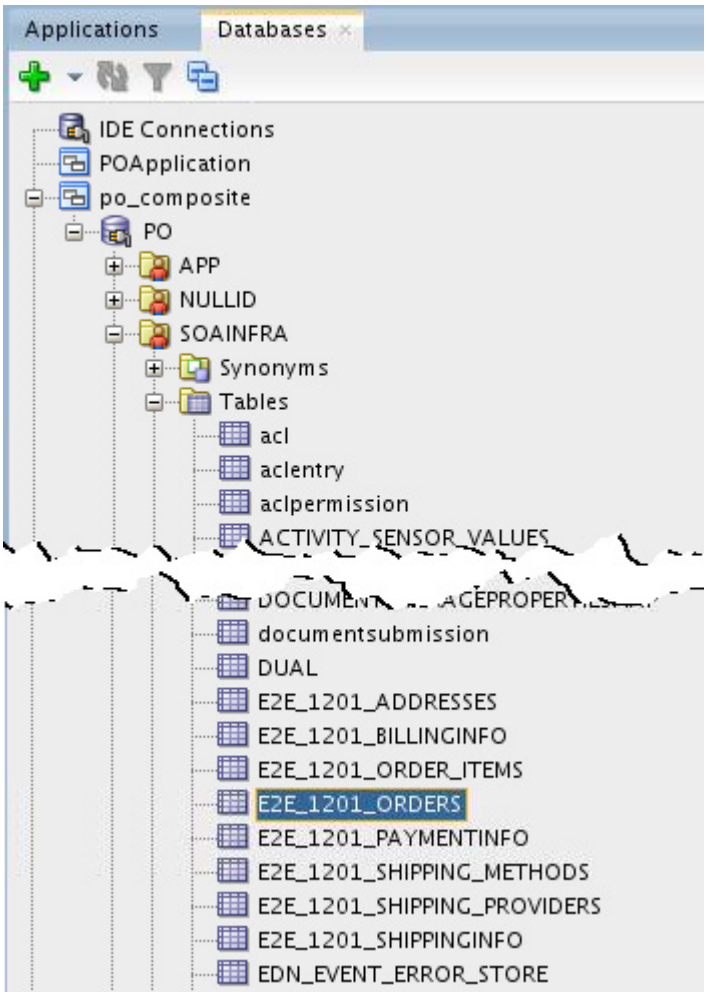


7. Verify the outcome of the test in the database.

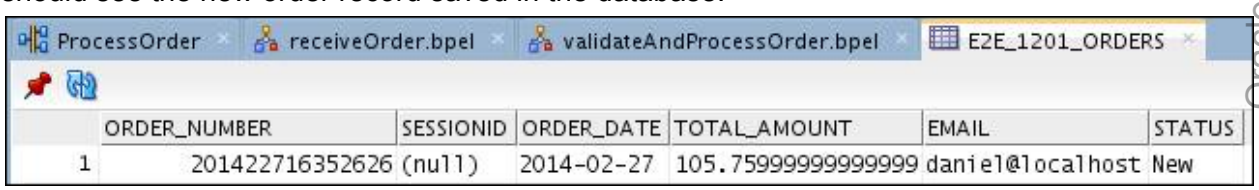
To see whether the new order record has been created in the database, perform the following steps:

a. From the JDeveloper main menu, select Window > Database >Databases. The Databases window is displayed in the upper-left corner.

b. Navigate to po_composite > PO >SOAINFRA > Tables, and locate the E2E_1201_ORDERS table in the list.



c. Right-click the table, and select **Open Object Viewer** from the context menu. You should see the new order record saved in the database.



	ORDER_NUMBER	SESSIONID	ORDER_DATE	TOTAL_AMOUNT	EMAIL	STATUS
1	201422716352626	(null)	2014-02-27	105.75999999999999	daniel@localhost	New

Keep this window open, because you will come back to verify the status update later.

Practice 5-2: Invoking the ValidatePayment service

Overview

The order will only be processed if the payment is valid. Remember that you implemented the payment validation in the `ValidatePayment` composite. Now you add a call to invoke this payment validation from `ProcessOrder`.

Looking at the `ProcessOrder` composite, you will see that the BPEL process `receiveOrder` only sets the order number (which is provided back to the client) and the order date and then calls `validateAndProcessOrder` to validate and process the order.

Assumptions

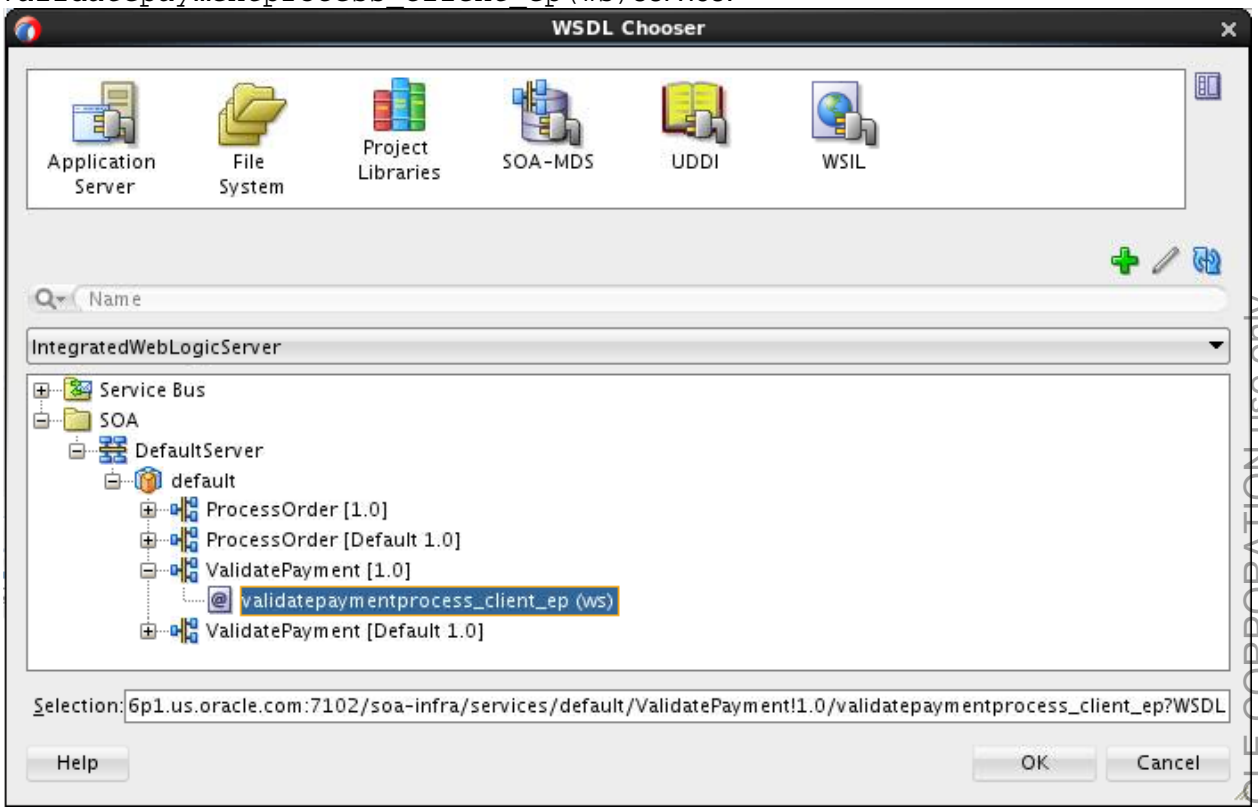
The `ValidatePayment` service is deployed and running on the SOA server.

Tasks

1. Make sure that the Composite editor of your `ProcessOrder` composite application is open and the Design view displayed.
2. Drag a SOAP web service from the Components palette (in the Technology section) into the External References swimlane.

   The Create Web Service dialog box is displayed.
3. Define a reference to the `ValidatePayment` service, by doing the following:
   a. In the Create Web Service window, enter the following information:
      - Name: validatePaymentService
      - Type: Reference
      - Click the “Find existing WSDLs” icon next to the WSDL URL field to open the WSDL Chooser window.

b. In the WSDL Chooser window, click the Application Server tab, and select the `validatepaymentprocess_client_ep(ws)` service.

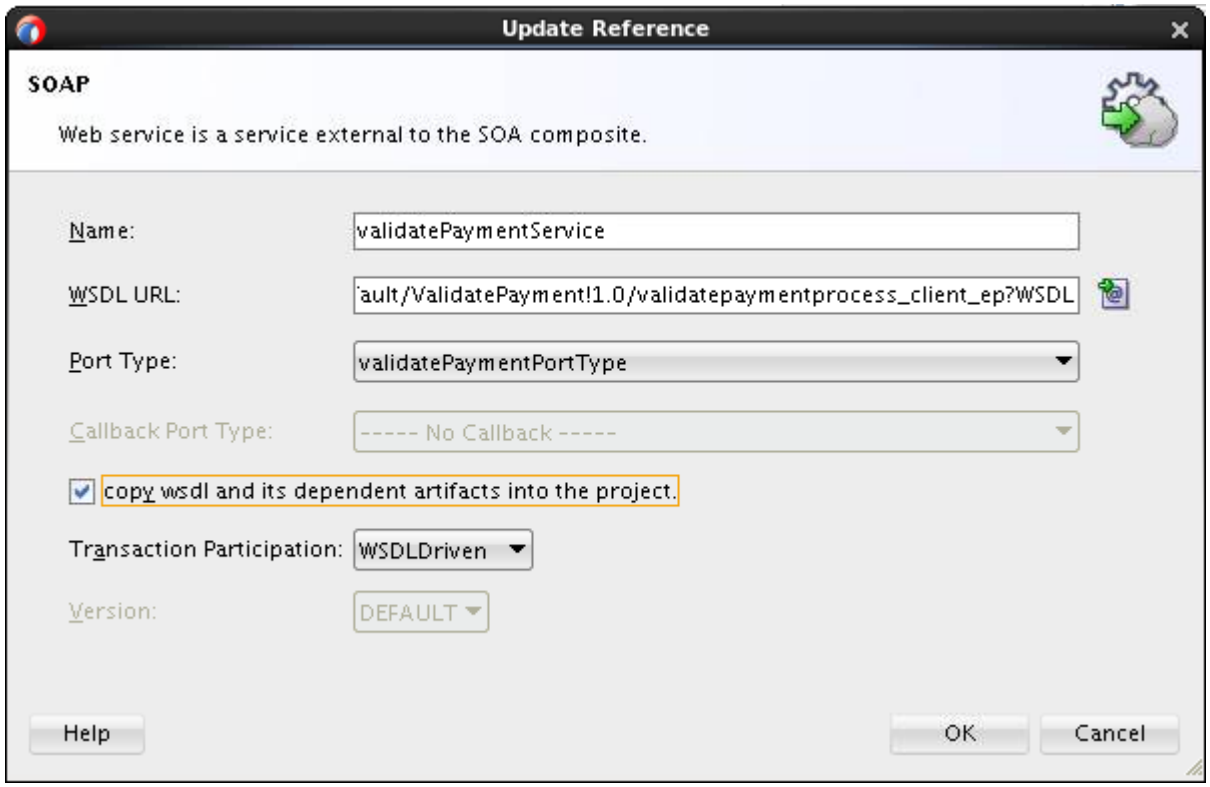


c. In the Selection field, note that it uses SSL connection. This design is based on security considerations. Now change to the regular HTTP connection by modifying the URL to **`http://<yourhostname>:7101/soa-infra/...`** Click **OK**.

d. Make sure that the **copy the WSDL file and its dependent artifacts into the project** option is selected.

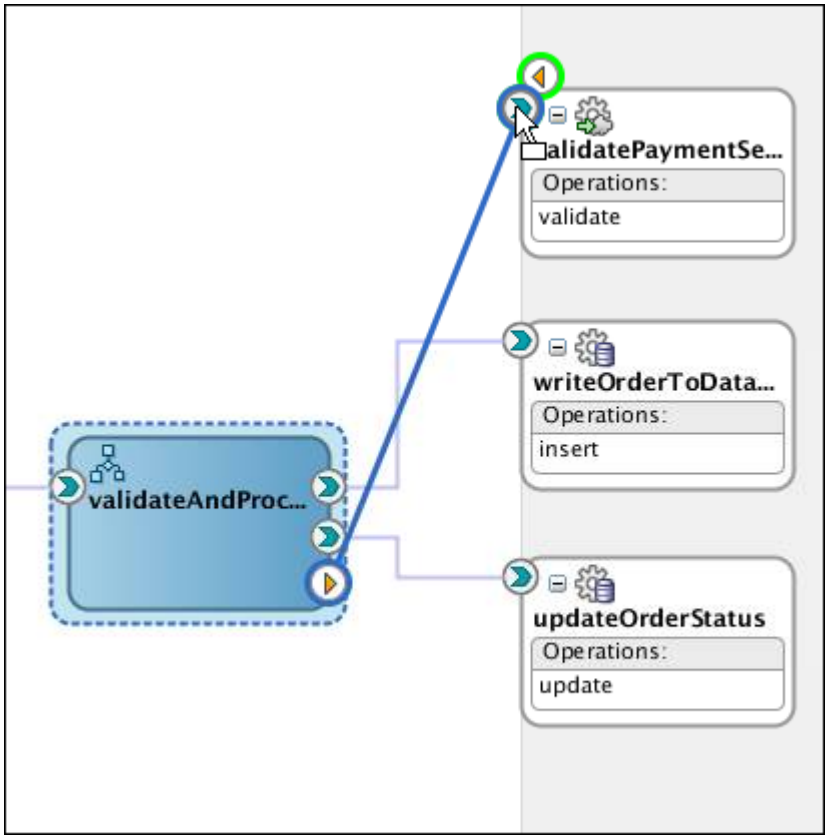
   This option ensures that the designer does not throw an error if the web service is not available because the server is down or the service is not deployed.

e. Click **OK**.



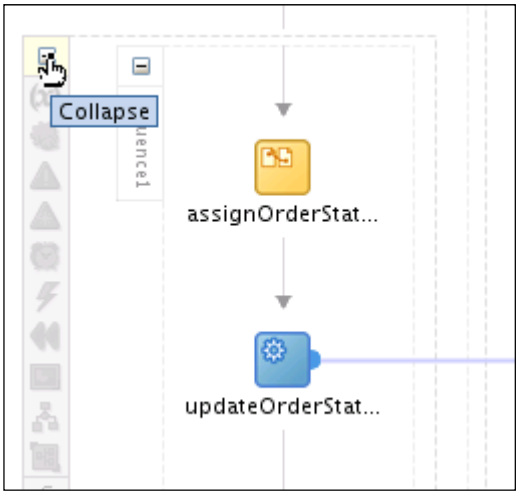
f. On the Localize Files screen, deselect all three options: Maintain original directory structure for imported files, Localize external references and Rename duplicate files. Click **OK**.

4. Wire the `validateAndProcessOrder` BPEL process to the newly created SOAP reference.



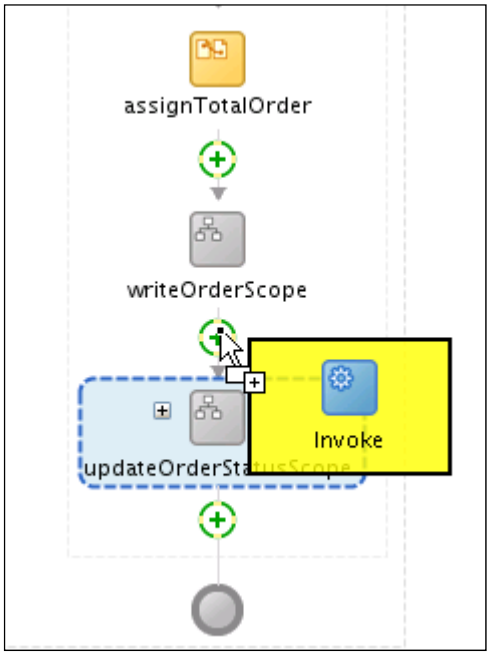
5. Open the `validateAndProcessOrder` BPEL process. You should see that there is now a new partner link in the BPEL process for the payment validation.

   Tip: You can collapse the scopes to make the design look less crowded.

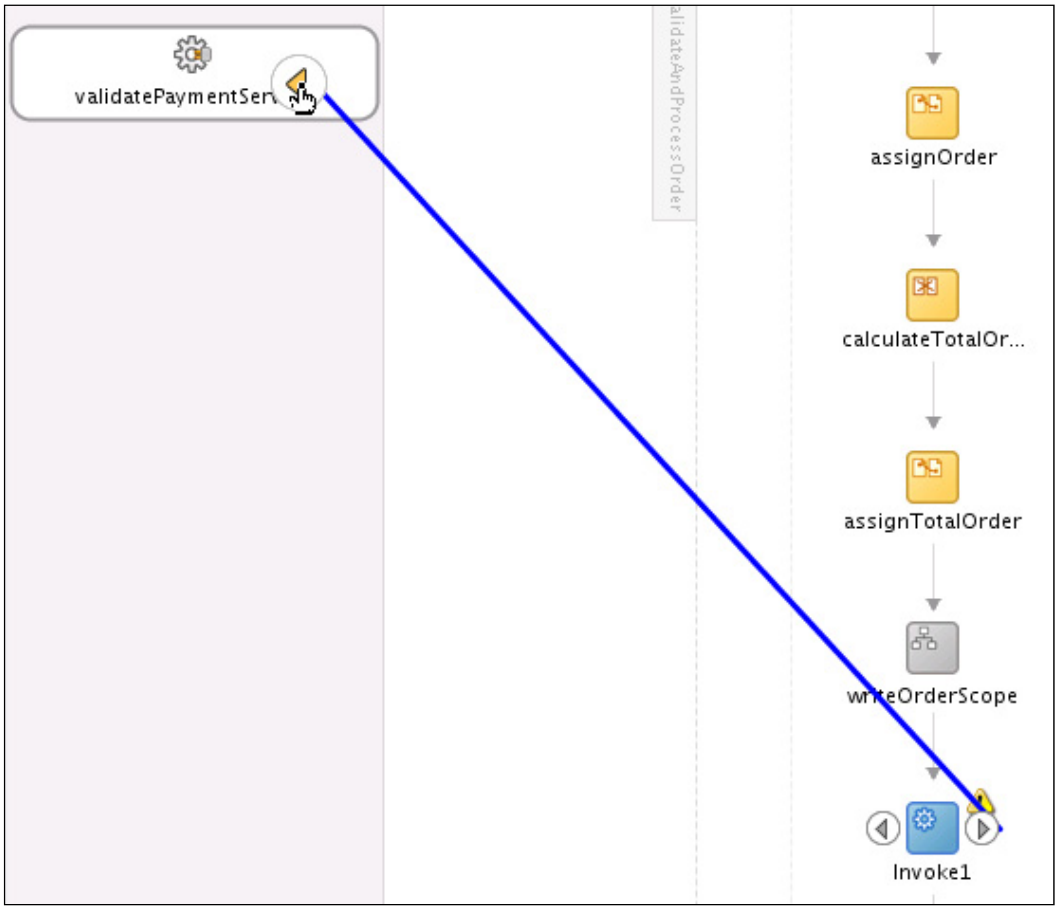


   To add the call to validatePaymentService, do the following:

   a. Add an invoke activity between `writeOrderScope` and `updateOrderStatusScope`.



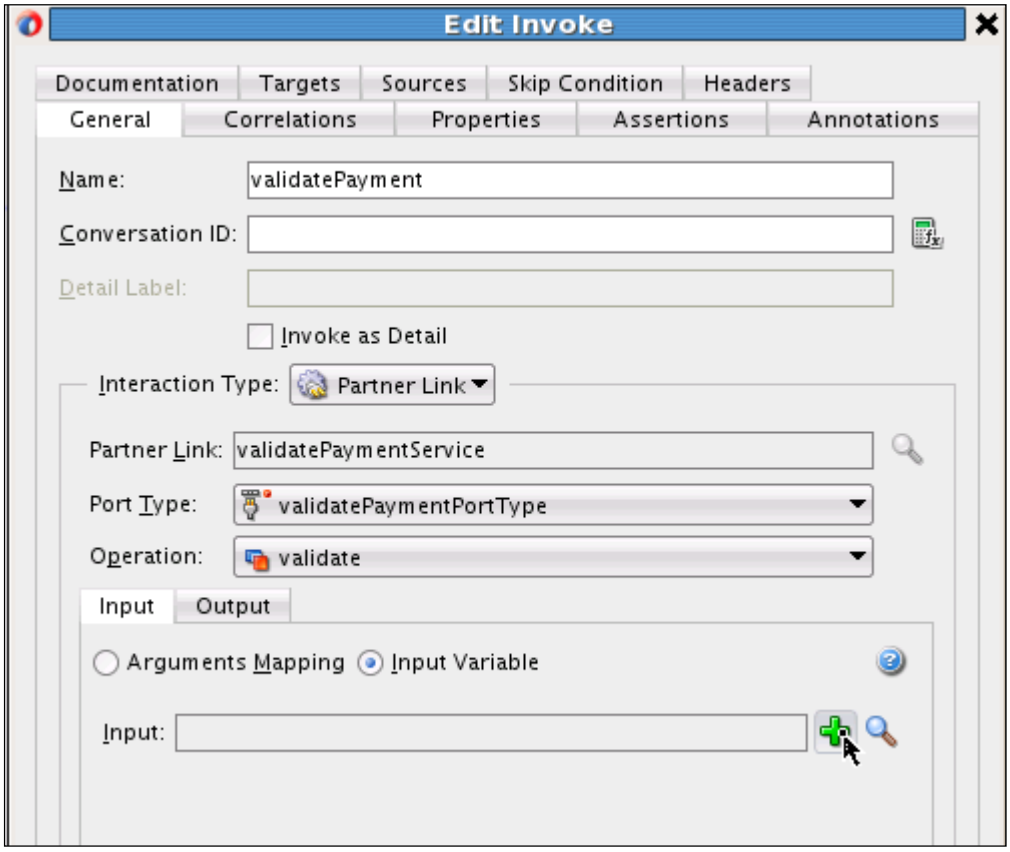
b. Wire the invoke activity to the `validatePaymentService` partner link.



This opens the invoke activity for editing. You can choose to continue editing in this dialog box or close the dialog box and edit in the property editor. In this practice, you use the open dialog box.

c. Change the name of the invoke activity to `validatePayment`.

d. Click the **Add** icon ("+" sign) next to the Input field to automatically create the input variable for the invoke activity.



The Create Variable dialog box is displayed.

e. Accept the default value, and click **OK**.

This will create a new global variable.

Note: This variable contains the data that will be sent to the validatePayment service or the input to the service.

f. Select the Output tab.

g. Click the Add icon ("+" sign) next to Output.

h. Accept the default value for output variable, and click OK.

i. In the Edit Invoke window, click OK.

The invoke activity is now complete. Before the web service can be invoked, the right values must be assigned to the input variable for the web service call.

5. Create and configure an Assign activity.

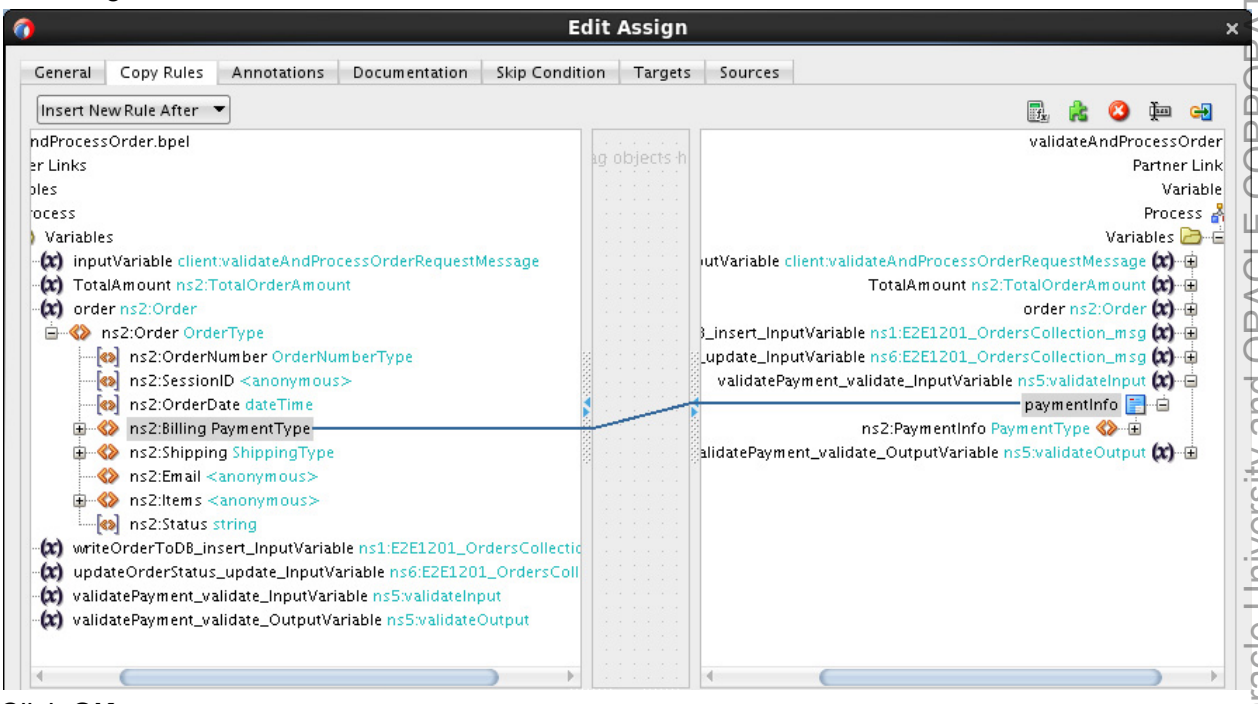
You created the variables that are used when interacting with the validatePayment service, but they have not been populated. The output variable will automatically be populated when the service returns a result, but you need to populate the input variable that is going to be passed to the service. In this case, you assign the billing data that is passed into the `validateAndProcessOrder` BPEL component to the `validatePayment` service.

a. Drag an Assign activity into the process, before the just added Invoke activity.

b. Double-click the Assign activity to edit it. The Assign activity uses an XPath expression to extract values from the input variable of the `validateAndProcessOrder` service—in this case, it is Billing—and assigns it to the input variable of the validatePayment service.

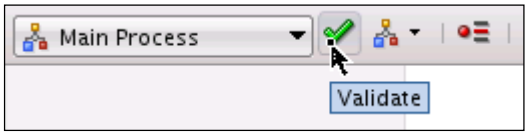
1) In the Edit Assign window, click the General tab and change the name to `assignPaymentInformation`.

2) Click the Copy Rules tab, to assign the payment information (Billing) from the order variable to the input variable of the `validatePayment` service (`validatePayment_validate_inputVariable`) by dragging a map from left to right.



c. Click **OK**.

d. Click the **Validate** button at the top of the BPEL editor.

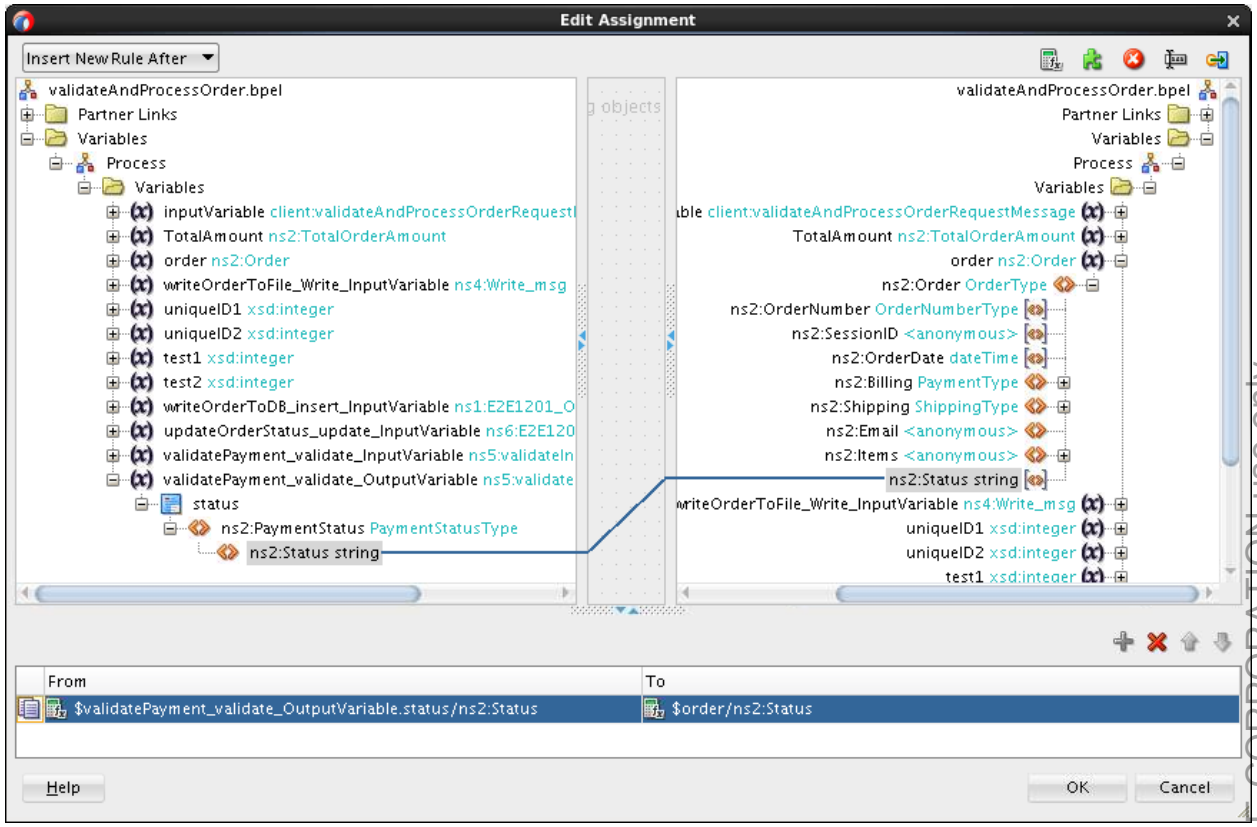


6. Create and configure an Assign activity for status.

a. Drag and drop an assign activity after the validatePayment invoke activity.

b. Name it `setPaymentStatus`.

c. In the Assign Editor, map the payment status in the output variable of the invoke activity to the status in the order variable.



d. Click **OK**.
   The Assign Editor is closed.

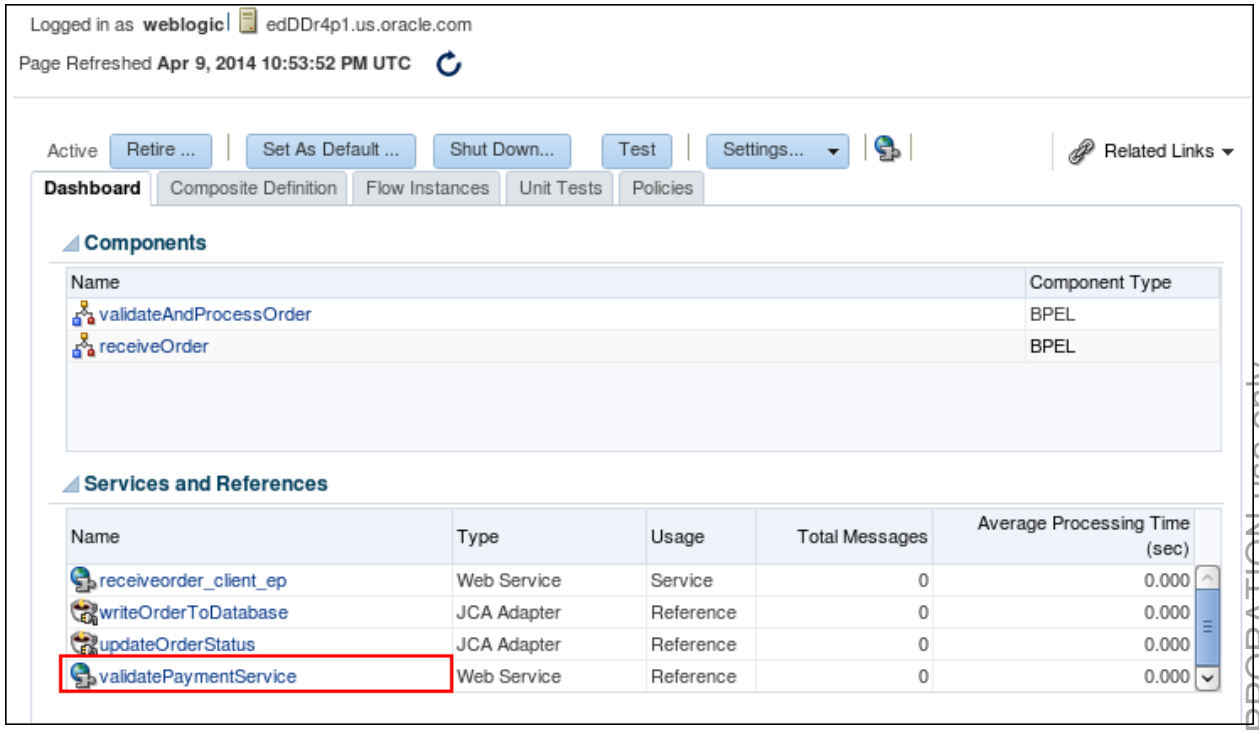
7. Save your work.

Deploy and Test the Project

8. Deploy the project.
   a. Right-click `ProcessOrder` in the project menu.
   b. Select **Deploy > ProcessOrder**.
      The Deploy ProcessOrder dialog box is displayed.
   c. Select the **Deploy to Application Server** option, and click **Next**.
      The SOA Deployment Configuration dialog box opens.
   d. Change the revision ID to **2.0**, and click **Next**.
   e. Choose **IntegratedWebLogicServer**, and click **Next**.
   f. Click **Next**.
      The deployment summary is displayed.
   g. Click **Finish**.

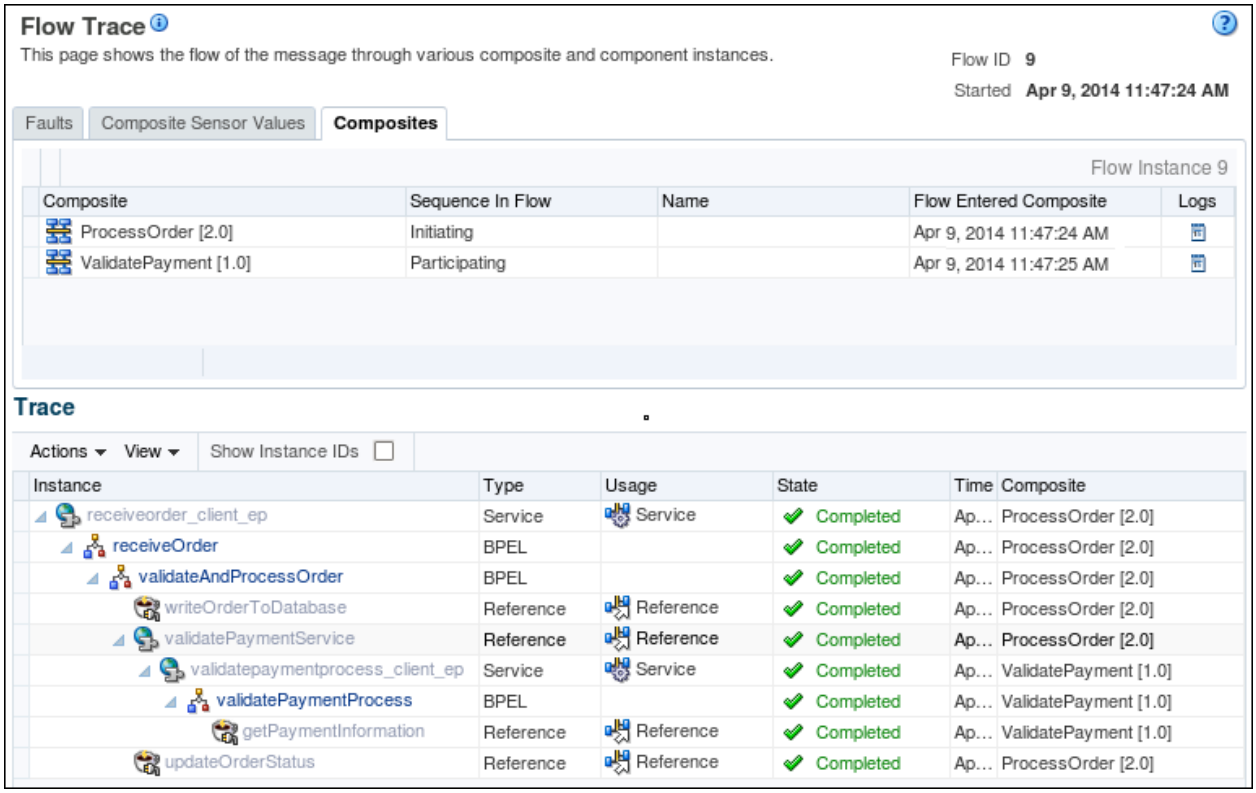
   The application is built and deployed. In the SOA log, at the bottom of JDeveloper, the message BUILD SUCCESSFUL is displayed, and the deployment begins. In the Deployment log, you can view the details of the deployment.

9. Test the project.
   a. In the Enterprise Manager navigator, select the **`ProcessOrder [2.0]`** composite.
   b. In Dashboard, note the newly added validatePayment service reference.



c. Click the **Test** button at the top of the window.
   The Test page opens.
d. Scroll down to the **Input Arguments** section.
e. Click **Browse**. In the File Upload window, select the `/labs/resources/sample_input/OrderSample_soap.xml` file, and open it.
f. Click **Test Web Service**.

10. Verify the outcome of the test.
    When you now open the flow instance in EM FMWC and select the Composites tab, you will see both composites within one flow instance.
    - `ProcessOrder` is the initiating composite.
    - `ValidatePayment` is participating.
    - Verify that there are no errors in the flow trace.



11. To verify the database update, go to the Databases window in JDeveloper. Refresh the E2E_1201_ORDERS table view, and you should see a new row added with the status **Authorized**.

Practice 5-3: Adding an Inline BPEL Subprocess

Overview

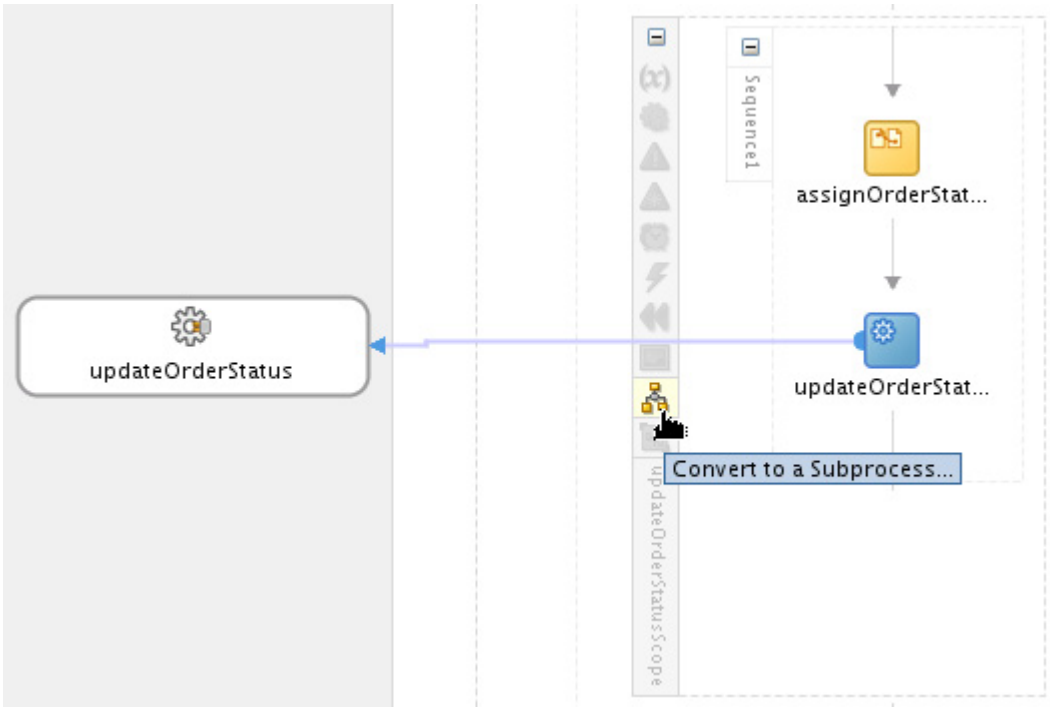
The order status update will be used at least once more in the BPEL process. For this, you have to repeat the same assign and invoke activities that we already used. This is an error prone process, and every time a change is necessary it has to be implemented in all those places. To avoid this, you can use a new feature introduced in SOA Suite 12*c*, a *BPEL subprocess*. There are two varieties of BPEL subprocesses: *standalone* and *inline.* In this practice, you will create an inline subprocess for updating order status in the database.

Tasks

1. Convert the `updateOrderStatusScope` to a subprocess.

   The scope `updateOrderStatusScope` will be converted into an inline BPEL subprocess because it is only used within the same BPEL process.

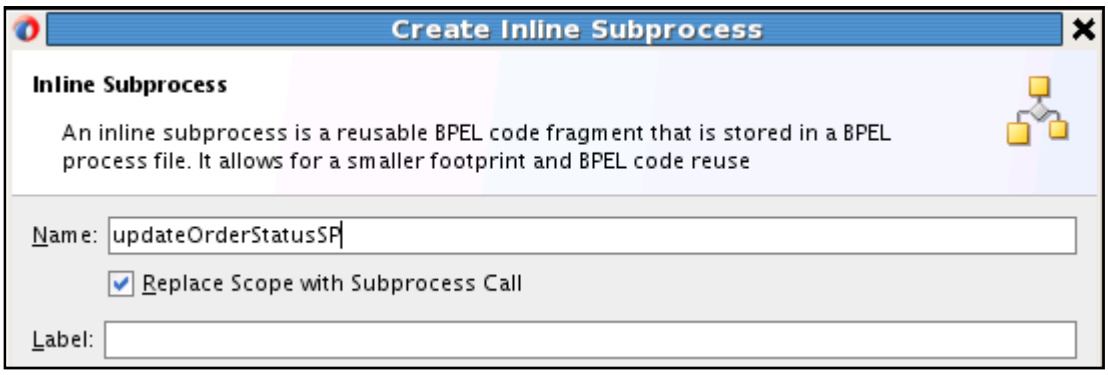
   a. In the `validateAndProcessOrder` BPEL editor, in the `updateOrderStatusScope` scope activity, select Convert to a Subprocess.



   b. Name it `updateOrderStatusSP`. (By default it uses the scope name.)

   Leave **Replace Scope with Subprocess Call** selected. This will automatically replace the current scope with a `call` activity.

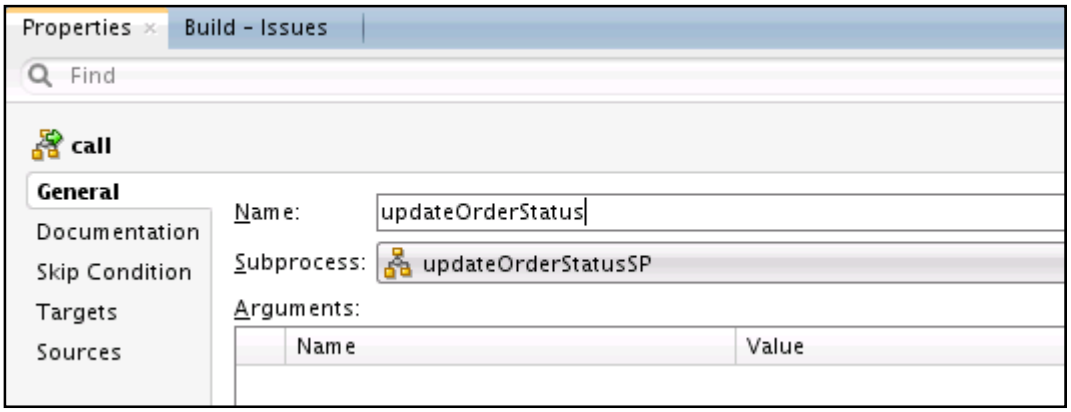
   Optionally, you can set a label, comment and choose an image for the new subprocess.



c. Click **OK**.

A new `call` activity is added to the process. The subprocess appears in the component palette under Subprocesses. You can see and edit the definition of the subprocess when switching from Main Process to the subprocess on top of the BPEL editor. In the subprocess editor, you can see all partner links that are available in the main process, in case you want to add another invoke. You can make changes to this subprocess, which will be reflected in every call.

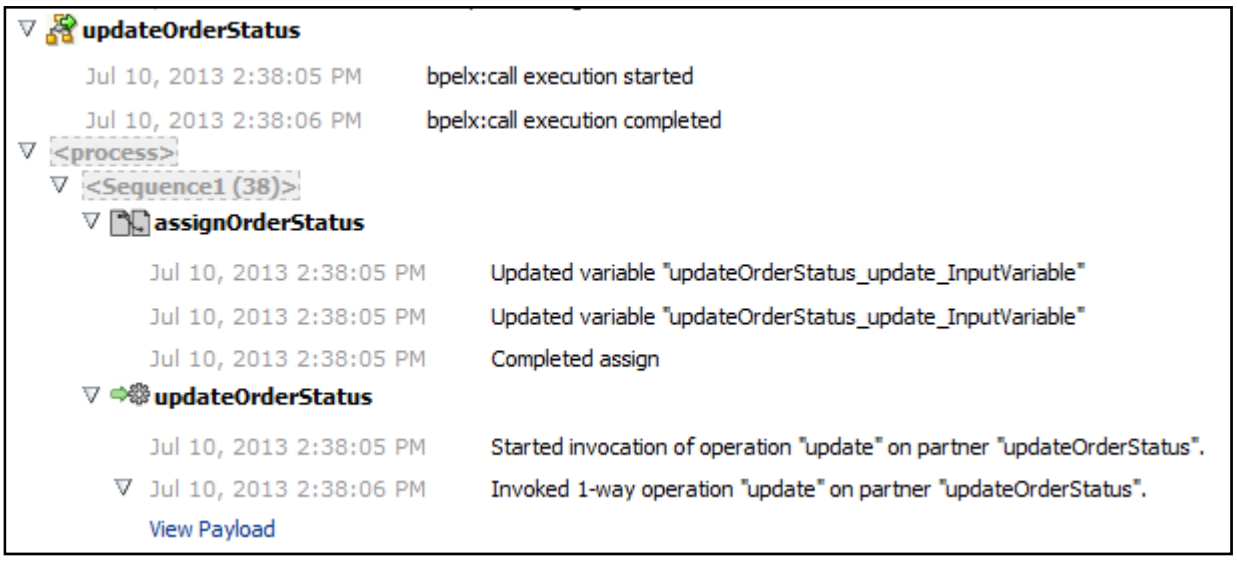
d. Change the name of the call activity by selecting the call activity and changing the name in the property inspector to `updateOrderStatus`.



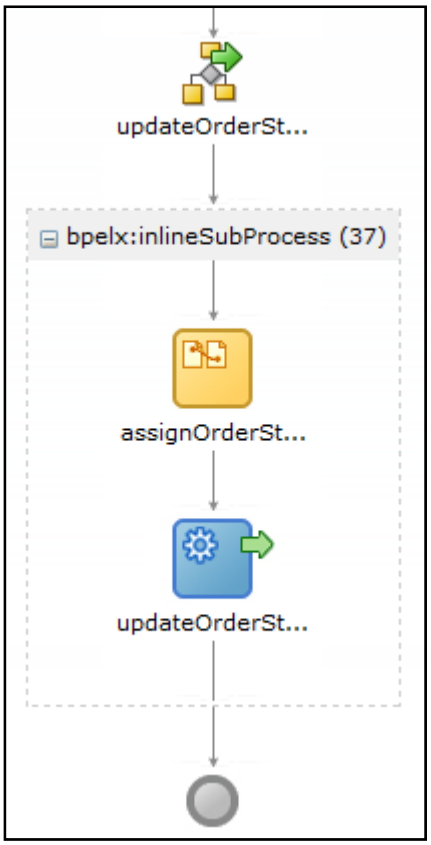
2. Redeploy the process.
   a. Right-click `ProcessOrder` in the project menu.
   b. Select **Deploy > ProcessOrder to IntegratedWeblogicServer**.
   c. In Deployment log, make sure that you see that the deployment finished successfully.
3. Repeat step 9 in Practice 5-2 to test the composite.

   The actual result should not change.

   - The call activity is shown in the audit trail with the activities included in the subprocess.



- Similarly, in the flow trace, the call activity and all activities included in the subprocess below it are shown.



4. At this point, you can either move on to the next practice, an optional lab exercise, or end this practice by closing the po_composite application.

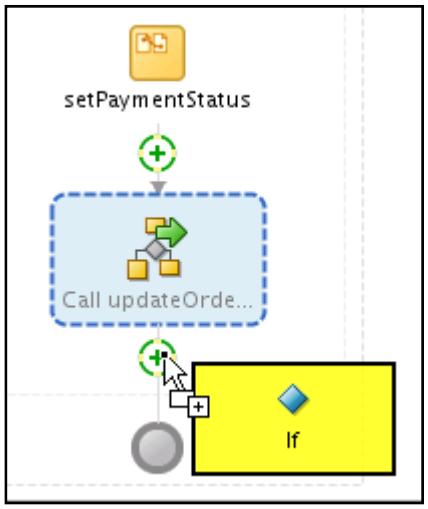
Practice 5-4: Adding a Call to the BPEL Subprocess (Optional)

Overview

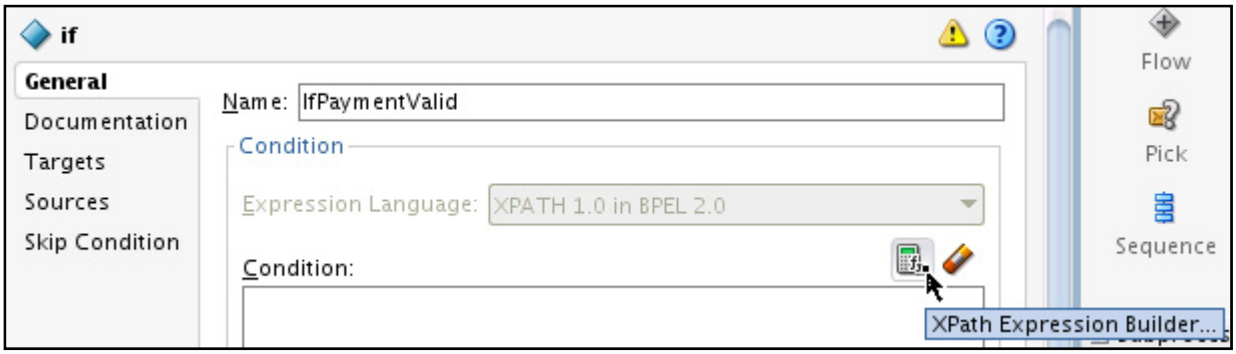
If the payment is valid, the order status is set to ReadyForShip in the database. This status update will trigger the order fulfillment process. If the payment is not valid, processing ends here. First we add an `If` statement. (`If` in BPEL 2.0 is the equivalent of `switch` in BPEL 1.1.) The order will only continue if the payment was authorized. Otherwise, processing is stopped and an email is sent to the customer informing him of his unauthorized payment.

Tasks

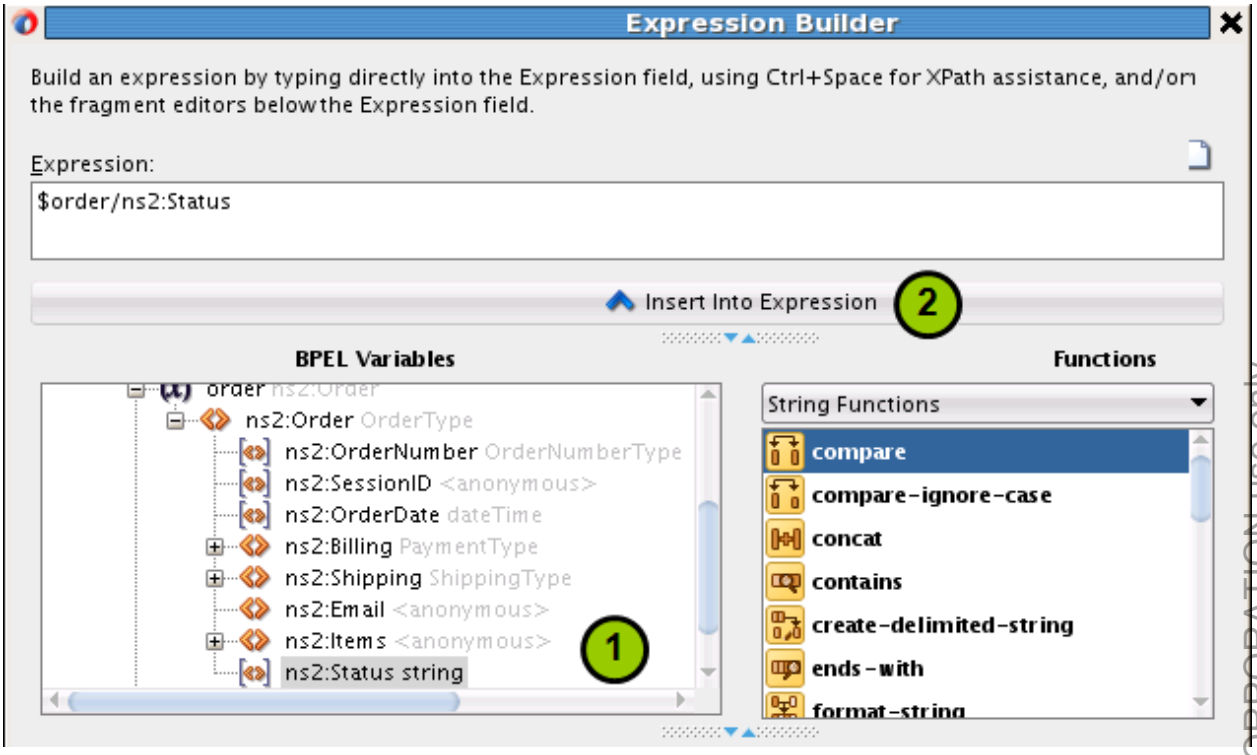
1. Create and configure an **if** statement.
   a. Open the `validateAndProcessOrder` BPEL process if it is not already open.
   b. Add an **If** under the subprocess call. (The **If** activity is found on the component palette under *BPEL Constructs – Structured Activities*.)



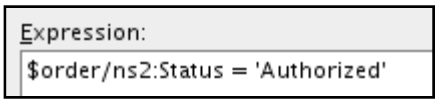
   c. Select the **If** construct and open the property inspector window, if it is not already open.
   d. Change the name to `IfPaymentValid`.
   e. Edit the condition by clicking the **XPath Expression Builder** icon.



   f. Expand the `Order` variable and enter the `Status` element into the Expression field by clicking **Insert Into Expression**.



g. Set the `Status` element to **`'Authorized'`**. Click **OK**.



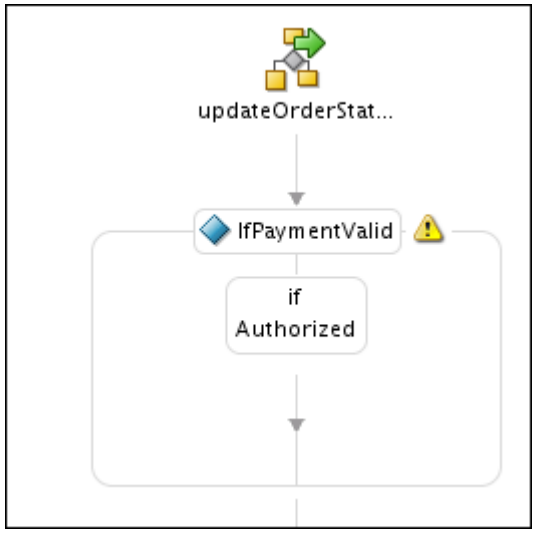
h. Back in the BPEL process, change the **If** label to `Authorized` by clicking on the label.

i. Click **OK**.

j. Delete the **else** branch.

The else branch is not necessary, because the processing will just stop if the payment has been denied.

k. Confirm the deletion.



2. Create and configure an assign activity in the **If** branch.

If the payment has been authorized, the processing of the order is complete and you update the order status to `ReadyForShip`.

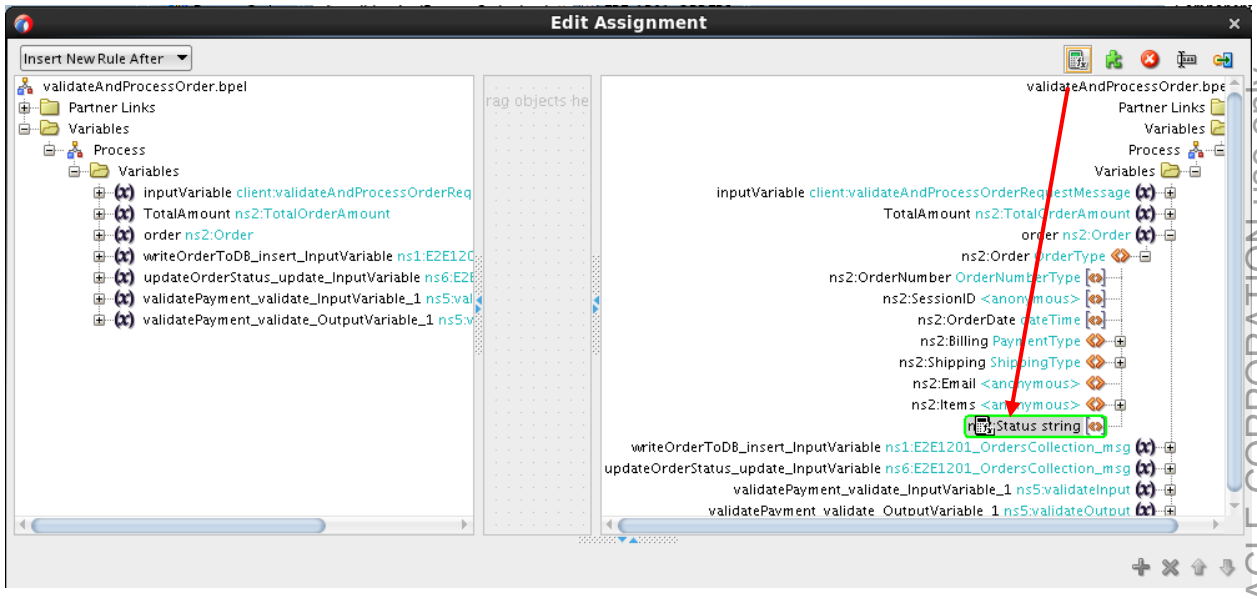
a. Add an assign activity to the **If** branch.

b. Name it `assignReadyForShipStatus`.

c. Open the Assign Editor.

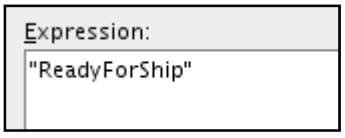
d. Expand the `Order` variable on the right side of the `Status` element.

e. Drag the **Expression** icon from the toolbar onto the `Status` element.



The Expression Builder is displayed.

f. Enter `"ReadyForShip"`.



g. Click **OK** to close the Expression Builder.

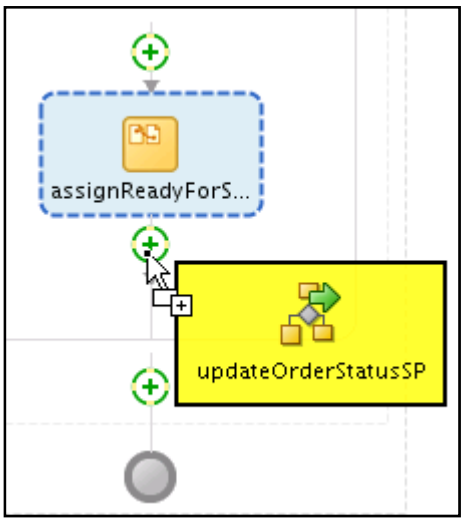
h. Click **OK** to close the Assign Editor.

i. Save your work.

3. Add a call to the subprocess that you just created.

The subprocess will update the order status in the database. As before, because it reads the status from the order message, no additional assign activities are needed.

a. Drag the updateOrderStatusSP subprocess from the component palette to the business process, just after the assign.



A call activity is created.

b. Rename the call activity updateOrderStatusToReadyForShip.

4. Redeploy and test the project.

   The order status in the database will be either Denied or ReadyForShip.

5. When you are done, close the po_composite application in JDeveloper (including all the open windows).

Practices for Lesson 6: Connecting with Binding Components

Chapter 6

Practices for Lesson 6

Practices Overview

Business trends indicate that AviTrec must launch a mobile application soon and the new payment validation service must support access through RESTful APIs.

In these practices, you will learn how to expose the existing ValidatePayment composite application as a REST service.

When, in the practices for the lesson titled “Building SOA Composite Applications,” you examined the structure of the ValidatePayment composite, you saw that the composite uses the database adapter to access credit card information stored in a database. In the current practices, you will examine how the database adapter is configured to access data in the database.

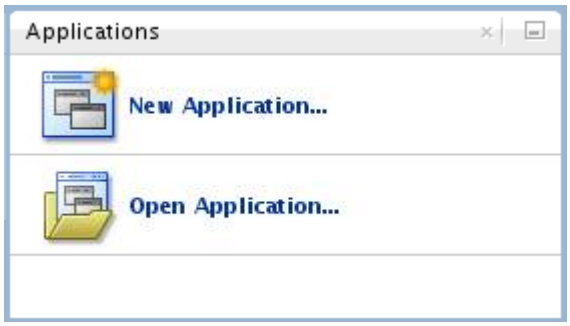
Practice 6-1: Exposing a Composite as a REST Service

High-level steps:

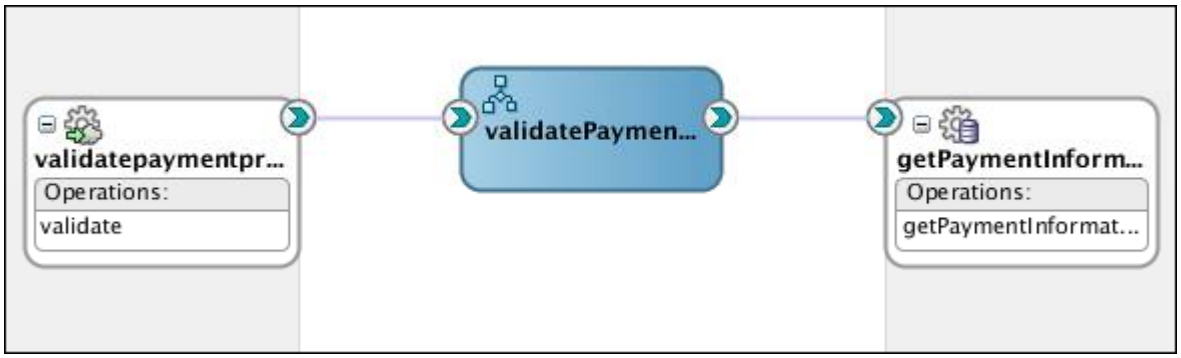
- Define a REST interface for the ValidatePayment application.
- Deploy the service as revision 2.0.
- Test the REST service by using the HTTP analyzer.

Open ValidatePayment project

1. Open JDeveloper if it is not already open. (You can double-click the JDeveloper shortcut on the desktop to open JDeveloper.)
2. In the Applications pane, click **Open Application** (or select **File** > **Open**).



3. In the Open Application(s) dialog box, navigate to the `/labs/lessons/Lesson06/po_composite/` directory, and open the `po_composite.jws` file.
4. Double-click ValidatePayment (composite.xml) in the ValidatePayment project folder. The application components are displayed on the canvas of the composite editor as shown in the following:



Note: This application currently is exposed as a SOAP interface, validatepaymentprocess_client_ep.

Define a REST interface

5. Define a REST interface for the ValidatePayment project.

   The validate payment service expects a payment resource that includes all necessary information for validation.

   a. Drag a REST Binding Adapter from the Components palette (in the Technology section) onto the Exposed Services lane.

   b. In the Create REST binding window, set the following fields:

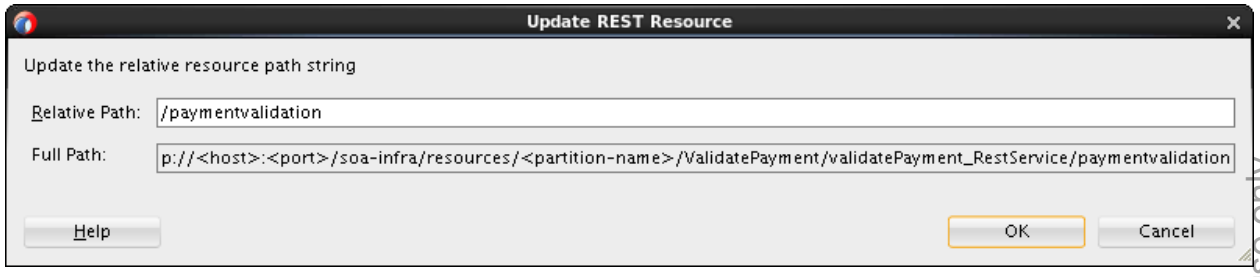
      - Name: `validatingPayment_RestService`

- Description: A REST interface to the ValidatePayment application

c. Add a new resource.

1) Click the green Add icon in the Resources section. The Create REST Resource dialog box is displayed.

2) Enter /`paymentvalidation` as the relative path to the resource.



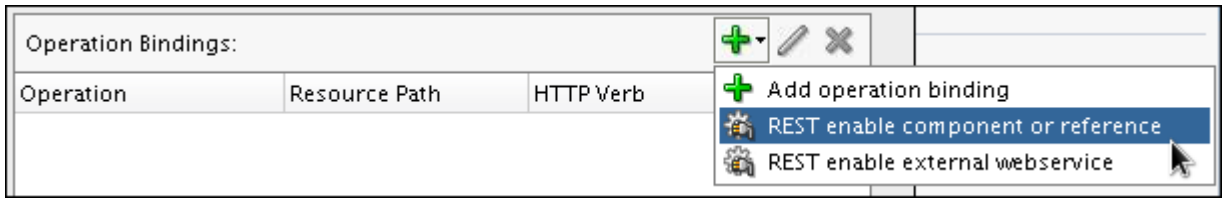
3) Click OK.

d. Delete the default resource by selecting it and clicking the delete icon.

e. Add an operation binding.

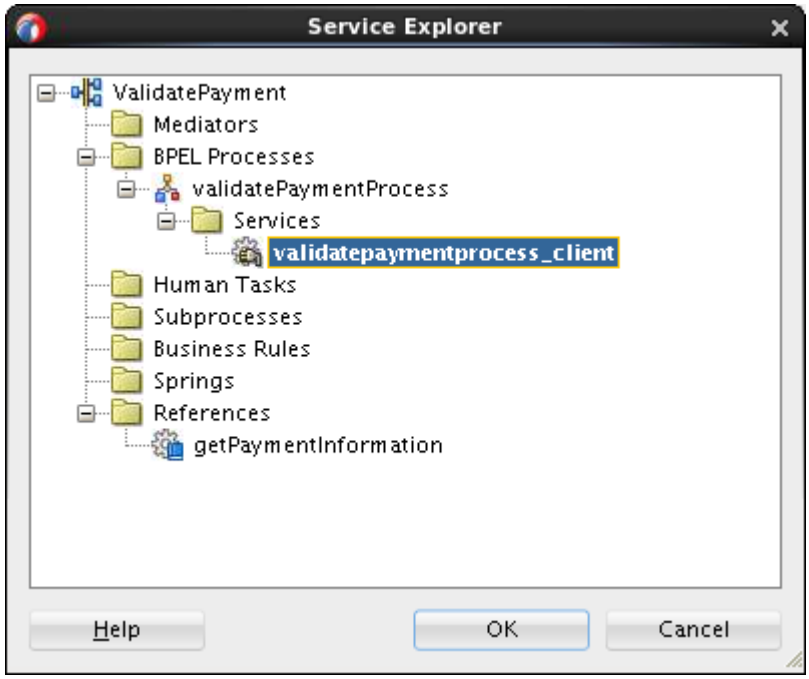
In this step, you define the REST operations based on the WSDL of the component with which the REST service interfaces.

1) Click the Add icon in the Operation Bindings panel, and select **REST enable component or reference** from the drop-down list.



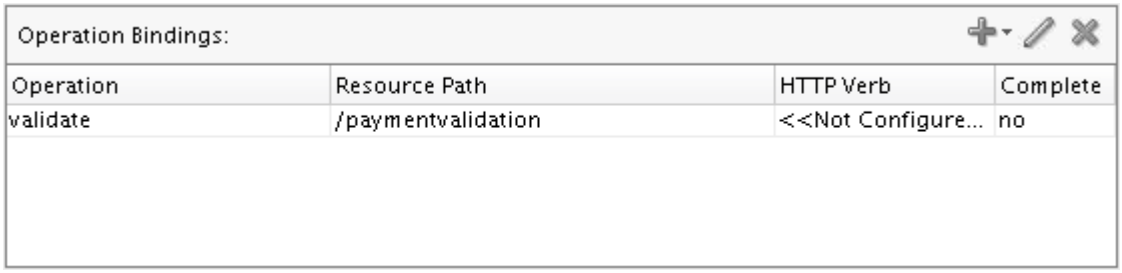
The Service Explorer dialog box is displayed. You select the WSDL of the service component from which to map WSDL operations to resource paths and HTTP verbs. In this practice, the service component is the BPEL process.

2) Expand ValidatePayment > BPEL Processes > validatePaymentProcess > Services, and select **validatepaymentprocess_client**.



3) Click OK.

The Create REST binding window is updated with the `validate` WSDL operation and its mapping resource path, `/paymentvalidation`, as shown in the following screenshot:



Note that HTTP Verb is not defined yet. This is indicated by the value `no` in the Complete column.

4) Map an HTTP verb to the WSDL operation.

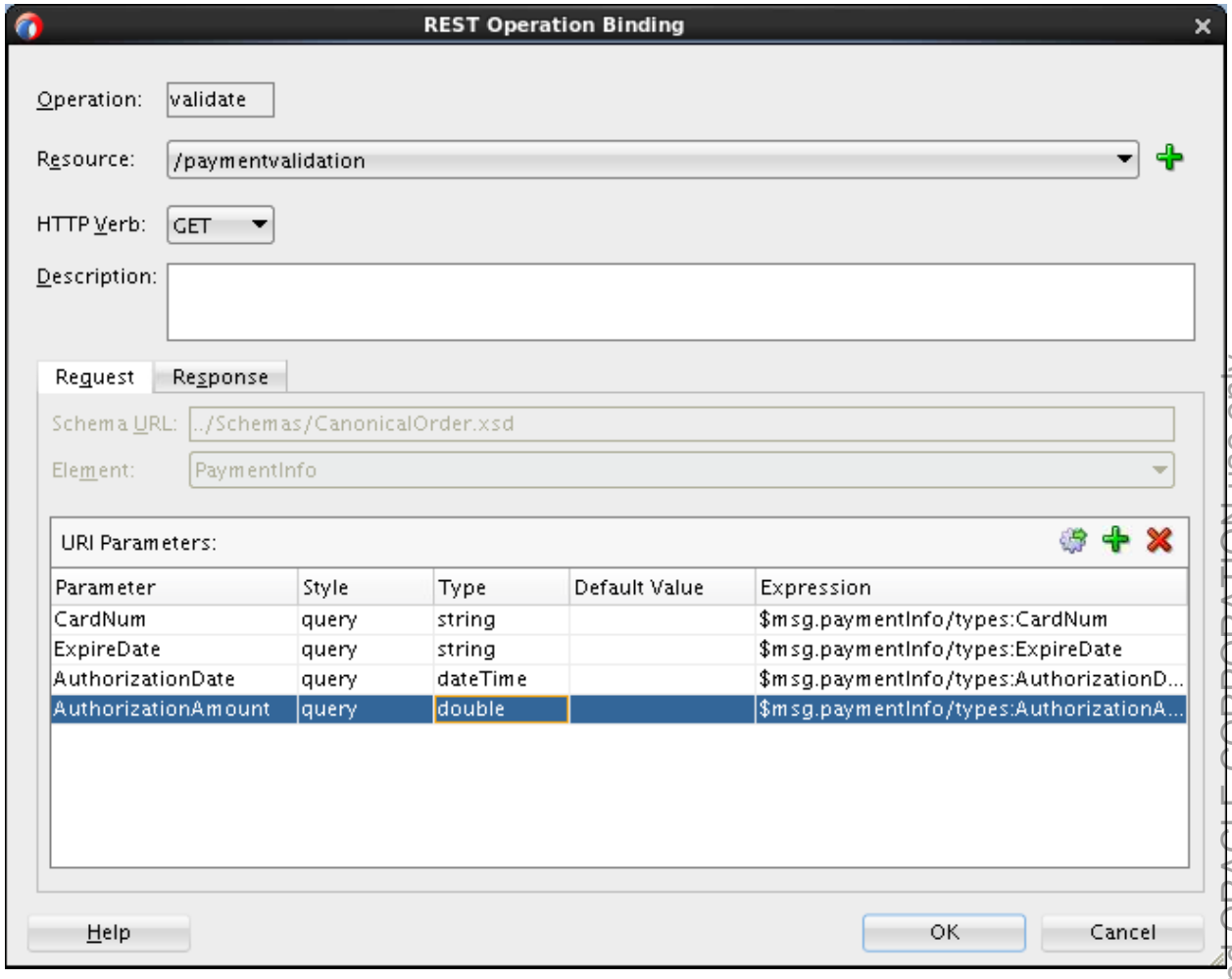
a) Select the `validate` operation, and click the Edit icon (the pen icon).

The REST Operation Binding dialog box is displayed.

b) Use the default value `GET` as the HTTP verb for the operation.

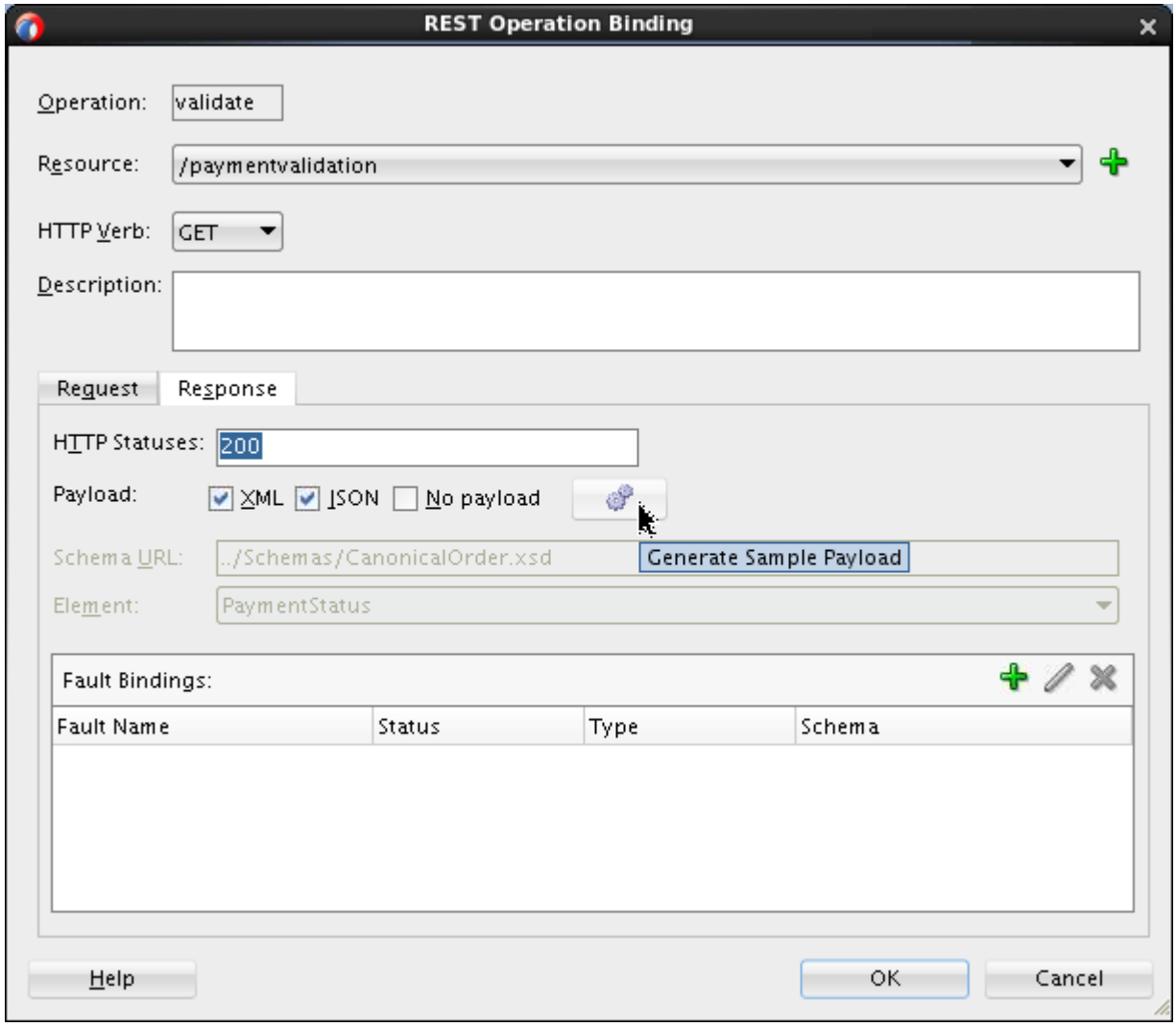
The URI Parameters section is automatically populated when a schema is specified (depending on the verb) in the HTTP Verb list.

c) Remove unneeded parameters. The URI Parameters field in the REST Operation Binding dialog box lists the kept parameters.



Note: With URI Parameters, you can specify the mapping from the REST query parameters to the WSDL schema here.

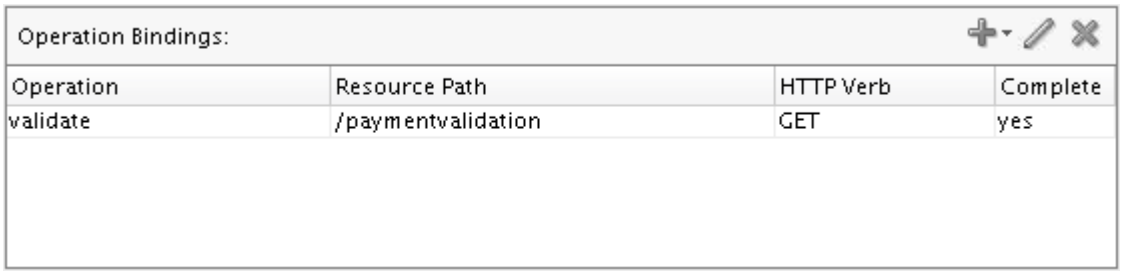
d) Click the Response tab. Accept the default HTTP status code and Payload selection.



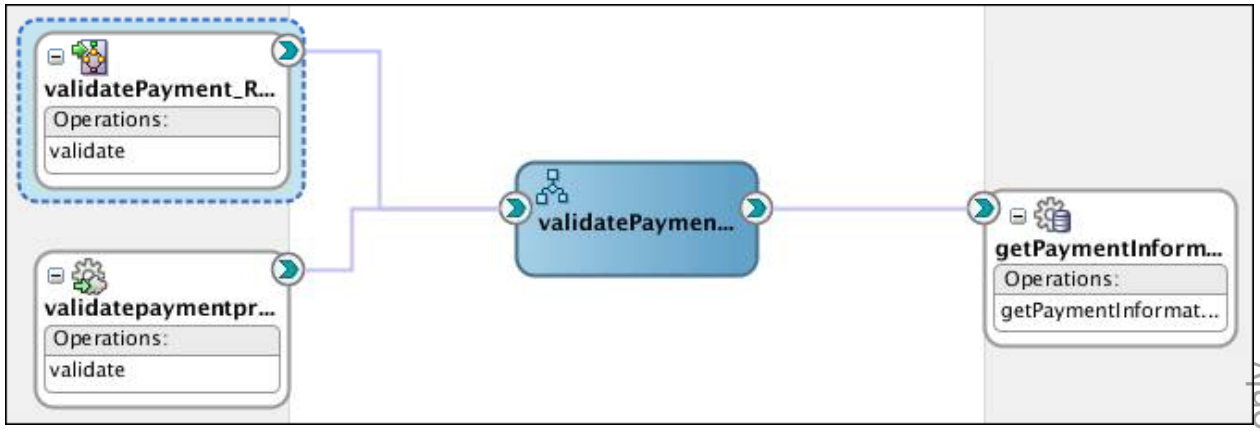
e) Select JSON.

f) Click Generate Sample Payload to view a sample of the selected response payload.

5) Click OK. The Create REST binding window is updated with the defined HTTP verb, as shown in the following screenshot:



f. Click OK to complete the definition of the REST service. The canvas is updated with this newly added REST service, and a new file, `validatePayment_RestService.wadl`, is added in the SOA > Adapters folder.



Deploy and test the REST service

Testing a REST service is different than testing a SOAP service. SOA REST services expose WADL files instead of WSDL files to define their interface. In order to be able to test the REST service, you will need to get the WADL location from EM.

6. Deploy a new version (v2.0) of the ValidatePayment application.

   Note: Make sure the Integrated WebLogic Server is up and running before the deployment.

   a. In the Applications pane, right-click ValidatePayment, and select Deploy > ValidatePayment from the context menu.

      In the Deploy ValidatePayment dialog box window, enter the information specified in the following table:

Step	Window/Page Description	Choices or Values
1	Deployment Action	**Deploy to Application Server** Click **Next**.
2	Deploy Configuration	New Revision ID: **2.0** Deselect "Mark composite revision as default." Click **Next**.
3	Select Server	**IntegratedWebLogicServer** Click **Next**.
4	SOA Servers	Use the default server and click **Next**.
5	Summary	Review the summary and click **Finish**.

   b. Open the Deployment – Log tab window, monitor the deployment progress and make sure that you see "Deployment finished successfully."

7. Test the REST service.

   a. Open Enterprise Manager Fusion Middleware Console: `http://localhost:7101/em`.

   b. In the Target Navigation pane, expand the SOA > soa-infra > default folder. You should see a new version of ValidatePayment [2.0] added to the deployed composite application list.

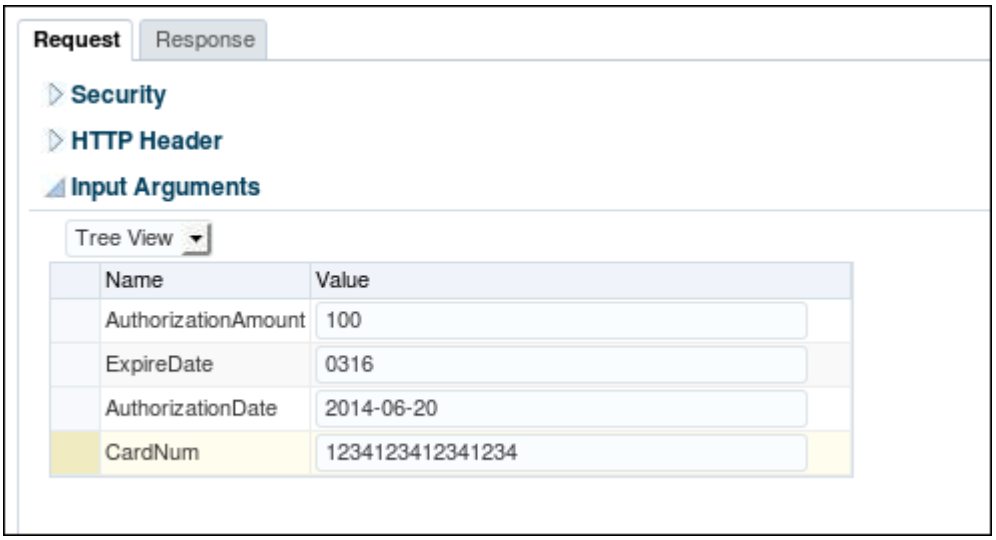
c. Click the ValidatePayment [2.0] link, and view the composite information on the ValidatePayment [2.0] > Dashboard tab page and compare it to the earlier version [1.0]. Describe the differences:

_____

d. Click **Test**, and select **validatingPayment_RestService** from the drop down menu. The Test Web Service page is displayed.

e. On the Request tab, enter XML payload data in the Input Arguments section as specified in the following table:

Name	Value
Authorization Amount	**100**
ExpireDate	**0316**
AuthorizationDate	**2014-06-20**
CardNum	**123412341341234**



f. Click **Test Web Service.**

g. On the Response tab, you should see the request successfully received and the payment status updated to Authorized.

Note: Alternatively, you can test the REST service with the HTTP Analyzer in JDeveloper.

Practice 6-2: Examining a Database Adapter

The database adapter in SOA Suite helps to expose data and operations from relational databases in a service-compliant fashion. In this practice, you examine the preconfigured database adapter of ValidatePayment. This adapter performs a Select operation on the PAYMENTINFO table to return the expiry date and daily limit information for the given credit card number.

1. In the ValidatePayment Composite Editor, right-click the `getPaymentInformation` database adapter (in the External Reference swimlane), and select **Edit** from the context menu.

   The Oracle Database Adapter Configuration wizard is displayed. This wizard takes you through the steps of configuring the database adapter. Because the adapter is pre-created, let's take a look at some key configuration steps, described in the following table:

Step	Window/Page Description	Choices or Values
a.	Service Connection	The database adapter needs a connection that contains all the details needed to connect to the database. You can pre-create a database connection or create one on the fly. The database adapter connects to any relational database using JDBC. For our practices, we use the embedded Java Derby database. **Note:** The JNDI Name must match a JNDI connection factory resource configured for the database adapter deployed in the Oracle WebLogic Server runtime environment.
b.	Operation Type	Specify the database operation that this service will provide. Here we need to perform a query against a table, so the Select operation is used.
c.	Select Table	The E2E_1201_PYMENTINFO table is in the SOAINFRA schemas.
d.	Relationships	Defined when there are multiple tables to be joined together, or a table with self-referencing relationships.
e.	Attribute Filtering	Only select the attributes that must be queried from the database. Excluding information that you do not need can lower the load on both the database and the SOA Suite run time.
f.	Define Selection Criteria	Here you can create query parameters and conditions for your Select operation.

2. When you are done with the examining, click Cancel to close the wizard window.
3. Now close the po_composite application in JDeveloper (including all the open windows).

Practices for Lesson 7: Mediating Messages with Mediator Components

Chapter 7

Practices for Lesson 7

Practices Overview

At this point, AviTrec has built the core order system. There is a new requirement to extend and add a new interface so that legacy systems or trusted users may place orders using files.

In these practices, you add a new file order channel for the ProcessOrder composite leveraging the work done in previous practices. The main tasks are adding:

- A new file adapter to accept purchase order from a file
- A Mediator component to provide mapping between the message structure delivered by the file adapter service and the data model of the service interface published for the BPEL service component

By doing so, you get to understand:

- How a file adapter can be used together with Mediator to read incoming files as well as initiate new instances of composite applications to process the contents from these files
- How Mediator takes the messages from the client, after transforming them into the appropriate structure, and routes it to the target service

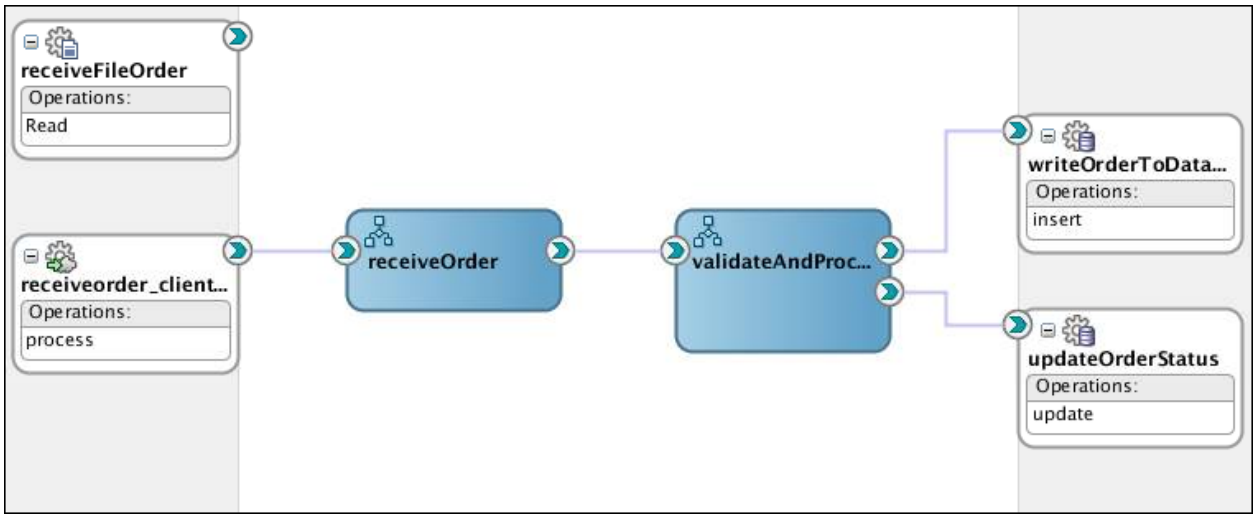
Practice 7-1: Inspecting the File Adapter That Reads Purchase Orders from Files

Overview

For this practice, a File Adapter service reading purchase order from files has already been added for you (to save your time). Therefore, you just need to examine the configuration of the file adapter and the XML schema that defines the message structure of the incoming order files.

Tasks

1. In the Applications pane, click **Open Application** (or select **File** > **Open**).
2. In the Open Application(s) dialog box, navigate to the `/labs/Lessons/lesson07/po_composite` directory, and open the `po_composite.jws` file. The ProcessOrder project folder displays in the Applications pane.
3. In SOA Content folder, open the ProcessOrder file in the Design view of the Composite Editor.



In the practices for the lesson titled “Orchestrating Services with BPEL Process Components,” you created the ProcessOrder composite that accepts purchase orders via a SOAP invocation. This assembly model is based on the composite version 1.0 created in the practices for the lesson titled “Orchestrating Services with BPEL Process Components” with one difference made—adding a file adapter, `receiveFileOrder`. This file adapter service reads an incoming purchase order from a file and initiates new instances of composite applications to process the order.

Note: The assembly model (composite) provides two service entry points. You can still initiate the ProcessOrder application from the receiverorder_client service entry point through the Oracle Fusion Middleware Control Console Composite Test Web Service page tool. The newly added file adapter will provide the receiveFileOrder service entry point to read the purchase order as an input file. The file adapter service interface avoids too much typing to initiate the application, and simulates data input similar to the way an external client application would provide input.

Inspecting the File Adapter

Let's take a look at the configuration of this File Adapter service:

4. In the Design view of the Composite Editor, double-click the receiveFileOrder file adapter (Exposed Service).
5. In the File Adapter Configuration Wizard, review the information presented in each step and answer the following questions:

   Q: What is the operation name of this file adapter service interface?

   A: `Read`

   Q: From where should the File Adapter service read the files?

   A: The directory from where the service reads the files is `/labs/test`.

   Q: What is the name pattern for the files to process?

   A: `po*.xml`. This name pattern is used as a filter that determines which of the files in the directory should be processed by the file adapter.

   Q: How does the message structure of the file adapter service interface get defined?

   A: The message structure is defined by the XSD file, `FileOrder.xsd`, in the `Schemas` directory of the SOA project folder.

   Note: With the above configuration, the File Adapter service is kicked off whenever polling reveals that a new file is present to be processed. Files that have been processed by the File Adapter service are removed from the directory where they were placed; this behavior is configured through the check box labeled "Delete files after successful retrieval."
6. Click Cancel when you are done.

Examining the File Order XML Schema

The input message is produced by the File Adapter service according to the XML schema, `FileOrder.xsd`. Let us take a look at the structure of this XSD file.

Note: This XSD file was imported into the project folder from the file system by the File Adapter Configuration Wizard when creating the File Adapter service.

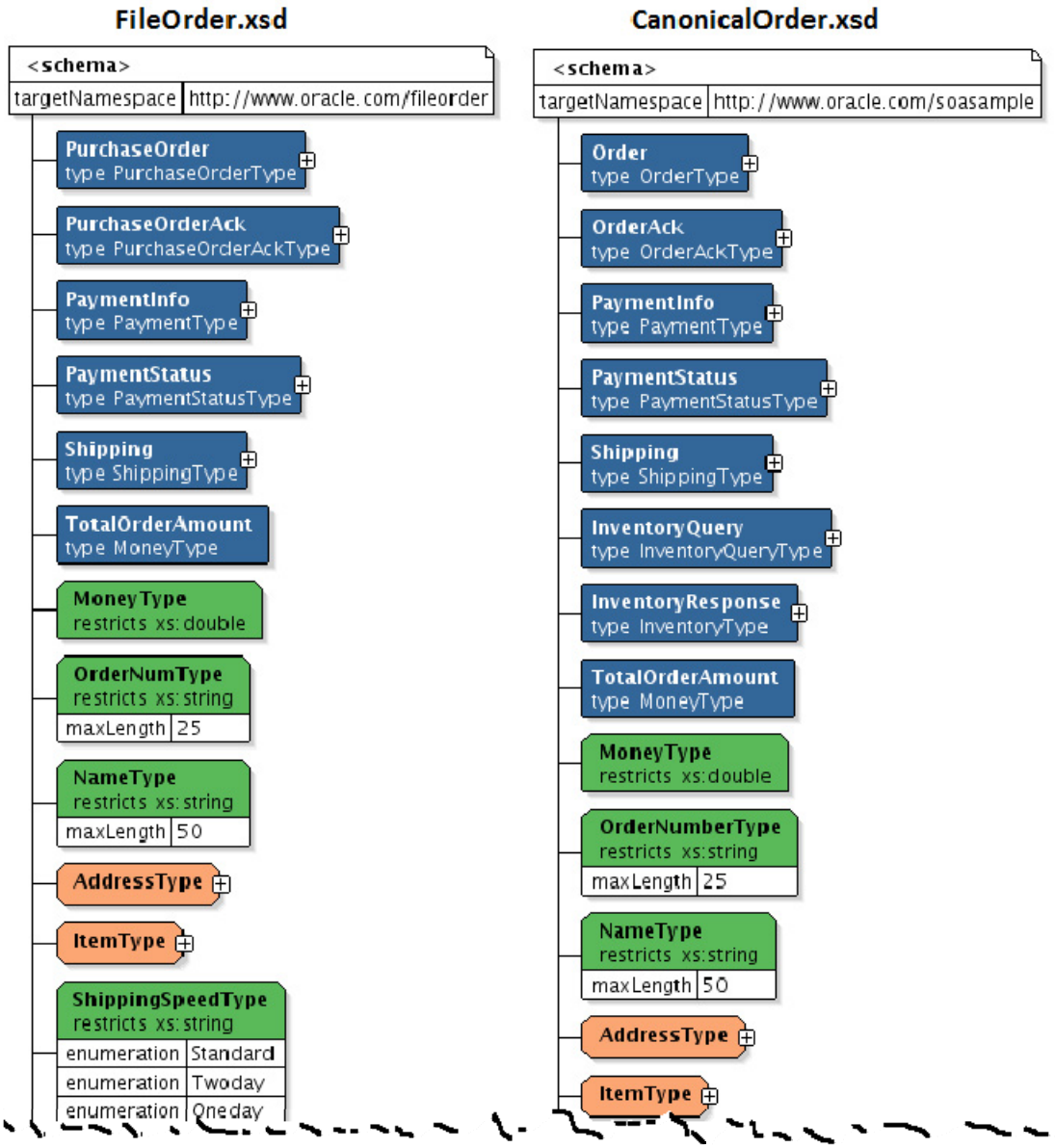
7. In the ProcessOrder > SOA> schemas folder, double-click the `FileOrder.xsd` file.
8. The file opens in the XSD Editor, in the Design view. Examine the purchase order XML schema and answer the following questions:

   Q: What elements does the schema declare?

   A: The XSD declares the PurchaseOrder element.

   Q: What type is the element based on?

   A: A complex type named PurchaseOrder.
9. Expand the PurchaseOrder type to view the definition of this complex type. It contains a number of child elements.
10. Compare this file order schema with the schemas of the `CanonicalOrder.xsd` file, which is located in the same folder.



Comparing the two schemas, can you describe the differences?

Practice 7-2: Creating a Mediator to Route Order Requests

Overview

In this practice, you create a Mediator component to route the purchase order retrieved from the files to the receiveOrder BPEL service component. You need to configure the routing rules to:

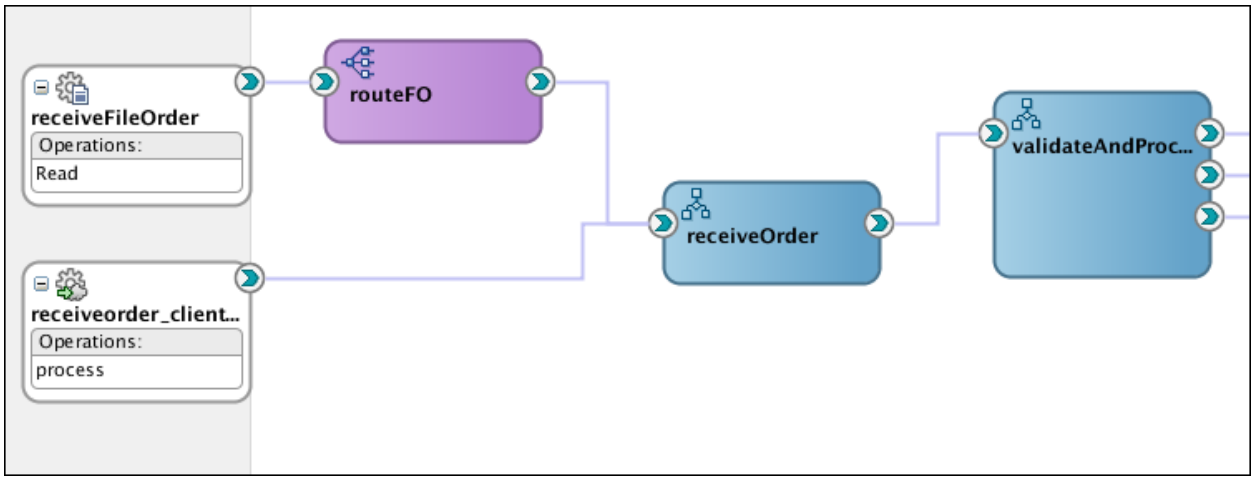
- Instruct the Mediator where to send XML messages
- Perform some message transformation, because the structure and some of the data elements in the files are not matched with the BPEL component service interface

Tasks

Creating a Mediator Component

1. Make sure that the Composite Editor of your POApplication composite application is open and displays the Design view.
2. Drag a Mediator component from the Component palette, and drop it into the Components lane in the Composite Editor.
3. In the Create Mediator window, specify the following options, and click OK.
   - Name: `routeFO`
   - Template: Define Interface Later
4. Wire the components:
   - From receiveFileOrder File Adapter to routeFO Mediator
   - From routeFO Mediator to receiveOrder BPEL component

   Use the following screenshot as a guide:

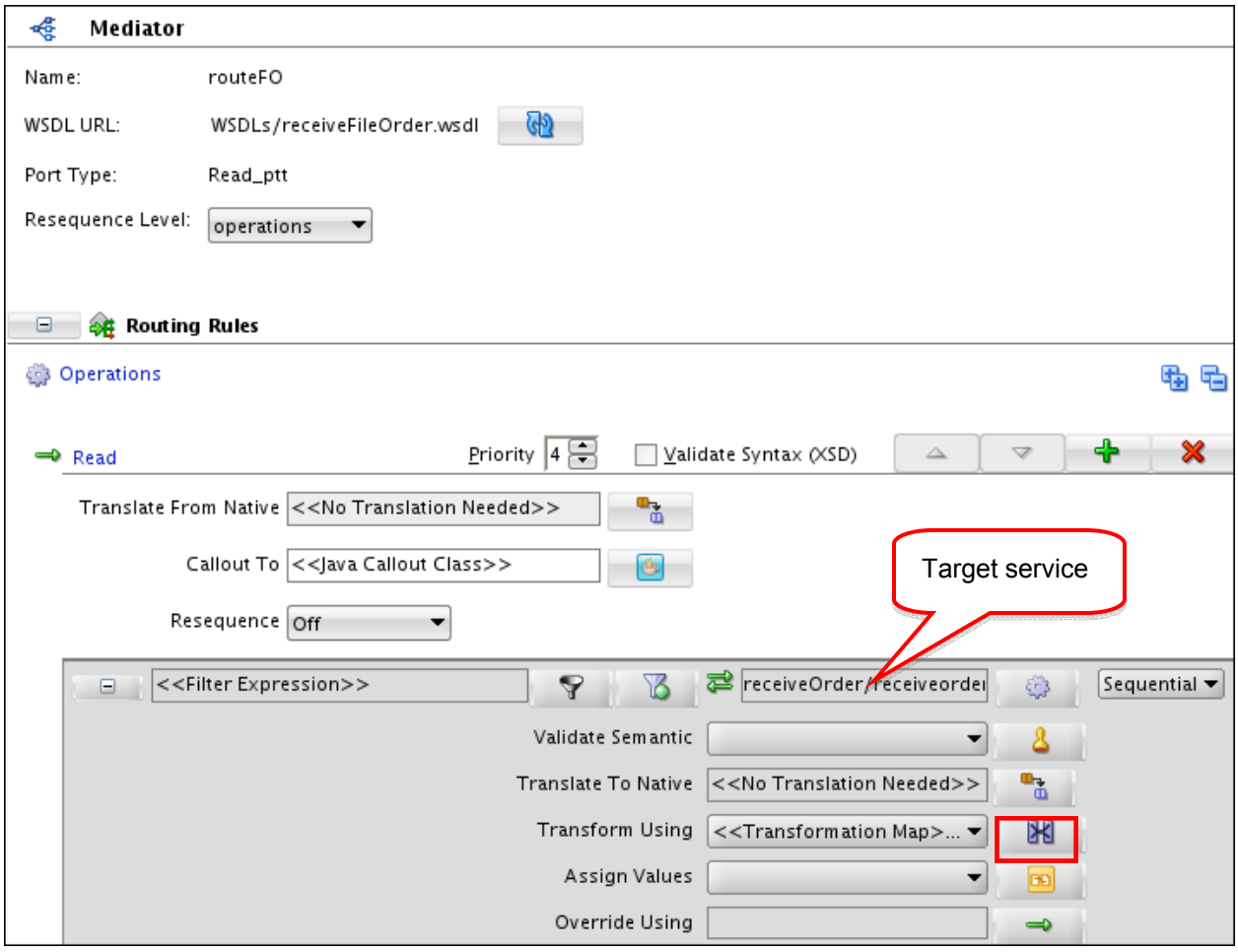


Configuring Routing Rules

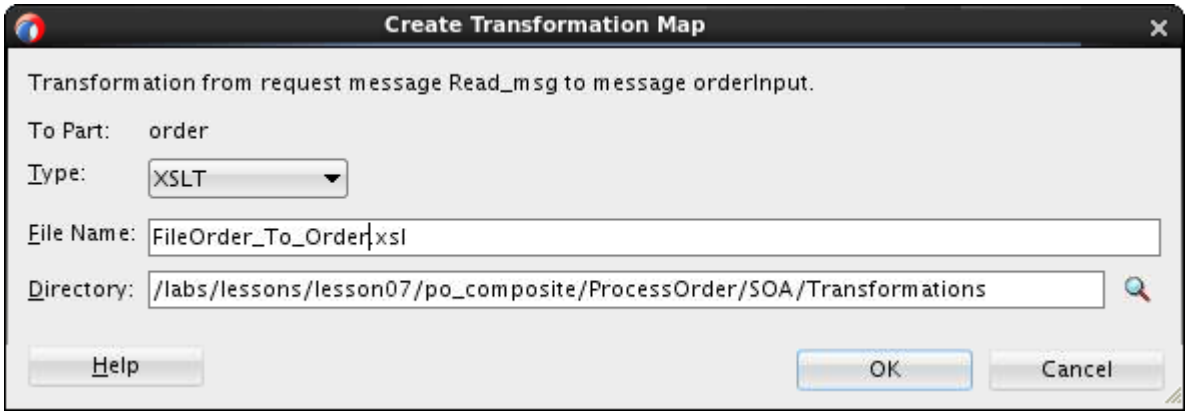
By wiring the routeFO Mediator component to the receiveOrder BPEL component, you create a routing rule with the receiveOrder BPEL process as the target service. In this section, you configure this new routing rule to transform the purchase order, an XML input based on the `FileOrder.xsd` file, into an XML format understood by the ReceiveOrder BPEL process component.

5. To further configure the routing rule, in the Composite Editor, double-click the routeFO Mediator to open the Mediator Editor.

6. In the Mediator Editor (routeFO.mplan window), click the mapping icon to specify the mapping as shown in the following screenshot:



7. In the Request Transformation Map dialog box, click the Create Mapping icon. Enter `FileOrder_To_Order.xsl` as File Name, and click OK.

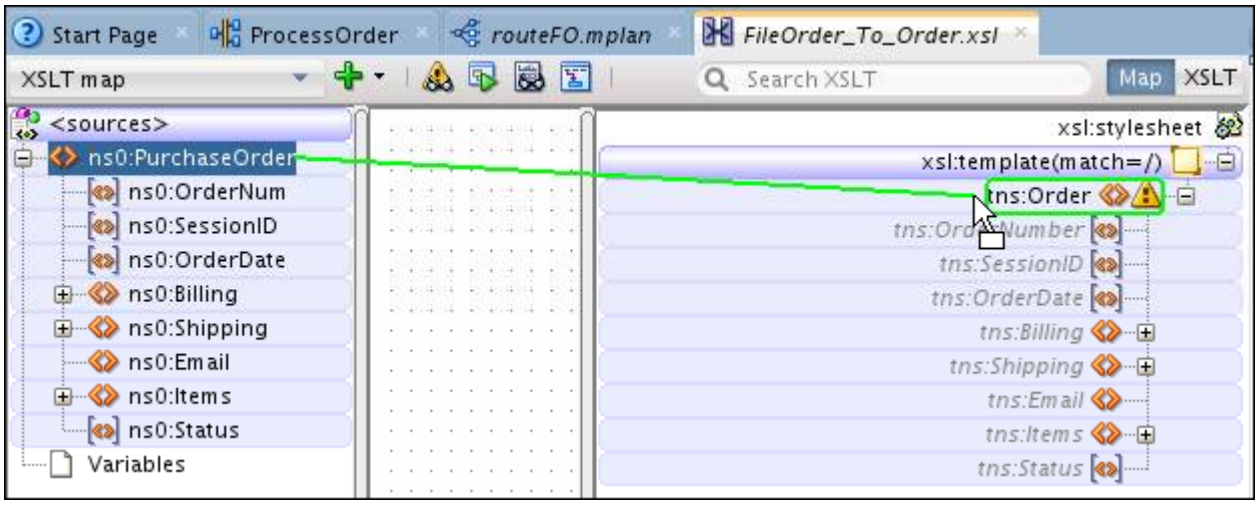


Note: Be sure to provide a meaningful name for the Mapper file, one that clearly indicates what is being mapped to what.

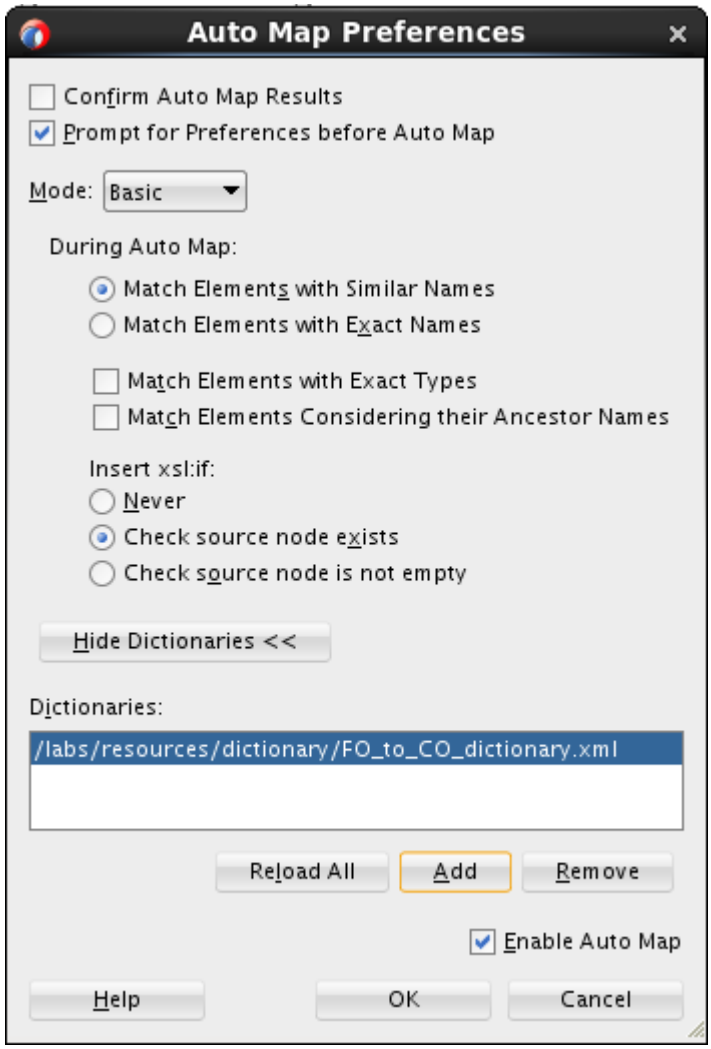
The mapping editor opens in the main window.

8. Click OK to close the Request Transformation Map dialog box.

9. When creating a Mapper file, you basically specify the XML conversion path from the input XSD to the target service's XSD. In the mapping editor, drag PurchaseOrder node from the source side to Order node on the target side. Use the following image as a guide:

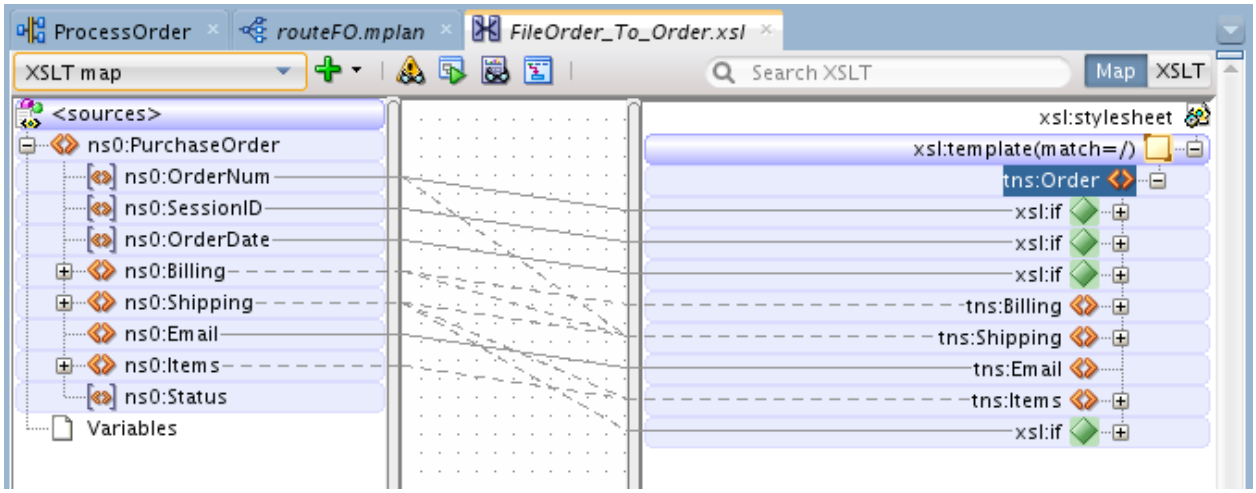


10. You will be prompted for auto-mapping preferences. In the Auto Map Preferences window, perform the following steps:
    a. Deselect the "Match Elements Considering their Ancestor Names" check box and click Show Dictionaries.
    b. Under the Dictionaries field, click the Add button.
    c. Select `/labs/resources/dictionary/FO_to_CO_dictionary.xml` and click Open.
    d. The Auto Map Preferences window is updated with the dictionary information as shown in the following screenshot:

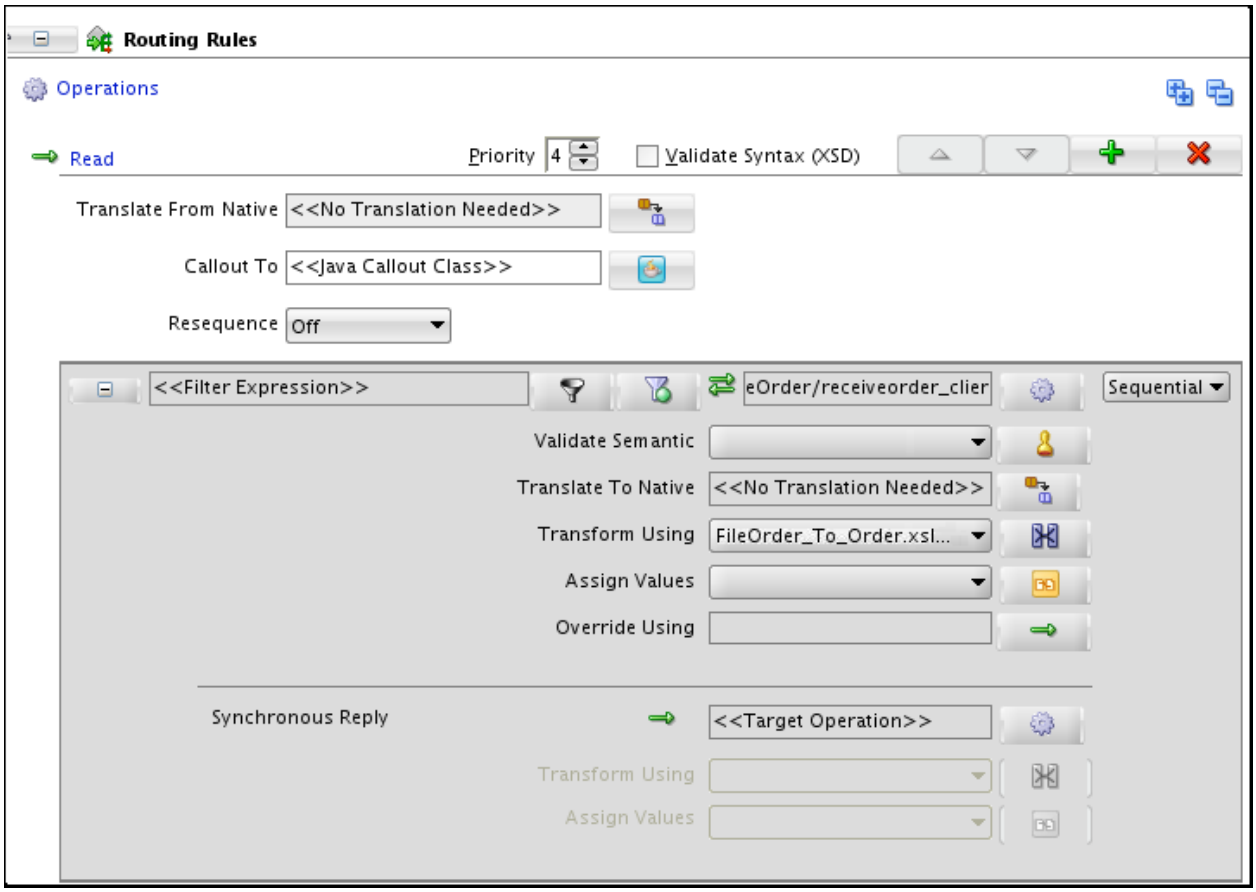


Note: The dictionary is usually created by business analysts that lists common synonyms in use across data objects (such as “qty” being used instead of “quantity,” and “CardNumber” instead of “CardNum”). The dictionary is not mandatory, and even without it, the auto-mapping feature identifies and enables mapping of these fields. However, customizing a dictionary to a specific company helps improve its accuracy.

11. Click OK to close the Auto Map Preferences window.
12. In the mapping editor, verify that the resultant mapping resembles the following screenshot:



13. Save and close the `FileOrder_to_Order.xml` file XSLT mapping editor.
14. The Mediator editor is updated with the XSLT mapping file just created.



Practice 7-3: Deploying and Testing

Overview

In this practice, you deploy the ProcessOrder composite application to the SOA server and test the service by feeding the sample purchase order files to it.

Tasks

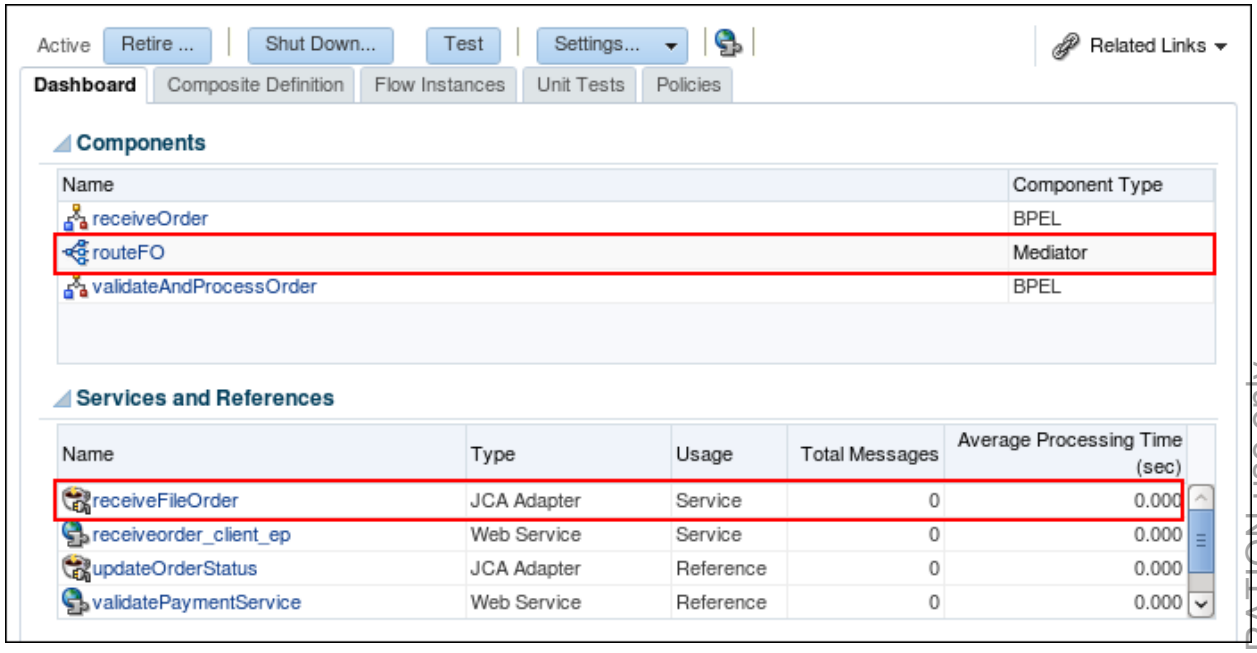
Deploy the ProcessOrder composite to application server.

1. Deploy the project.
   a. Right-click `ProcessOrder` in the project menu.
   b. Select **Deploy > ProcessOrder**.
      The Deploy ProcessOrder dialog box is displayed.
   c. Select the **Deploy to Application Server** option, and click **Next**.
      The SOA Deployment Configuration dialog box opens.
   d. Change the revision ID to **3.0**, and click **Next**.
   e. Select **IntegratedWebLogicServer**, and click **Next**.
   f. Click **Next**.
      The deployment summary is displayed.
   g. Click **Finish**.
2. Deployment processing starts. Monitor deployment progress and check for successful compilation in the SOA – Log window, and verify that deployment is successful in the Deployment – Log window.

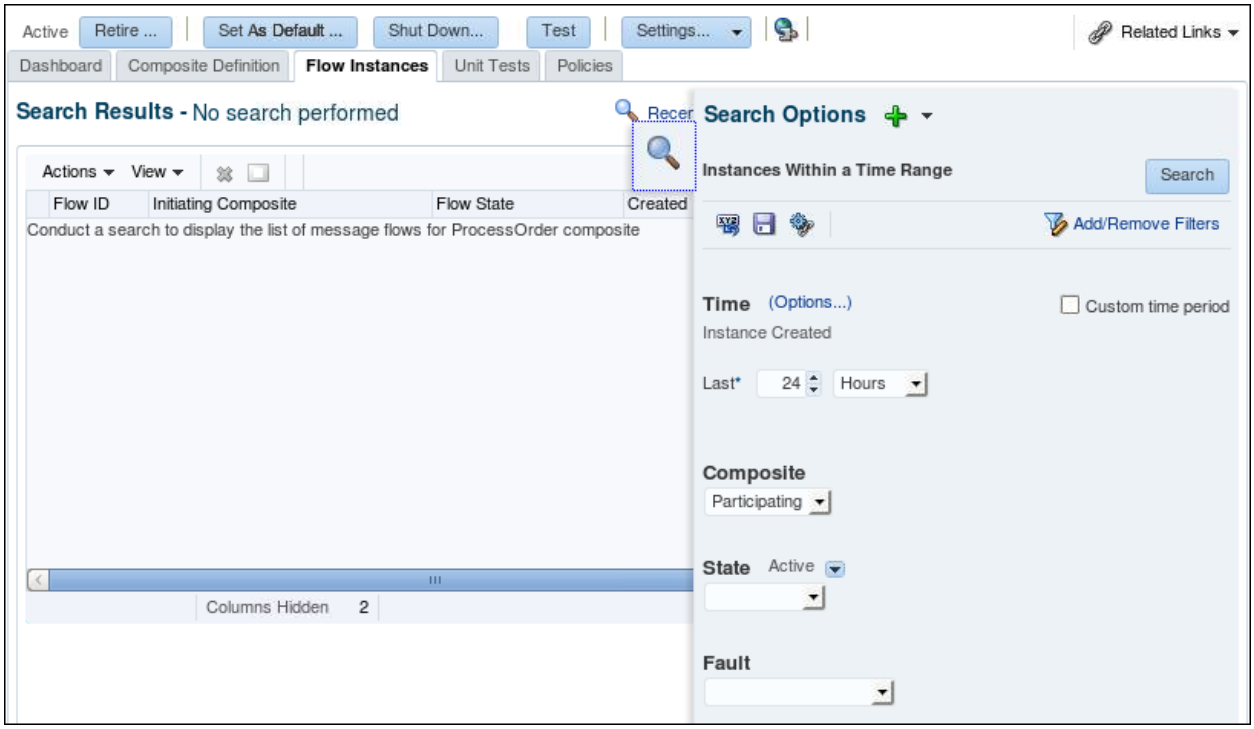
Test ProcessOrder Composite by using the sample purchase order files

In this section, you copy an XML file into the folder specified as the input folder for the purchase orders to be processed by the ProcessOrder composite application, monitor the execution of the process, and observe the results in the form of a file created by the application in a folder specified as its output folder.

3. Test the project.
   a. In the Enterprise Manager navigator, select the **`ProcessOrder [3.0]`** composite.
   b. In Dashboard, note the newly added service component and service reference.



4. Click the Flow Instances tab. In the Search Options pop-up panel, click Search.



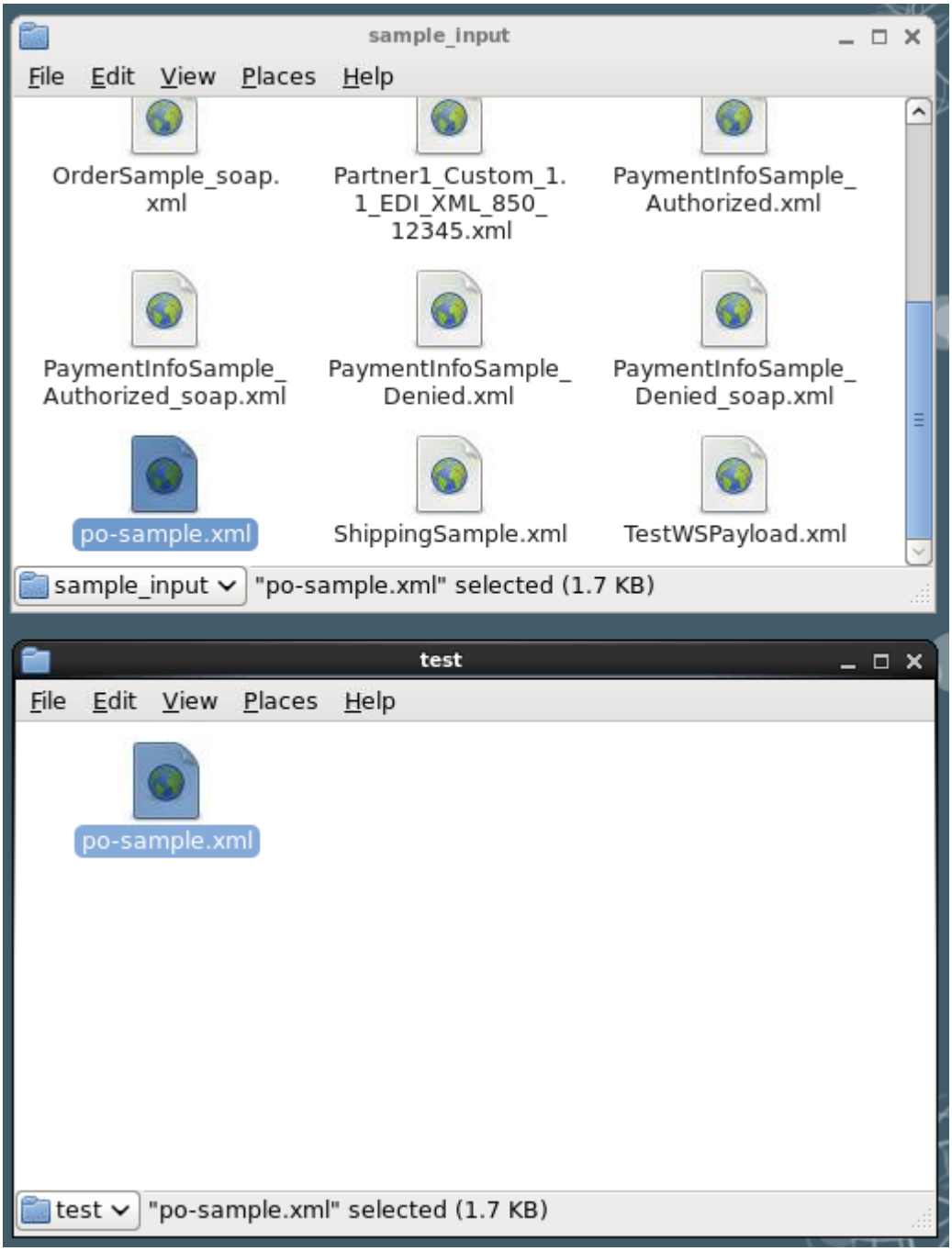
At this point, you should not see any instances.

5. To initiate the ProcessOrder [3.0] service:
   a. Open a terminal window.
   b. Copy the `/labs/resources/sample_input/po-sample.xml` file to the `/labs/test` folder:

      ```
      cp /labs/resources/sample_input/po-sample.xml /labs/test
      ```

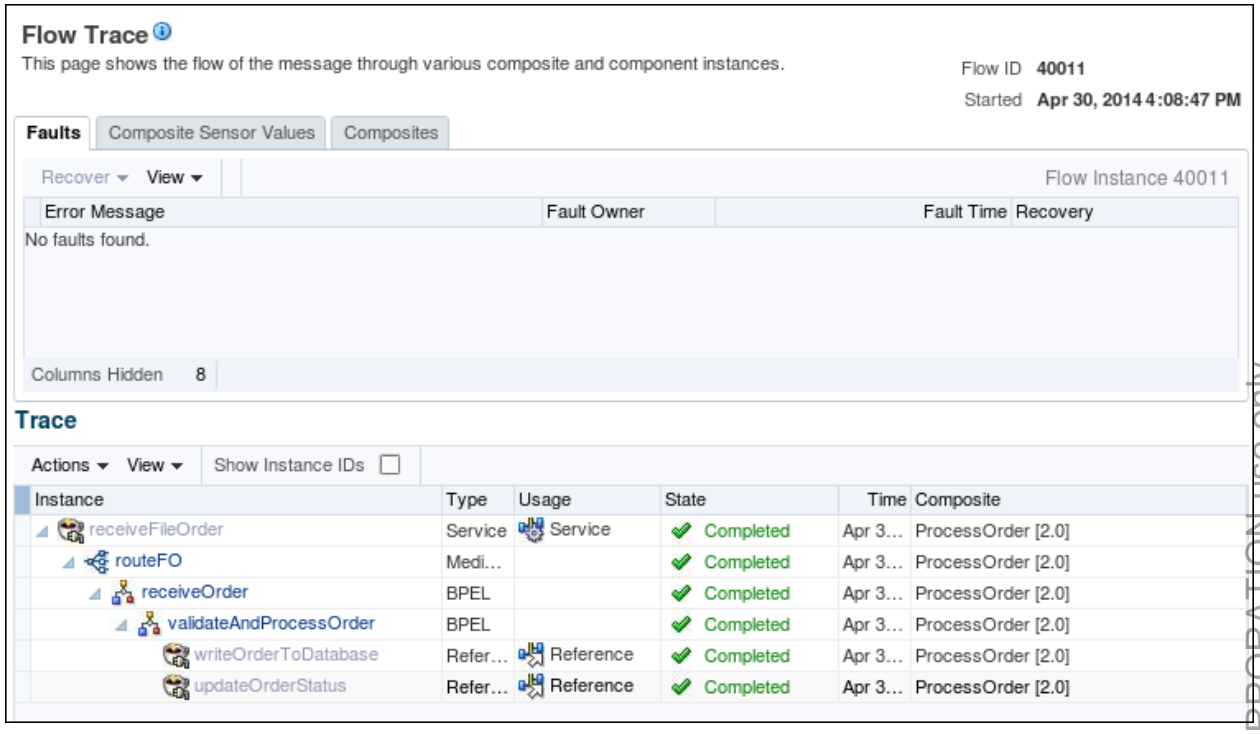
      Or

You can open these two folders on the desktop and then do a copy-paste of the `po-sample.xml` file as shown in the following screenshot:



Warning: You *must copy (*do not *move)* the file because, the input file adapter removes the file from the input folder after it has been read (consumed) by the file adapter service.

6. In the `test` folder, wait for the input file to be removed as an indication that the application has started to process the file, and then perform the following steps:
   a. In the EM console, on the ProcessOrder [3.0] home page, click the Refresh button in the top right corner of the page, a new instance appears with flow state Completed.
   b. Click the Flow ID link. The Flow Trace page should resemble the following screenshot:



c. You can click the component link to view the details of each component (routeFO, receiveOrder, and updateAndProcessOrder) in the audit trail page.

Note: You can find out how data is passed within the process by looking at the payload of each step.

7. You can also verify the result by checking the database to see whether the new order record has been created. Perform the following steps:
   a. From the JDeveloper main menu, select Window > Database >Databases. The Databases window is displayed in the upper-left corner.
   b. Navigate to po_composite > PO >SOAINFRA > Tables, and locate the E2E_1201_ORDERS table in the list.
   c. Right-click the table, and select **Open Object Viewer** from the context menu. You should see the new order record saved in the database.

	ORDER_NUMBER	SESSIONID	ORDER_DATE	TOTAL_AMOUNT	EMAIL	STATUS
1	201422716352626	(null)	2014-02-27	105.75999999999999	daniel@localhost	New
2	20143410535454	(null)	2014-03-04	105.75999999999999	daniel@localhost	New
3	20143611102020	(null)	2014-03-06	105.75999999999999	daniel@localhost	New
4	20143611164141	(null)	2014-03-06	105.75999999999999	daniel@localhost	New
5	20143613431414	(null)	2014-03-06	105.75999999999999	daniel@localhost	New
6	201443019454747	(null)	2014-04-30	105.75999999999999	jl@localhost	New
7	201443020474747	(null)	2014-04-30	105.75999999999999	jl@localhost	New
8	20144302184747	(null)	2014-04-30	105.75999999999999	jl@localhost	New

8. When you are done, close the po_composite application in JDeveloper (including all the open windows).

Practices for Lesson 8: Encapsulating Business Logic with Business Rules Components

Chapter 8

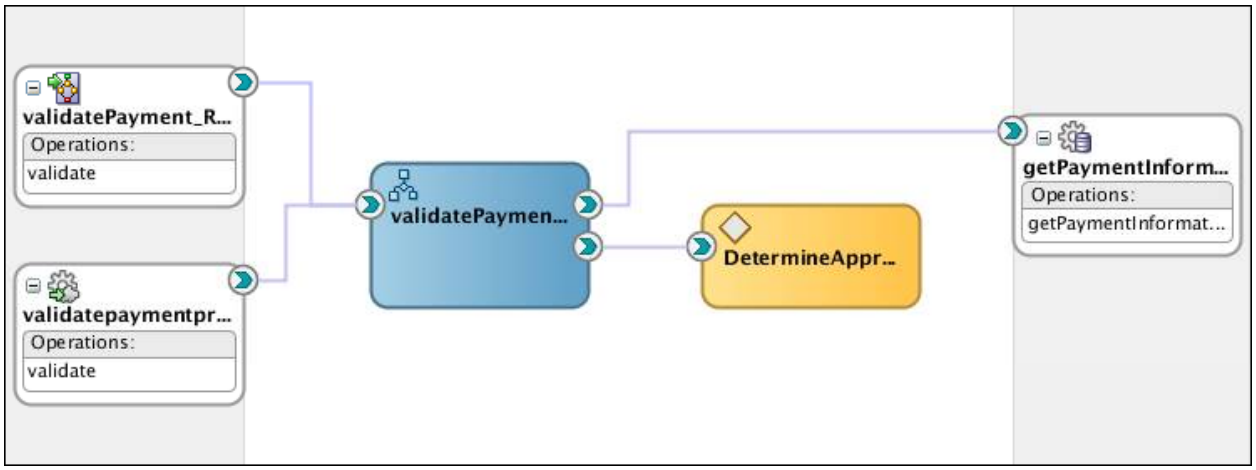
Practices for Lesson 8

Practices Overview

During holiday seasons, AviTrec wants to manually override the daily-limit rule if a customer's total order amount is marginally over the daily limit.

In these practices, you will learn how to implement manual overrides by using a Business Rule service component (with predefined Business Rules) in the ValidatePayment composite application. If an order's total amount is over the daily limit but within a 10% range, the rule triggers a manual approval process. Later, if AviTrec changes its policy (for example, it increases the range to 15%), you can use Oracle SOA Composer to make the adjustment without modifying the composite application.

When completed, the project will look like this:



Practice 8-1: Adding the Business Rules

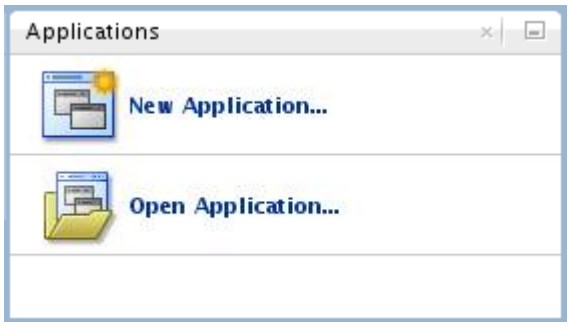
Overview

In this practice you add a Business Rule component that determines whether the order is over the daily limit but within a 10% range. If so, it updates the order status to Manual Approval Required.

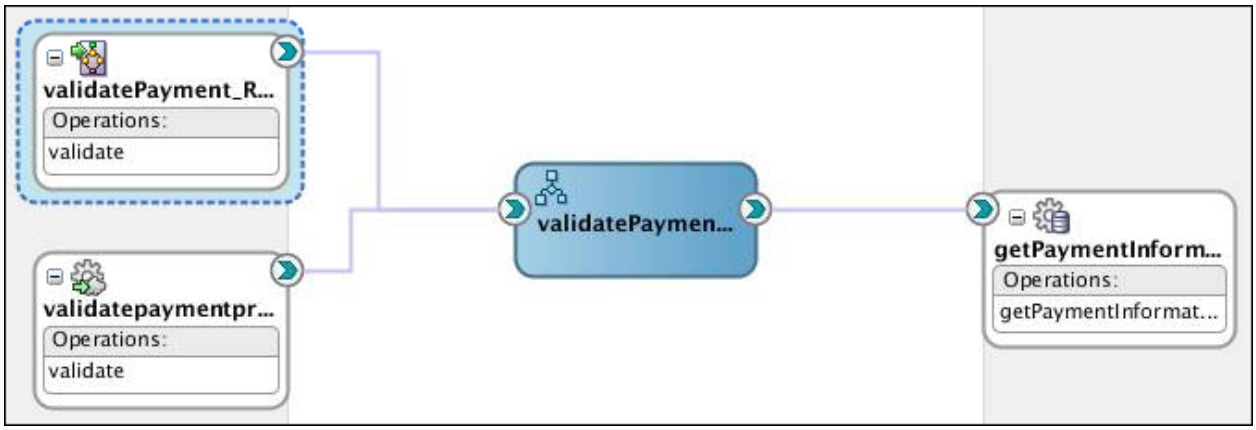
Tasks

Open the ValidatePayment project

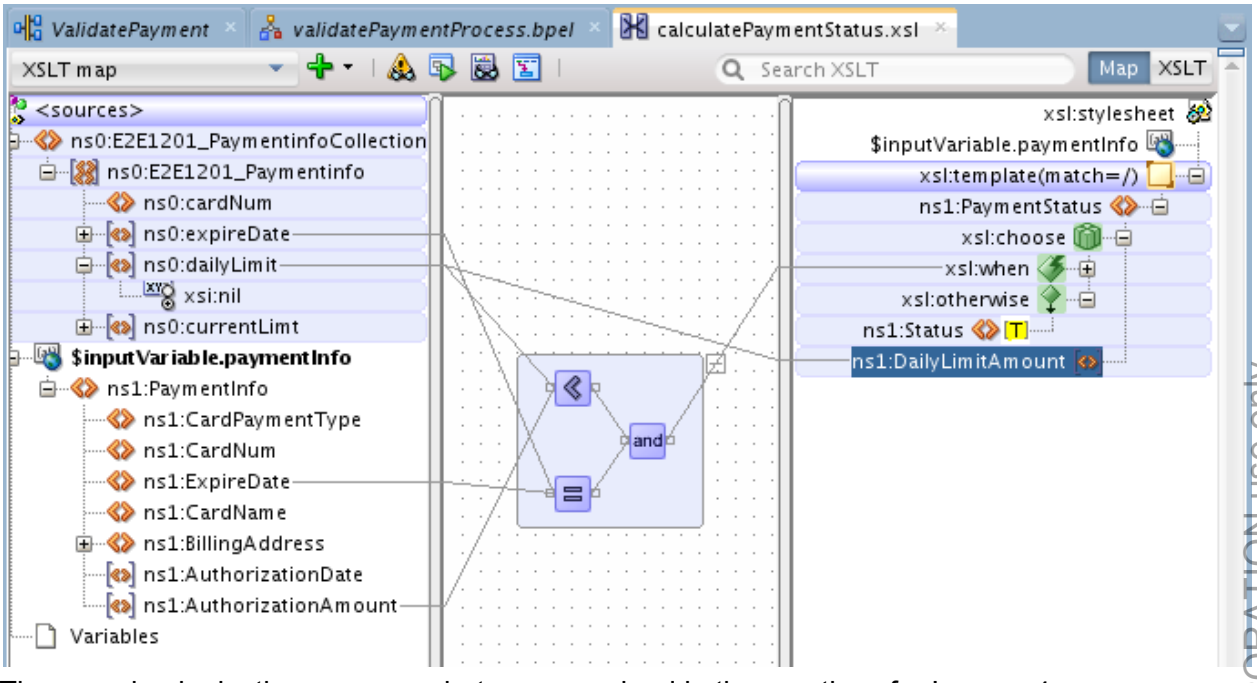
1. Open JDeveloper if it is not open. (You can double-click the JDeveloper shortcut on the desktop).
2. In the Applications pane, click **Open Application** (or select **File** > **Open**).



3. In the Open Application(s) dialog box, navigate to the `/labs/lessons/Lesson08/po_composite/` directory, and open the `po_composite.jws` file. The Validatepayment project folder is displayed in the Applications pane.
4. In the SOA folder, double-click ValidatePayment (composite.xml), the application components are displayed in the Design view of the Composite Editor:



5. Inspect a change made in the `calculatePaymentStatus.xsl` transformation stylesheet.
   a. Open `validatePaymentProcess` in the BPEL designer.
   b. Right-click the `caculatePaymentStatus` transformation activity, and select Edit.
   c. Click the Edit Mapping icon next to the Mapper File field. The XSLT Map Editor is displayed.



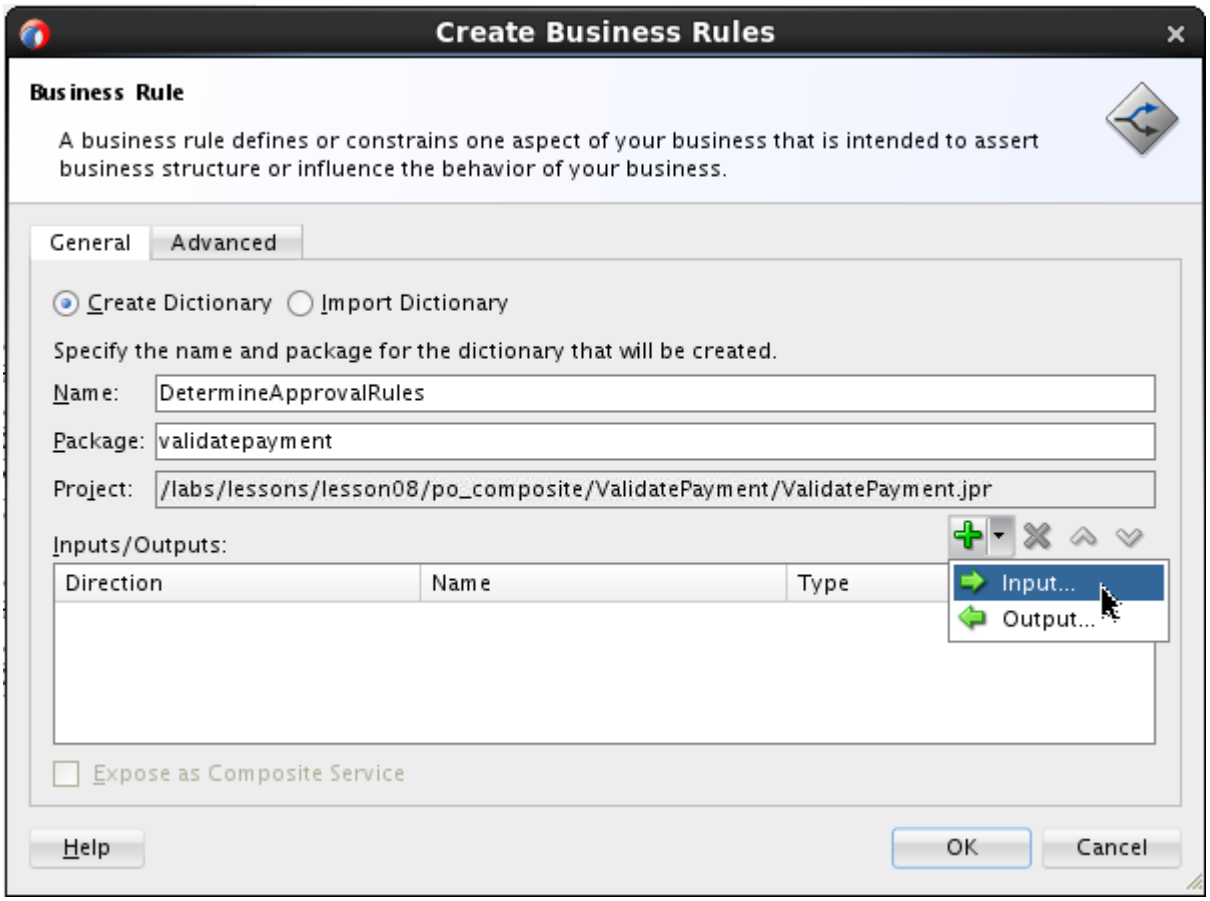
The mapping looks the same as what you examined in the practices for Lesson 4 except for one change: it adds a new child element `DailyLimitAmount` in PaymentStatus, and maps the `dailyLimit` (returned from database) to it.

d. Close the XSLT Map Editor.

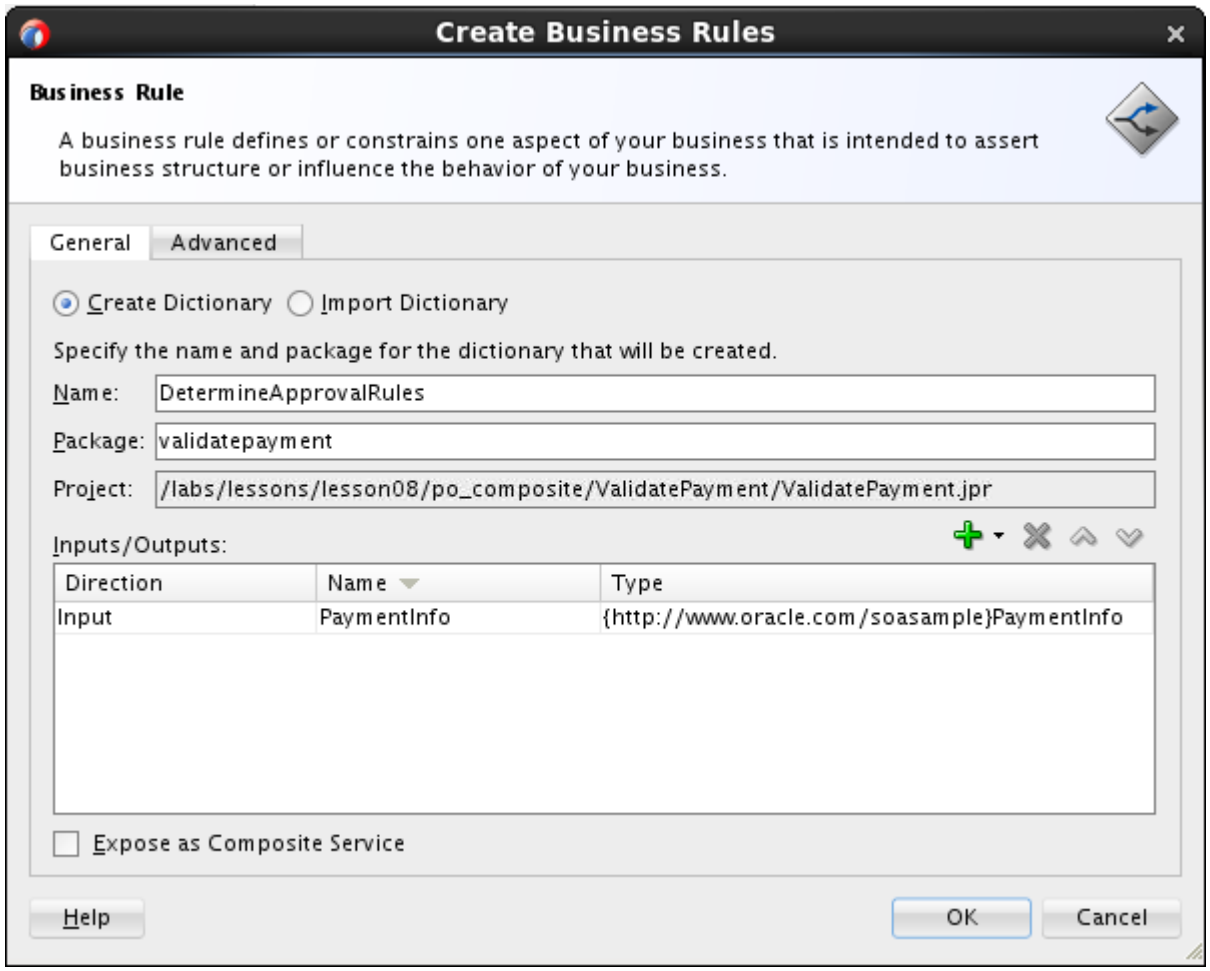
Tip: You can also open the XSLT stylesheet directly from the project's `SOA/Transformations` folder.

Add a business rule to the project

6. In the composite editor, drag a Business Rule component to the Components swim lane.
   The Create Business Rules dialog box is displayed.
7. Name it `DetermineApprovalRules`.
8. Specify input and output types for the business rule.
   The rule takes the total order amount (AuthorizationAmount) and dailyLimit (retrieved from the database) as input, and returns the status.
   a. Click the green + icon in Inputs/Outputs and select Input.



b. In Type Chooser window, expand `CanonicalOrder.xsd` under Project Schema Files and select `PaymentInfo` (AuthorizationAmount is a child element of PaymentInfo). Click OK.

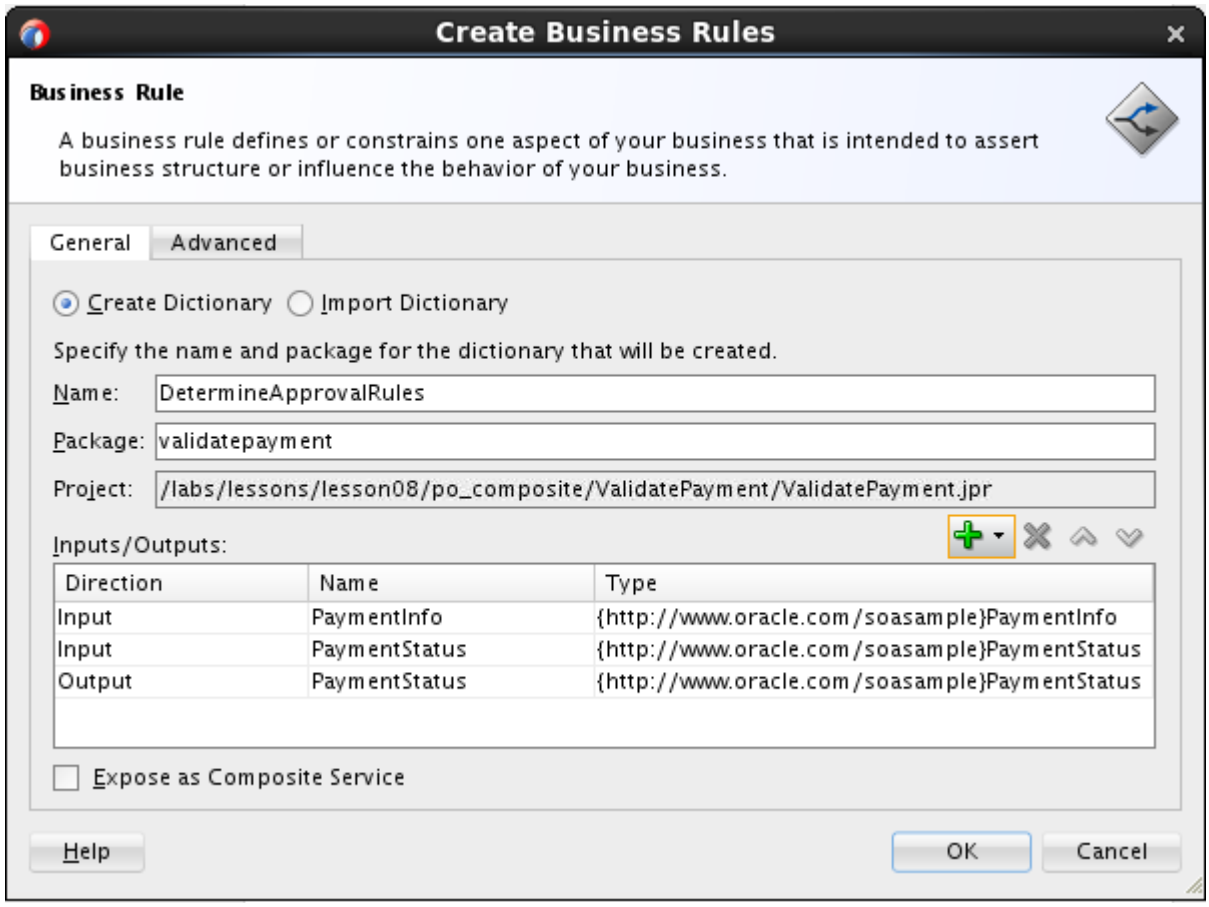


c. Similarly, add daily limit (returned from database) as another input to the rule. Click Add and select input. In the Type Chooser window, this time select `PaymentStatus` in `CanonicalOrder.xsd`. Click OK.

d. Click the green + icon in Inputs/Outputs and select Output.

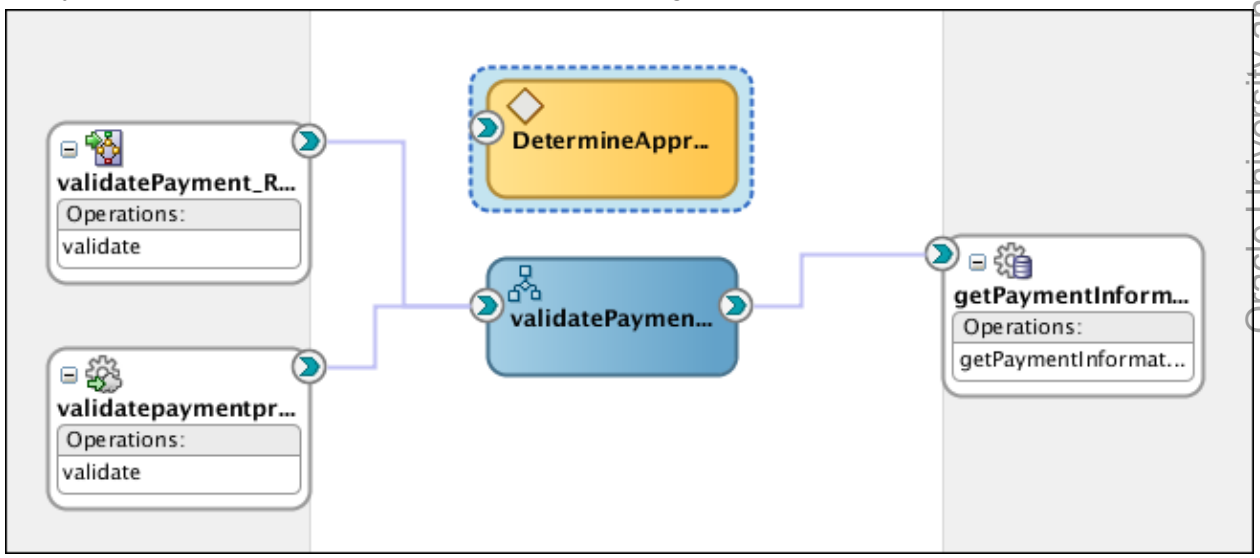
e. Again, expand `CanonicalOrder.xsd` under Project Schema Files and select PaymentStatus. Click OK.

The Create Business Rules dialog box should now look like the following screenshot:



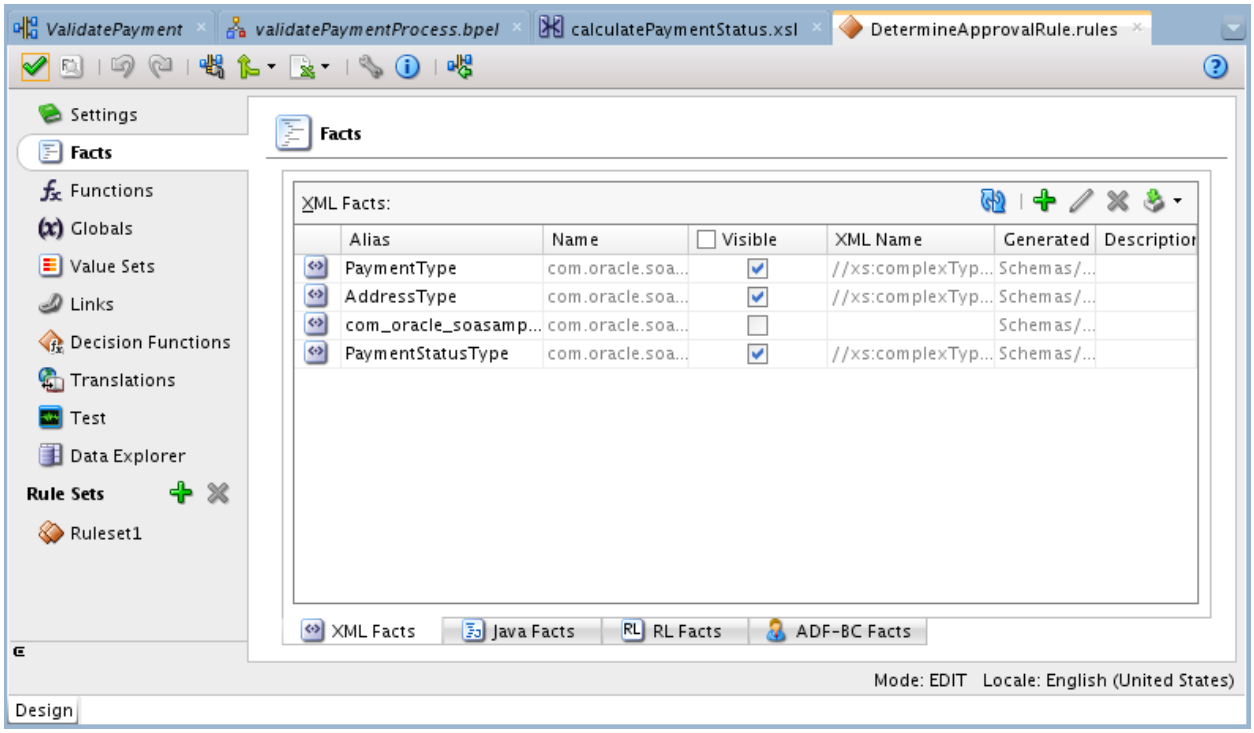
f. Click OK.

g. Verify that the composite now looks like the following screenshot:



Configure the Business Rule component

9. Double-click the Business Rule component in the composite.
   The rules editor opens.
10. Click **Facts**. On the XML Facts tab, you should see the types based on the elements you selected for the rule's input and output.

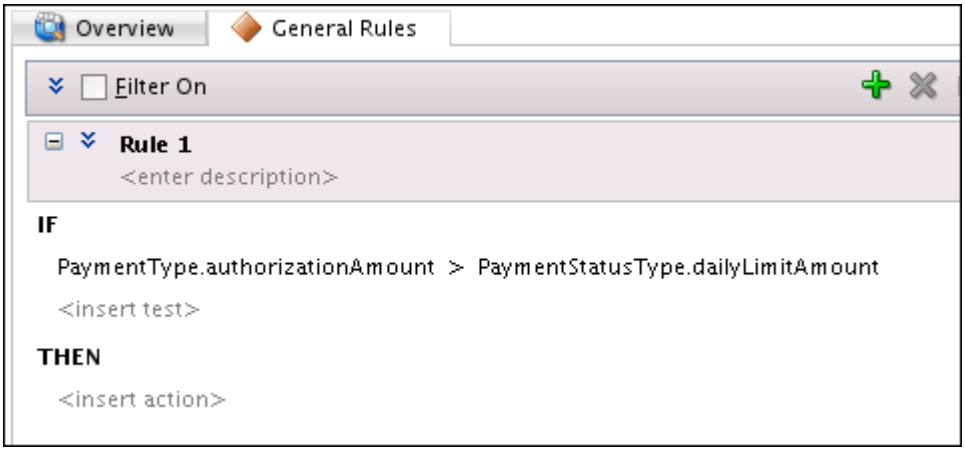


Create a ruleset

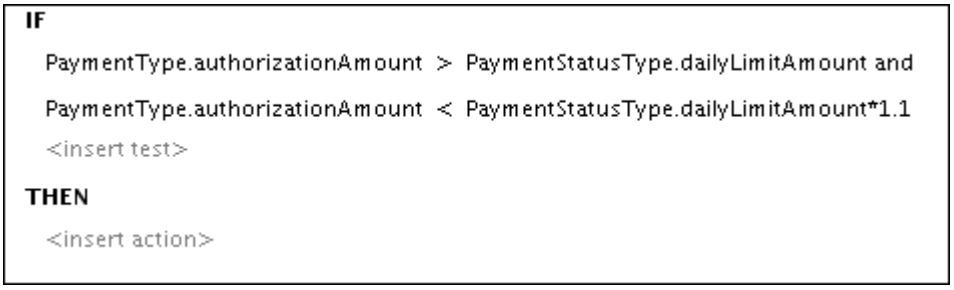
11. Name the ruleset.
    a. Click **Ruleset1**.
    b. Change the name to OverrideRuleset, and click Create (the green + icon) in the General Rules panel.

    A new view opens where you can specify the conditions and actions (THEN) to take based on those conditions.

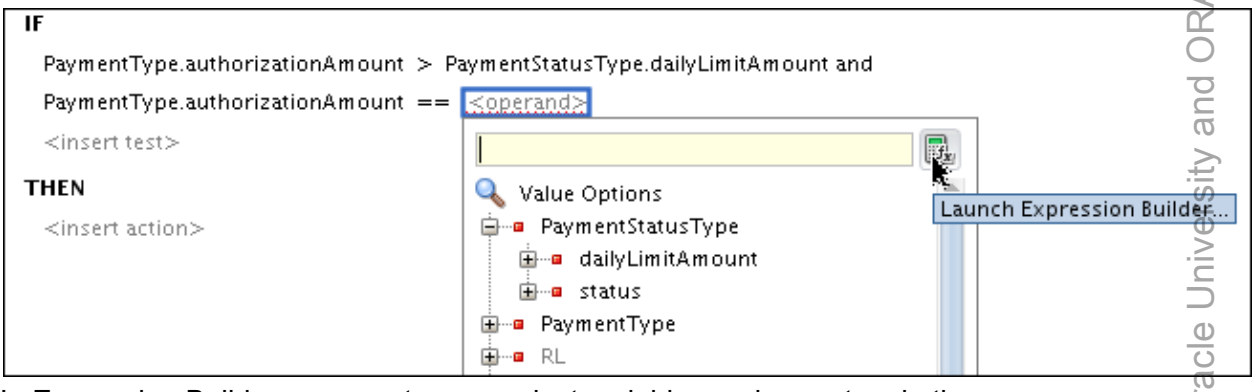
    When the condition evaluates (by using test) to true—in this case, total order amount is greater than daily limit but less than daily limit x 1.1—a match has been made, and then its action (THEN) is triggered or executed. In this case, the action is setting the payment status (fact of the output type) to Manual Approval Required.
12. Create Rule 1 for manual approval.
    a. Define the conditions:
        1) In the IF statement of Rule1, click <insert test> and select **simpletest** from the drop-down list.
        2) You can click **<operand>** and the operator (==), and then select the values from the drop-down list. The first test should look like the following screenshot after your selection:



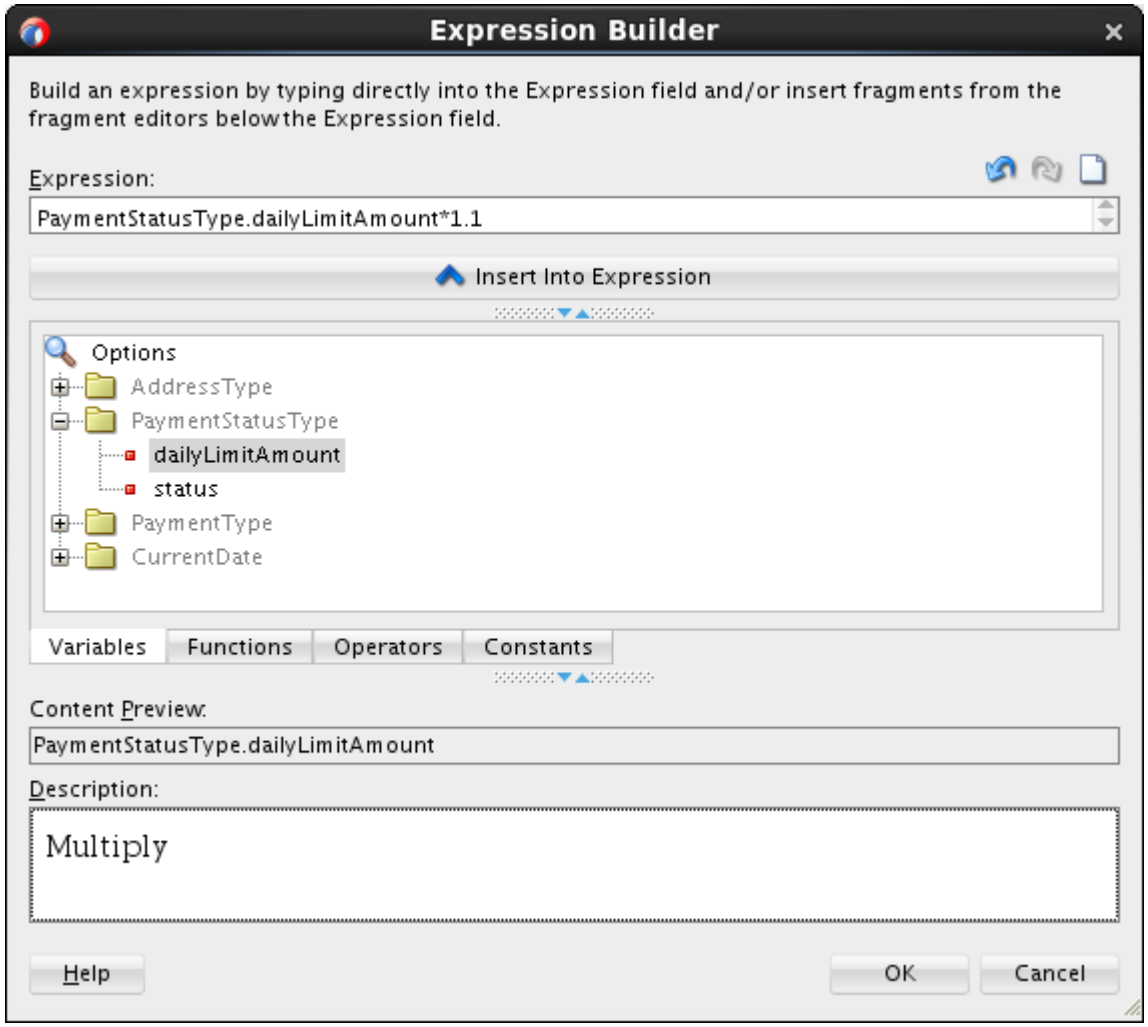
3) Click <insert test> (under the first test) and select **simpletest** from the drop-down list to add another test.



To get `PaymentStatusType.dailyLimitAmount*1.1`, you need to use Expression Builder.

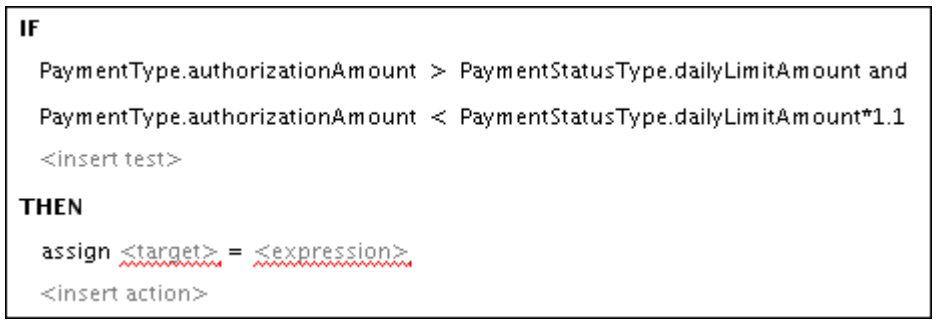


In Expression Builder, you can type or select variables and operators in the Expression field.



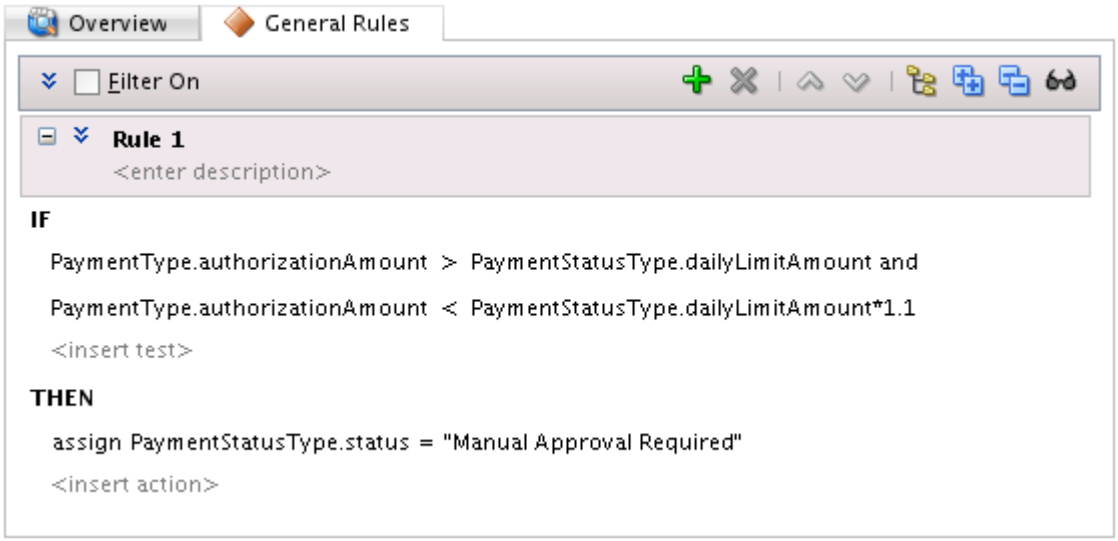
b. Define the action to take for each condition.

1) In the THEN statement of Rule1, click <insert action> and select **assign** from the drop-down list.

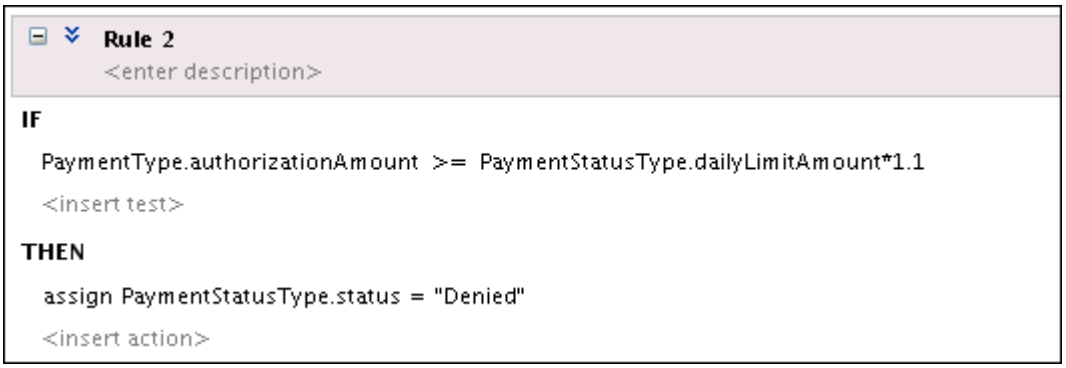


2) For <target>, select PaymentStatusType > status, and for <expression>, just type `"Manual Approval Required"` in the field.

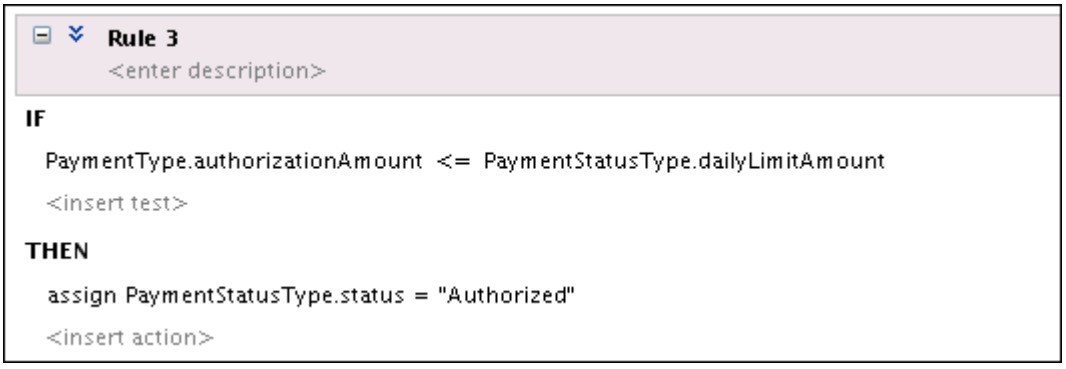
Your final rule should like the following screenshot:



13. Similarly, create two rules for authorized and denied conditions.
    To add a new rule, click Create (green + icon) at the top of the General Rules tab.
    a. For Rule 2, add conditions and actions as shown in the following screenshot:

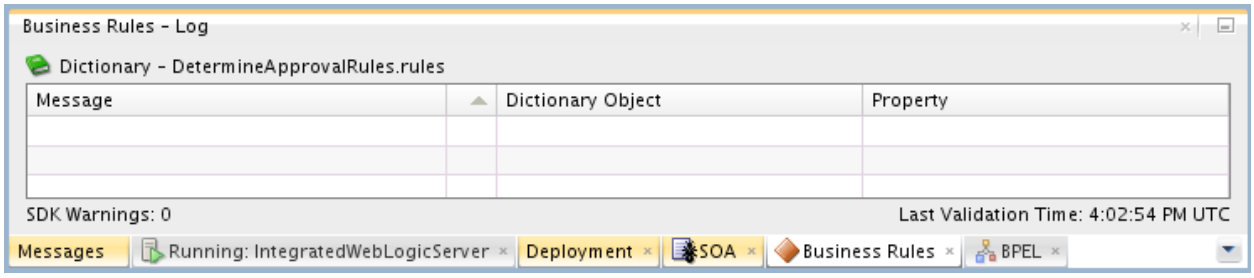


    b. For Rule 3, add conditions and actions as shown in the following screenshot:



14. To validate the rules, click the Validate Dictionary icon in the toolbar at the top of the rule editor.

    Make sure that you do not see any errors in the Business Rules – Log.



15. Save your rule file, DetermineApprovalRule.rules.
    At this point, you have created a rule dictionary.
16. Close the rule editor window.

Practice 8-2: Integrating the Business Rule Component in the BPEL Process

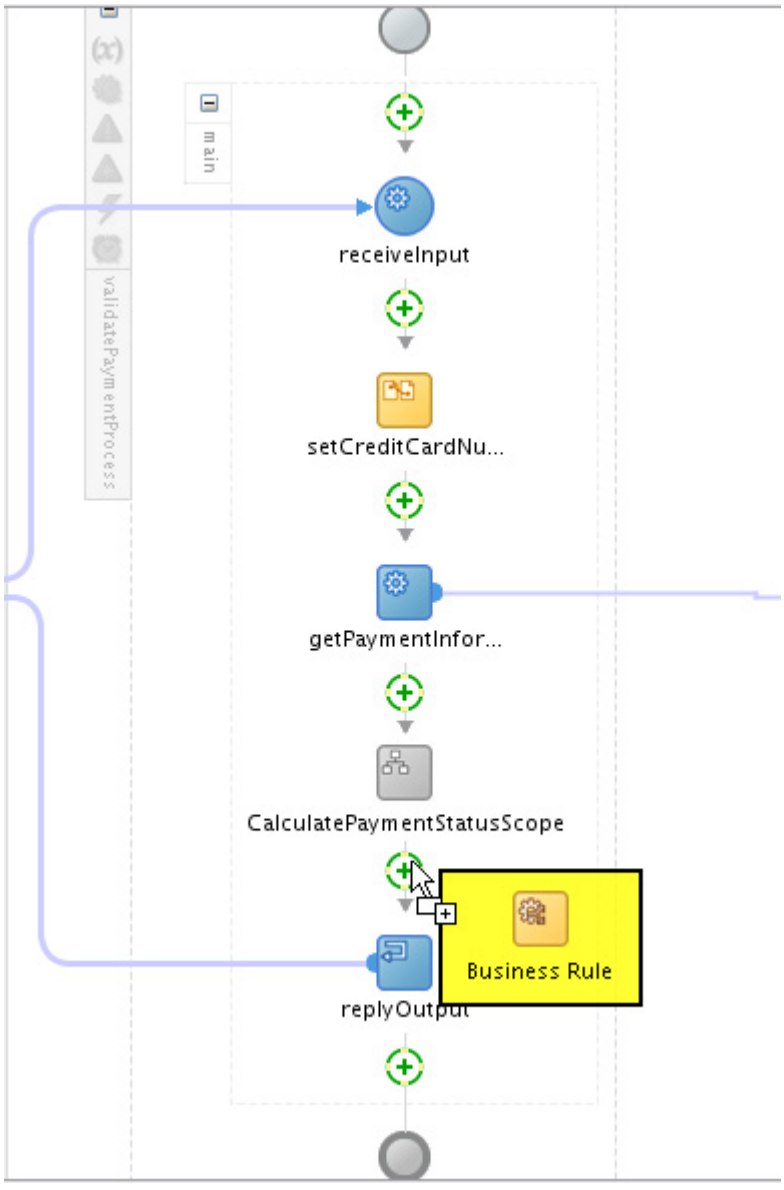
Overview

In this practice, you integrate the DetermineApprovalRule decision service in the ValidatePaymentProcess BPEL process by adding and configuring a Business Rule activity in the BPEL process.

Tasks

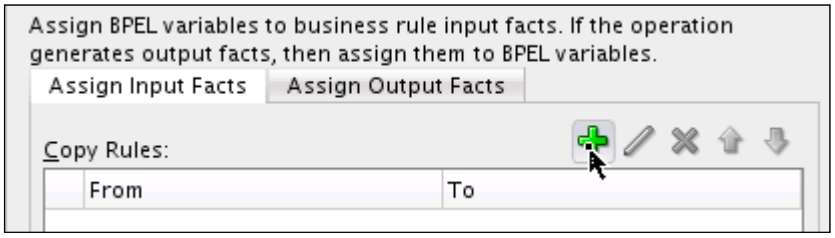
Adding a business rule to the BPEL process

1. In the design view of the Composite editor, double-click the ValidatePaymentProcess BPEL component to open it in the BPEL editor.
2. Drag a Business Rule activity (in SOA Components) from the Component Palette and drop it between CaculatePaymentStatusScope and replyOutput.

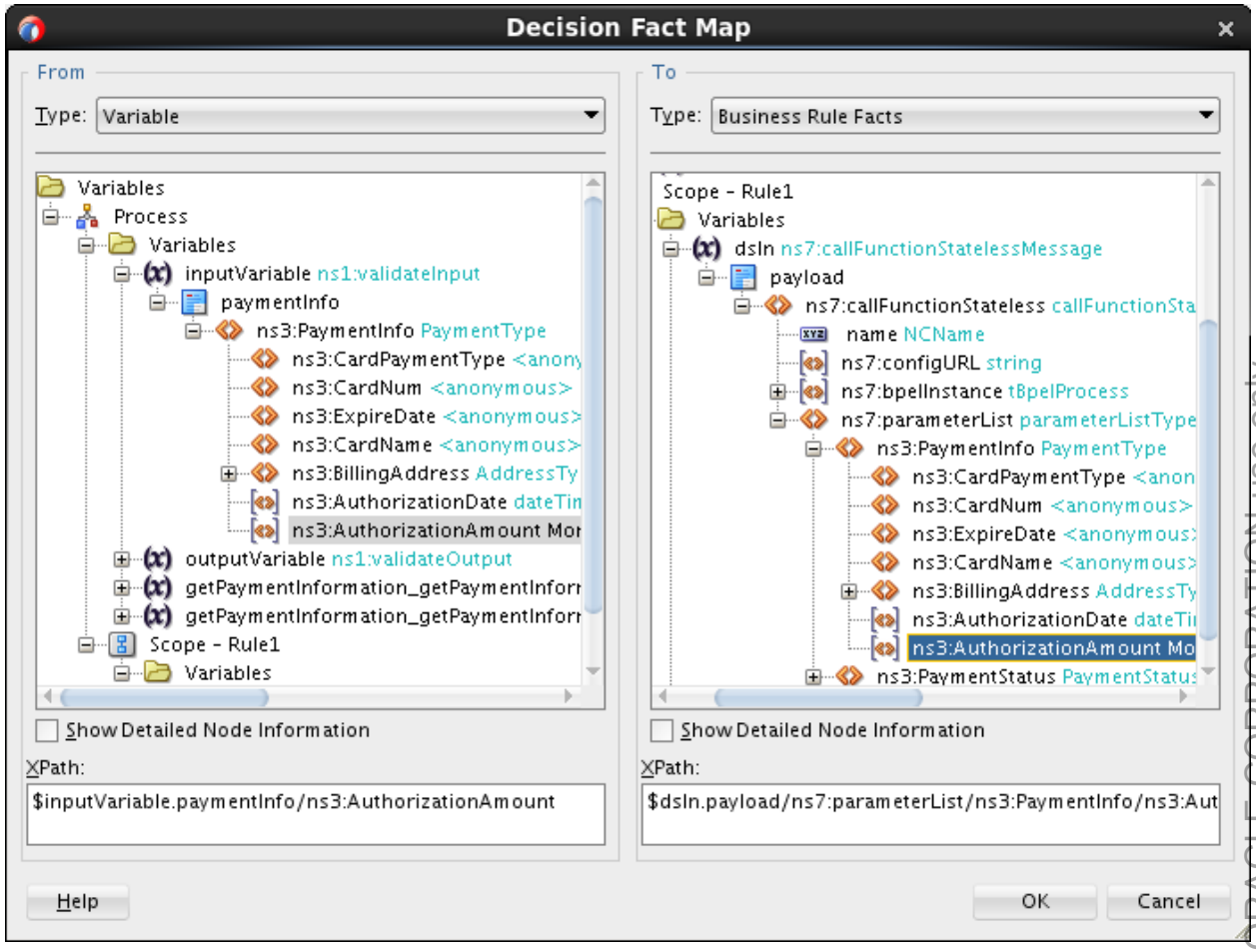


3. Double-click the rule activity (Rule1).
   The Edit Rule window is displayed.
4. Click the General tab, and change the name to **DetermineApprovalRule**.
5. Click the Dictionary tab, and select the dictionary or business rule that this activity invokes. In this case it is the `DetermineApprovalRules` dictionary.
6. Next define the copy rules for the input and output facts:
   - For the input:
     - Copy the `AuthorizationAmount` of `PaymenInfo` from the process `inputVariable` to the `dsIn` variable of the business rule.
     - Copy the `dailyLimitAmount` of `PaymentStatus` from the process `outputVariable` to the `dsIn` variable of the business rule.
   - For the output, copy `PaymentStatus` from the `dsOut` variable of the business rule to the process `outputVariable` variable.

   To do so, perform the following steps:
   a. Click the + icon for Assign Input Facts.



   The Decision Fact Map window is displayed. On the left, you have the source or From. On the right, you have the destination or To.
   b. On the left, under Process > Variables, expand inputVariable > paymentInfo > paymentInfo, and select AuthorizationAmount.
   c. On the right, under Scope – Rule1 > Variables, expand dsIn > payload > callFunctionStateless - parameterList > paymentInfo, and select AuthorizationAmount.

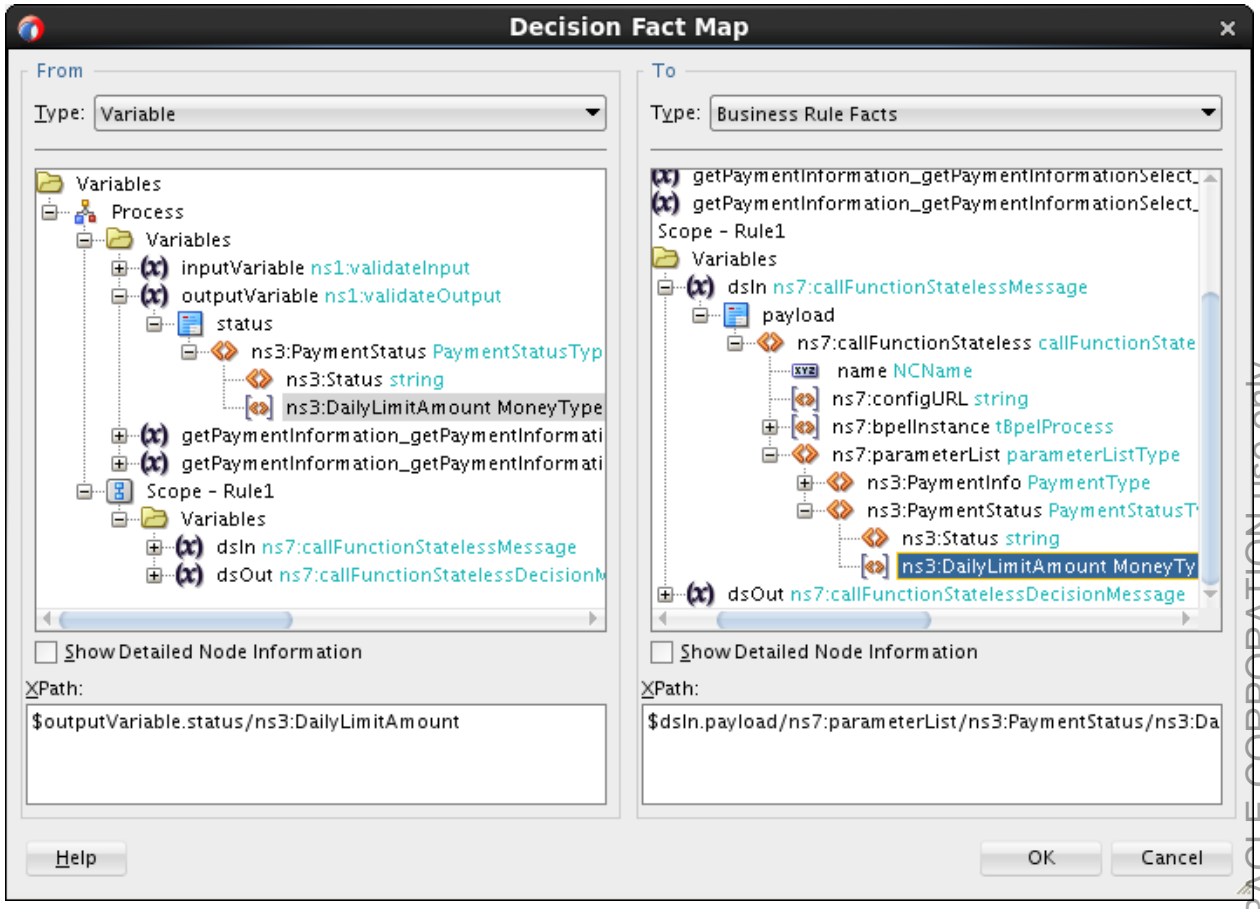


Click OK.

Verify that you see that the first input fact is assigned in the Edit Rule window.

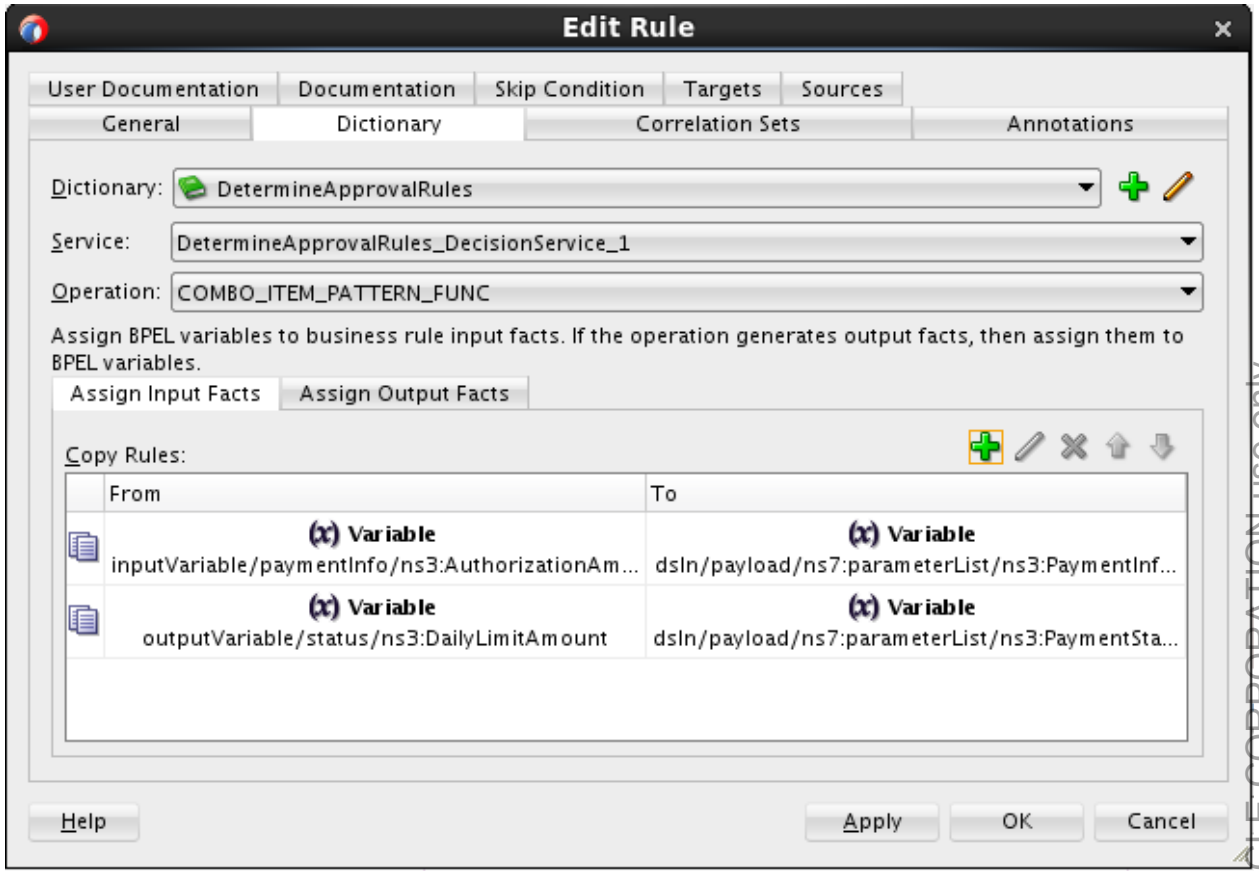
d. Repeat step 6a to 6c to assign BPEL variable to the second input fact, but this time select the following:

- DailyLimitAmount under Process > Variables > outputVariable > status > paymentStatus in the source column
- DailyLimitAmount under Scope – Rule1 > Variables > dsIn > payload > callFunctionStateless - parameterList > paymentStatus in the destination column



Click OK.

The Edit Rule window is updated with the newly added Copy Rule as shown in the following screenshot:

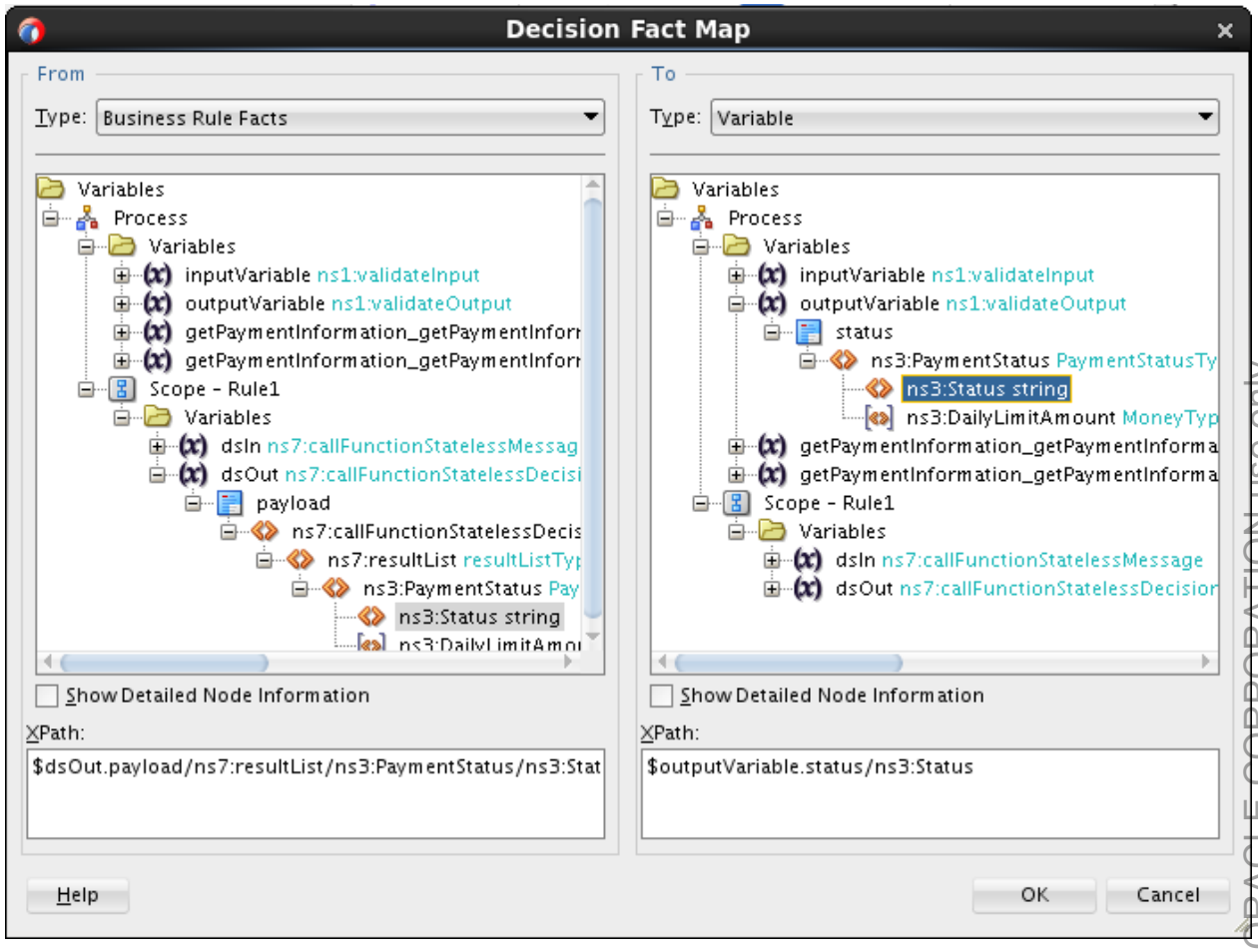


Similarly, assign business rule output facts to BPEL variables.

e. Click the Assign Output Facts tab, and click the green + icon.

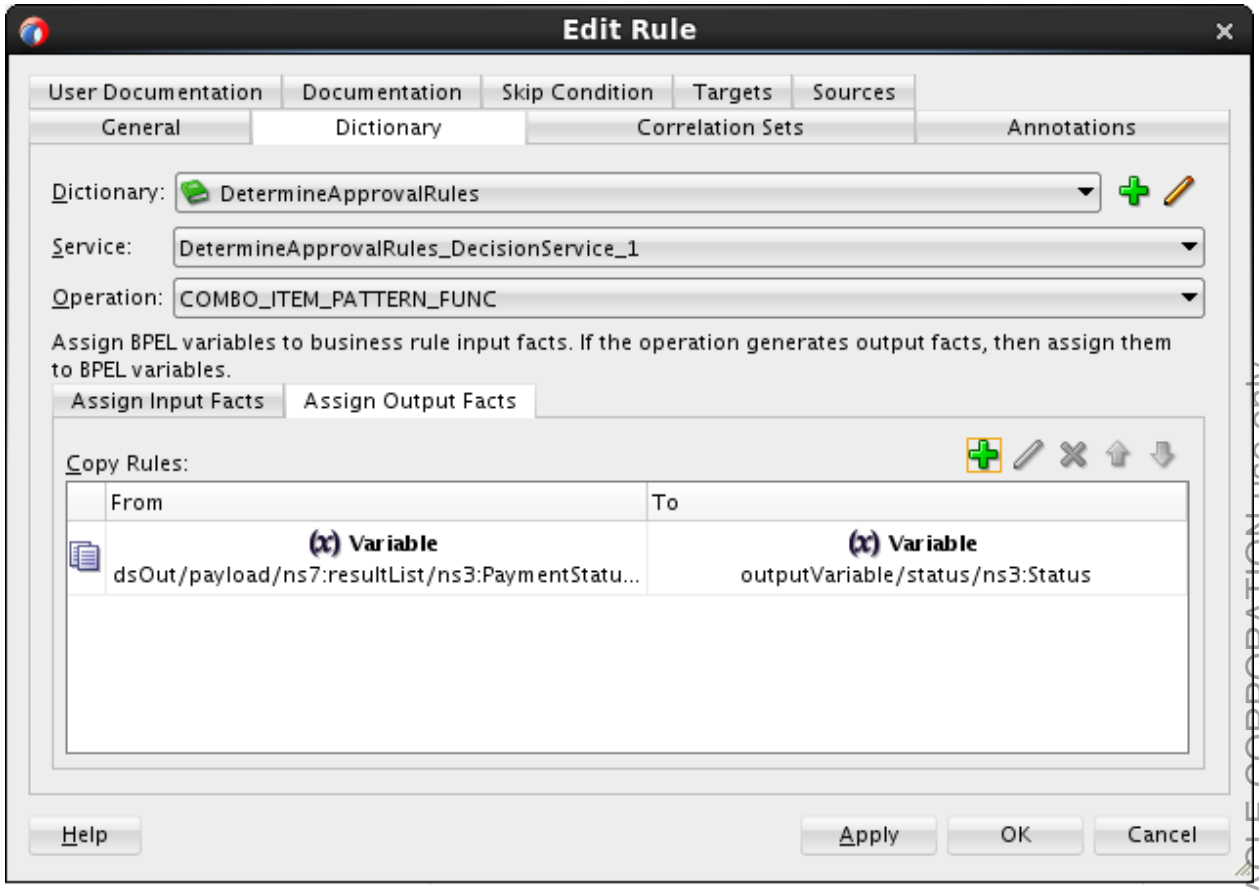
   The Decision Fact Map window is displayed. This time, the source or From is the Business Rule Facts and the destination or To is the (BPEL) Variables.

f. Make the source (under `Scope`) and destination (under outputVariable) selections as shown in the following screenshot:



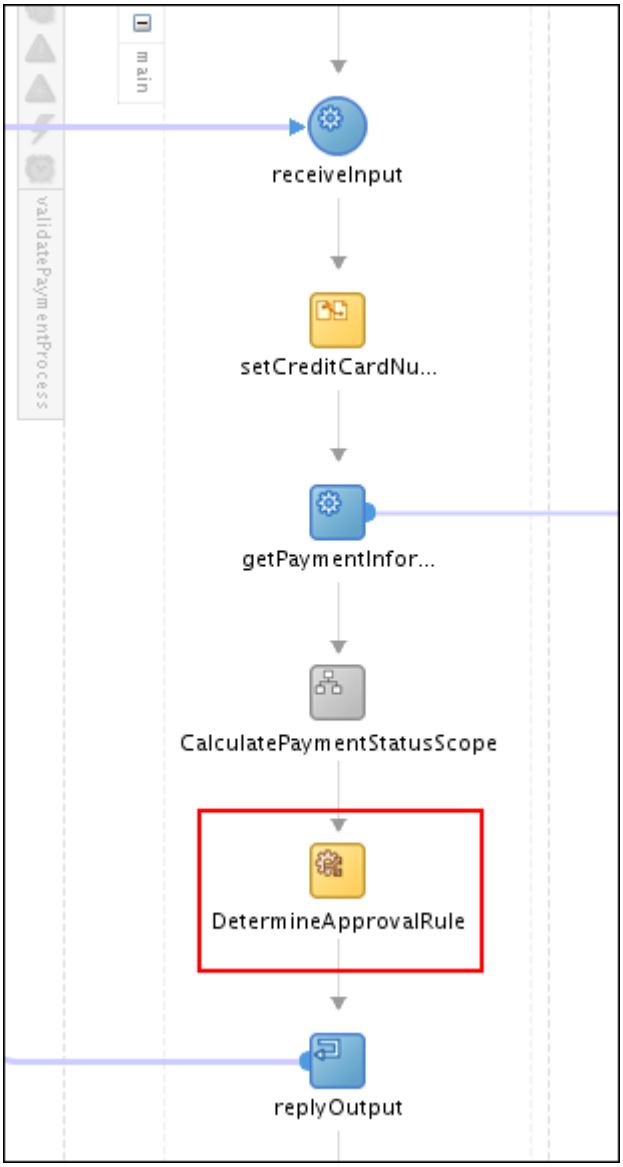
g. Click OK.

The Edit Rule window is updated with the new copy rule as shown in the following screenshot:

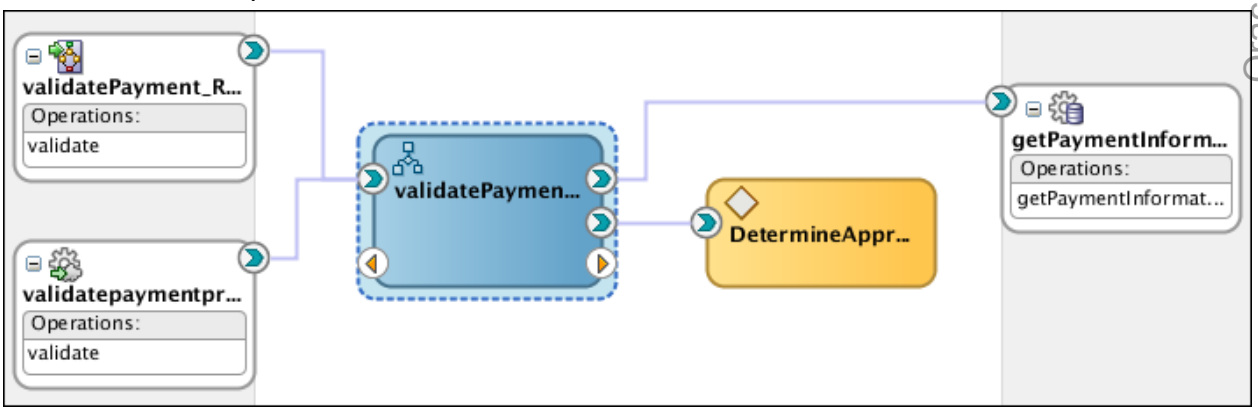


h. Click OK.

i. In the BPEL editor, the BPEL workflow is updated with the new rule activity.



j. In the composite editor, you should see that the BPEL component is now wired with the Business Rule component.

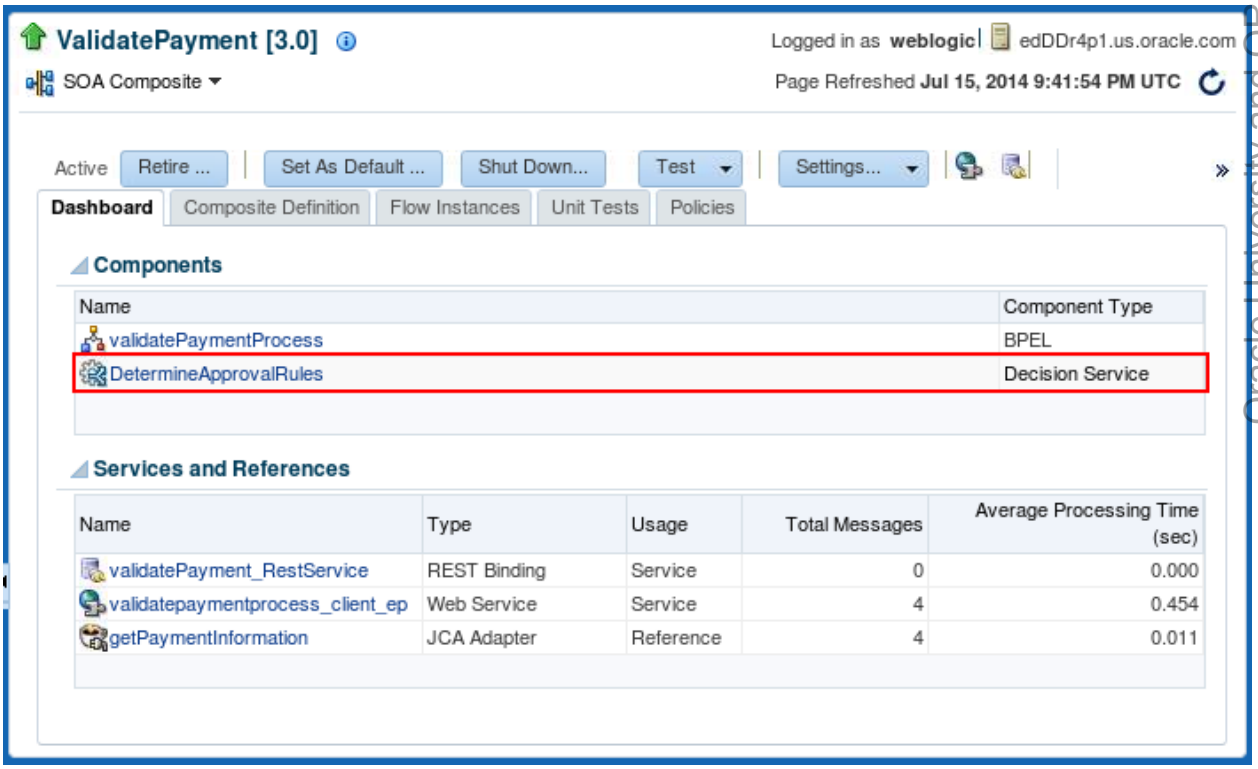


k. Save your work.

Deploying the composite application

7. Deploy the project.
   a. Right-click `ValidatePayment` in the project menu.
   b. Select **Deploy > ValidatePayment**.
      The Deploy ValidatePayment page is displayed.
   c. Select the **Deploy to Application Server** option, and click **Next**.
      The SOA Deployment Configuration page is displayed.
   d. Change the revision ID to **3.0**, and deselect "Mark composite revision as default." Click **Next**.
      The Select Server page is displayed.
   e. Choose **IntegratedWebLogicServer**, and click **Next**.
      The SOA Servers page is displayed.
   f. Click **Next**.
      The deployment summary is displayed.
   g. Click **Finish**.

   Deployment processing starts. Monitor the deployment progress and check for successful compilation in the SOA – Log window, and verify that deployment is successful in the Deployment – Log window.
8. Test the project
   a. In the Enterprise Manager navigator, select the **`ValidatePayment [3.0]`** composite.
   b. In Dashboard, note the newly added decision service.



   c. Click **Test**, and select **validatepaymentprocess_client_ep** from the drop-down menu.
      The Test Web Service page is displayed.

d. Scroll down to the **Input Arguments** section.

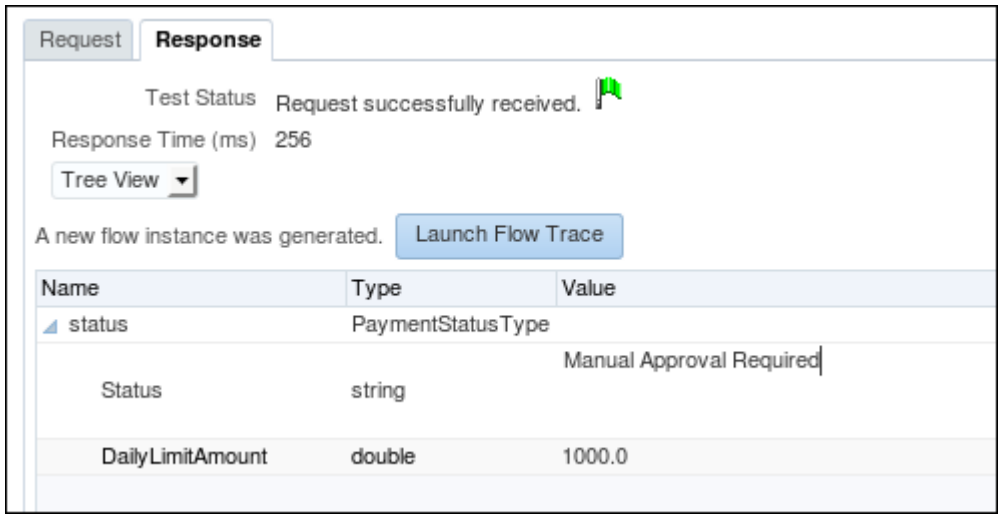
e. Click **Browse**. In the File Upload window, select the `/labs/resources/sample_input/PaymentInfoSample_Authorized_soap.xml` file, and open it.

f. Click **Test Web Service**.

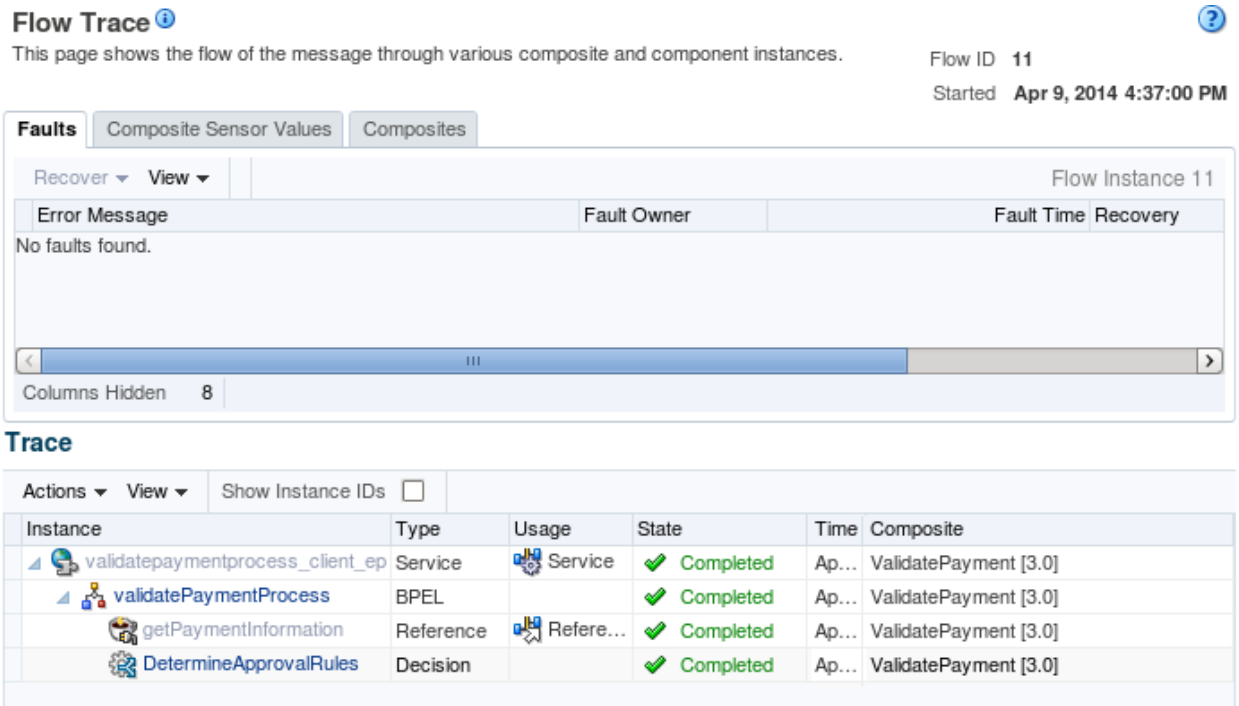
g. The Response shows the status is Authorized along with the daily limit amount.

h. Go back to the Request tab, change the AuthorizationAmount to a number between 1000 and 1100 (for example, 1050), and then Click **Test Web Service**.

This time the response shows the status is Manual Approval Required.



9. You can launch the flow trace to view the details in the audit trail.



10. When you are done, close the po_composite application in JDeveloper (including all the open windows).

Practices for Lesson 9: Implementing Human Activities with Human Workflow Service Components

Chapter 9

Practices for Lesson 9

Practices Overview

In these practices, you will create and define a human task service to make decision on the orders that are marginally over the daily limit.

Practice 9-1: Inspecting the Human Task Service Component

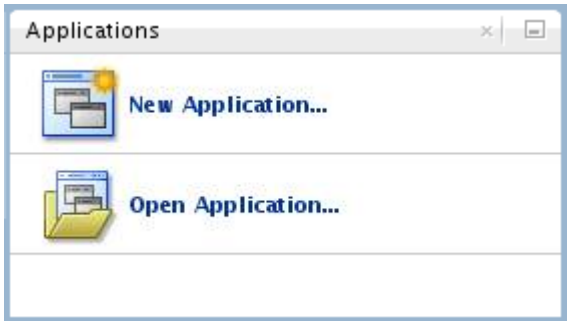
Overview

In this practice you inspect a preconfigured human task service component that allows the assignee to approve or reject a marginally over limit order. This simple human task is defined with just a single user, an expiration date, and a default (out-of-box) task form. It has no routing, escalation, or delegation; no notifications; and no fancy task form.

Tasks

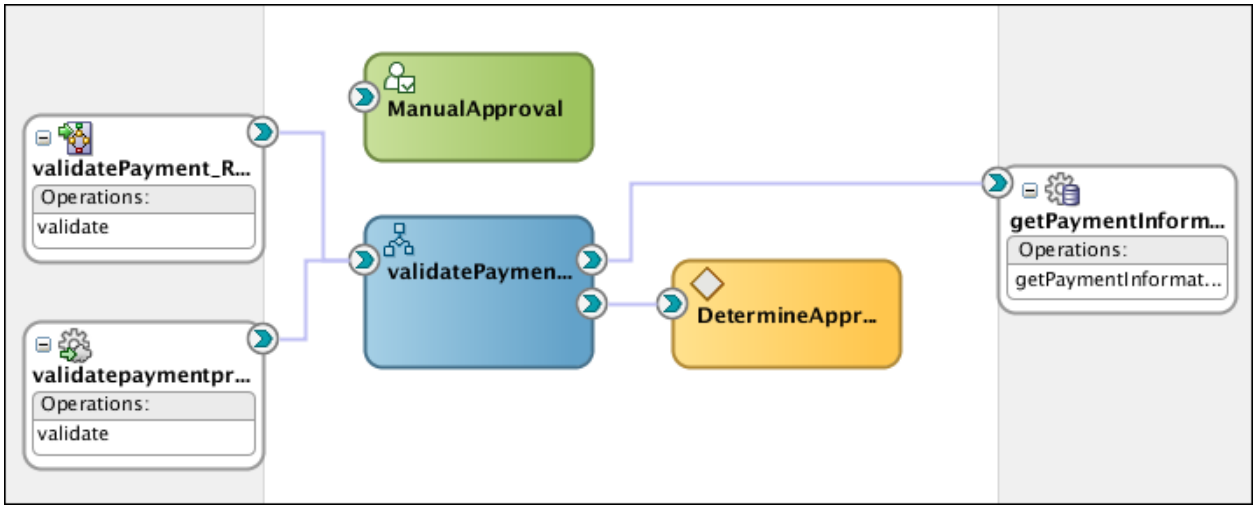
Open ValidatePayment project

1. Open JDeveloper if it is not open. (You can double-click the JDeveloper shortcut on the desktop.)
2. In the Applications pane, click **Open Application** (or select **File** > **Open**).



3. In the Open Application(s) dialog box, navigate to the `/labs/lessons/Lesson09/po_composite/` directory, and open the `po_composite.jws` file. Two project folders, Validatepayment and ManualApproveOrder are displayed in the Application pane.

   Note: ManualApproveOrder is the project folder for Task Form.
4. In the SOA folder, double-click ValidatePayment (composite.xml). The application components are displayed in the Design view of the Composite Editor.

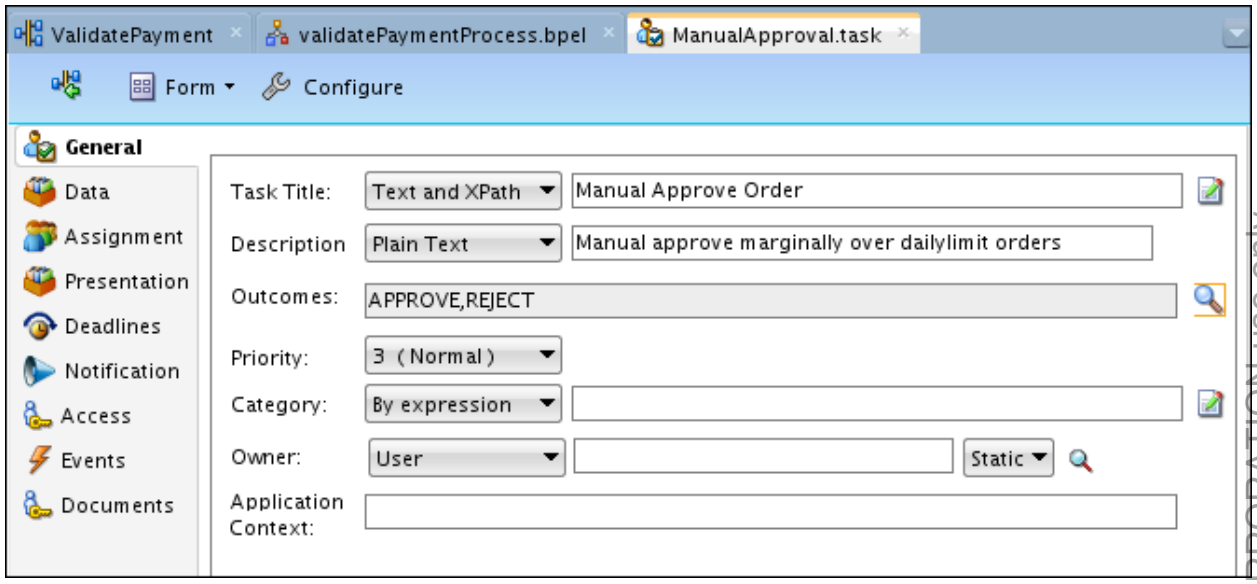


Inspect the human task service component

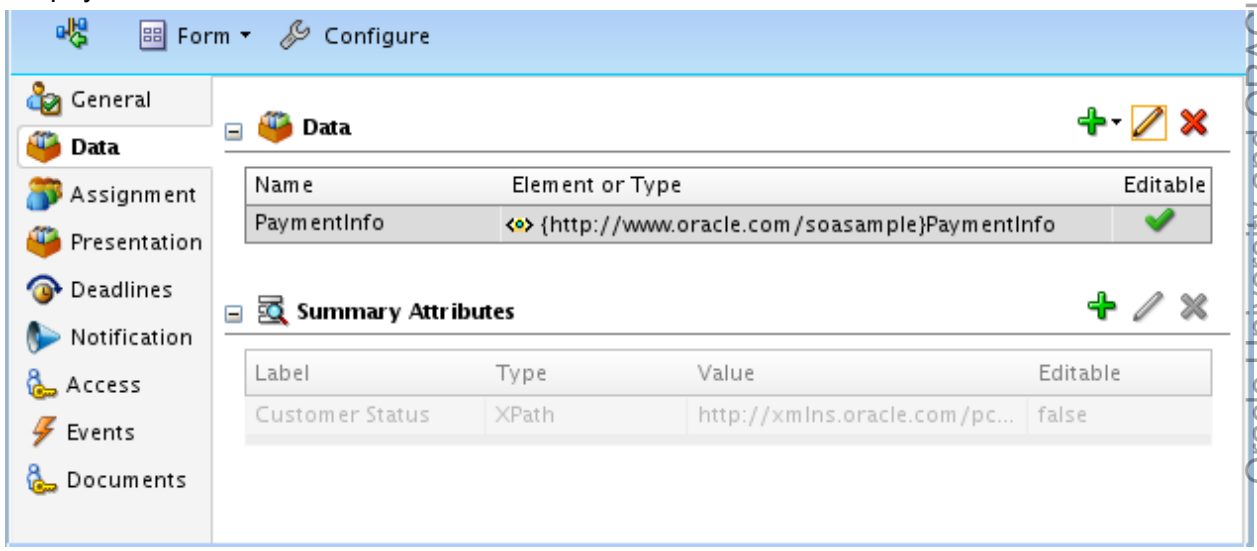
5. Double-click the ManualApproval human task component. The task definition file, ManualApproval.task, is opened in the Human Task editor.

The Human Task editor consists of the main sections shown on the left side of the following screenshots. These sections enable you to design the metadata of a human task.

a. Click **General**. In this section, you specify details such as the task title, description, task outcomes, task category, task priority, and task owner.

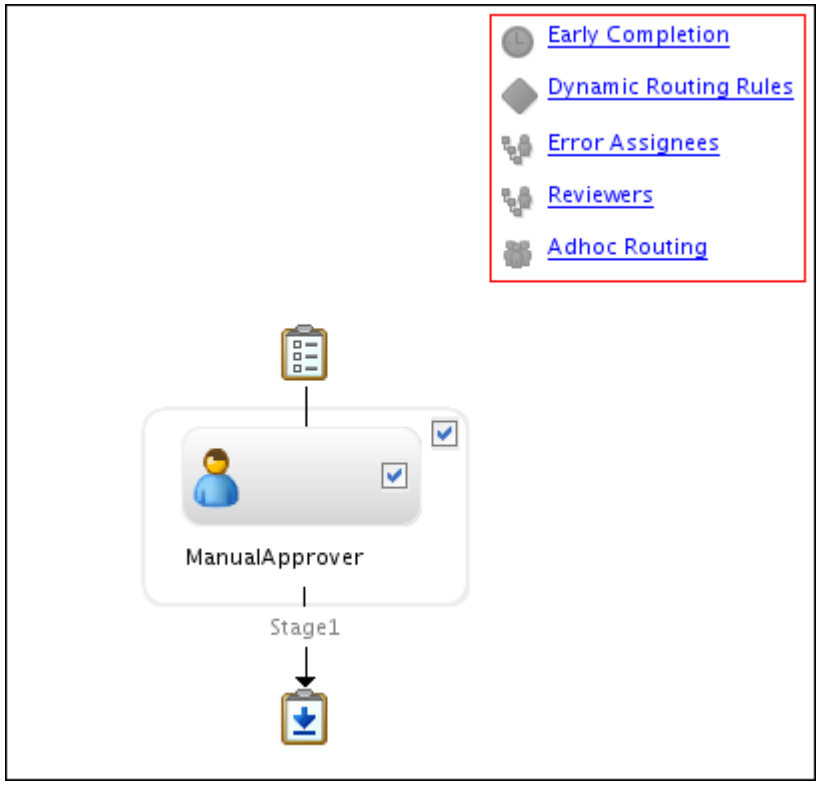


b. Click **Data**. In this section, you specify the structure (message elements) of the task payload. You create parameters to represent the elements in the XSD file. This makes the payload data available to the workflow task.

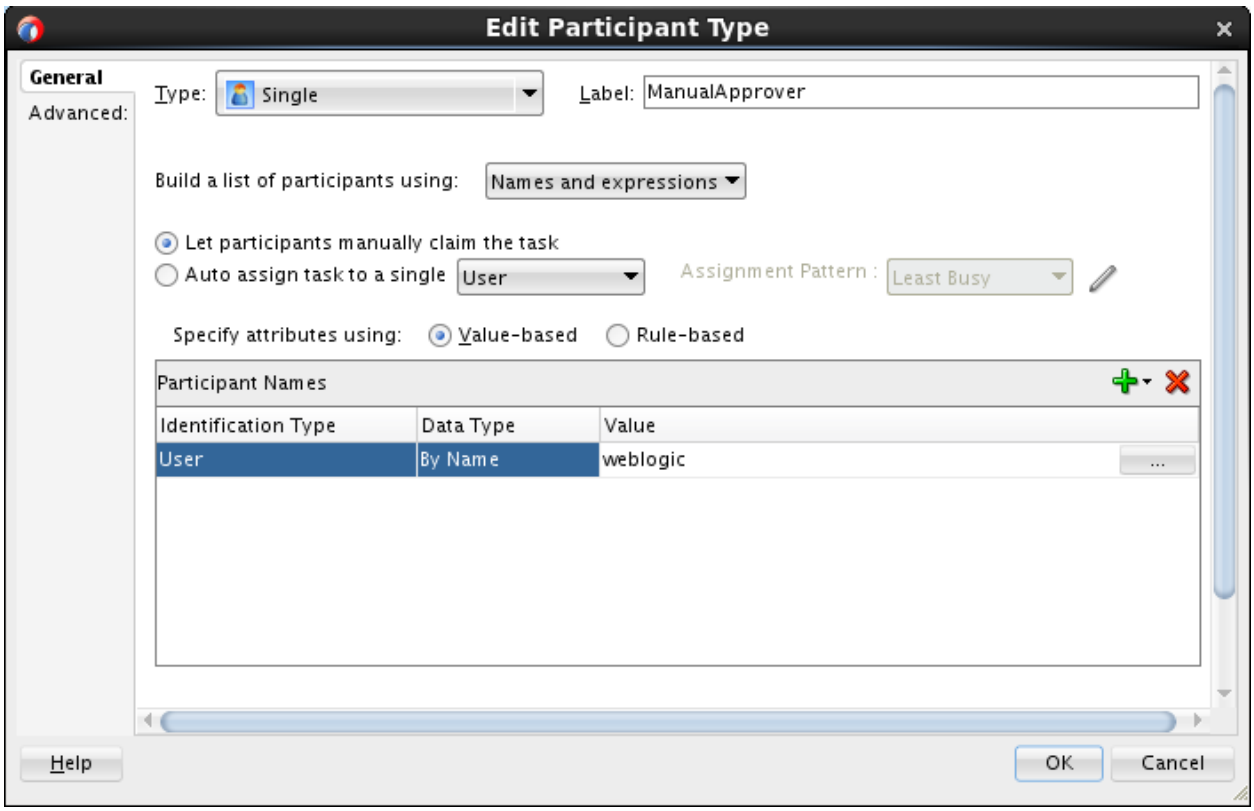


1) Task payload data consists of one or more elements or types. Click the Edit Task Parameter icon to view the selection for this task payload. It should be PaymentInfo Element.
2) Click OK.

c. Click **Assignment** to view the participants assigned to the task.

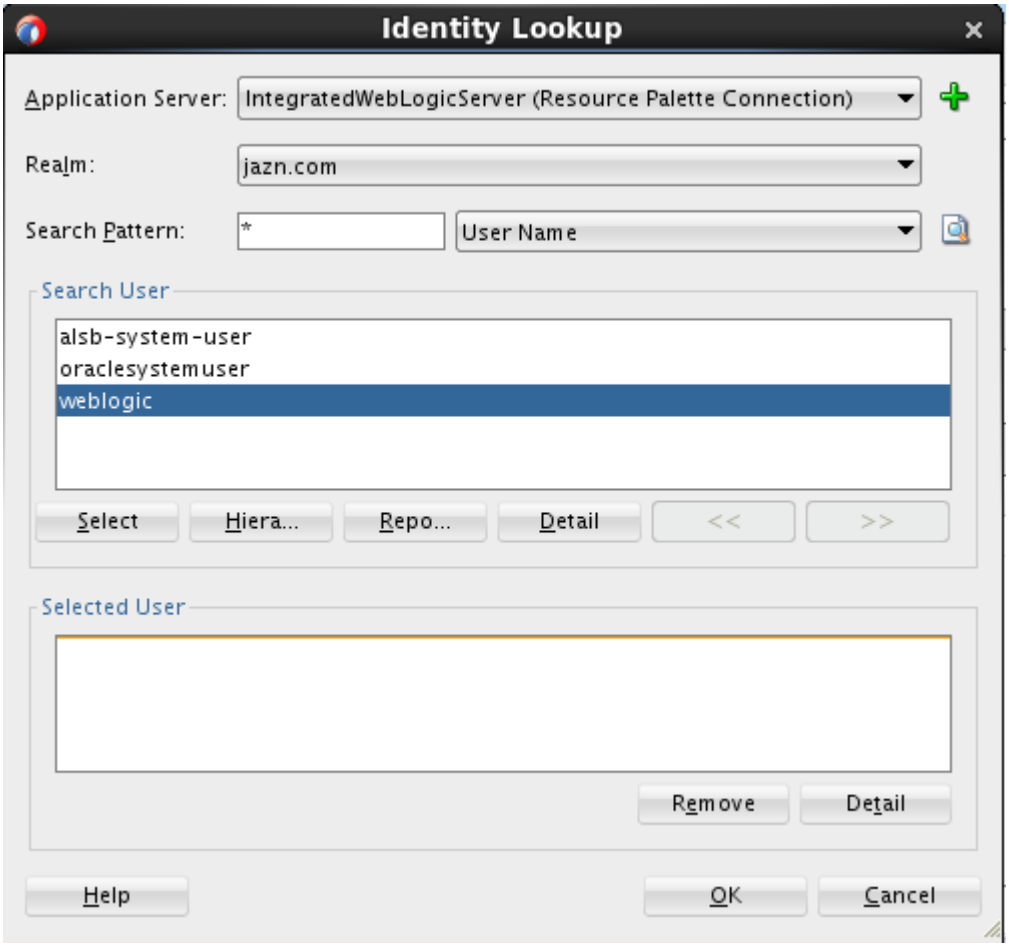


1) Double-click the ManualApprover icon.
   The Edit Participant Type dialog box is displayed.



   In this practice, we use the single user `weblogic` that comes with the QuickStart installation as the task assignee.

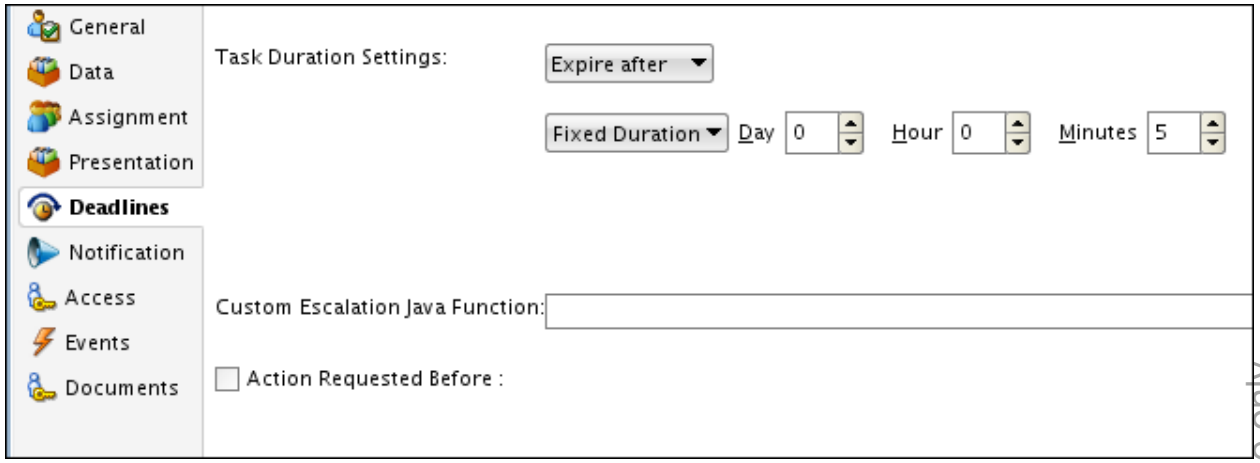
2) Click the ellipsis (…) next to weblogic to open the Identity Lookup dialog box.
3) Select IntegratedWebLogicServer for Application Server, and click search. You should see the users that come with the product installation.



Hint: If you have LDAP configured, you can add LDAP users in this dialog box.

4) Click Cancel to close the Identity Lookup dialog box.
5) Click Cancel to close the Edit Participant Type dialog box.

d. Click **Deadlines** to set an expiration time for the task.
   1) In Task Duration settings, select **Expire after** from the drop-down list.
   2) Set the duration to 5 minutes.



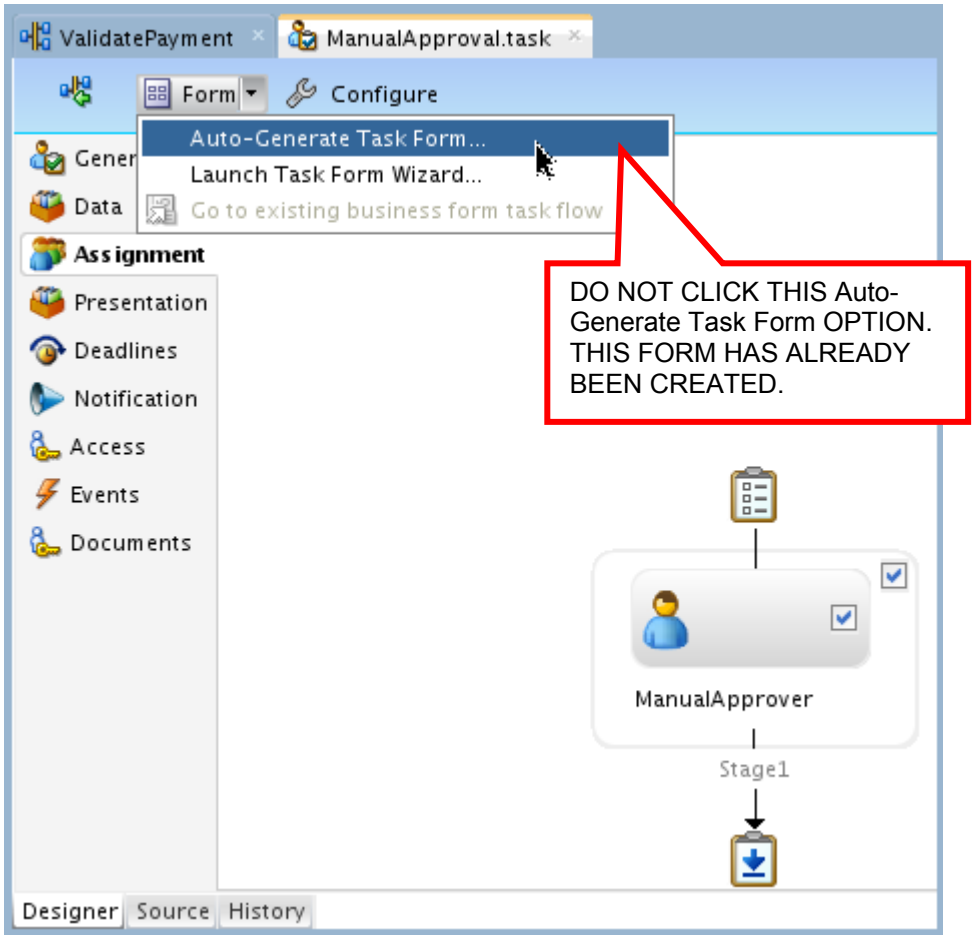
With this configuration, if the assignee does not take any act on the assigned task within 5 minutes, the task will be expired.

e. Save your work. At this point, you have finished reviewing and modifying the simple task definition.

Examine the task display form

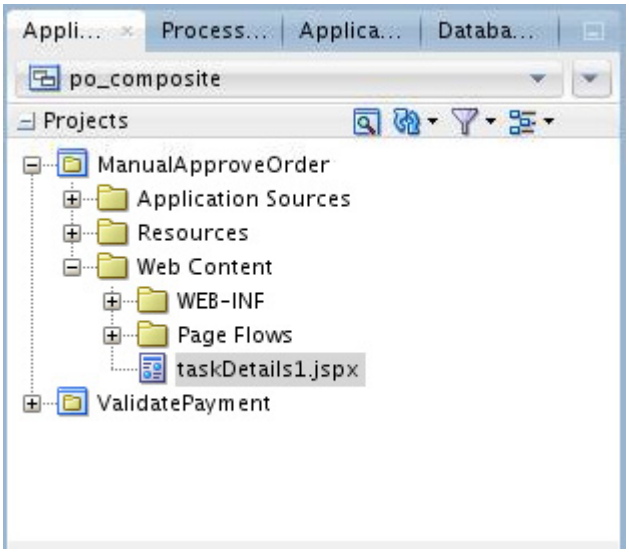
When a SOA composite includes a human task, you need a task form for users to interact with the task.

6. A task display form can be created from your human task definition. The following screenshot shows how to do it in the Human Task editor. Because the form has already been created, you do not need to do anything here.

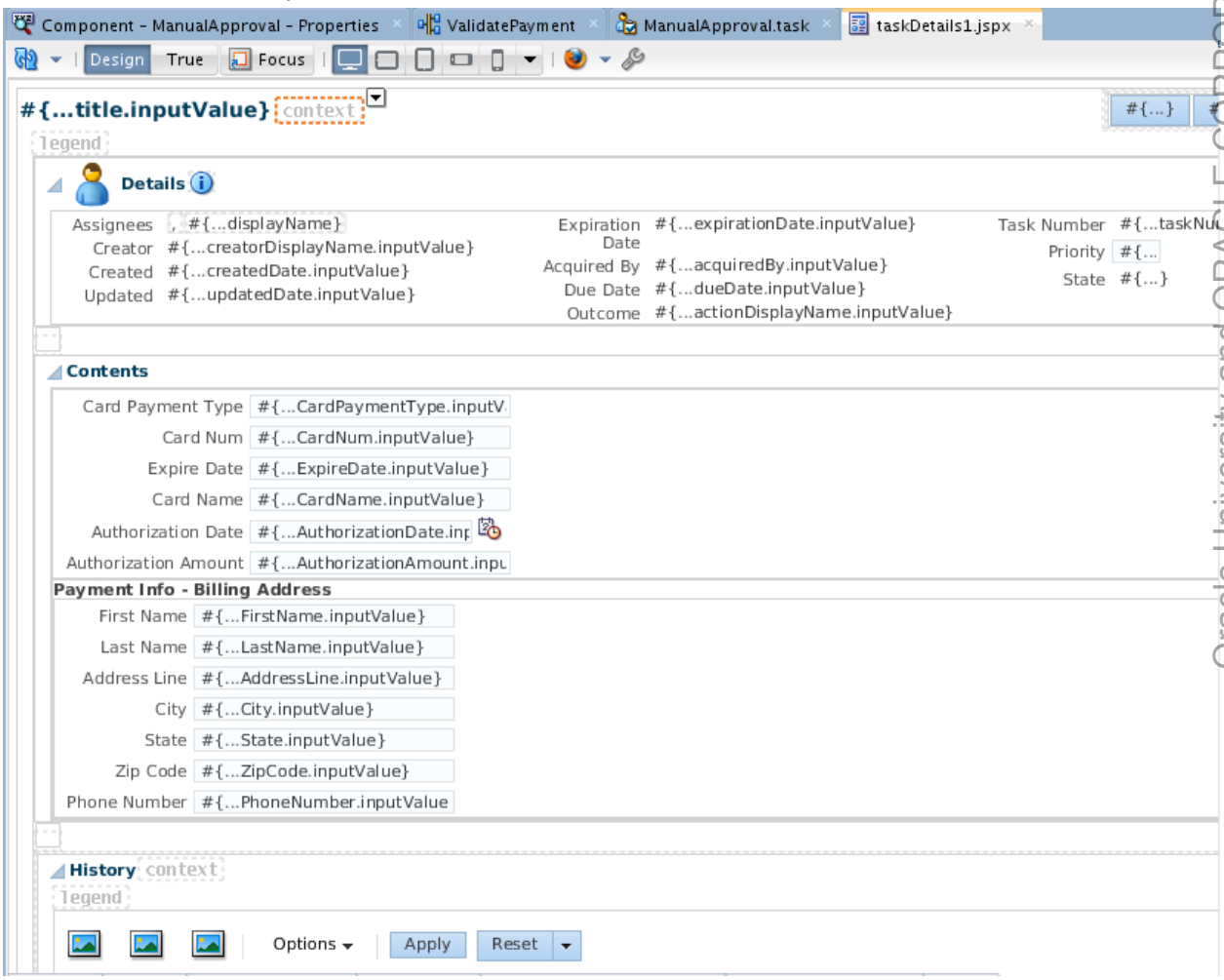


Note: The Auto-Generate Task Form option generates a default task form for your composite. This generated task form lives in its own project. The form is created using ADF Faces Rich Client components. The Launch Task Form Wizard option is a semi-automated way of creating a task form. It guides you through a multistep wizard that allows configuration of many aspects of the generated task form. Alternatively, you can create a JSF project to manage the task form and point it to the task file that you create in your composite.

a. In the Application panel, expand ManualApproveOrder > Web Content folder. You should see the taskDetails1.jspx file, which is a Java Server Page XML (`.jspx`) file.



b. Double-click taskDetails1.jspx.
The task form is displayed in a new window.



c. Review the contents. In the task form, the task details include the task payload and actions that you defined in the task.

7. When you are done, close the task form, task flow and Human Task editor, and return to the Composite editor.

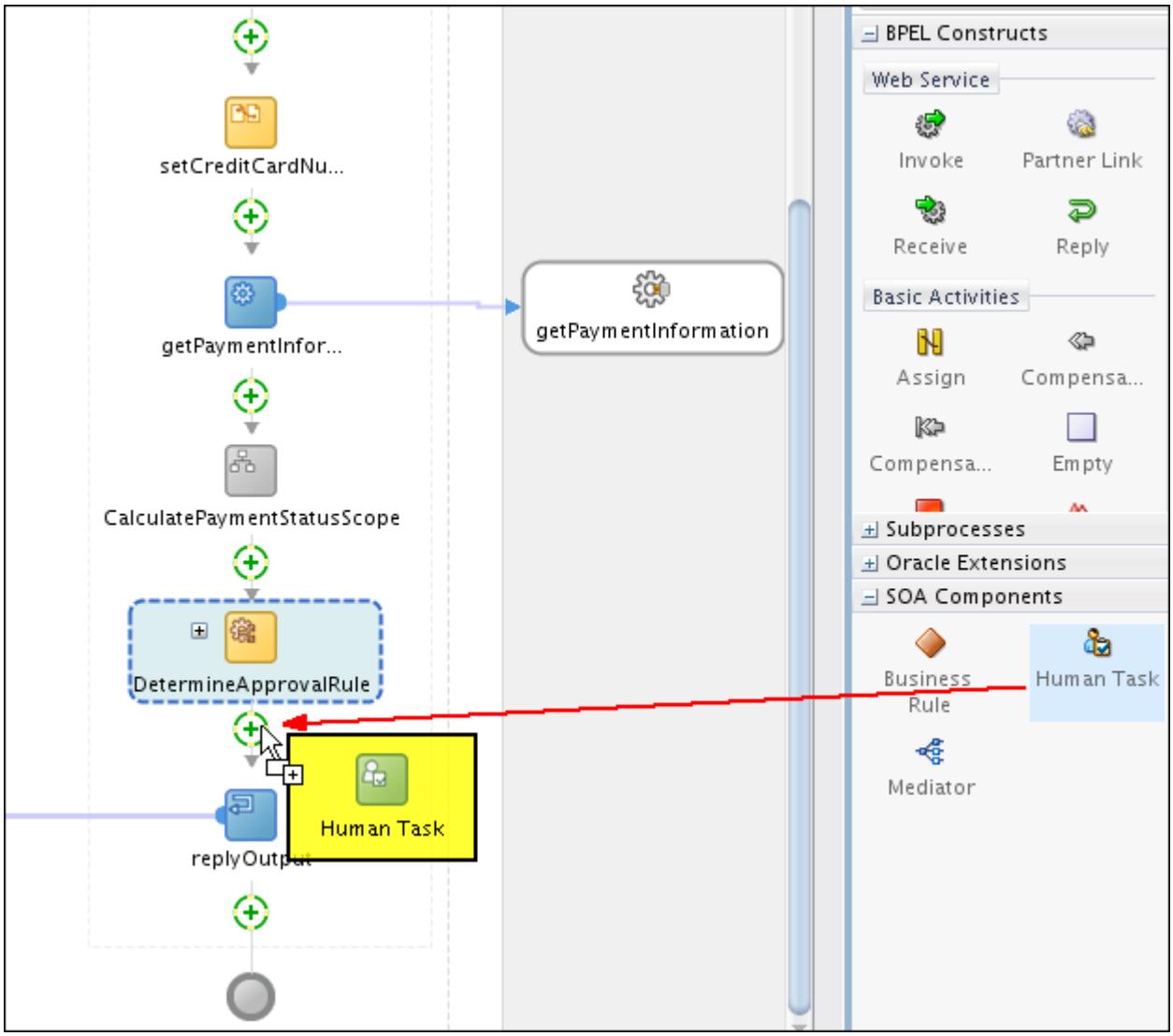
Practice 9-2: Integrating the Human Task Component in the BPEL Process

Overview

In this practice, you integrate the ManualApproval service component into the ValidatePaymentProcess BPEL process by adding and configuring a human task activity in the BPEL process.

Tasks

1. In the Design view of the Composite editor, double-click the ValidatePaymentProcess BPEL component to open it in the BPEL editor.
2. Drag a Human Task activity (in SOA Components) from the Component Palette and drop it right after DetermineApprovalRule. Use the following screenshot as a guide:



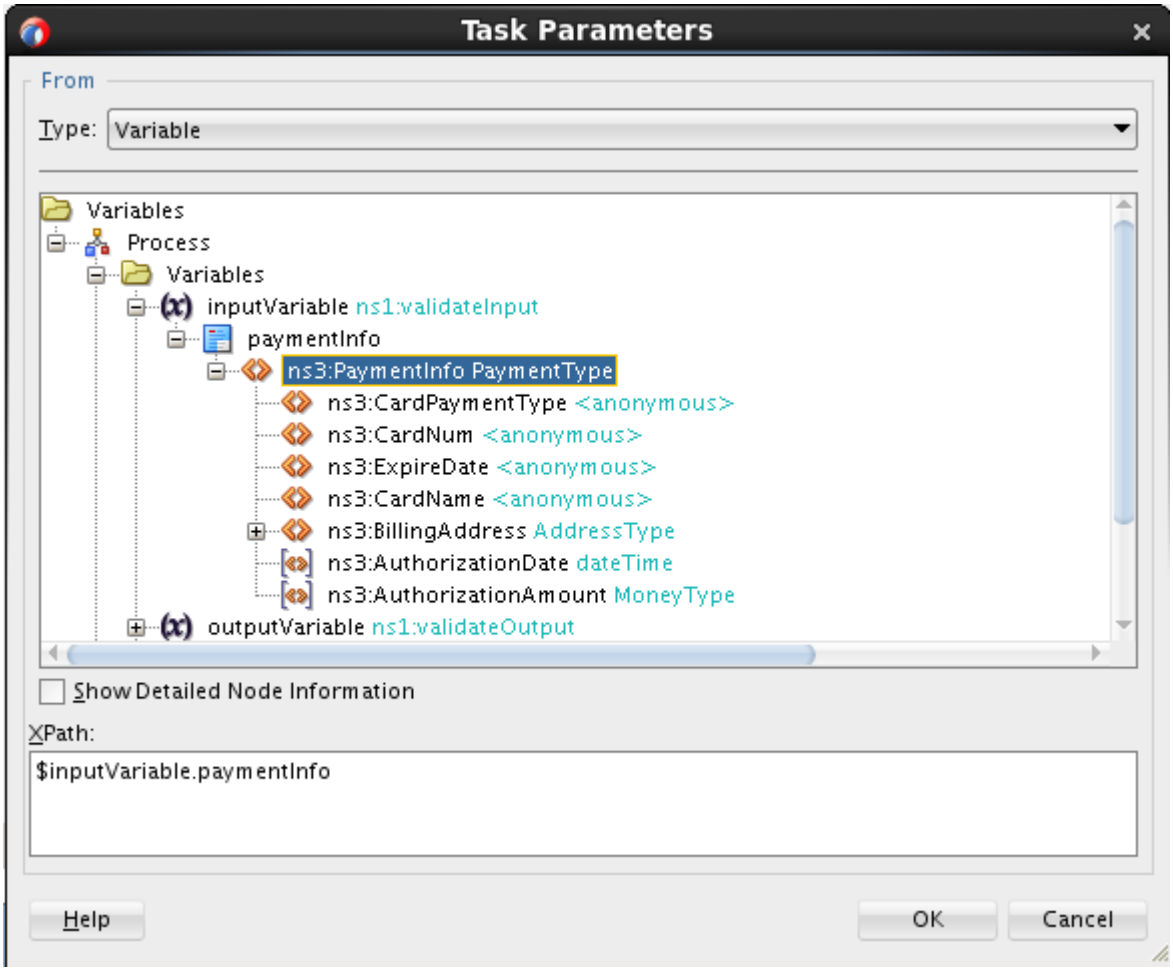
3. Double-click the Human Task activity, HumanTask1.
4. In the Edit Human Task dialog box, do the following:
   a. Select ManualApproval from the Task Definition drop-down list.

b. Enter `Approve Marginally Over limit Order` in the Task Title field.

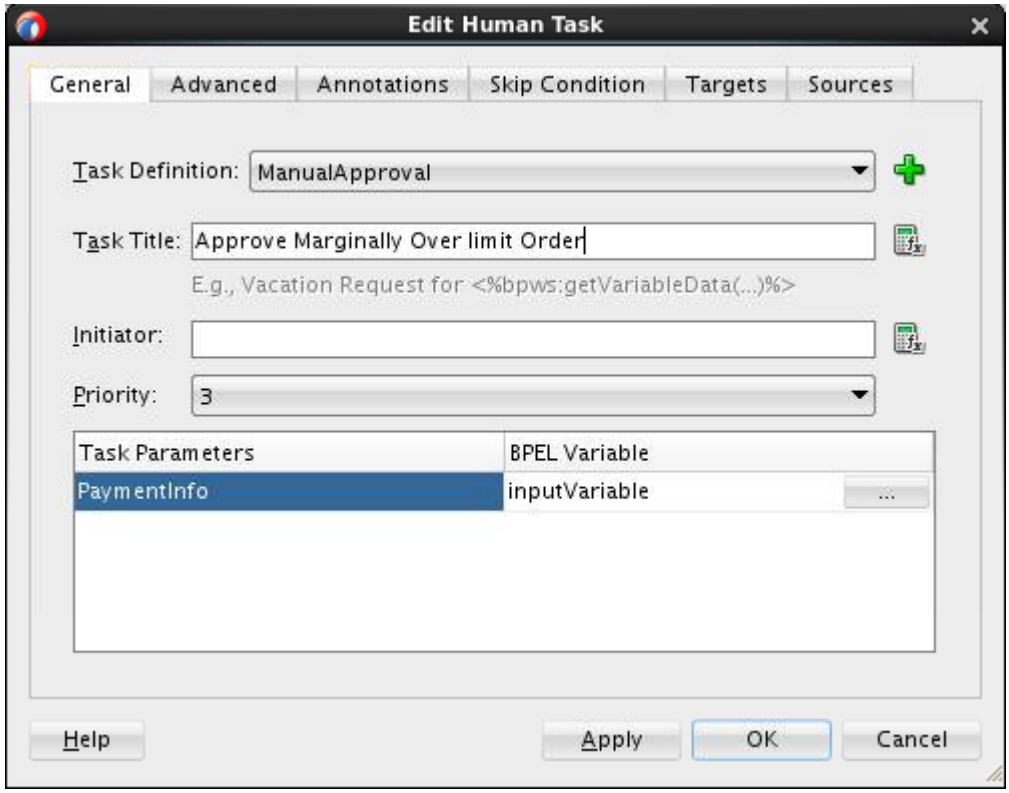
c. Select the BPEL variable that needs to be passed as the input parameter:

1) Click the ellipsis (…) button to the right of the PaymentInfo parameter.
The Task Parameters dialog box is displayed.

2) To Configure the mapping between the task parameters and the BPEL variables, ensure that the Type is set to the Variable option, then select Variables > Process > Variables > inputVariable > paymentInfo, and click OK.

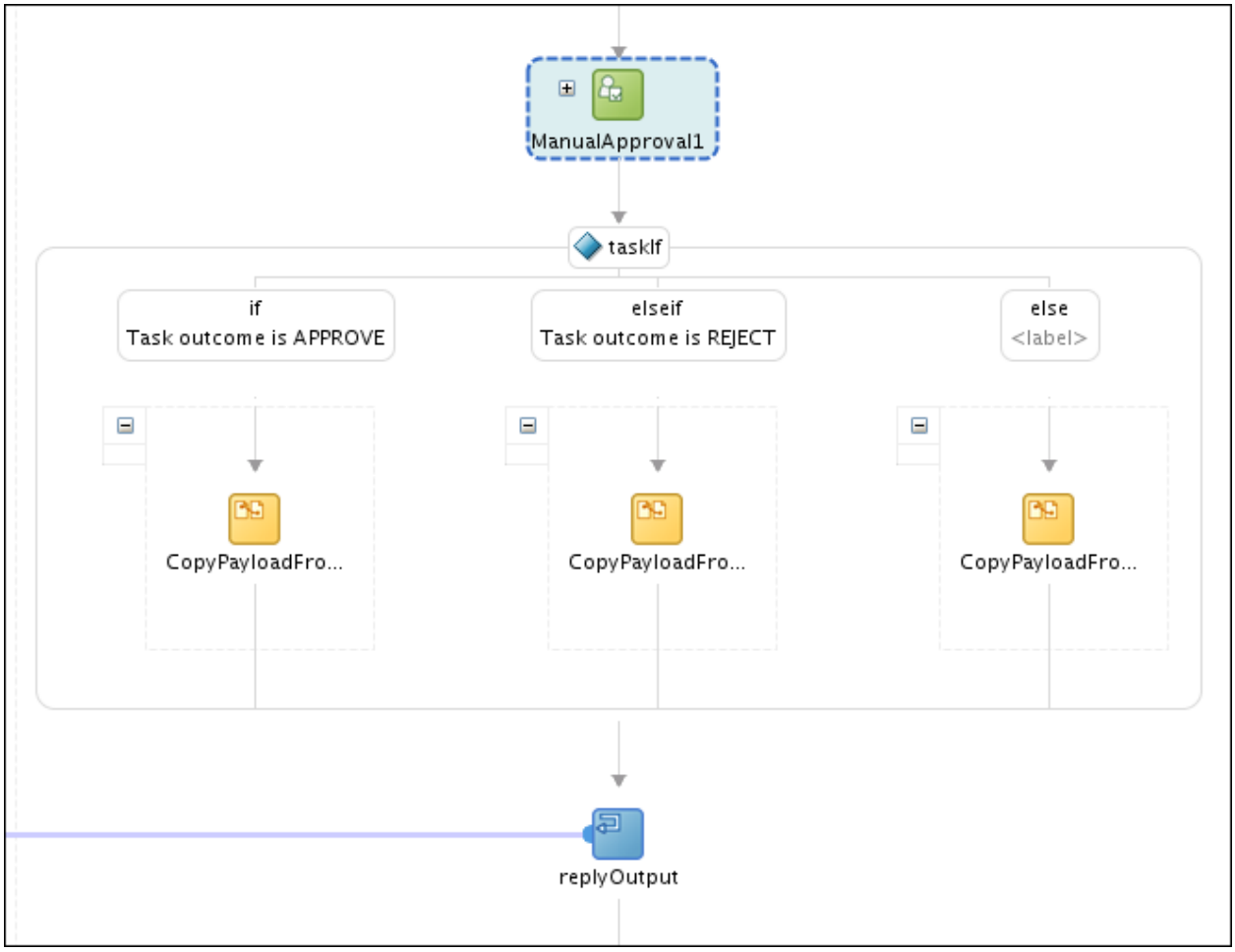


d. The Human Task dialog box is updated with the options that you specified, as shown in the following screenshot:



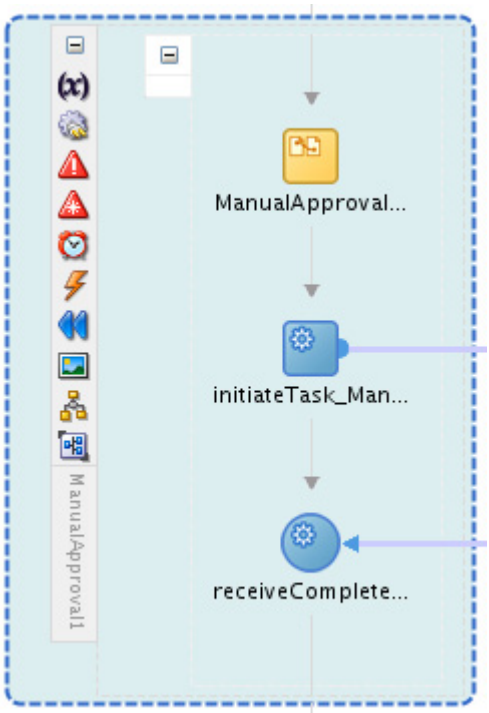
Click OK to close the dialog box.

5. In the BPEL editor, note that:
   - The HumanTask1 was updated to ManualApproval1.
   - An IF activity was added—with branches for each of the outcomes configured for the selected task definition as well as a branch for the otherwise case. For this task, the outcomes are REJECT and APPROVE, and, therefore, the switch has three branches.



Note: If the task definition specifies task parameters that can be modified by the task service, then the CopyPayloadFromTask Assign activities are generated with a single copy operation to copy modified task payload data to the BPEL variable defined as a task parameter in the task definition.

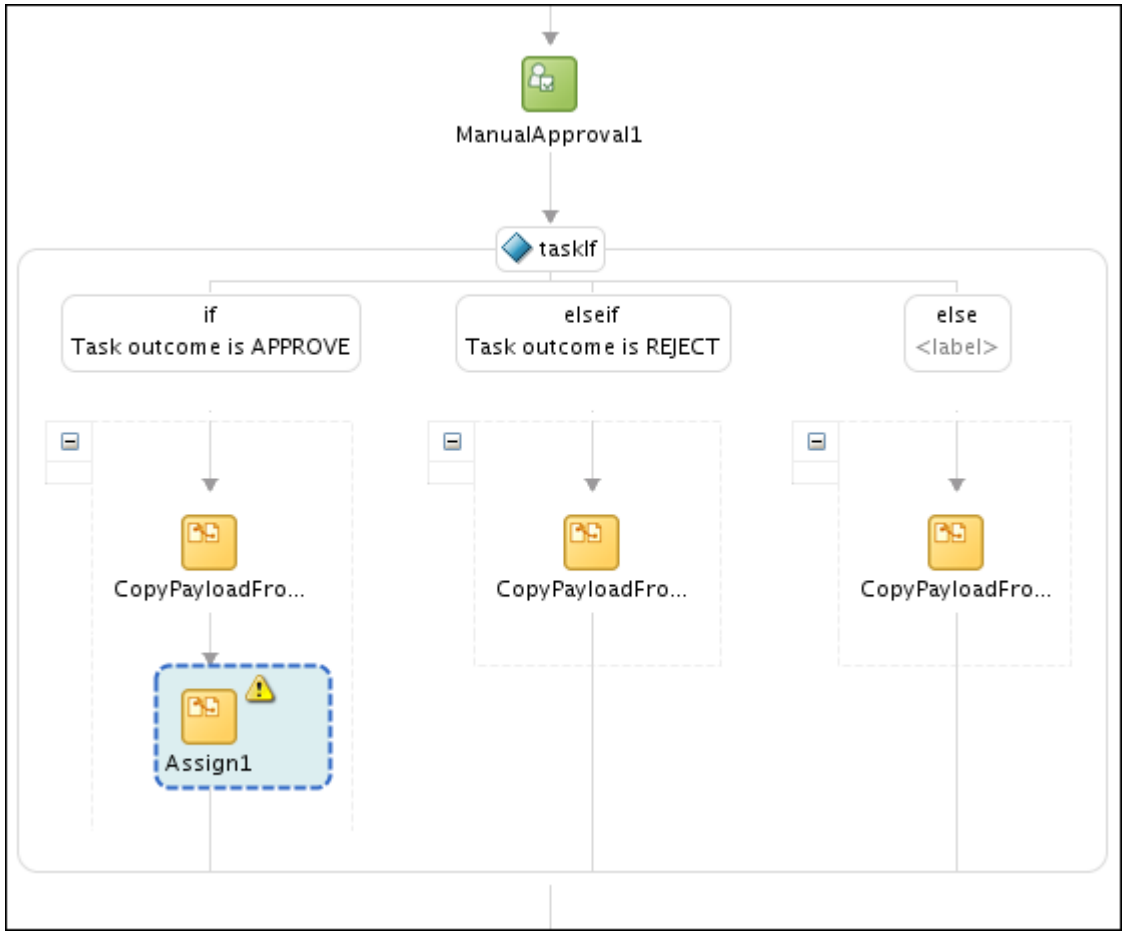
a. Expand the ManualApproval1 activity. It contains a combination of an Assign, an Invoke, and a Receive activity. This is similar to an asynchronous Invoke from the BPEL component to a partner link.



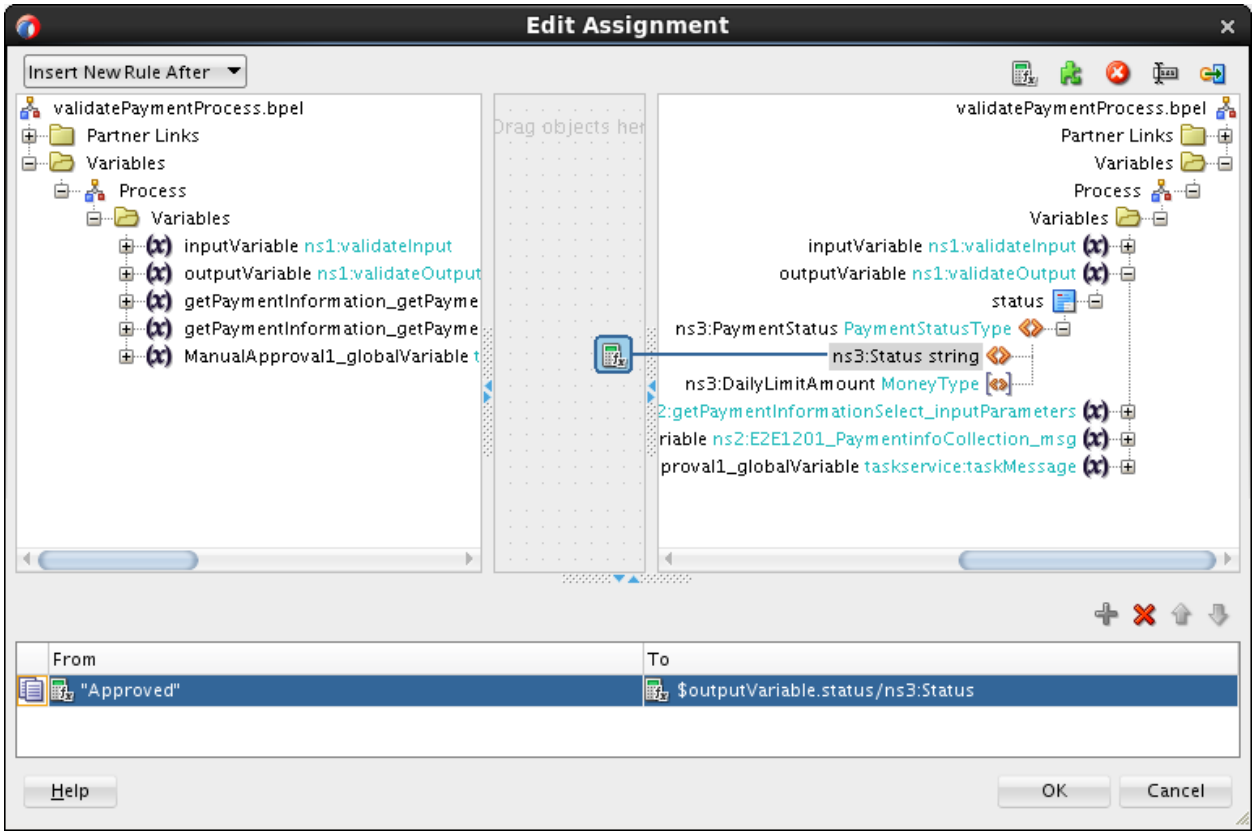
These three activities prepare the data to be passed to the task, invoke the Task Service to create an instance of the ManualApproval task, and receive its asynchronous result.

b. Collapse the activity.

6. Add an Assign activity in the outcome is APPROVE branch of the IF activity to assign the decision result "`Approved`" to the status element of the outputVariable to populate the outgoing message.
   a. Drag the Assign activity from the Component Palette and drop it in the "if Task outcome is APPROVE" branch, right after CopyPayloadFromTask.

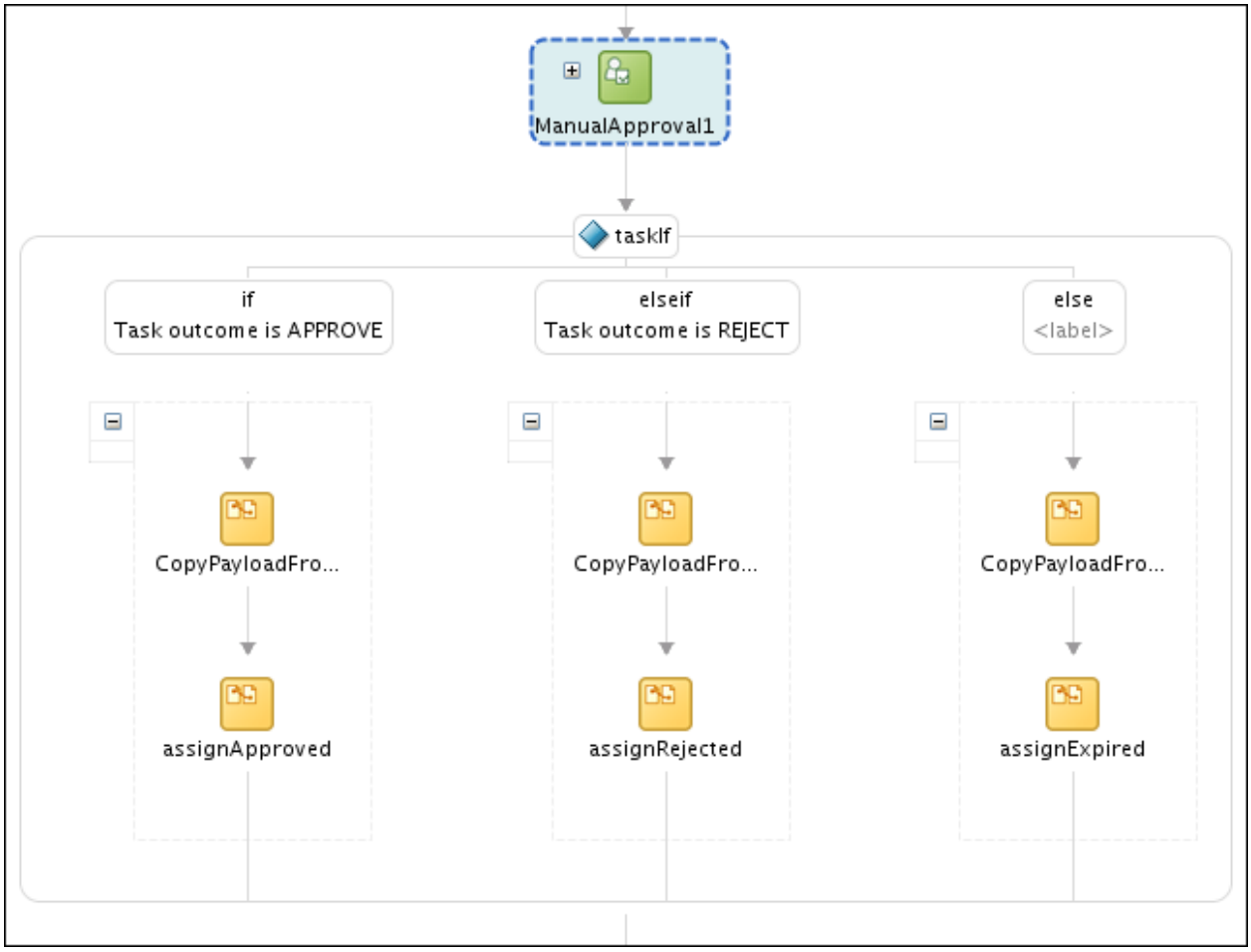


b. To edit the activity, double-click the Assign1 icon, and do the following:
   1) In the Edit Assign window, click the General tab and change the name to `assignApproved`.
   2) Click the Copy Rules tab.
   3) In the target column (right), expand Variables > Process > Variables > OutputVariable > status > PaymentStatus, and select the status node.
   4) Drag the Expression icon ( ), and drop it on the status node.
      The Expression Builder window is displayed.
   5) In the Expression field, enter `"Approved"`, and click OK.
   6) The Edit Assign window is updated with the XPath expression. See the following screenshot of the Edit Assign window:

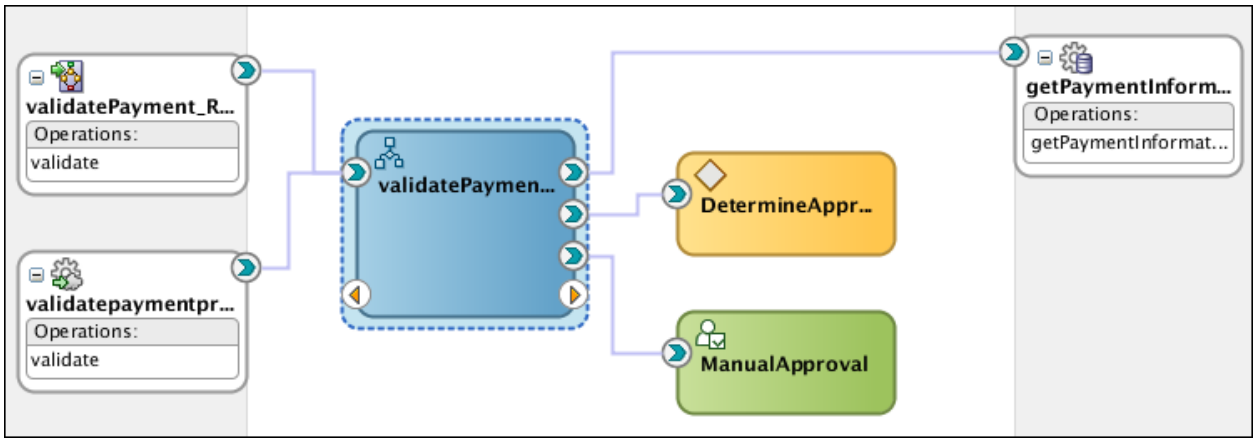


7) Click OK to close the Edit Assign window.

7. Repeat step 6 to add the assignRejected assign activity in the REJECT branch of the IF activity to assign the decision result `"Rejected"`, to the status element of the outputVariable.
8. Repeat step 6 to add the assignExpired assign activity in the “else” branch for the otherwise case to assign the value `"Expired"` to the status element of the outputVariable.
9. After you complete the above steps, the Switch activity (after the ManualApproval1) in the BPEL process flow should resemble the following screenshot:



10. Save and close the BPEL editor and return to the Composite editor. The ValidatePayment composite is updated.



11. Save the project.

Practice 9-3: Deploying and Testing

Overview

In this practice, you deploy the ValidatePayment composite application and test the service by feeding the sample purchase order.

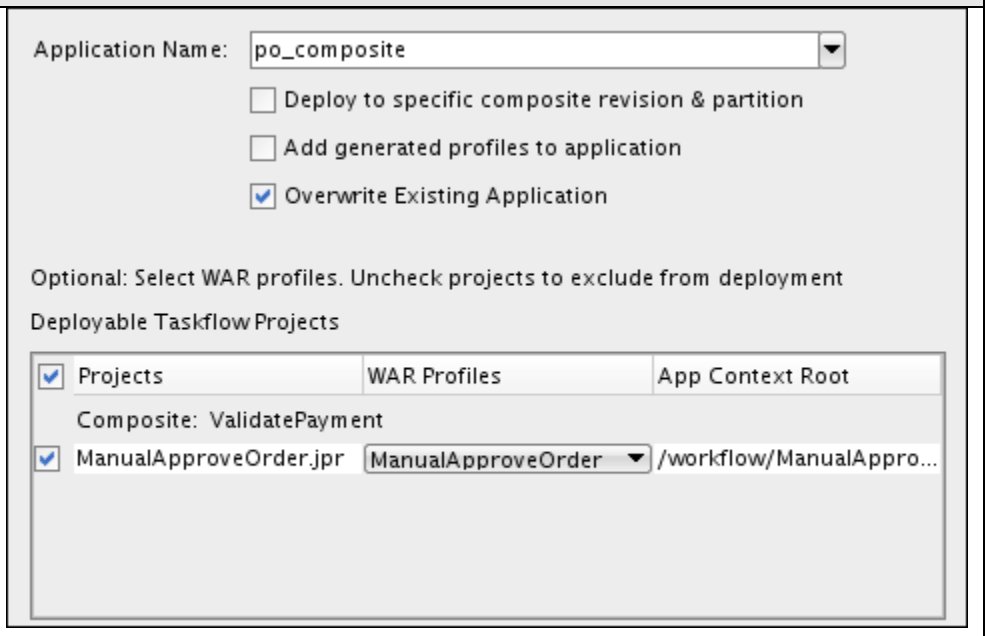
In a real world situation, when the BPEL process is initiated and executes the Human Task service, the users involved in the Human Workflow interaction should ideally be notified that a task has been assigned to them. The users can then log in to the Worklist application to view their task assignments and the associated data, and perform a task-based action to enable the workflow pattern to come to completion. Due to the complexity of configuring a mail server and email account, the lab setup is simplified without notification configuration; therefore, you will directly log in to the Worklist application as the authorized user and perform an action on the assigned task.

Tasks

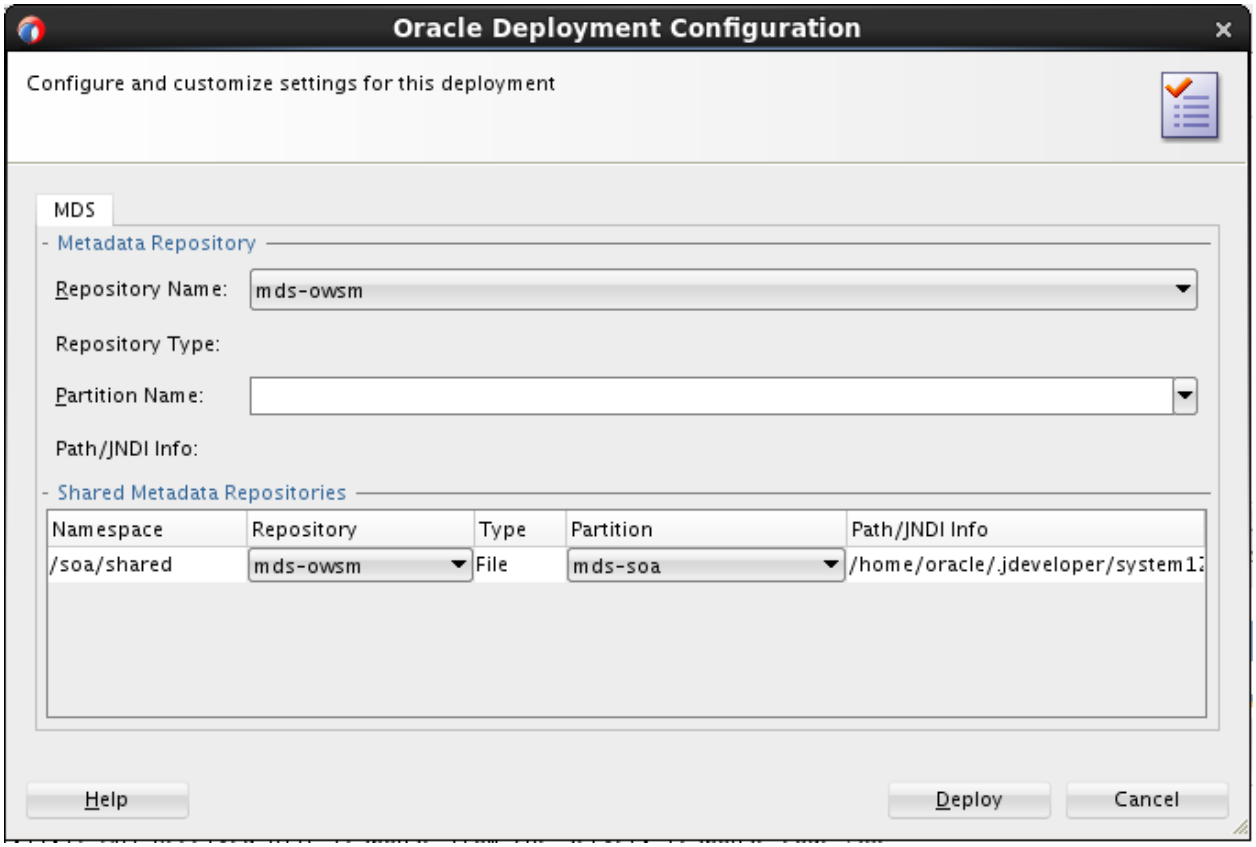
Deploy the ValidatePayment composite to the application server

1. In the Applications pane, right-click the ValidatePayment project, and select Deploy > ValidatePayment. The Deploy ValidatePayment window is displayed.
2. In the Deploy ValidatePayment window, use the instructions in the following table to configure the deployment settings and deploy the composite:

Step	Screen/Page Description	Choices or Values
a.	Deployment Action	Make sure that “Deploy to Application Server” is selected. Click Next.
b.	Deployment Configuration	Use a new version for this deployment by changing the revision ID to **4.0**. Deselect “Mark composite revision as default.” Click Next.
c.	Task flow deployment	Use the default po_composite for Application Name. Select the Overwrite Existing Application option. Select Projects in the Deployable Taskflow Projects table. Use the following screenshot as a guide:

Step	Screen/Page Description	Choices or Values
		 **Note:** The TaskDetails project with the generated task form is deployed as a stand-alone web application that will be linked in FMW Control with the human task that it supports. Click Next.
d.	Select Server	Select IntegratedWebLogicServer. Click Next.
e.	SOA Servers	Accept the default settings. Click Finish.

When prompted for configuring and customizing settings for the deployment, accept the default, and click Deploy.

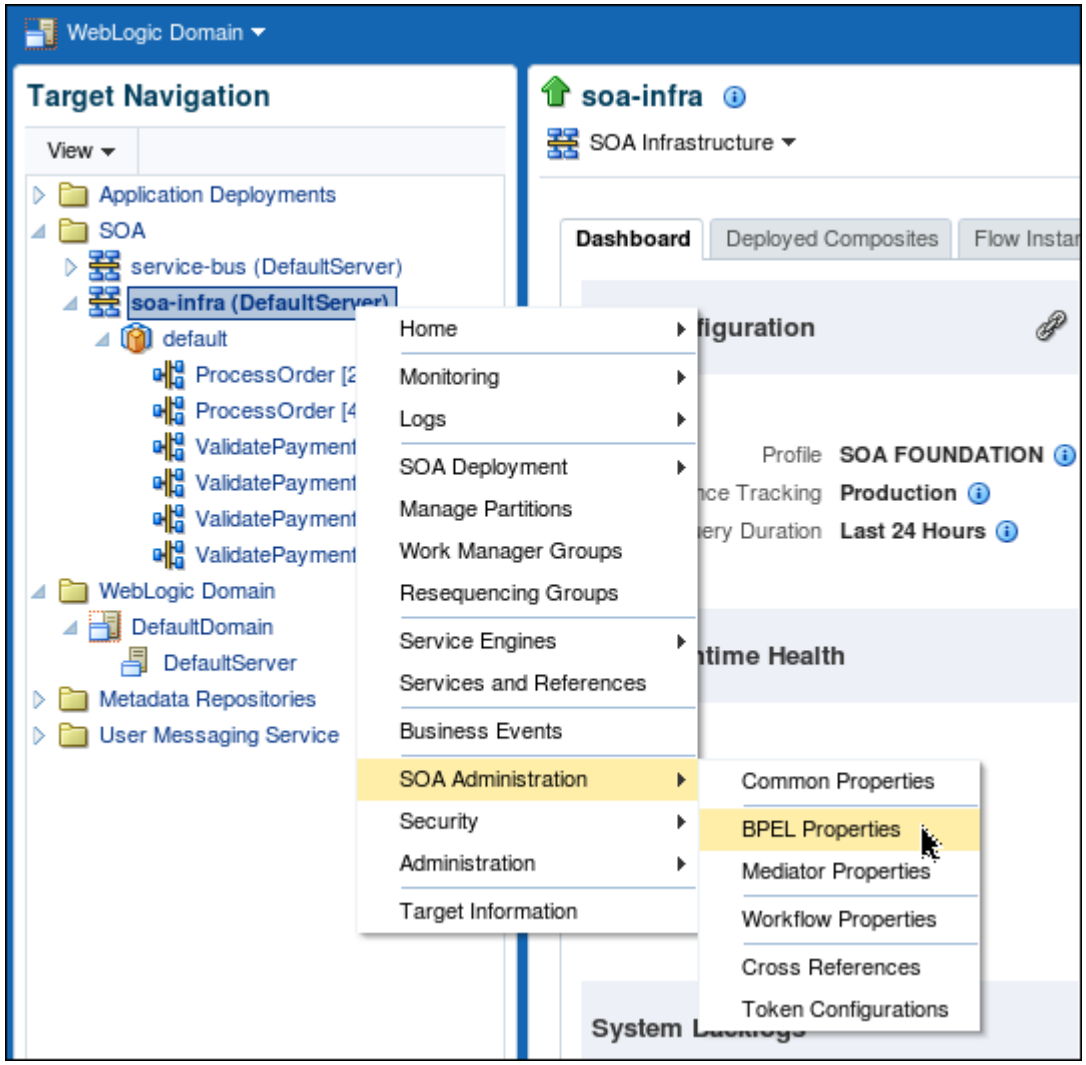


3. Deployment processing starts. Monitor the deployment progress and check for successful compilation in the SOA – Log window, and verify that deployment is successful in the Deployment – Log window.

Pretest Configuration

Before you test the deployed composite, you need to increase the timeout value of the BPEL process by using the SyncMaxWaitTime property. This property defines the maximum time a request and response operation takes before timing out. If the BPEL process service component does not receive a reply within the specified time, then the activity fails. You can specify timeout values with the SyncMaxWaitTime property in the System MBean Browser of Fusion Middleware Control Console.

4. To change timeout values, do the following:
   a. In the Enterprise Manager navigator, right-click soa-infra, select SOA Administration > BPEL Properties.



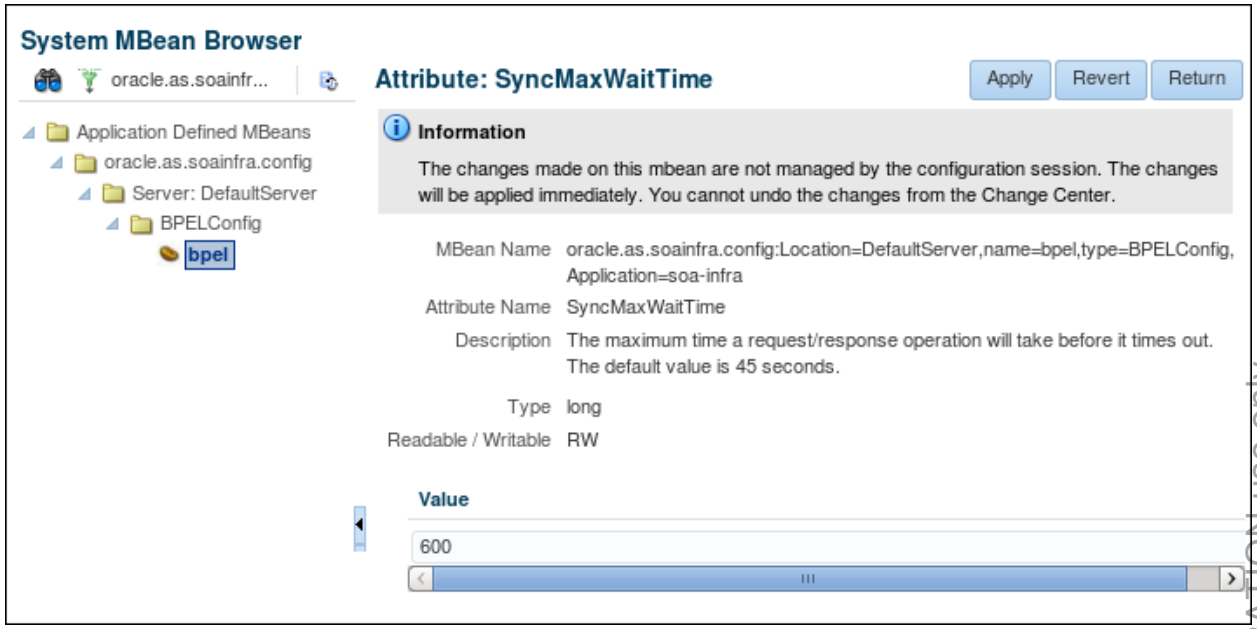
b. At the bottom of the BPEL Service Engine Properties page, click More BPEL Configuration Properties.

c. Click **SyncMaxWaitTime**.

You should see the default value is 45 seconds.

d. In the Value field, change the value to 600 seconds (5 minutes).

Note: The time out value should be longer than the expiration time that you specified for the task; otherwise, an exception will be thrown by the BPEL process.



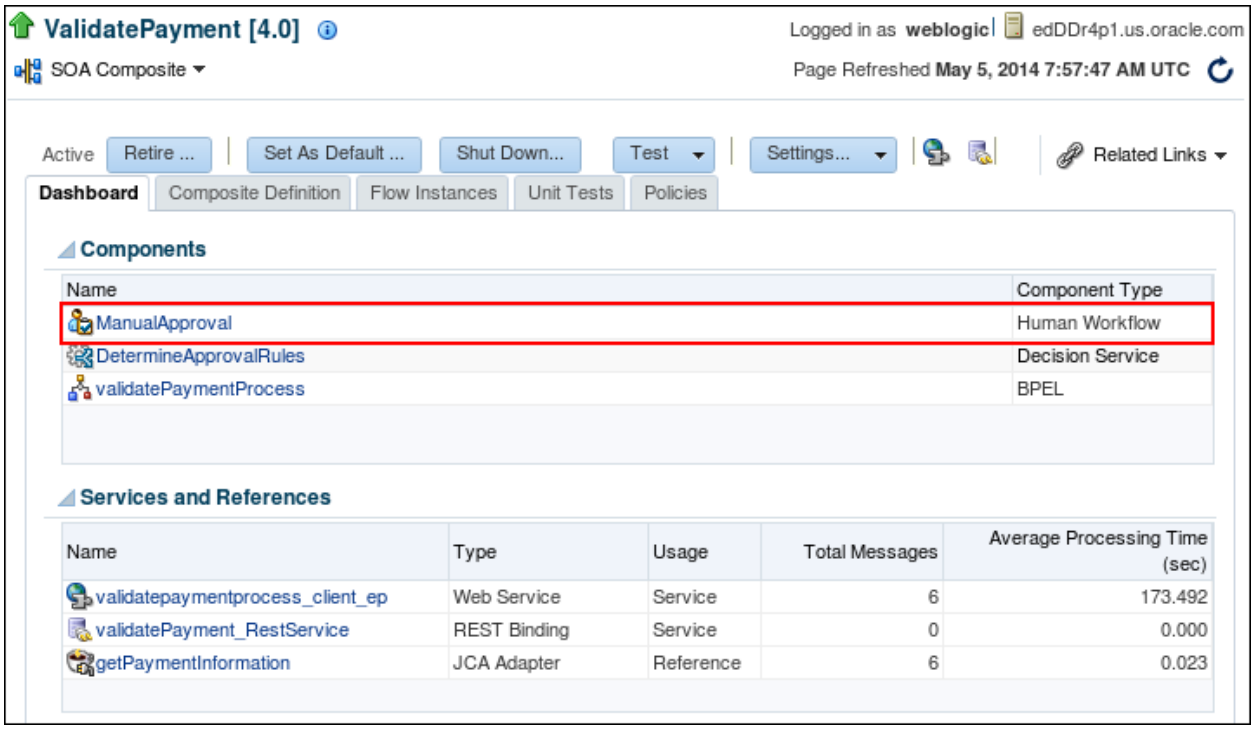
e. Click Apply.

f. Click Return.

Test the ValidatePayment composite

5. In the Enterprise Manager navigator, expand the SOA > soa-infra > default folder. You should see ValidatePayment [4.0] in the list of deployed applications.

6. In Dashboard, note the newly added Human Task service component.



7. Click the **Test** button (near the top of the window) and select **validatepaymentprocess_client_ep**.

   The Test page opens.

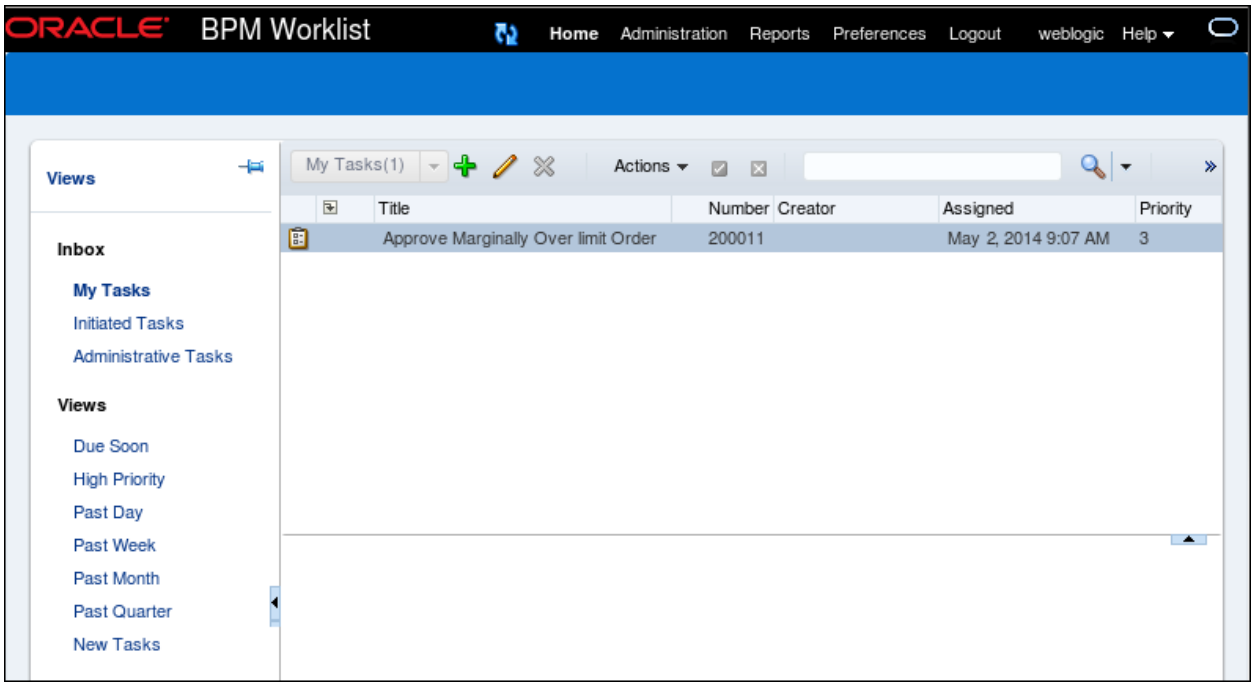
8. Scroll down to the **Input Arguments** section.
9. Click **Browse**. In the File Upload window, select the `/labs/resources/sample_input/PaymentInfoSample_ManualOverride_soap.xml` file, and open it.
10. Click **Test Web Service**.

    You will see an indication that the process is running. However, at this point, you will not get any response, because the task is pending approval or rejection.

Approve/reject the order by using the Oracle BPM Worklist application

11. In the Firefox browser, open a new tab and access the Oracle BPM Worklist application with the following URL:

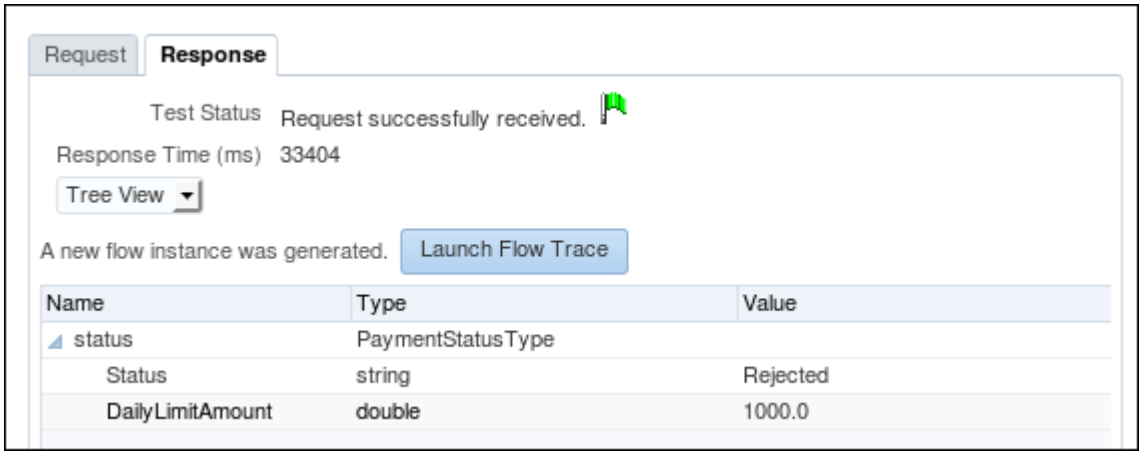
    `http://localhost:7101/integration/worklistapp`
12. If prompted for login, use the same credentials as you use for Fusion Middleware Control.
13. On the Oracle BPM Worklist page, you should see an Approve Marginally Over Limit Order task entry on the My Tasks subtab page.



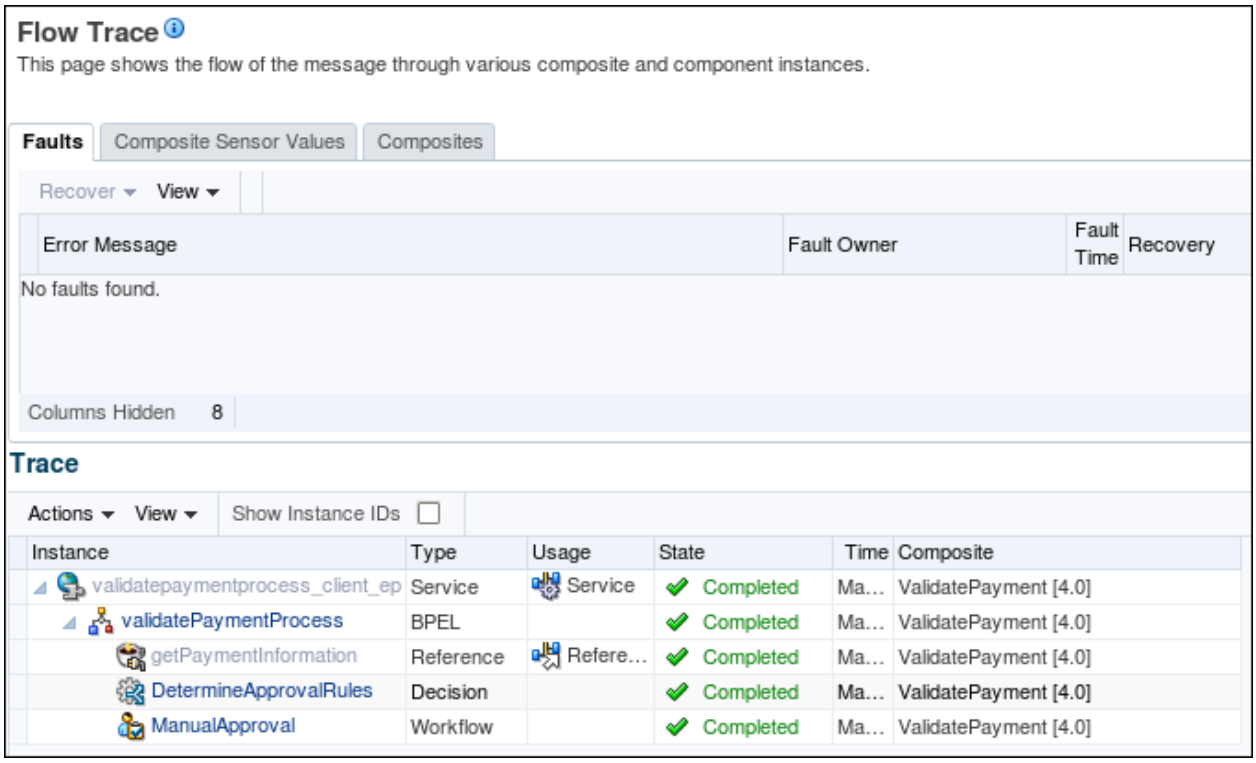
14. Select the task.
15. Click Actions, and select either Approve or Reject from the drop-down list.

    Once you made the selection, the task disappears on the page.

    Note: If you do not act on the task within 5 minutes, the task will be expired, and then the Approve and Reject options will not be available.
16. Go back to Fusion Middleware Control console and refresh the Flow Trace page. You should see the response with a status based on your decision along with the daily limit amount.



17. You can launch the flow trace to view the details in the audit trail.
18. Click the instance ID link, the Flow Trace page resembles the following screenshot:



19. When you are done, do the following:
    - Close the flow trace window in Fusion Middleware Control console.
    - Close the po_composite application in JDeveloper (including all the open windows).

Practices for Lesson 10: Virtualizing and Securing Services

Chapter 10

Practices for Lesson 10

Practices Overview

As you have seen, your credit validation application has gone through several updates. Every time there is a change in this application, you need to update the services (for example, the order processing service) that consume it to point to its updated revision. How do you protect the consumers of your service from the changes?

In addition, the consolidated credit validation service is hosted in-house to control quality for now; however, once its interface is stabilized, this service can be outsourced to a third-party provider. In the future, when AviTrec decides to outsource credit validation to an external provider, this can be accomplished without impacting existing applications.

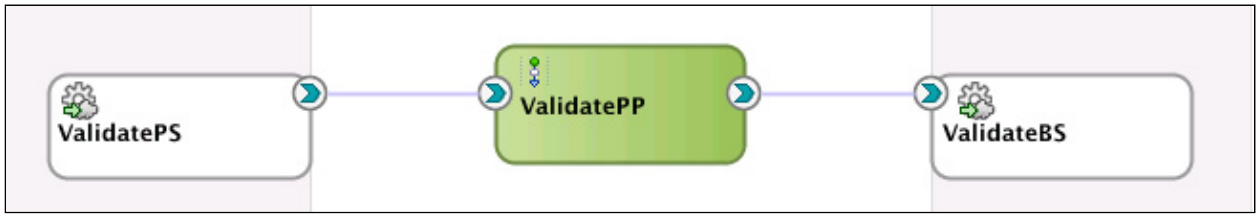
Practice 10-1: Registering the Composite on Service Bus

Overview

In this practice, you register the `validatePayment` composite on Oracle Service Bus. Service Bus will protect consumers of the `validatePayment` composite from routine changes such as changes of deployment location and implementation updates. Service Bus will help scale the service to handle higher volumes of requests and provide resiliency for the service if it needs to be taken down for routine maintenance.

Start by first creating a business service to register the composite URI. Then add a simple pipeline and proxy. Pipelines contain actions performed on the Service Bus—typically reporting, data transformation, and validation—before invoking the backend service. Consumers of the `validatePayment` service will invoke via a proxy rather than connect directly to the composite, allowing more agility and flexibility in managing change.

After completing this practice, the Service Bus application will look like the following screenshot:



High-Level Steps

In this lab, you accomplish the following tasks:

- Create a new Service Bus application, `po-servicebus`, and a new project, `ValidatePayment`.
- Create folders and import WSDL and XSD resources.
- Configure a business service for the `ValidatePayment` composite and review its properties.
- Create a proxy and a pipeline and wire them to the business service.
- Test and debug the end-to-end application.

Assumptions

`ValidatePayment [1.0]` and `ValidatePayment [3.0]` have been deployed and are running on the SOA server.

Tasks

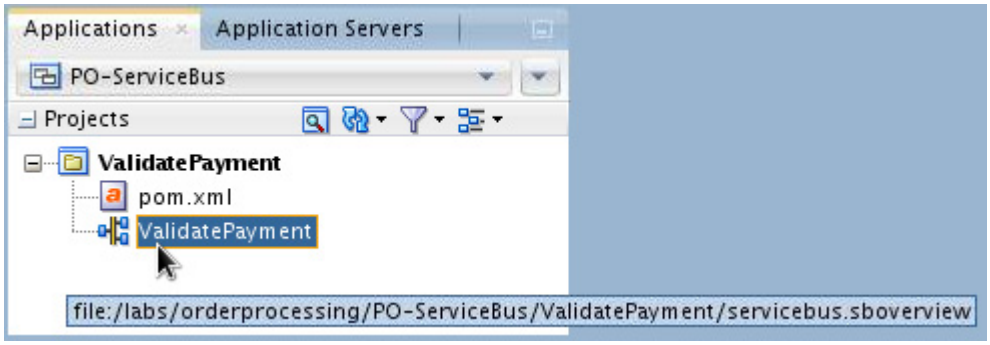
Create a Service Bus application and project

The first steps in building a new application are to assign it a name and to specify the directory in which to save source files. By creating an application using application templates provided by Oracle JDeveloper, you automatically get the organization of the workspace into projects, along with the project overview file.

1. Create a new Service Bus application.
   a. Select **File > New > Application** from the menu.
   b. From the Categories tree, select **General > Applications**.
   c. Select **Service Bus Application with Service Bus Project** from the **Items** field.

d. Click **OK**.
   The Create Service Bus Application With Service Bus Project dialog box opens.
e. Set the application name to `po-ServiceBus`.
f. Enter `/labs/lessons/lesson10/po-ServiceBus` for the directory.
g. Click **Next**.
   You are prompted to create a new project.
h. Set the project name to `ValidatePayment`.
i. Click **Finish**.

2. Double-click the `ValidatePayment` > `ValidatePayment` Service Bus icon in the Applications pane.



   The Services Bus Overview editor opens on the right. The Overview Editor is a new view for Service Bus in 12*c*, and it is modeled on the SOA Composite Editor. This view allows construction of Service Bus projects using a top-down, drag-and-drop approach.

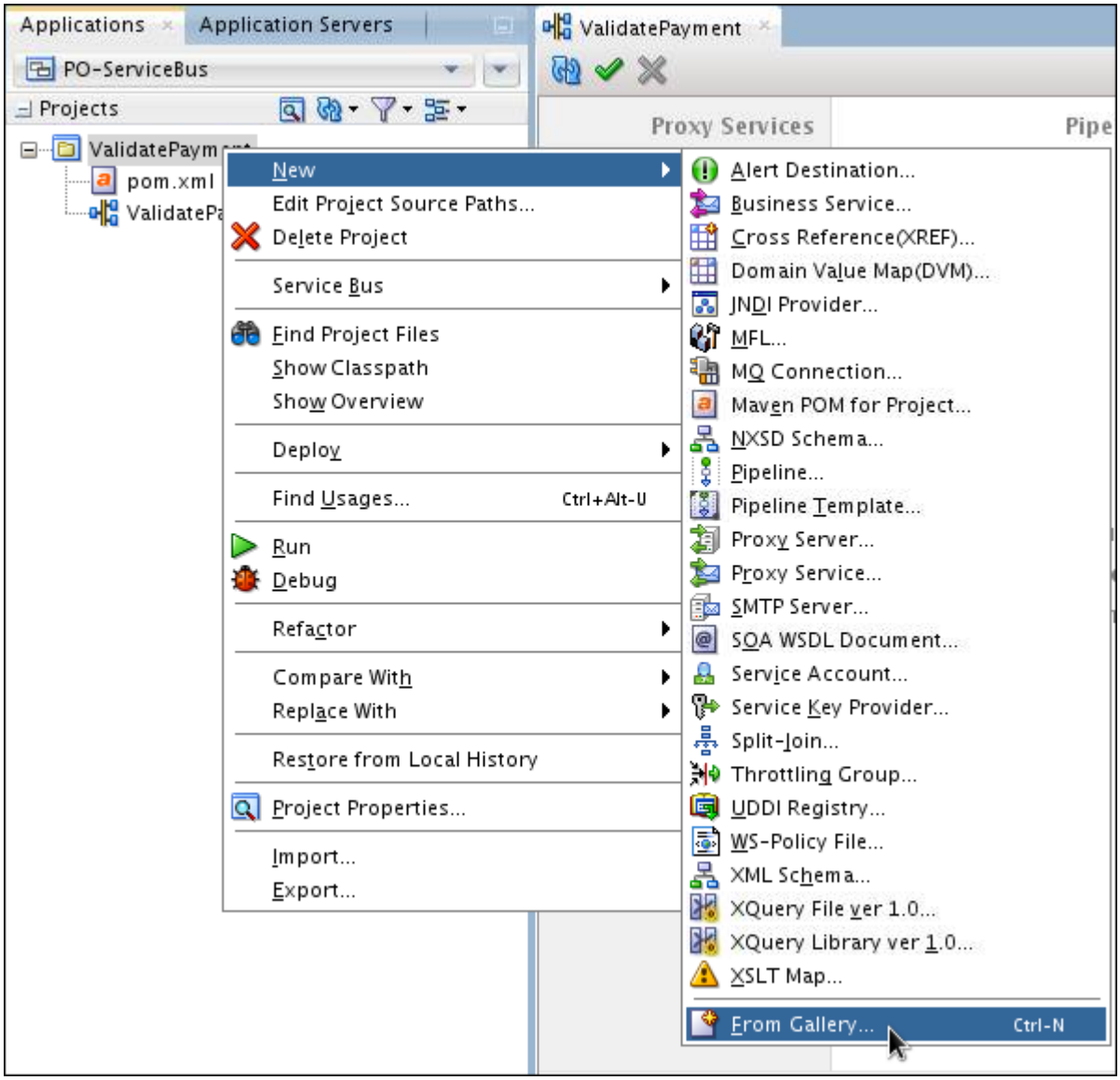
3. Take a few moments to browse the Applications pane in the left side of the window and the component palette on the right.
   - In the component palette, notice that the resources category contains pipeline and Split-Join icons. These are the components for a Service Bus application. In 12*c*, the pipeline has been split from the proxy to allow it to be a re-usable component.
   - Other palette categories—Technology, Adapters, and Advanced—contain adapters and transports for building business services (External References) and proxies (Exposed Services).
   - If the Properties window is on the bottom right of the JDeveloper window, drag and position it to the bottom center. This will make editing properties of pipeline actions easier.

Create folders and import artifacts

In Service Bus applications, folders are leveraged to organize artifacts within a project. For new applications, you are encouraged to create folders that align with the default folders in the composite application. Folders will not be automatically created in Service Bus applications, so that backward compatibility of Service Bus projects imported from previous releases is maintained.

For this practice, you will keep the structure simple, because there are only a few artifacts to manage. As the projects grow, you can add folders for categorizing artifacts into business service, proxy, and templates.

4. Create two folders.
   a. Right-click the `ValidatePayment` project icon, and select **New > From Gallery**.



b. In the New Gallery window, from the Categories tree, select **General > Folder**.

c. Click **OK**.

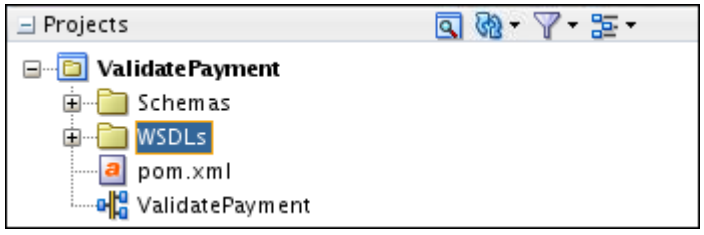
This is only necessary the first time a folder is added. Thereafter, it will be listed on the menu by default.

The Create Folder dialog box is displayed.

d. Name the folder `Schemas`, and click **OK**.

e. Similarly, add another folder named `WSDLs`.

f. Verify your work.



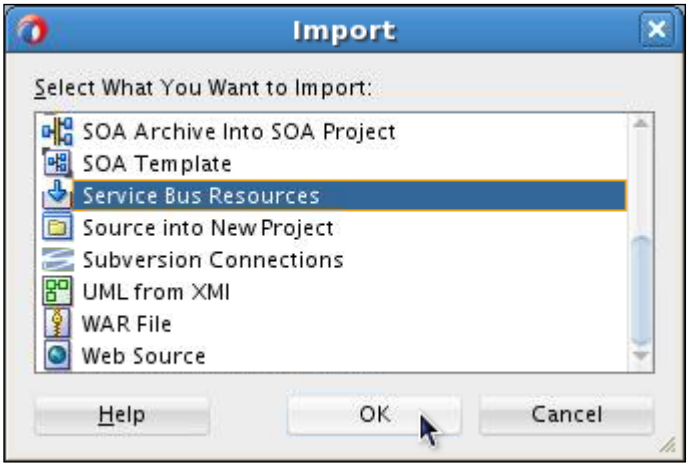
5. Import Artifacts. This step is to register the WSDL for the ValidatePayment service that you want to use in the business service that you will create in the next step.

   There are many ways to share artifacts between Service Bus and Composite applications. (For example: source control, MDS backed by source control, etc.) In this practice, the artifacts will be imported from the file system.

   a. Right-click the **WSDLs** folder that was just created in the left-hand navigation pane, and select Import from the context menu.

      The Import dialog box is displayed.

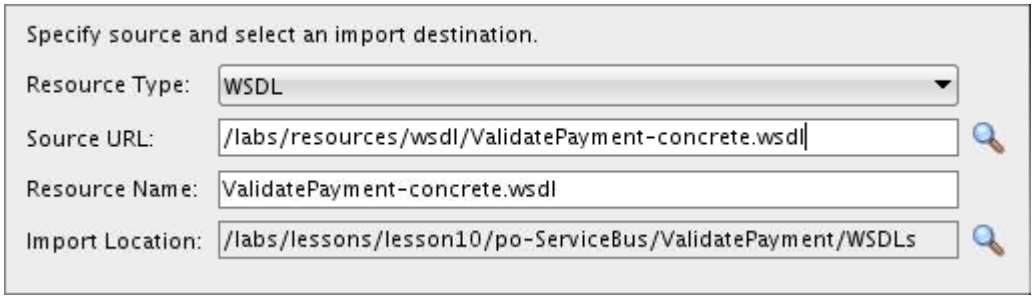
   b. Select **Service Bus Resources**.



   c. Click **OK**.

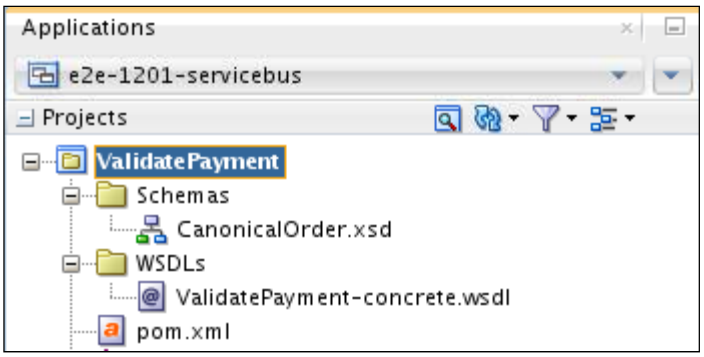
   The Import Service Bus resources wizard is displayed. The title bar of the wizard dialog box shows the step number.

   d. Complete the steps in the wizard by following the instructions in the following table:

Step	Window/Page Description	Choices or Values
1)	Type	Select **resources from URL.** Click Next.
2)	Source	Click the Browse icon next to the Source URL field. In the Select WSDL dialog box, select File System in the top box, navigate to the `/labs/resources/wsdl` folder, and select `ValidatePayment-concrete.wsdl`.  Click Next.
3)	Configuration	Accept the default. Make sure that you see a Schemas folder at the same level as the wsdl folder

Step	Window/Page Description	Choices or Values
		containing CanonicalOrder.xsd. This directory structure is required to successfully import the WSDL. Click Finish.

The Applications pane resembles the following:



Service Bus supports the ability to import and export artifacts and projects at a fine-grain level. You may import artifacts individually, such as a WSDL or schema, or whole projects. You may control whether dependencies are included.

By default, when importing individual artifacts, Service Bus will attempt to import all dependencies declared in the artifact. For example, if a WSDL includes a schema, the schema will also be imported if the paths are relative.

Also note that you can control whether environmental settings, security policies, and credentials are preserved on import. If you are bringing artifacts from a production environment for testing and editing, you may not want all the same policies applied in the development environment, for example.

Configure a business service

In this section, you create and configure a business service to represent the backend `validatePayment` composite.

There are different ways to create artifacts in Service Bus in 12c JDeveloper:

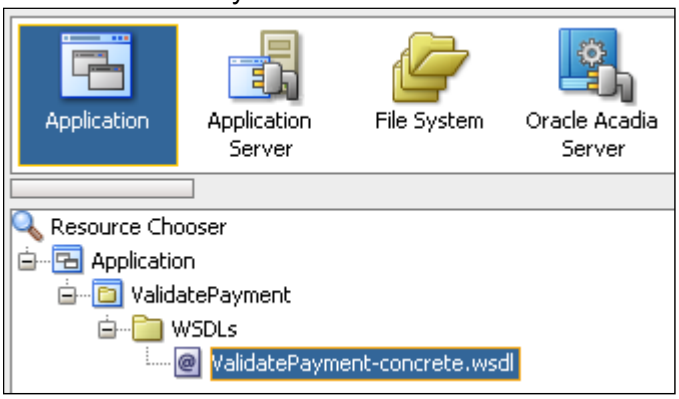
- Use the right-click menu in the left Applications pane.
- Drag-and-drop icons from the component palette onto the Overview canvas.
- Right-click directly on the overview canvas to insert artifacts.

You will use the drag-and-drop component palette to build the Service Bus project; however, feel free to experiment with other mechanisms.

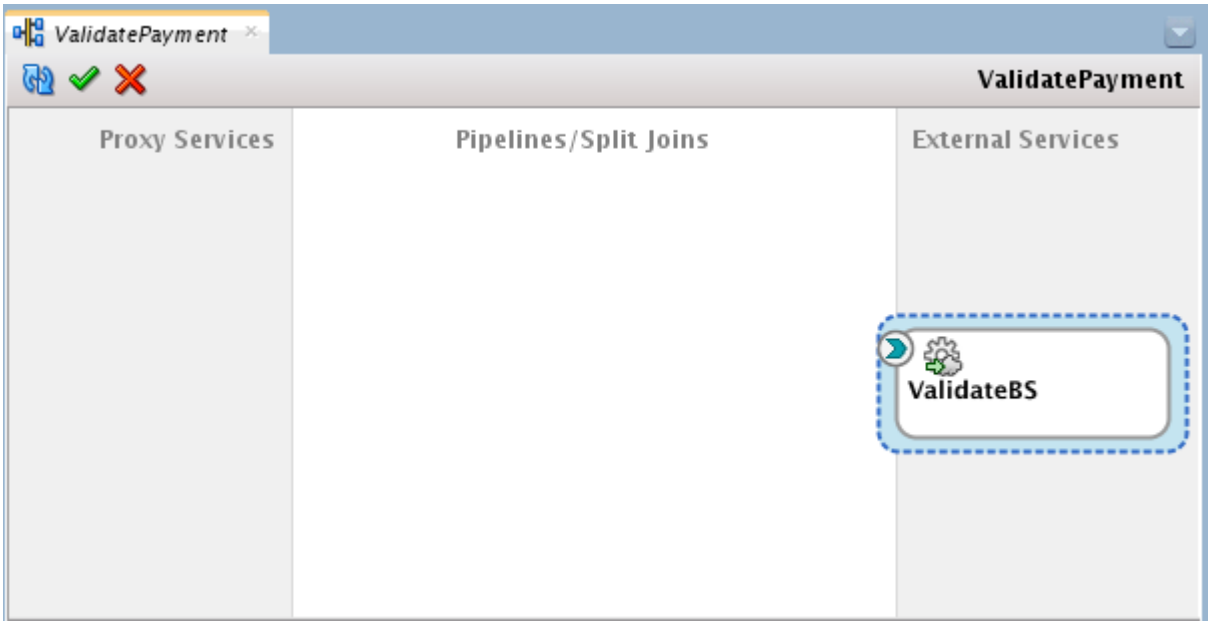
6. Create and configure a business service.
   a. Drag the HTTP icon from the **Technology** section of the component palette onto the **External Services** lane.

      The Create business service wizard is displayed.

b. Complete the following steps in the wizard by using instructions in the following table and associated screenshots:

Step	Window/Page Description	Choices or Values
1)	Create Service	Name the service `ValidateBS`. Click Next.
2)	Type	1) Select **WSDL** for Service Type. 2) Click the Browse WSDLs icon to the right of the WSDL choice. 3) In the Select WSDL dialog box, click the **Application** icon in the top box. 4) Expand Application > ValidatePayment > WSDLs. 5) Select ValidatePayment-concrete.wsdl.  6) Click OK. You are returned to the wizard. 7) Confirm that ValidatePaymentPort is selected in the Port field. 8) Click **Next**.
3)	Transport	Verify that http is selected in the Transport field. Confirm that Endpoint URI is set to the validatePayment composite. It should look similar to this: http://localhost:7101/soa-infra/services/default/ValidatePayment/validatepaymentprocess_client_ep Click Finish.

c. Verify your work.



Note

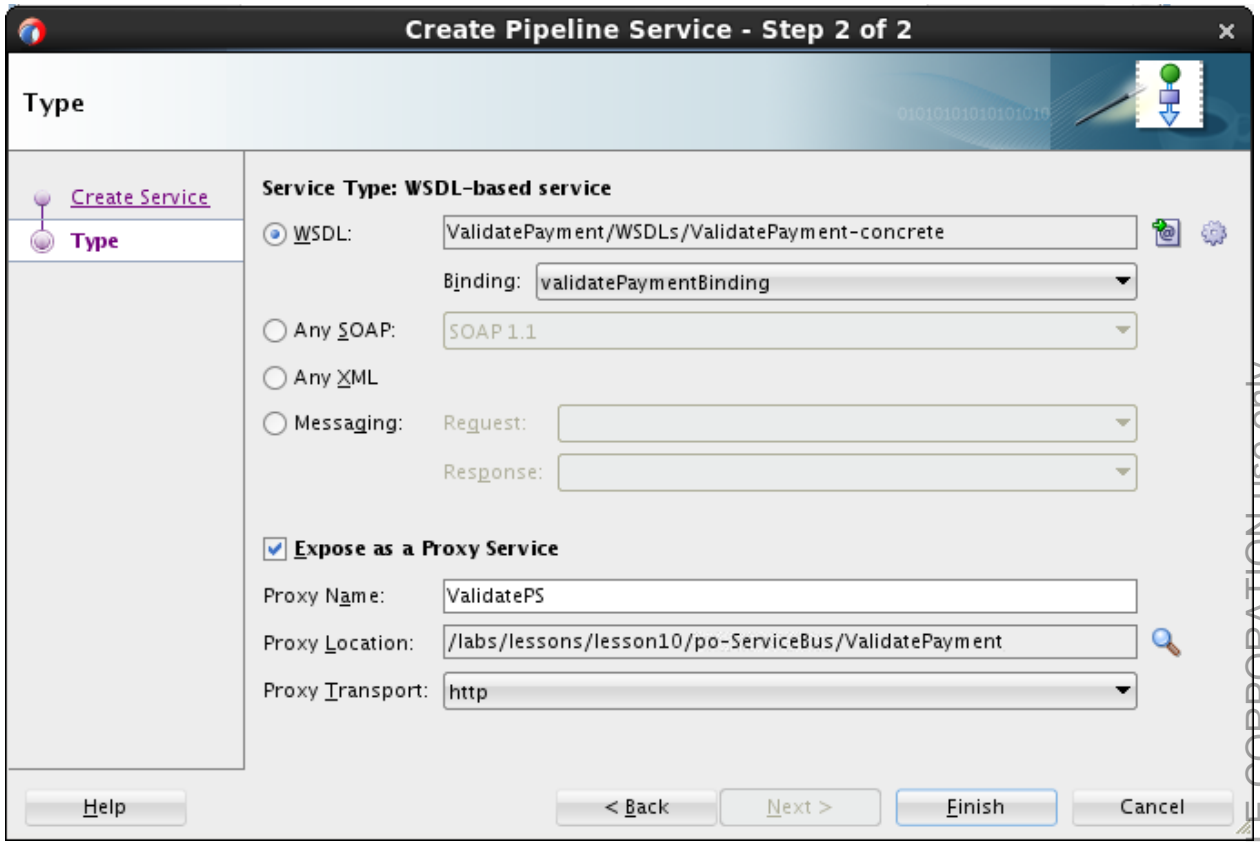
There are several ways to check the deployed endpoint of a Composite. One way is to visit the EM console (http://localhost:7101/em) and navigate to the composite. Click the Test icon near the top right of window. This will bring up a Web Service test page that lists the deployed endpoint.

If needed, the endpoint URI can be updated as you review the business service properties on the Transport tab.

Configure the proxy and pipeline

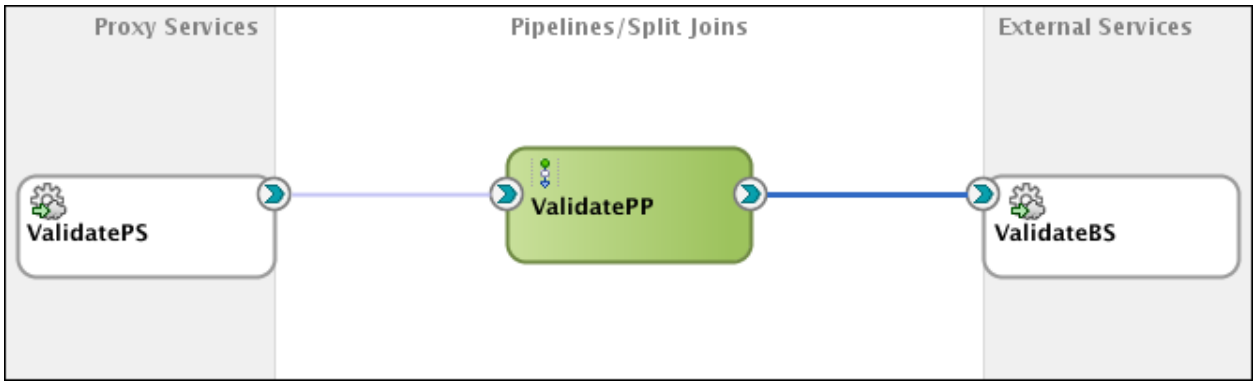
Now you create the proxy and pipeline to invoke the `ValidateBS` business service. The proxy will be the interface to the service from external consumers. The pipeline contains actions that must be performed before invoking the composite. Typical actions are data transform and validation, reporting with error handling. For this practice, you will create a simple pipeline.

7. Create and configure a pipeline.
   a. Drag the pipeline icon from the resources section of the component palette onto the **components** lane.
      The Create Pipeline Service wizard is displayed. The title shows the current step.
   b. Name the service `ValidatePP`.
   c. Click **Next**.
      The wizard advances to step 2.
   d. Select service type WSDL.
   e. Click the **WSDL chooser** icon on the right.
      The WSDL chooser is displayed.
   f. Navigate to Application > ValidatePayment > WSDLs directory, and select **`ValidatePayment-concrete.wsdl`**.
   g. Click **OK**.
      You are returned to the wizard.
   h. Ensure that the “Expose as a Proxy Service” check box is selected (the default).
   i. Name the proxy `ValidatePS`.



j. Click **Finish**.

8. To wire the pipeline to invoke the business service, select `ValidatePP` and drag the arrow to `ValidateBS`.



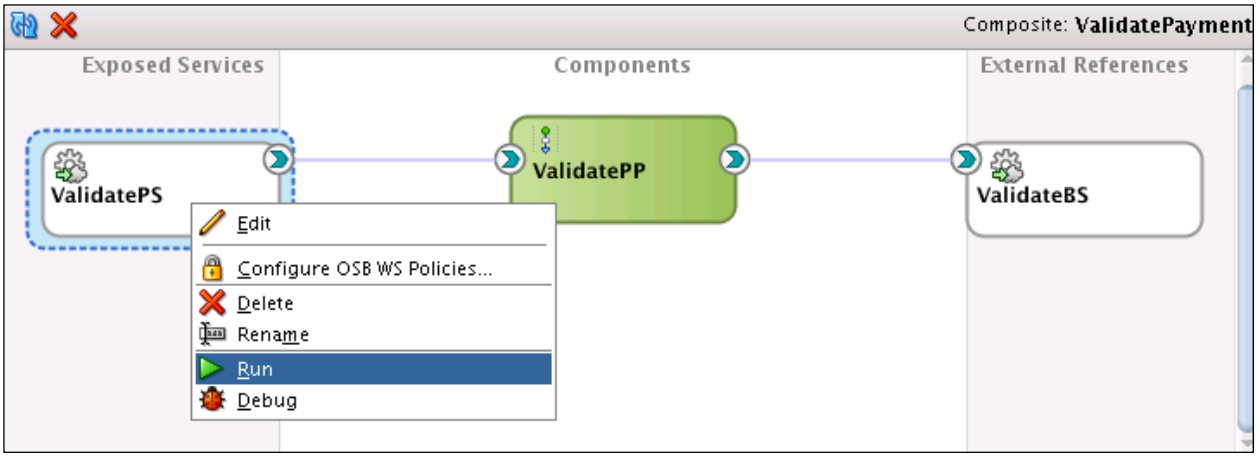
The Routing action is automatically configured in the pipeline. Normally, you would add some actions to the pipeline to validate, transform the payload, or report for auditing.

9. Save your work.

Deploy and test

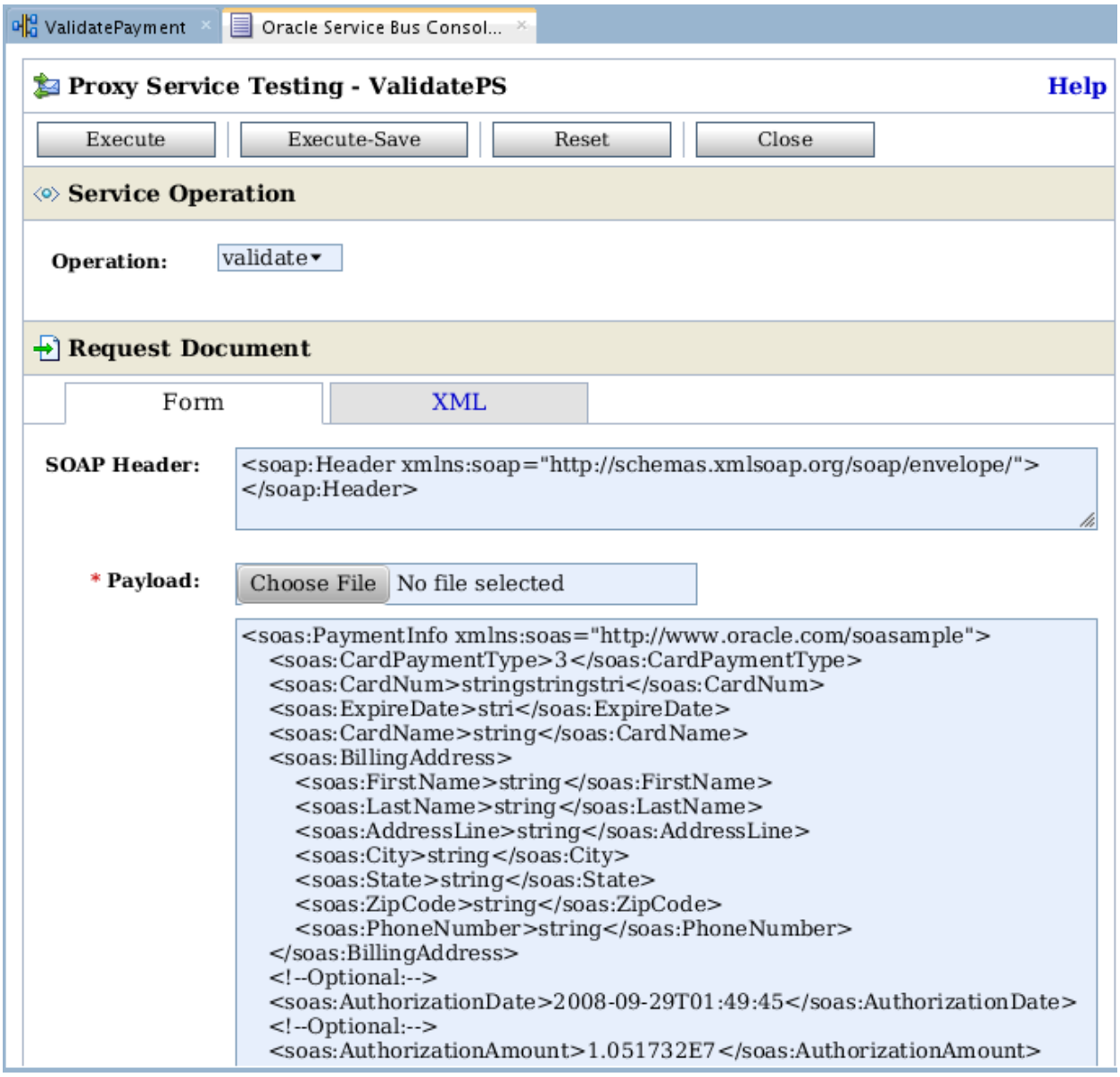
Now you are ready to deploy and test end-to-end. Make sure that the Integrated Server is running.

10. On the canvas, right-click the `ValidatePS` exposed service, and select **Run**.

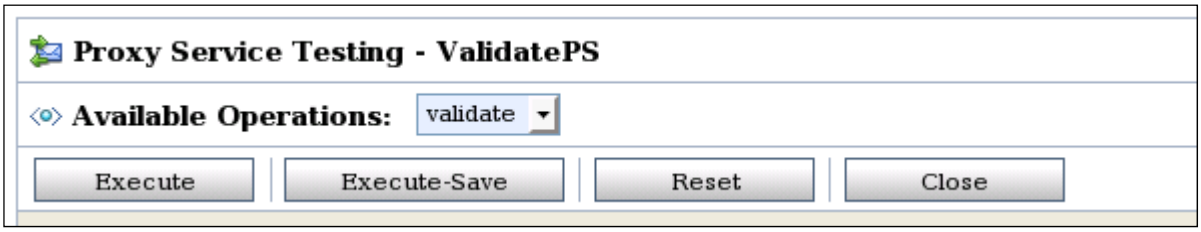


The OSB Test Console window is displayed in JDeveloper.

By default, a sample payload will be generated. However, you will test with a specific file.



11. Select a file.
    a. Click the **Choose File** button.
    b. In the pop-up file explorer window, select File System, navigate to `/labs/resources/sample_input/`, and select **`PaymentInfoSample_Authorized.xml`**
    c. Click **Open**.
12. On the Test Console, click the **Execute** button.

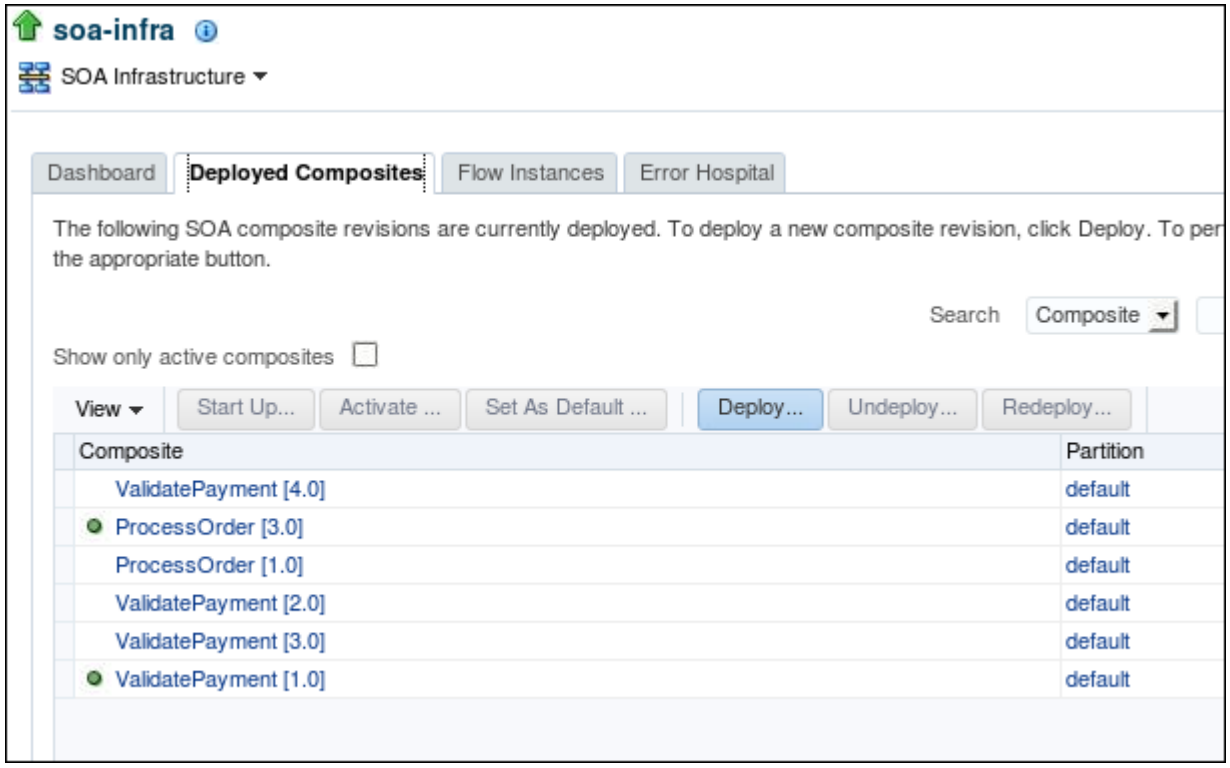


The next screen, in the Response Document section, indicates that the payment has been authorized.

```
<env:Body>
    <PaymentStatus xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tns="http://xmlns.oracle.com/pcbpel/adapter/db/e
                   1201-composites/ValidatePayment/PaymentInfoDB" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
                   xmlns:ns2="http://www.oracle.com/ValidatePayment" xmlns:ns1="http://www.oracle.com/soasample"
                   xmlns="http://www.oracle.com/soasample" xmlns:plt="http://schemas.xmlsoap.org/ws/2003/05/partner-link/"
                   xmlns:jca="http://xmlns.oracle.com/pcbpel/wsdl/jca/">
        <Status>Authorized</Status>
    </PaymentStatus>
</env:Body>
```

Do you know which composite revision is used for this test?

13. Examine the service revision that serves the test
    a. In the Enterprise Manager navigator, select SOA > soa-infra.
    b. In the soa-infra home page (right pane), click Deployed Composites. You should see a list of all composite applications currently deployed in the SOA Infrastructure. Use the following screenshot as a guide:



The green dot to the left of the composite name indicates that this is the default revision of the application.

You should see ValidatePayment revision 1.0 is marked as the default revision.

c. Click the ValidatePayment [1.0] link.

The ValidatePayment [1.0] home page is displayed.

d. Click the Flow Instances tab.

e. In the Search Options pop-up panel, click Search.

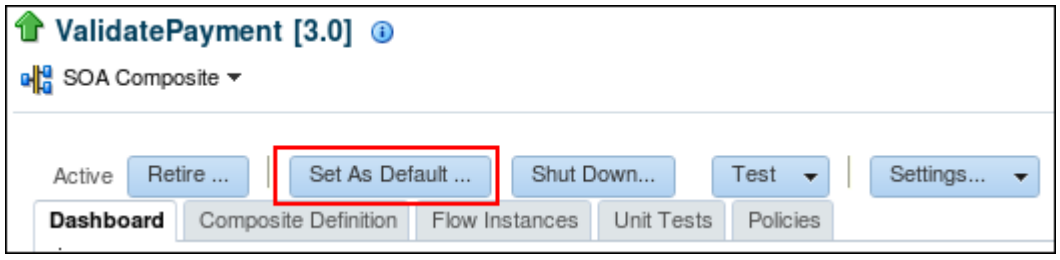
You should see the instance generated from the recent Service Bus test.

14. Change the default revision of the application and retest.

a. In the Enterprise Manager navigator, select ValidatePayment [3.0] (added a business rule).

The ValidatePayment [3.0] home page is displayed.

b. Click Set As Default at the top of the page, and click Yes when prompted for confirmation.



You should see a success confirmation message.

c. Go back to the OSB Test Console window opened in JDeveloper.

d. Select a new test file.

1) Click the **Choose File** button.

2) In the pop-up file explorer window, select File System, navigate to `/labs/resources/sample_input/`, and select **`PaymentInfoSample_ManualOverride.xml`**
3) Click **Open**.

e. On the Test Console, click the **Execute** button.

f. The next screen, in the Response Document section, you should see that the payment needs manual approval.

```
<env:Body>
    <PaymentStatus xmlns:jca="http://xmlns.oracle.com/pcbpel/wsdl/jca/" xmlns:wsdl="http://schemas.xmls
                   xmlns:ns3="http://xmlns.oracle.com/pcbpel/adapter/db/po_composite/ValidatePayment/
                   xmlns:tns="http://xmlns.oracle.com/pcbpel/adapter/db/e2e-1201-composites/ValidatePa
                   xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:oraxsl="http://www.oracle.
                   xmlns:plt="http://schemas.xmlsoap.org/ws/2003/05/partner-link/" xmlns:ns1="http://ww
        <Status>Manual Approval Required</Status>
        <DailyLimitAmount>1000.0</DailyLimitAmount>
    </PaymentStatus>
</env:Body>
```

Explore the Oracle Service Bus Console

You can use the Oracle Service Bus Console to configure services and other Service Bus resources.

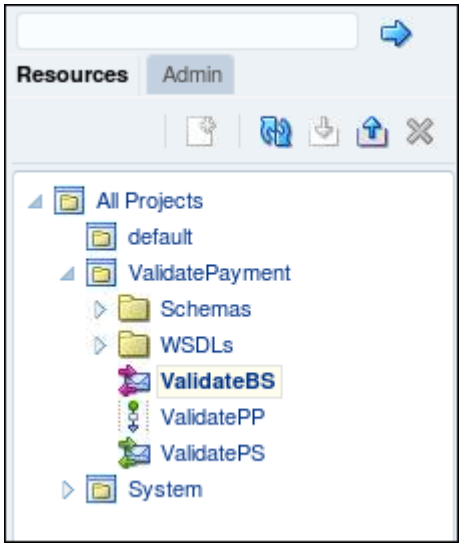
15. In the Firefox browser, open a new tab and access the Oracle Service Bus Console with the following URL:

    `http://localhost:7101/sbconsole`

16. If prompted for login, use the same credentials as used for Fusion Middleware Control.

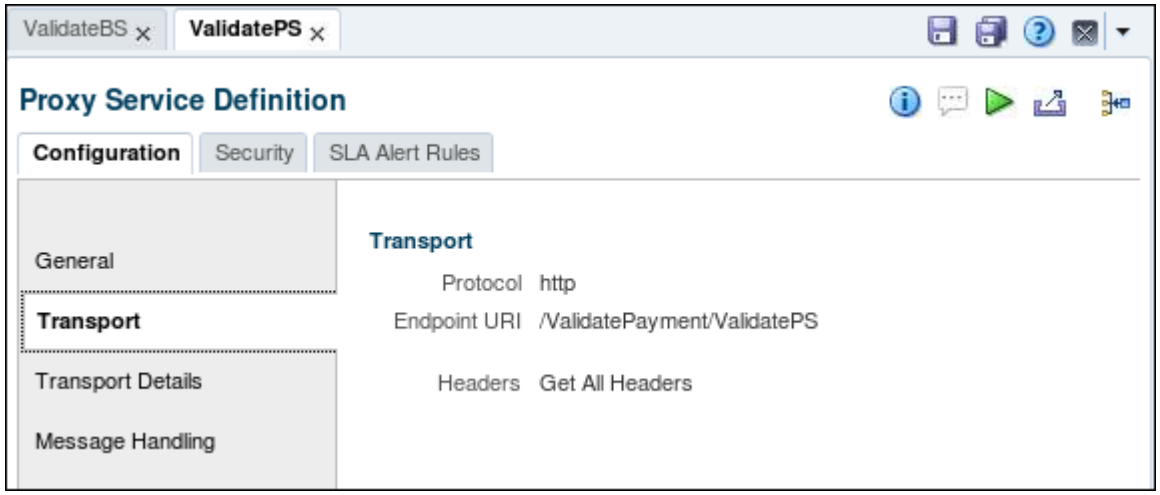
    When you first log in, Fusion Middleware Control is displayed with all projects, folders, and resources in the Project Navigator in a tree view on the left.

17. Expand the ValidatePayment project, you should see a folder structure similar to the one in JDeveloper.



18. You can select the business service, pipeline, and proxy service to view their definitions on the right.

    a. Select ValidatePS, and click the Transport tab on the Proxy Service Definition page. You should see the transportation details of the proxy service as shown in the following screenshot:



Note: The Endpoint URI is what service consumers will use to access your ValidatePayment service.

b. To verify,

1) Copy Endpoint URI on the page.

2) Open another tab in Firefox browser, and type the following in the URL:
`http://localhost:7101/<paste_Endpoint_URI_here>?wsdl`
The wsdl of the ValidatePS proxy service is displayed.

```
−<WL5G3N0:definitions targetNamespace="http://www.oracle.com/ValidatePayment">
  −<WL5G3N0:types>
    −<xsd:schema>
       <xsd:import namespace="http://www.oracle.com/soasample"/>
       <xsd:import namespace="http://www.oracle.com/soasample" schemaLocation="http://localhost:7101
       /ValidatePayment/ValidatePS?SCHEMA%2FValidatePayment%2FSchemas%2FCanonicalOrder"/>
     </xsd:schema>
   </WL5G3N0:types>
  −<WL5G3N0:message name="validateInput">
     <WL5G3N0:part element="WL5G3N1:PaymentInfo" name="paymentInfo"/>
   </WL5G3N0:message>
  −<WL5G3N0:message name="validateOutput">
     <WL5G3N0:part element="WL5G3N1:PaymentStatus" name="status"/>
   </WL5G3N0:message>
  −<WL5G3N0:portType name="validatePaymentPortType">
    −<WL5G3N0:operation name="validate">
       <WL5G3N0:input message="WL5G3N2:validateInput"/>
       <WL5G3N0:output message="WL5G3N2:validateOutput"/>
     </WL5G3N0:operation>
   </WL5G3N0:portType>
  −<WL5G3N0:binding name="validatePaymentBinding" type="WL5G3N2:validatePaymentPortType">
     <WL5G3N3:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    −<WL5G3N0:operation name="validate">
       <WL5G3N3:operation soapAction="validate" style="document"/>
      −<WL5G3N0:input>
         <WL5G3N3:body use="literal"/>
       </WL5G3N0:input>
      −<WL5G3N0:output>
         <WL5G3N3:body use="literal"/>
```

c. You can explore more services and pipeline.

Question: How to make your Process Order service call the registered ValidatePayment service (Proxy) for payment validation?

19. When you are done, do the following:
    - Close Oracle Service Bus Console in the browser.
    - Close the po_ServiceBus application in JDeveloper (including all the open windows).

Practices for Lesson 11: Managing, Monitoring, and Troubleshooting Composite Applications

Chapter 11

Practices for Lesson 11: Overview

Practices Overview

In the practices for this lesson, the key tasks are to:

- Deploy a new version of ProcessOrder composite application through EM console.
- Initiate a ProcessOrder composite instance, monitor the process execution and activity, and recover the fault using the EM Control console.

Practice 11-1: Deploying the Composite Through EM Console

Overview

In this practice, you deploy the archive of the ProcessOrder composite application by using the Oracle Enterprise Manager Fusion Middleware Control console.

In this revision, ProcessOrder invokes the proxy service to access the backend credit card validation service.

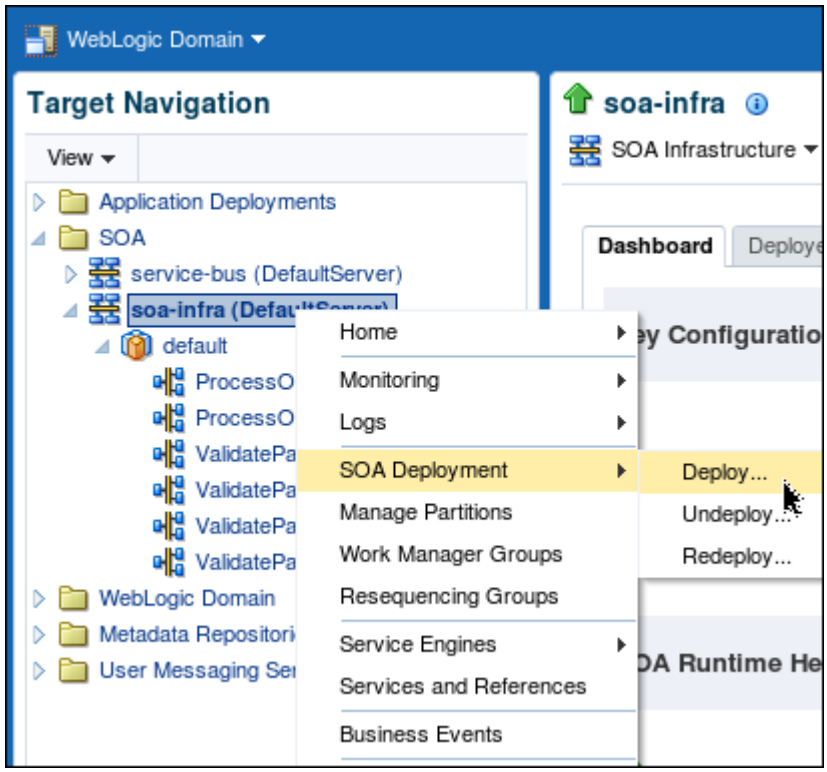
Assumptions

The `ValidatePayment` service has been deployed and is running on the SOA server.

Tasks

To deploy the ProcessOrder application through the Oracle Enterprise Manager Fusion Middleware Control console, perform the following steps:

1. Log in to the Oracle Enterprise Manager Fusion Middleware Control console.
   a. If the console is not already visible, open a web browser window and enter the URL `http://localhost:7101/em`.
   b. On the Oracle Enterprise Manager Fusion Middleware Control console login page, enter the username `weblogic` and password `welcome1`, and click Login.
2. Deploy the ProcessOrder application to the default partition in the SOA infrastructure.
   a. In Target Navigation panel, expand the SOA folder, right-click soa-infra, and select SOA Deployment > Deploy.



   b. On the Deploy SOA Composite : Select Archive page, perform the following steps:
      1) In the “Archive or Exploded Directory” section, accept the default “Archive is on the machine where this web browser is running” option and click Browse.

2) In the “Choose file” dialog box, navigate to the `/labs/lessons/lesson11/po_composite/ProcessOrder/deploy` folder, select `sca_ProcessOrder_rev4.0.jar`, and click **Open**.
3) Verify that the selected JAR file is shown in the “Archive is on the machine where this web browser is running” option field, and click **Next**.

c. On the Deploy SOA Composite: Select Target page, in the SOA Partition section, make sure that `default` is selected from the drop-down menu, and click **Next**.
d. On the Deploy SOA Composite: Confirmation page, accept the default settings and click **Deploy**.

   Note: A deployment progress dialog box appears, showing the progress of the deployment operation until deployment succeeds (as in this case) or fails.
e. Close the Deployment Succeeded dialog box.
f. In the (left) Navigator frame, the deployed ProcessOrder[4.0] is selected, and the ProcessOrder [4.0] application page appears in the frame at the right.

Practice 11-2: Testing the Application with a Fault Scenario

Overview

In this practice, you create a fault scenario by sending a bad order to the ProcessOrder, examine the instance Flow Trace pages to track the message flow through the composite application, check the error message, and take corrective action to recover the faulted instance.

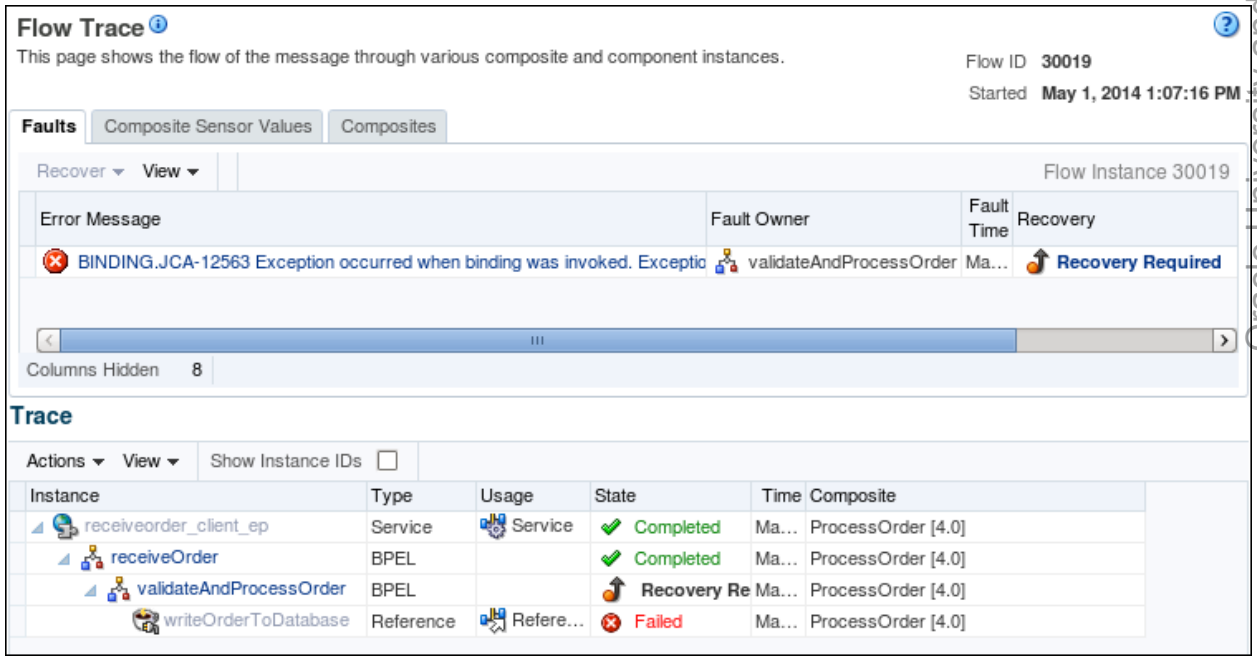
Assumptions

You have successfully deployed the ProcessOrder[4.0] application as described in the previous practice.

Tasks

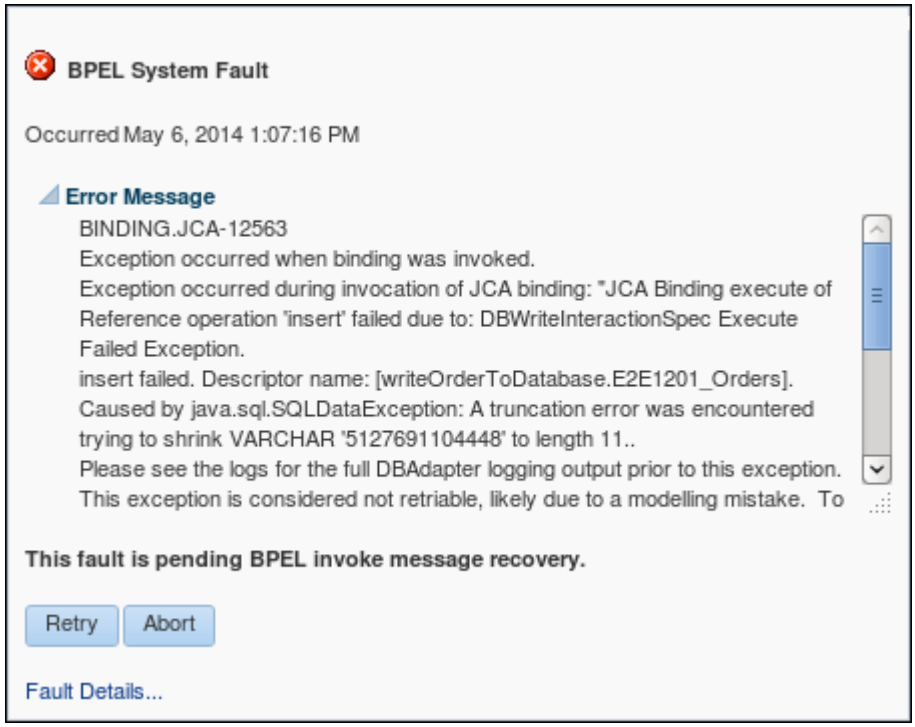
1. Test with a bad order
   a. In Target Navigation panel, click the ProcessOrder[4.0] link. The composite application home page (on the Dashboard tab) opens in the right pane.
   b. Click the **Test** button at the top of the screen.
   c. The Test page opens.
   d. Scroll down to the **Input Arguments** section.
   e. Click **Browse**. In the File Upload window, select the `/labs/resources/sample_input/BadOrderSample.xml` file, and open it.
   f. Click **Test Web Service**.

   The Response shows an order number.
2. Check the outcome of the test.
   a. Click Launch Flow Trace. You should see a recoverable fault in the instance trace as shown in the following screenshot:



   b. Click the Recovery Required link.

   An error message dialog box is displayed. Read the details to help you troubleshoot the problem.

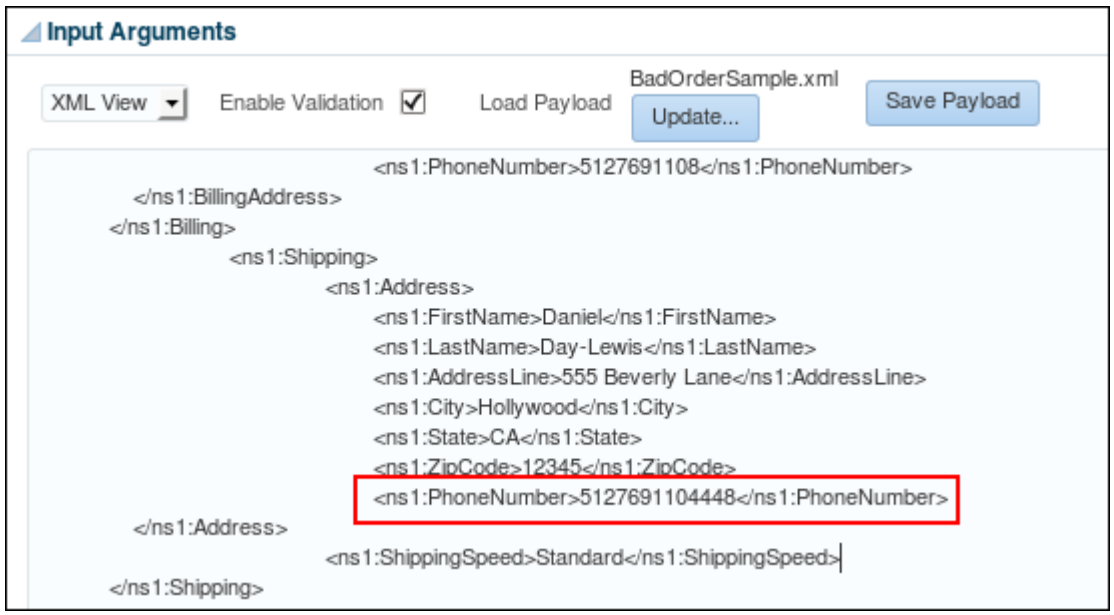


The error is caused by the wrong length of an element with value of "5127691104448" in the order file.

c. Close the flow trace window.

3. Recover the faulted instance

a. Go back to the EM console, click the Request tab, and change to XML View in the Input Arguments section. You should see a phone number in the shipping address.



b. Check the defined length of the phone number element.

1) Open the gedit text editor.

2) Select File > Open from the main menu.
The Open Files window is displayed.

3) Open the `/labs/resources/Schemas/CanonicalOrder.xsd` file in the editor.
4) Search for the element with name “PhoneNumber,” and you should see its maximum length is defined as 11.
5) Close the gedit editor.

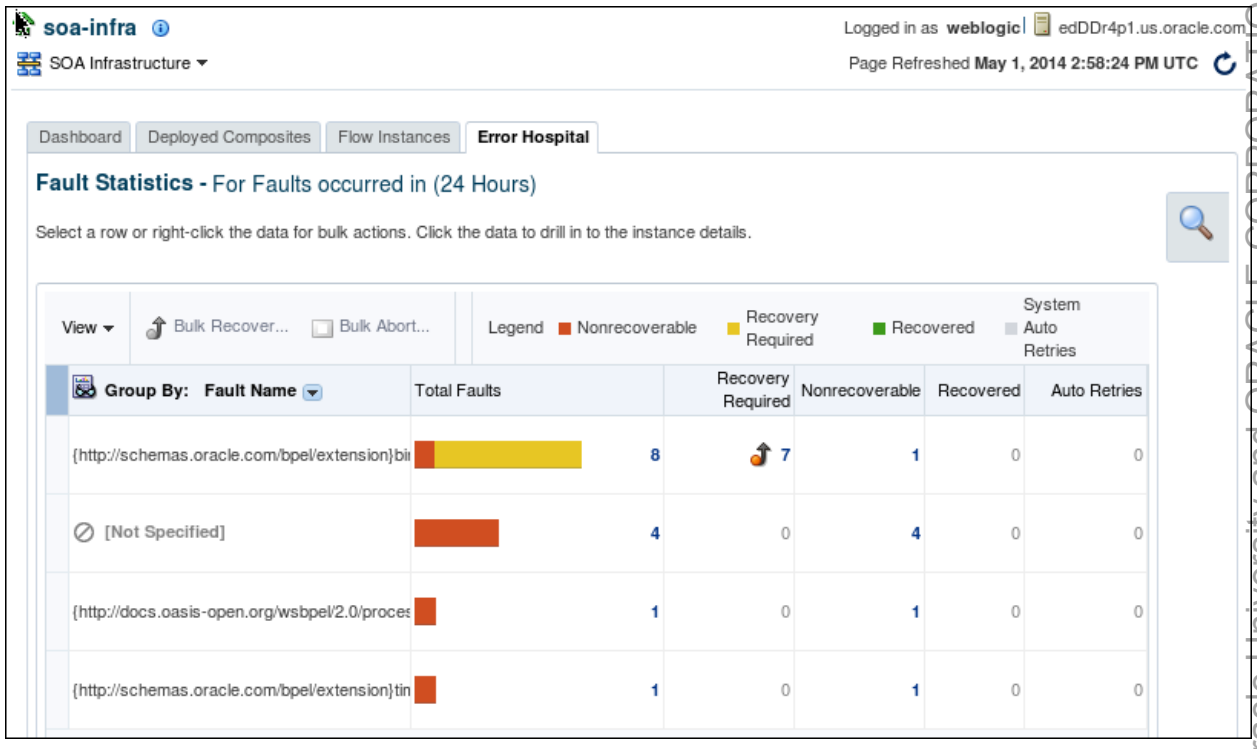
c. Go back in the Request tab, change the phone number to less than 11 digit (for example, 51276911044), and then click **Test Web Service**.

d. Launch the flow trace. This time you should see that the instance is completed without any error.

4. View fault data in Error Hospital.

a. In the Enterprise Manager navigator, select SOA > soa-infra.

b. In the soa-infra home page (right pane), click the Error Hospital tab. In the popup Report Filters panel, click Search with the default settings. You should see a chart of Fault Statistics similar to the following screenshot:



Practices for Lesson 12: Enabling On-Premises Integration

Chapter 12

Practices for Lesson 12

Practices Overview

There are no practices for Lesson 12.

Practices for Lesson 13: Enabling Mobile and Cloud Integration

Chapter 13

Practices for Lesson 13

Practices Overview

There are no practices for Lesson 13.